

RTL Design for SoC & ASIC

The Complete Guide - From Beginner to Expert

Digital Logic · Verilog & SystemVerilog · Simulation Semantics · FSMs · Datapaths · FIFOs · Pipelines · Clock-Domain Crossing · Resets · APB / AHB / AXI · Interconnect · Registers & Interrupts · Peripherals · Synthesis · STA & SDC · Low Power & UPF · DFT · PPA · Testbenches · SVA · UVM · Formal · Projects · Career Roadmap

Contents	30 chapters in 6 parts, plus 4 appendices
Level	From your first module to micro-architecture, sign-off and tape-out readiness
Goal	Every RTL topic you need on the road to SoC and ASIC design, in the order you need it
Every example runs	All synthesizable RTL and testbenches were simulated with Icarus Verilog 12 ; the output on green cards is the output it actually produced
Languages	Verilog-2005 and the synthesizable subset of SystemVerilog (IEEE 1800), plus SVA, UVM, SDC, UPF and Tcl where the flow needs them
Assumed	Curiosity and basic programming. No prior hardware design experience.

How To Use This Book

RTL - register-transfer level - is the level at which almost every digital chip is designed today. An RTL description says which **registers** exist, and what **combinational logic** transforms the values moving between them on every clock edge. Everything upstream of RTL (architecture, performance models) decides *what* to build; everything downstream (synthesis, place and route, sign-off) turns your RTL into transistors. The RTL is the contract between the two, and it is where most of a chip's bugs, most of its timing problems and most of its power are decided.

This book follows that contract from both sides. Parts I and II teach the language and the building blocks until you can write any block correctly. Part III assembles blocks into an SoC. Part IV shows what the back end does with your code, so that you write RTL that closes timing, saves power and can be tested. Part V covers how the design is proven correct, and Part VI puts it all together in two complete, simulated projects and a career roadmap.

The six parts

Part	Chapters	What you get out of it
I - Foundations	1-4	Where RTL sits in the SoC/ASIC flow; the digital logic you must know cold; Verilog and SystemVerilog for design; and the event scheduler that explains blocking vs non-blocking assignments.
II - Core Building Blocks	5-10	Combinational and sequential logic, resets, FSMs, arithmetic datapaths, memories and FIFOs, pipelines and valid/ready flow control.
III - SoC Architecture	11-16	Clock-domain crossing, reset architecture, AMBA buses (APB, AHB, AXI), interconnect and arbitration, register maps and interrupts, peripherals, DMA and processor integration.
IV - Implementation-Aware RTL	17-21	Synthesis, static timing and SDC, low-power design and UPF, design for test, PPA trade-offs and the physical-design hand-off.
V - Verification	22-26	Self-checking testbenches, assertions and coverage, UVM, static sign-off (lint, CDC, formal, LEC), debug, gate-level simulation, FPGA prototyping and emulation.
VI - Projects and Roadmap	27-30	Coding guidelines and reviews; an APB UART subsystem; an AXI-Stream int8 MAC accelerator for on-device ML; and the skills, study plan and interview preparation for an RTL/SoC career.

Reading paths

- **Complete beginner:** read Parts I and II in order, typing and simulating every example. Chapter 4 is the one most people skip and most regret skipping.
- **Software engineer moving to hardware:** 1, 2, 3, 4, 6, 10 - then Chapter 11 before you connect anything to a second clock.
- **FPGA designer moving to ASIC:** 1, 11, 12, 17-21 and 25 - the topics FPGA tools hide from you.
- **Verification engineer:** 3, 4, 10, 13, then all of Part V.
- **Preparing for interviews:** Chapters 2, 4, 7, 9, 11, 18, then Appendix C and Chapter 30.
- **Building an SoC for the first time:** 11-16 in order, then the projects in Chapters 28 and 29.

Conventions

Style	Meaning
A grey card	RTL, a testbench, a constraint file or a script, exactly as you would save it
A green card	The output the code above it produced when simulated with Icarus Verilog 12 while this book was written

Style	Meaning
An ASCII figure	The hardware a piece of code infers, a block diagram, or a timing diagram
Coloured boxes	KEY IDEA = the one thing to remember. MATH = a derivation or equation. INTUITION = the mental picture. PRACTICAL TIP = what to do. PITFALL = a mistake that reaches silicon. EXPERT CORNER = depth you can skip on a first read.
<code>_q / _d / _n</code>	Register output / next-state input / active-low signal (Chapter 27 gives the full naming convention)

SIMULATE AS YOU READ

Install a free simulator - `iverilog` (Icarus Verilog) and `gtkwave` are one package-manager command away on Linux and macOS, and Verilator is the fastest open-source option. Type the examples, run them, change them and look at the waveforms. RTL design is learned at the simulator and the whiteboard, not by reading alone.

THE ONE HABIT THAT SEPARATES GOOD RTL DESIGNERS

Before writing a line of code, **draw the hardware**: the registers, the logic between them, and the clock that drives them. RTL is a description of a circuit, not a program. Every chapter in this book shows the circuit behind the code for exactly this reason.

Table of Contents

PART I - Foundations	14
1. RTL in the SoC/ASIC Flow, and the Roadmap	15
1.1 What 'register-transfer level' means	15
1.2 Levels of abstraction	15
1.3 The ASIC flow, from specification to silicon	16
1.4 Anatomy of a System-on-Chip	17
1.5 FPGA versus ASIC	18
1.6 The people around the RTL designer	19
1.7 The roadmap: how this book is organised	19
1.8 Summary	20
1.9 Exercises	21
2. Digital Logic Essentials for RTL Designers	22
2.1 Number systems and notation	22
2.2 Two's complement	22
2.3 Boolean algebra and the gates it maps to	23
2.4 Multiplexers, decoders and encoders	24
2.5 Karnaugh maps, briefly	25
2.6 Latches and flip-flops	26
2.7 Setup, hold and clock-to-Q	27
2.8 The critical path and maximum frequency	27
2.9 Metastability: a preview	28
2.10 Fan-out, glitches and hazards	28
2.11 Summary	29
2.12 Exercises	29
3. Verilog and SystemVerilog for Design	30
3.1 Modules, ports and hierarchy	30
3.2 Data types: wire, reg, logic, and four-state values	31
3.3 Vectors, part-selects and arrays	32
3.4 Operators	33
3.5 Expression sizing and extension rules	33
3.6 Parameters, localparams and generate	34
3.7 Functions and tasks	36
3.8 typedef, enum, struct and packages	36
3.9 always_comb, always_ff and always_latch	37
3.10 The synthesizable subset	37
3.11 Summary	38
3.12 Exercises	38
4. Simulation Semantics: The Event Scheduler, Blocking vs Non-Blocking, and Races	39

4.1 Processes, events and time steps	39
4.2 The stratified event queue	39
4.3 Delta cycles	40
4.4 Blocking versus non-blocking assignment	40
4.5 Why blocking for combinational logic	43
4.6 Races between always blocks and between testbench and DUT	43
4.7 Sensitivity lists and simulation/synthesis mismatch	44
4.8 X-propagation: optimism and pessimism	45
4.9 Summary	46
4.10 Exercises	46

PART II - Core RTL Building Blocks 47

5. Combinational Logic	48
5.1 assign versus always_comb	48
5.2 Priority logic versus parallel logic	49
5.3 unique, priority, and the dangers of parallel_case	49
5.4 casez, casex and don't-cares	50
5.5 Decoders, encoders and one-hot logic	51
5.6 Unintended latches, and how never to write one	52
5.7 Parity, comparators and other everyday functions	54
5.8 Combinational loops and logic depth	54
5.9 Summary	55
5.10 Exercises	55
6. Sequential Logic: Flip-Flops, Resets and Enables	56
6.1 Coding the D flip-flop	56
6.2 Synchronous versus asynchronous reset	57
6.3 Flops without reset	58
6.4 Enables, and a first look at clock gating	58
6.5 Shift registers	58
6.6 Counters	58
6.7 Edge detectors, pulse stretchers and debouncers	60
6.8 Register retiming	62
6.9 Summary	62
6.10 Exercises	63
7. Finite State Machines	64
7.1 Moore versus Mealy	64
7.2 Coding styles: one, two or three always blocks	64
7.3 State encoding	67
7.4 Safe FSMs and illegal-state recovery	67
7.5 Registered outputs	68
7.6 FSMD: a state machine with a datapath	69

7.7 FSM pitfalls	70
7.8 Summary	70
7.9 Exercises	71
8. Datapath and Arithmetic	72
8.1 Adders: from ripple-carry to parallel prefix	72
8.2 Subtraction, overflow and saturation	73
8.3 Signed and unsigned: the rules that cause real bugs	74
8.4 Multipliers	75
8.5 Shifters and the barrel shifter	75
8.6 Division	76
8.7 Fixed-point arithmetic and Q formats	77
8.8 Resource sharing	78
8.9 A verified saturating MAC	78
8.10 Summary	80
8.11 Exercises	80
9. Memories, Register Files and Synchronous FIFOs	81
9.1 Flop-based register files	81
9.2 SRAM macros versus inferred memory	82
9.3 Single-port and dual-port RAMs, and read-during-write	82
9.4 Parity and ECC basics	83
9.5 The synchronous FIFO	84
9.6 Standard versus first-word-fall-through (FWFT)	86
9.7 Content-addressable memory (CAM), briefly	87
9.8 Summary	87
9.9 Exercises	87
10. Pipelining, Valid/Ready Handshakes and Flow Control	88
10.1 Latency versus throughput	88
10.2 Pipeline registers and valid bits	88
10.3 Hazards in datapath pipelines	89
10.4 The valid/ready handshake	89
10.5 A pipeline register with ready propagation	90
10.6 The skid buffer	90
10.7 Credit-based flow control	92
10.8 Elastic buffers and bubble collapsing	93
10.9 Throughput math for handshaked pipelines	93
10.10 Summary	94
10.11 Exercises	94
PART III - SoC Architecture in RTL	95
11. Clocks and Clock-Domain Crossing	96
11.1 Where clocks come from	96

11.2 How two clocks can be related	96
11.3 Metastability and MTBF	97
11.4 The two-flop synchronizer	97
11.5 What must never cross unsynchronized	98
11.6 Pulse (toggle) synchronizer	98
11.7 Request/acknowledge handshake	99
11.8 Multi-bit crossings and reconvergence	101
11.9 The asynchronous FIFO	101
11.10 Glitch-free clock multiplexing	104
11.11 Summary	105
11.12 Exercises	105
12. Reset Architecture and RDC	106
12.1 Kinds of reset in an SoC	106
12.2 Asynchronous assert, synchronous de-assert	106
12.3 Reset distribution trees	107
12.4 Reset sequencing	108
12.5 Reset-domain crossing (RDC)	108
12.6 Which flops need a reset?	109
12.7 Reset and DFT	109
12.8 X-propagation and reset	109
12.9 Summary	109
12.10 Exercises	110
13. On-Chip Buses: APB, AHB, AXI4, AXI4-Lite, AXI4-Stream	111
13.1 The AMBA family at a glance	111
13.2 APB: the peripheral bus	111
13.3 AHB-Lite: pipelined address and data	114
13.4 AXI4: five independent channels	114
13.5 An AXI4-Lite slave	116
13.6 AXI4-Stream	118
13.7 Choosing a protocol	118
13.8 Bridges	118
13.9 Summary	119
13.10 Exercises	119
14. Interconnect, Arbitration, Memory Maps and NoCs	120
14.1 Memory maps and address decoding	120
14.2 Shared bus versus crossbar	120
14.3 Arbiters	121
14.4 Starvation, QoS and deadlock at the transaction level	123
14.5 Bridges and converters	123
14.6 Networks-on-chip	123
14.7 Cache coherency in one page	124

14.8 Summary	124
14.9 Exercises	125
15. Control/Status Registers, Interrupts and Register Generation	126
15.1 Register access types	126
15.2 Documenting a register map	126
15.3 A CSR block with interrupts	127
15.4 Interrupts and interrupt controllers	129
15.5 Register generation: SystemRDL and IP-XACT	129
15.6 Verification tie-in: the UVM register layer	130
15.7 Summary	130
15.8 Exercises	131
16. Peripherals, DMA and Processor Integration	132
16.1 GPIO	132
16.2 Timers, PWM and watchdog	132
16.3 UART	133
16.4 SPI	134
16.5 I2C	135
16.6 DMA controllers	136
16.7 Integrating a RISC-V core	137
16.8 Caches and the memory hierarchy, from the RTL side	137
16.9 Putting it together: an example SoC	138
16.10 Summary	139
16.11 Exercises	139

PART IV - Implementation-Aware RTL 140

17. Logic Synthesis: What Your RTL Becomes	141
17.1 The synthesis pipeline	141
17.2 Standard-cell libraries: what the tool maps to	142
17.3 What infers what	143
17.4 Constructs synthesis ignores or rejects	144
17.5 full_case and parallel_case: pragmas that lie	145
17.6 Hierarchy, uniquification and boundary optimization	146
17.7 A synthesis script in three dialects	147
17.8 Reading QoR reports	148
17.9 Summary	149
17.10 Exercises	149
18. Timing: STA, SDC and Closing Timing from RTL	151
18.1 Timing paths	151
18.2 Setup and hold, with skew and uncertainty	151
18.3 SDC: telling the tools what you mean	153
18.4 Closing timing from RTL	155

18.5 Summary	158
18.6 Exercises	159
19. Low-Power RTL and UPF	160
19.1 Where the power goes	160
19.2 Clock gating	160
19.3 Operand isolation and data gating	162
19.4 Multi-voltage design and power gating	163
19.5 UPF: describing power intent	166
19.6 DVFS: dynamic voltage and frequency scaling	167
19.7 Power estimation flow	167
19.8 Summary	168
19.9 Exercises	168
20. Design for Test: Scan, BIST, JTAG	169
20.1 Fault models	169
20.2 Scan: turning sequential test into combinational test	169
20.3 DFT-friendly RTL rules	171
20.4 Memory BIST	172
20.5 Logic BIST	174
20.6 JTAG: the TAP controller and boundary scan	174
20.7 Summary	177
20.8 Exercises	177
21. PPA Trade-offs and the Hand-Off to Physical Design	179
21.1 The PPA triangle	179
21.2 Estimating area from RTL	179
21.3 Floorplan-aware RTL	180
21.4 Congestion: when RTL makes the chip unroutable	181
21.5 The hand-off to physical design	182
21.6 ECOs: changing a design after hand-off	183
21.7 Signoff, seen from the RTL seat	183
21.8 The tapeout checklist	183
21.9 Summary	184
21.10 Exercises	184

PART V - Verification 185

22. Self-Checking Testbenches in SystemVerilog	186
22.1 Why self-checking, and what it must check	186
22.2 Testbench architecture	186
22.3 Clock and reset generation, and driving without races	187
22.4 Tasks for bus transactions	187
22.5 Reporting: \$display, \$error, \$fatal and friends	189
22.6 File I/O: vectors in, logs out	190

22.7 Randomization: \$urandom and constrained-random classes	190
22.8 A complete self-checking testbench: synchronous FIFO	191
22.9 Class-based transactions, a reference function and file vectors	193
22.10 Interfaces, modports, clocking blocks and virtual interfaces	195
22.11 Summary	198
22.12 Exercises	198
23. SystemVerilog Assertions and Functional Coverage	199
23.1 Why assertions pay for themselves	199
23.2 Immediate versus concurrent assertions	199
23.3 Immediate assertions in practice: a protocol checker	200
23.4 Anatomy of a concurrent assertion	201
23.5 Sequence operators	202
23.6 Implication: -> and =>	202
23.7 Sampled-value functions	203
23.8 assert, assume, cover - and disable iff	203
23.9 Binding assertions without touching the RTL	203
23.10 A library of useful assertions	204
23.11 Functional coverage: covergroups, coverpoints, bins and crosses	205
23.12 Coverage closure	206
23.13 Code coverage types	206
23.14 Summary	207
23.15 Exercises	207
24. UVM Methodology	208
24.1 Why UVM	208
24.2 The class hierarchy	208
24.3 Phases	209
24.4 Testbench topology	209
24.5 Sequence items and sequences	210
24.6 Driver, monitor and the sequencer handshake	211
24.7 Agent, scoreboard, environment and TLM connections	212
24.8 Tests, config_db, the factory and objections	214
24.9 The register abstraction layer (RAL)	215
24.10 Debugging UVM testbenches	216
24.11 Summary	217
24.12 Exercises	217
25. Static Sign-off: Lint, CDC/RDC, Formal and Equivalence Checking	218
25.1 Where static checks fit	218
25.2 Lint: rule categories	218
25.3 CDC verification	220
25.4 RDC verification	221
25.5 Formal property verification	222

25.6 Equivalence checking	224
25.7 X-propagation checking	226
25.8 A sign-off checklist	226
25.9 Summary	227
25.10 Exercises	227
26. Debug, Gate-Level Simulation, FPGA Prototyping and Emulation	228
26.1 A debugging methodology	228
26.2 Waveforms: \$dumpfile, \$dumpvars and GTKWave	228
26.3 Common bug patterns	229
26.4 X: optimism in RTL, pessimism in gates	230
26.5 Gate-level simulation	231
26.6 FPGA prototyping of ASIC RTL	233
26.7 Hardware emulation	235
26.8 Hardware/software co-verification	235
26.9 Post-silicon debug	236
26.10 Summary	236
26.11 Exercises	236
PART VI - Projects and the Roadmap	238
27. RTL Coding Guidelines and Design Reviews	239
27.1 Why guidelines exist	239
27.2 Naming conventions	239
27.3 File and module organization	240
27.4 Parameterization	241
27.5 Reset and clock rules	241
27.6 A module written to the guideline	242
27.7 Rules that prevent silicon bugs	244
27.8 Comments and documentation	244
27.9 The micro-architecture specification	244
27.10 Design reviews	245
27.11 Common review findings	246
27.12 A git workflow for RTL	246
27.13 Regression discipline	247
27.14 Summary	247
27.15 Exercises	247
28. Project: An APB UART Subsystem, Designed and Verified	248
28.1 Specification	248
28.2 Block diagram	248
28.3 Baud generation and the 16x oversampling budget	249
28.4 The FIFOs	249
28.5 The transmitter	250

28.6 The receiver: synchronize, find the edge, sample the middle	251
28.7 The APB interface, registers and interrupt	252
28.8 A self-checking testbench	254
28.9 What it costs	257
28.10 From project to production	257
28.11 Summary	257
28.12 Exercises	258
29. Project: An AXI-Stream MAC Accelerator for On-Device ML	259
29.1 Why int8, and what the hardware must compute	259
29.2 Specification	259
29.3 Micro-architecture	260
29.4 The RTL, stage by stage	260
29.5 Verification against a golden model	263
29.6 PPA of the engine	266
29.7 Exploiting sparsity for speed, not just power	266
29.8 Scaling up: from a dot-product engine to a systolic array	266
29.9 Integrating the engine into an SoC	267
29.10 Summary	268
29.11 Exercises	268
30. The RTL/SoC Career Roadmap: Skills, Study Plan, Portfolio and Interviews	269
30.1 The four stages	269
30.2 Skills by stage, in detail	269
30.3 A 48-week study plan built on this book	270
30.4 Portfolio projects that get attention	270
30.5 Tools to learn	271
30.6 How RTL interviews are structured	272
30.7 Soft skills that separate good engineers from great ones	273
30.8 Specializations	274
30.9 Final checklist	274
30.10 Summary	275
30.11 Exercises	275

APPENDICES

276

Appendix A. Verilog/SystemVerilog Quick Reference	277
Data types	277
Operators and precedence	278
Expression sizing and signedness rules	278
Procedural blocks and assignments	279
case variants	279
System tasks and functions	279
Modules, parameters and generate	280

Packages, typedef, enum, struct	280
Interfaces and modports	281
Synthesizable subset at a glance	281
Common idioms (all verified with Icarus)	282
Appendix B. Glossary	284
Appendix C. 100 RTL Interview Questions with Answers	290
Digital logic fundamentals	290
Verilog and SystemVerilog semantics	291
Finite state machines	292
Datapath, pipelining and FIFOs	292
Clock-domain crossing and resets	293
Static timing analysis	295
Low power	296
Design for test	296
On-chip buses	297
Verification	297
Design puzzles	298
Appendix D. Tools, Standards and Resources	301
IEEE and industry standards	301
AMBA and other bus specifications	301
Open-source tools	302
Commercial tools by flow step	302
Papers and books	303
Open-source SoC and processor projects to study	304
Getting real silicon: shuttles and open PDKs	304
A suggested order of use	305

PART I

Foundations

Where RTL sits in the journey from a product idea to working silicon; the digital-logic facts every RTL line depends on; the synthesizable core of Verilog and SystemVerilog; and the simulation semantics - the event scheduler, blocking versus non-blocking assignment, and races - that separate RTL that merely simulates from RTL that is correct.

1. RTL in the SoC/ASIC Flow, and the Roadmap

Every modern chip - the application processor in a phone, the controller in an SSD, the neural accelerator in a smart camera - began life as text. Not as a schematic, and not as transistors, but as tens or hundreds of thousands of lines of **register-transfer level** (RTL) code written in Verilog, SystemVerilog or VHDL. That text is the single most important artefact in a chip project: it is the contract between the architects who decided what the chip does and the implementation tools that decide how it is built.

This book teaches you to write that text well - from the first `always_ff` to AXI interconnect, clock-domain crossings, low-power intent and timing closure - and to see, behind every line, the hardware it becomes. This first chapter draws the map: what RTL is, where it sits in the flow from specification to silicon, who the people around you are, and which chapter teaches which skill.

1.1 What 'register-transfer level' means

A synchronous digital design is a set of **registers** (flip-flops) separated by clouds of **combinational logic**. On every active clock edge each register captures a new value, computed by the logic from the current register values and the primary inputs. RTL describes a design in exactly those terms: *which registers exist*, and *what function of the current state is transferred into each register on each clock edge*. It deliberately says nothing about which gates, which transistors, or where on the die.

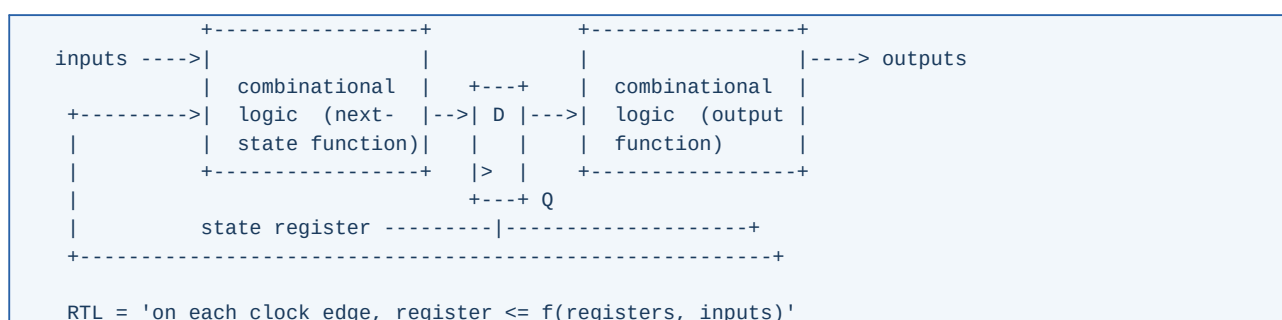


Figure 1.1 - The register-transfer abstraction: registers hold state, combinational logic computes the next state between clock edges.

A one-line example makes the idea concrete. The statement below says: on each rising edge of `clk`, the register `count` takes the value `count + 1` unless `clear` is asserted. From it a synthesis tool infers an adder, a 2:1 multiplexer and a bank of flip-flops - and chooses the adder architecture (ripple, carry-lookahead, prefix) that meets your clock period.

```
always_ff @(posedge clk)
  if (clear) count <= '0;
  else      count <= count + 1'b1;
```

The essence of RTL: state (`count`), a clock edge, and a transfer function. Chapter 6 builds on exactly this pattern.

1.2 Levels of abstraction

Hardware is described at several levels, each trading detail for speed of thought and simulation. RTL occupies a sweet spot: detailed enough that a tool can build the hardware automatically and predict its timing, abstract enough that one engineer can describe a million gates.

Level	What is described	Typical language / artefact	Who works here
System / algorithm	Functions and data flow, no timing or clocks	C/C++, Python, MATLAB, SystemC untimed	Architects, algorithm engineers
Transaction level (TLM)	Components exchanging transactions (a whole burst, a whole packet); approximate timing	SystemC TLM-2.0, SV classes	Architects, virtual-platform and software teams

Level	What is described	Typical language / artefact	Who works here
Register-transfer (RTL)	Registers, clocks, and the combinational logic between them; cycle-accurate	Verilog, SystemVerilog, VHDL; also HLS output, Chisel	RTL designers , DV engineers
Gate level	A netlist of standard cells (NAND2, DFF, MUX2) from a specific library	Verilog netlist, .lib timing models	Synthesis, DFT, STA engineers
Transistor level	MOSFETs, sizes, parasitics	SPICE netlists	Circuit and library designers
Layout / physical	Polygons on mask layers; placement and routing	GDSII / OASIS, DEF, LEF	Physical-design and layout engineers

Table 1.1 - Abstraction levels. Each step down is performed largely automatically by tools, guided by constraints.

RTL IS THE LAST LEVEL A HUMAN WRITES BY HAND

Below RTL, almost everything is generated: synthesis produces the gate netlist, place-and-route produces the layout. That is why RTL quality dominates chip quality. A clumsy RTL architecture cannot be rescued by a tool; a well-structured one lets every downstream tool do its best work.

1.3 The ASIC flow, from specification to silicon

A chip project is a pipeline of transformations, each with its own specialists, tools and sign-off criteria. The diagram below is the backbone of this whole book; keep it in mind, because every RTL decision is judged by what it does to the stages after it.

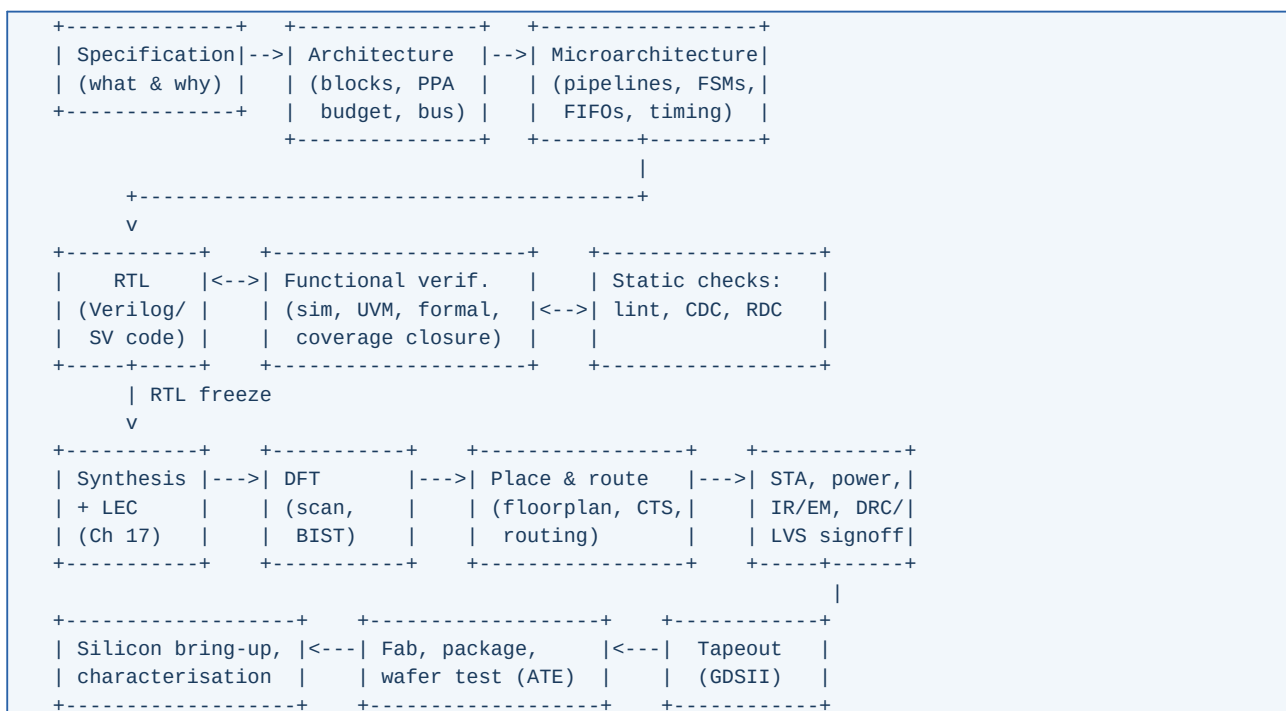


Figure 1.2 - The ASIC flow. Arrows between RTL and verification iterate hundreds of times; after RTL freeze, changes become ECOs (engineering change orders) and get very expensive.

Stage	Input -> output	Main question answered	Where in this book
Specification	Market/customer needs -> spec document	What must the chip do, at what cost, power and speed?	Ch 30
Architecture	Spec -> block diagram, memory map, PPA budgets	Which blocks, which buses, how much memory, what clock rates?	Ch 13-16, 21

Stage	Input -> output	Main question answered	Where in this book
Microarchitecture	Architecture -> per-block design doc	How many pipeline stages, which FSMs, how deep the FIFOs?	Ch 5-10
RTL coding	Microarchitecture -> synthesizable SV	Is the hardware described exactly, and only, as intended?	Ch 3-16, 27
Functional verification	RTL + spec -> coverage, bug reports	Does the RTL do what the spec says in all cases?	Ch 22-26
Synthesis + LEC	RTL + SDC + library -> gate netlist	Can this RTL be built at the target frequency and area?	Ch 17
DFT	Netlist -> scan-inserted netlist, test patterns	Can every manufactured die be tested for defects?	Ch 20
Place & route	Netlist -> placed, clock-treed, routed layout	Does it physically fit, and can the wires be routed?	Ch 21
STA / signoff	Layout parasitics -> timing, power, IR reports	Does it meet timing in every corner and mode?	Ch 18, 19
Tapeout	Signed-off GDSII -> mask set	Is everything clean? (a mask set can cost millions)	-
Bring-up	First silicon -> working product	Does the real chip boot, and do the numbers match?	Ch 26

Table 1.2 - Flow stages, their artefacts and the chapters that prepare you for each.

FRONT END AND BACK END

Industry splits the flow into the **front end** (spec, architecture, RTL, verification - up to a synthesizable, verified RTL release) and the **back end** (synthesis onwards). RTL designers live in the front end, but the best ones think constantly about the back end: timing paths, congestion, clock gating, testability. Part IV of this book exists to build that instinct.

WHY LATE BUGS COST SO MUCH

A bug found in simulation costs minutes to fix. After RTL freeze it needs a netlist ECO and re-verification of timing. After tapeout it needs a metal-layer respin (weeks, and a partial mask set) or a full respin (months, and millions of dollars) - or a software workaround that ships forever. This cost curve is why verification consumes more engineering effort than design on most projects, and why good RTL designers write code that is easy to verify.

1.4 Anatomy of a System-on-Chip

A **System-on-Chip (SoC)** integrates what used to be a board full of chips: processors, memory, interconnect and peripherals on one die. Most of an SoC's RTL is not the CPU - which is usually licensed IP - but the glue, the peripherals, the accelerators and the infrastructure around them. That is where most RTL designers work.

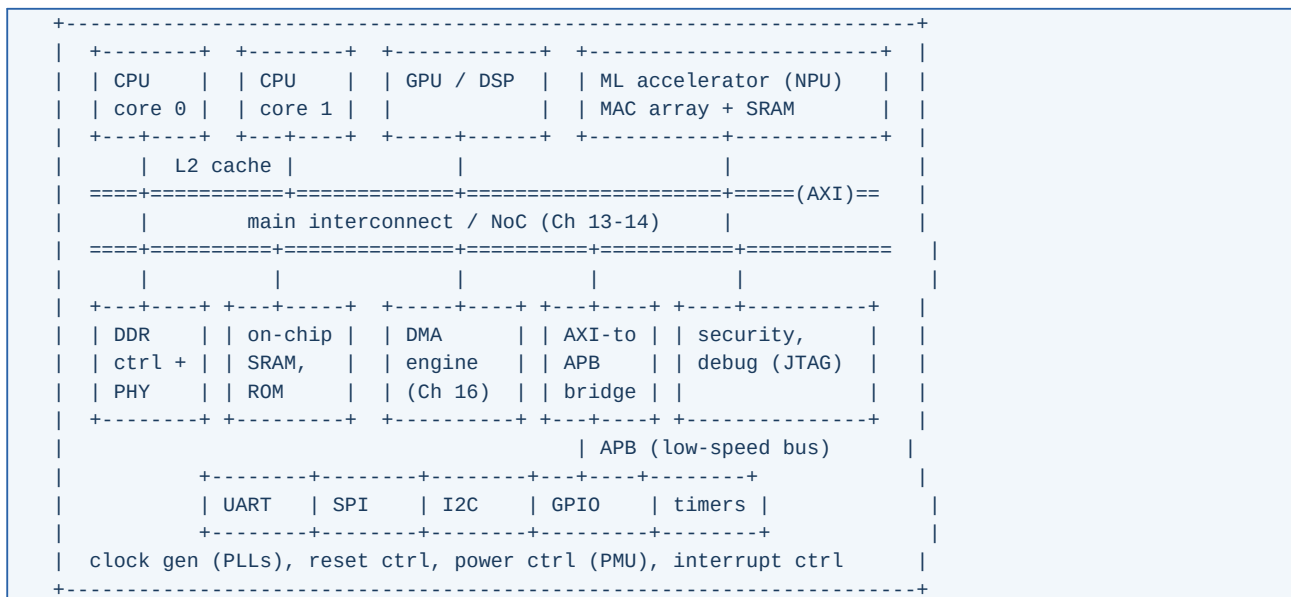


Figure 1.3 - A representative SoC. High-bandwidth masters and slaves sit on AXI; slow peripherals hang off APB behind a bridge.

Subsystem	Role	Typical RTL work
Processor complex	CPU cores, caches, interrupt controller	Integration, wrappers, debug and interrupt wiring (Ch 16)
Interconnect	Moves transactions between masters and slaves	Crossbars, arbiters, bridges, width/clock converters (Ch 13-14)
Memories	SRAMs, ROM, register files, DDR controller	Memory wrappers, ECC, FIFOs, arbitration (Ch 9)
Peripherals	UART, SPI, I2C, GPIO, timers, watchdog	Complete blocks with CSRs and interrupts (Ch 15, 28)
Accelerators	Fixed-function engines: crypto, video, ML	Datapaths, pipelines, streaming interfaces (Ch 8, 10, 29)
Clock / reset / power	PLLs, dividers, reset sequencing, power domains	CDC synchronizers, reset controllers, UPF (Ch 11, 12, 19)
Test and debug	Scan, MBIST, JTAG, trace	Mostly inserted by tools, but RTL must allow it (Ch 20)

1.5 FPGA versus ASIC

The same RTL can target an **FPGA** (a field-programmable gate array: a sea of lookup tables, flip-flops, block RAMs and DSP slices, configured by a bitstream) or an **ASIC** (application-specific integrated circuit: standard cells fabricated for you). The language is the same; the priorities are not.

Aspect	FPGA	ASIC
Up-front cost	Low: buy a board	Very high: masks, EDA licences, IP, team
Unit cost / power / speed	High / high / 100s of MHz	Low / low / GHz possible
Iteration time	Minutes to hours (rebuild the bitstream)	Months per silicon spin
Primitives	LUTs, FFs, BRAM, DSP48-style slices	Standard-cell library, SRAM compilers, analog IP
Resets	Global set/reset free at configuration; sync reset often preferred	Reset is a design decision with area and timing cost (Ch 12)
Clocking	Dedicated clock networks, MMCMs/PLLs; clock gating discouraged	Clock tree built by CTS; clock gating is the main power lever (Ch 19)
Testability	Not needed (the fabric is pre-tested)	Scan and BIST mandatory (Ch 20)

Aspect	FPGA	ASIC
Role in ASIC projects	Prototyping and emulation (Ch 26)	The product

WRITE PORTABLE RTL

Code that is clean for ASIC is almost always clean for FPGA, not the other way round. Keep technology-specific cells (clock gates, SRAM macros, synchronizers) behind small wrapper modules so the same RTL can be prototyped on an FPGA and taped out as an ASIC.

1.6 The people around the RTL designer

Role	Owns	What they need from your RTL
Architect	Spec, block partitioning, PPA budgets	Faithful implementation; early feedback when a budget is unrealistic
RTL / design engineer	Microarchitecture and RTL of blocks	- (this book)
Design verification (DV)	Testbenches, UVM, coverage, formal	Clear interfaces, assertions, a microarchitecture document, debuggable code
DFT engineer	Scan, compression, MBIST, JTAG, ATPG	Controllable clocks and resets, no untestable structures (Ch 20)
Synthesis / STA engineer	Constraints, netlist, timing closure	Sensible pipelining, clean SDC-friendly clocking, no combinational loops (Ch 17-18)
Physical design (PD)	Floorplan, placement, CTS, routing	Reasonable fan-out and congestion, registered block boundaries (Ch 21)
Firmware / software	Drivers, boot code	A sane register map, documented reset values and interrupts (Ch 15)
Post-silicon validation	Bring-up, characterisation	Debug visibility: status registers, counters, trace (Ch 26)

Table 1.3 - Your RTL is consumed by everyone in this table. Senior designers are defined by how well they serve them.

1.7 The roadmap: how this book is organised

The chapters follow the flow. Part I lays the foundations; Part II builds the blocks every design is made of; Part III assembles them into an SoC; Part IV makes the RTL implementable; Part V proves it correct; Part VI ties everything together in projects and a career plan. Each row below names the skill a chapter builds and the flow stage it serves.

Ch	Topic	Skill you gain	Flow stage
1	RTL in the SoC/ASIC flow	The map; vocabulary	All
2	Digital logic essentials	Timing, number systems, flops vs latches	Microarch., RTL
3	Verilog & SystemVerilog	The synthesizable language	RTL
4	Simulation semantics	Race-free coding; NBA vs blocking	RTL, DV
5	Combinational logic	Muxes, decoders, priority logic, latch avoidance	RTL
6	Sequential logic	Flops, resets, enables, counters	RTL
7	Finite state machines	FSM styles, encoding, safety	Microarch., RTL
8	Datapath & arithmetic	Adders, multipliers, fixed point	Microarch., RTL
9	Memories & FIFOs	SRAM inference, register files, FIFOs	Microarch., RTL
10	Pipelining & handshakes	valid/ready, back-pressure, skid buffers	Microarch., RTL
11	Clocks & CDC	Synchronizers, async FIFOs	Arch., RTL, signoff

Ch	Topic	Skill you gain	Flow stage
12	Reset architecture & RDC	Sync/async reset, sequencing	Arch., RTL
13	On-chip buses	APB, AHB, AXI4, AXI-Lite, AXI-Stream	Arch., RTL
14	Interconnect & NoCs	Arbitration, decoding, memory maps	Arch., RTL
15	CSRs & interrupts	Register maps, generation flows	RTL, firmware
16	Peripherals, DMA, CPU integration	SoC assembly	RTL, integration
17	Logic synthesis	What RTL becomes; QoR	Synthesis
18	Timing: STA & SDC	Constraints, timing closure from RTL	Synthesis, STA
19	Low-power RTL & UPF	Clock gating, power domains	RTL, implementation
20	Design for test	Scan, BIST, JTAG awareness	DFT
21	PPA & PD hand-off	Floorplan-aware RTL	Implementation
22	Self-checking testbenches	Stimulus, checkers, scoreboards	DV
23	Assertions & coverage	SVA, covergroups	DV, formal
24	UVM	Agents, sequences, environments	DV
25	Static signoff	Lint, CDC/RDC, formal, LEC	Signoff
26	Debug, GLS, FPGA, emulation	Finding bugs at every level	DV, bring-up
27	Coding guidelines & reviews	Professional RTL style	RTL
28	Project: APB UART	A complete verified peripheral	Full front end
29	Project: AXI-Stream MAC	An ML accelerator datapath	Full front end
30	Career roadmap	Study plan, portfolio, interviews	-
A-D	Appendices	Quick reference, glossary, 100 interview questions, tools	-

Table 1.4 - The roadmap. Read Parts I-II in order; later parts can be read by need.

HOW TO USE THIS BOOK

Type in and run the examples. Every design and testbench with a green output card was simulated with the free Icarus Verilog simulator ([iverilog -g2012 -Wall](#), then [vvp](#)); the output shown is the real output. Pair it with a waveform viewer (GTKWave or Surfer) and the free Yosys synthesizer to see the gates your code produces - Chapter 17 shows how.

1.8 Summary

- RTL describes a design as registers plus the combinational logic that computes their next values on each clock edge. It is the last level engineers write by hand.
- The ASIC flow runs spec -> architecture -> microarchitecture -> RTL -> verification -> synthesis -> DFT -> P&R -> STA/signoff -> tapeout -> bring-up. The cost of a bug rises steeply along it.
- An SoC is CPUs, interconnect, memories, peripherals and accelerators, held together by clock, reset and power infrastructure - most RTL work is in that infrastructure.
- FPGA and ASIC share the language but differ in cost, speed, resets, clocking and testability.
- Good RTL serves its consumers: DV, DFT, synthesis, STA, PD and firmware.

1.9 Exercises

- 1 Name the six abstraction levels and, for each, one artefact a chip project produces at that level.
- 2 Draw the ASIC flow from memory. Mark where an RTL bug would be found by (a) simulation, (b) equivalence checking, (c) STA, (d) silicon bring-up, and estimate the relative cost of each fix.
- 3 For the SoC of Figure 1.3, list which blocks are masters and which are slaves on the interconnect. Why does a DMA engine need to be both?
- 4 Give three reasons an FPGA prototype of an ASIC might run a design at 50 MHz when the ASIC targets 1 GHz, and one bug class the prototype can still find.
- 5 Pick a chip you use daily (phone SoC, microcontroller, SSD controller). Using public datasheets, sketch its block diagram in the style of Figure 1.3.

2. Digital Logic Essentials for RTL Designers

RTL is a notation for digital logic, and every line you write inherits the physics and arithmetic underneath it. This chapter is a focused refresher on the facts that matter most when writing and reviewing RTL: how numbers are represented, which gates and building blocks your code turns into, how latches differ from flip-flops, and the timing equations that decide how fast a design can run. If you have taken a digital-logic course, skim it - but do not skip the timing section.

2.1 Number systems and notation

Hardware stores bits; humans read decimal. RTL uses binary for bit patterns, hexadecimal for wide buses (one hex digit = 4 bits), and decimal for counts and constants. Verilog literals carry an explicit width and base: `8'hA5`, `4'b1010`, `12'd100`. The underscore is a visual separator: `32'hDEAD_BEEF`.

Decimal	Binary (8-bit)	Hex	Verilog literal
5	0000_0101	05	<code>8'd5</code> , <code>8'h05</code> , <code>8'b101</code>
165	1010_0101	A5	<code>8'hA5</code>
255	1111_1111	FF	<code>8'hFF</code> , <code>'1</code> (all ones, any width)
-1 (signed)	1111_1111	FF	<code>-8'sd1</code>
-128 (signed)	1000_0000	80	<code>-8'sd128</code>

UNSIZED LITERALS ARE 32 BITS

A literal without a width, such as `5` or `'hFF`, is at least 32 bits wide and (for plain decimals) signed. This silently widens expressions, as Chapter 3 shows. In RTL, size every literal: `4'd5`, not `5`. The fill literals `'0`, `'1`, `'x`, `'z` are the exception - they deliberately stretch to the width of their context.

2.2 Two's complement

Nearly all hardware represents signed integers in **two's complement**. An N-bit pattern represents the value in which the most significant bit has *negative* weight:

$$\begin{aligned}
 \text{value} &= -b[N-1] \times 2^{(N-1)} + \sum_{i=0..N-2} b[i] \times 2^i \\
 \text{range} &= -2^{(N-1)} \quad \dots \quad 2^{(N-1)} - 1 && (8 \text{ bits: } -128 \dots 127) \\
 -x &= \sim x + 1 && (\text{invert, add one})
 \end{aligned}$$

Two's-complement definitions.

Its great virtue is that **addition and subtraction are the same circuit for signed and unsigned numbers** - only the interpretation of the result and the overflow rule differ. That is why an ALU needs one adder, not two. Its cost is asymmetry: there is one more negative number than positive, so negating the most negative value overflows back to itself.

```

module tb;
  logic signed [7:0] a, b, s;
  initial begin
    a = 8'sd100; b = 8'sd27; s = a + b;
    $display("100 + 27 = %0d (bits %b)", s, s);
    a = 8'sd100; b = 8'sd28; s = a + b;
    $display("100 + 28 = %0d (bits %b) <- overflow", s, s);
    a = -8'sd1;
    $display("-1 = %b = 0x%h", a, a);
    a = -8'sd128; s = -a;
    $display("-(-128) = %0d (no +128 in 8 bits)", s);
  end
endmodule

```

```

100 + 27 = 127 (bits 01111111)
100 + 28 = -128 (bits 10000000) <- overflow
-1 = 11111111 = 0xff
-(-128) = -128 (no +128 in 8 bits)

```

Output (Icarus Verilog 12). Signed overflow wraps silently - hardware never raises an exception.

Operation	Rule	Hardware
Zero extension (unsigned)	Pad with 0s on the left	Wires tied to 0
Sign extension (signed)	Replicate the MSB: $\{8\{x[7]\}, x\}$	Wires fanned out from the MSB - free
Unsigned overflow	Carry out of the MSB	<code>cout</code> of the adder
Signed overflow	Operands have equal signs and the result's sign differs	<code>cout XOR carry-into-MSB</code>
Saturation	Clamp to max/min on overflow	Overflow detect + mux (Ch 8)
Truncation	Drop MSBs (wrap) or LSBs (divide by 2^k)	Wires simply not connected

FIXED POINT IS JUST INTEGERS WITH AN AGREED BINARY POINT

An 8-bit signed value interpreted as Q1.7 (one integer bit including the sign, seven fraction bits) represents -1.0 ... +0.992 in steps of 1/128. The adder does not know or care; only the designer tracks where the binary point is. Quantized ML accelerators (Chapter 29) are built entirely on this idea, and Chapter 8 treats fixed-point arithmetic, rounding and saturation in depth.

2.3 Boolean algebra and the gates it maps to

Combinational logic is Boolean algebra. You rarely minimize by hand - synthesis does that far better - but you must be able to read an expression and see its cost, and recognise identities that make code clearer.

Law	AND form	OR form
Identity	$a \& 1 = a$	$a \mid 0 = a$
Null	$a \& 0 = 0$	$a \mid 1 = 1$
Idempotent	$a \& a = a$	$a \mid a = a$
Complement	$a \& \sim a = 0$	$a \mid \sim a = 1$
Distributive	$a \& (b \mid c) = a\&b \mid a\&c$	$a \mid (b \& c) = (a\mid b) \& (a\mid c)$
Absorption	$a \& (a \mid b) = a$	$a \mid (a \& b) = a$
De Morgan	$\sim(a \& b) = \sim a \mid \sim b$	$\sim(a \mid b) = \sim a \& \sim b$
Consensus	$a\&b \mid \sim a\&c \mid b\&c = a\&b \mid \sim a\&c$	(dual)

In CMOS the natural gates are **inverting**: NAND and NOR need four transistors, while AND and OR are a NAND/NOR plus an inverter. Standard-cell libraries also offer complex gates such as AOI (AND-OR-INVERT) and OAI, XOR/XNOR, multiplexers and full adders. Synthesis maps your Boolean expressions onto whichever cells give the best timing and area, so write for clarity, not for a particular gate.

Gate	Verilog	Relative cost (NAND2 = 1)	Notes
INV	<code>~a</code>	~0.7	Also used as a buffer stage
NAND2 / NOR2	<code>~(a & b) / ~(a b)</code>	1 / ~1	The universal gates
AND2 / OR2	<code>a & b / a b</code>	~1.3	NAND/NOR + INV
XOR2	<code>a ^ b</code>	~2-2.5	Adders, parity, CRC, comparators
MUX2	<code>s ? b : a</code>	~2-2.5	Every if/else and ?: becomes muxes
Full adder	<code>{co,s} = a+b+ci</code>	~5-7	Building block of arithmetic
DFF (no reset)	<code>always_ff</code>	~4-6	Flops dominate many designs' area
DFF with async reset	<code>always_ff ... or negedge rst_n</code>	~5-7	Reset costs area (Ch 12)

Table 2.1 - Typical relative cell areas. Exact numbers depend on the library, but the ratios are a useful mental model.

2.4 Multiplexers, decoders and encoders

Three building blocks appear inside almost every RTL block. A **multiplexer** selects one of N inputs; a **decoder** turns an n-bit index into a one-hot vector of 2^n bits; an **encoder** does the reverse, and a **priority encoder** resolves several active requests by picking the highest (or lowest) index. Here they are in SystemVerilog, with a testbench:

```

module mux4 (input logic [1:0] sel, input logic [3:0] d, output logic y);
    assign y = d[sel];
    // 4:1 mux
endmodule

module dec2to4 (input logic [1:0] a, input logic en, output logic [3:0] y);
    assign y = en ? (4'b0001 << a) : 4'b0000;
    // one-hot decoder
endmodule

module prio_enc (input logic [3:0] req, output logic [1:0] idx, output logic vld);
    always_comb begin
        vld = |req;
        idx = 2'd0;
        for (int i = 0; i < 4; i++)
            if (req[i]) idx = 2'(i);
    end
endmodule

```

Three combinational building blocks. The priority encoder's loop unrolls into a chain of muxes: later iterations override earlier ones.


```

module tb;
  logic [1:0] sel, idx; logic [3:0] d, dec, req; logic y, vld;
  mux4      u_m (.sel(sel), .d(d), .y(y));
  dec2to4   u_d (.a(sel), .en(1'b1), .y(dec));
  prio_enc  u_p (.req(req), .idx(idx), .vld(vld));
  initial begin
    d = 4'b1010;
    for (int i = 0; i < 4; i++) begin
      sel = i[1:0]; req = 4'(i * 5);
      #1 $display("sel=%0d y=%b dec=%b | req=%b idx=%0d vld=%b",
                  sel, y, dec, req, idx, vld);
    end
  end
endmodule

```

```

sel=0 y=0 dec=0001 | req=0000 idx=0 vld=0
sel=1 y=1 dec=0010 | req=0101 idx=2 vld=1
sel=2 y=0 dec=0100 | req=1010 idx=3 vld=1
sel=3 y=1 dec=1000 | req=1111 idx=3 vld=1

```

d[0] -- 0 \	req[3] -----+
d[1] -- 1	req[2] -----+
d[2] -- 2 -- y	req[1] --+
d[3] -- 3 /	v v v
	0 ->[mux]->[mux]->[mux]-> idx
sel[1:0]	1^ 2^ 3^
	(each mux selects its constant
4:1 mux: log2(4)=2 levels	when its req bit is 1 -> the
of MUX2 cells	highest index wins)

Figure 2.1 - Hardware inferred for mux4 (left) and prio_enc (right). The priority chain is a serial structure: its delay grows linearly with the number of requesters unless synthesis restructures it into a tree.

PRIORITY COSTS DELAY

An **if / else if / else if** chain describes priority, and priority is inherently serial logic. When the conditions are mutually exclusive (for example, one-hot selects), say so - with a **case** on a one-hot vector, **unique case**, or an AND-OR mux - and synthesis can build a parallel structure. Chapter 5 develops this.

2.5 Karnaugh maps, briefly

A Karnaugh map arranges a truth table in Gray-code order so that adjacent cells differ in one variable; circling groups of 1s of size 1, 2, 4, 8... yields a minimal sum of products. Below is the 3-input **majority** function (the carry-out of a full adder):

	bc =	00	01	11	10	
a = 0		0	0	[1]	0	groups:
a = 1		0	[1]	[1]	[1]	
						column bc=11 -> b&c
						a=1, bc=01,11 -> a&c
						a=1, bc=11,10 -> a&b

$\text{maj}(a,b,c) = a\&b \mid a\&c \mid b\&c$ (= cout of a full adder)

Figure 2.2 - K-map for the majority function.

K-maps stop being practical beyond about five variables, and synthesis tools use far more powerful algorithms (two-level minimization, BDDs, AIG rewriting). The value for an RTL designer is conceptual: they explain **don't-cares** (inputs that cannot occur let the tool pick 0 or 1 to simplify logic - the reason for **default: y = 'x** in some coding styles) and **hazards**, below.

2.6 Latches and flip-flops

Storage comes in two flavours. A **D latch** is *level-sensitive*: while its enable is high it is transparent (Q follows D); when the enable falls it holds. A **D flip-flop** is *edge-triggered*: it samples D only at the clock edge and holds Q constant for the rest of the cycle. A flip-flop is internally two latches back to back (master and slave) on opposite clock phases.

```
module d_latch (input logic en, d, output logic q);
    always_latch if (en) q = d;          // transparent while en=1
endmodule

module d_flop (input logic clk, d, output logic q);
    always_ff @(posedge clk) q <= d;    // samples d only at the rising edge
endmodule

module tb;
    logic clk = 0, d = 0, ql, qf;
    d_latch u_l (.en(clk), .d(d), .q(ql));
    d_flop u_f (.clk(clk), .d(d), .q(qf));
    always #5 clk = ~clk;
    initial begin
        $display(" t  clk d | latch flop");
        #2 d = 1; #1 $display("%2t  %b %b |  %b  %b", $time, clk, d, ql, qf);
        #4          $display("%2t  %b %b |  %b  %b", $time, clk, d, ql, qf);
        #1 d = 0; #1 $display("%2t  %b %b |  %b  %b", $time, clk, d, ql, qf);
        #1 d = 1; #1 $display("%2t  %b %b |  %b  %b", $time, clk, d, ql, qf);
        #6          $display("%2t  %b %b |  %b  %b", $time, clk, d, ql, qf);
        $finish;
    end
endmodule
```

```
t  clk d | latch flop
3   0  1 |   x   x
7   1  1 |   1   1
9   1  0 |   0   1
11  0  1 |   0   1
17  1  1 |   1   1
```

At t=9 the clock is still high: the latch follows d down to 0, the flop keeps the value captured at t=5. At t=11 the clock is low and both hold.

Property	Latch	Flip-flop
Sensitivity	Level (transparent while enabled)	Edge (samples at one instant)
Area	Smaller (~half a flop)	Larger
Timing analysis	Time borrowing across the transparent phase: powerful but complex	Simple: one launch, one capture per cycle
Glitch sensitivity	A glitch on D or enable passes through	Only the value at the edge matters
Use in ASIC RTL	Clock-gating cells, some high-speed custom designs, latch-based register files	The default for everything
How it appears by accident	Incomplete assignment in combinational <code>always</code> (Ch 5)	-

UNINTENDED LATCHES

If a combinational block does not assign an output on every path (an `if` without `else`, a `case` without `default`), the output must remember its old value - and synthesis builds a latch. It is one of the most common RTL bugs. Using `always_comb` makes tools warn about it; Chapter 5 shows how to avoid it by construction.

2.7 Setup, hold and clock-to-Q

A flip-flop only works if D is stable in a window around the clock edge. Three numbers from the cell library describe it:

- **Setup time** t_{su} : D must be stable this long *before* the edge.
- **Hold time** t_h : D must stay stable this long *after* the edge.
- **Clock-to-Q** t_{cq} : after the edge, Q changes this much later.

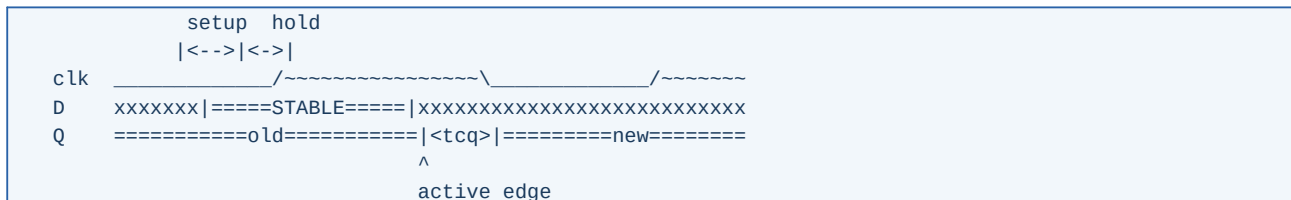


Figure 2.3 - The sampling window. A change of D inside the setup/hold window can make the flop metastable.

2.8 The critical path and maximum frequency

Consider two flops with combinational logic between them, both clocked by the same clock. Data launched at one edge must arrive at the capturing flop a setup time before the *next* edge; that is the **setup** (max-delay) constraint. Data launched at an edge must *not* arrive so quickly that it corrupts the value the capture flop is sampling at that same edge; that is the **hold** (min-delay) constraint.

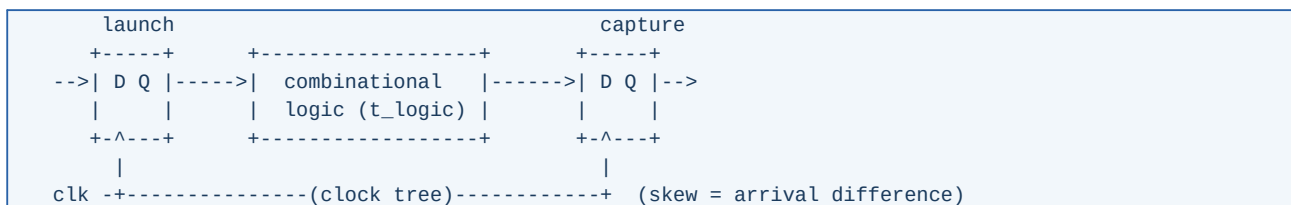


Figure 2.4 - A register-to-register timing path.

$$\text{Setup: } T_{clk} \geq t_{cq} + t_{logic(max)} + t_{su} - t_{skew}$$

$$\text{Hold: } t_{cq} + t_{logic(min)} \geq t_h + t_{skew}$$

$$f_{max} = 1 / (t_{cq} + t_{logic(max)} + t_{su} - t_{skew})$$

$$\text{Example: } t_{cq} = 80 \text{ ps, } t_{logic} = 600 \text{ ps, } t_{su} = 50 \text{ ps, } skew = 0$$

$$T_{min} = 730 \text{ ps} \rightarrow f_{max} = 1 / 730 \text{ ps} = 1.37 \text{ GHz}$$

The two fundamental timing equations (t_{skew} = capture clock arrival minus launch clock arrival; clock uncertainty is subtracted from the budget in the same way).

The **critical path** is the register-to-register path with the least slack; it sets f_{max} . Notice two deep facts:

- **Setup depends on the clock period; hold does not.** A setup violation can be fixed by slowing the clock (or by pipelining, Chapter 10). A hold violation cannot - the chip fails at any frequency - so back-end tools fix hold by inserting delay buffers.
- **Positive skew helps setup and hurts hold** by the same amount. Clock-tree synthesis balances skew; 'useful skew' deliberately exploits it.
- **RTL controls t_{logic} .** The number of logic levels you write between flops is the main lever the RTL designer has on frequency. A rule of thumb in a modern process at 1 GHz is roughly 20-30 gate levels per cycle; Chapter 18 makes this quantitative.

INTERVIEW FAVOURITE: 'CAN A DESIGN HAVE NEGATIVE HOLD SLACK AND STILL WORK IF YOU SLOW THE CLOCK?'

No. The hold check compares the earliest data arrival with the same clock edge's capture window, so the clock period does not appear in the inequality. Slowing the clock fixes setup, never hold. (Hold issues on real silicon are fixed by ECOs that add delay, or sometimes worked around by raising voltage, which changes the cell delays - not by frequency.)

2.9 Metastability: a preview

If D changes inside the setup/hold window, the flop's internal node can hang between 0 and 1 for an unbounded time before resolving randomly - **metastability**. Inside one synchronous clock domain, static timing analysis guarantees this never happens. But a signal arriving from another clock domain, or an asynchronous input such as a button or a reset release, has no fixed timing relationship to the capture clock and *will* eventually violate the window.

$$MTBF = \exp(t_r / \tau) / (T_0 \times f_{clk} \times f_{data})$$

t_r : time allowed for resolution (about one clock period per sync stage)

tau, T₀ : flop technology constants

Mean time between synchronizer failures. It grows exponentially with resolution time - which is why a second flop stage helps so much.

The standard cure is a **two-flop synchronizer**: the first flop may go metastable, but it has a whole clock period to resolve before the second flop samples it. Multi-bit values need more care (Gray codes, handshakes, asynchronous FIFOs). Chapter 11 is devoted to clock-domain crossing; Chapter 12 to the reset version of the same problem.

2.10 Fan-out, glitches and hazards

Fan-out

Fan-out is the number of gate inputs a signal drives. Every input adds capacitance, and delay grows with load. A single register driving 2,000 flops (a global enable, a reset, a mode bit) cannot switch quickly; synthesis and physical design build a **buffer tree** for it, adding delay and area. RTL designers help by keeping high-fan-out controls registered close to their loads and, for timing-critical paths, **replicating** the driving register.

Glitches and hazards

Different paths through combinational logic have different delays, so an output can pulse briefly to a wrong value while inputs change: a **glitch**. A **static hazard** is a glitch on an output that should have stayed constant. The classic example is a 2:1 mux built from gates, $y = a \& s \mid b \& \sim s$, with $a = b = 1$ while s switches: for an instant both AND terms can be 0, and y dips to 0.

```

a=1 ---[AND]---+
s ---/         |
               +--[OR]--- y
b=1 ---[AND]---+
~s ---/         |
               y
s      ~~~~~\_____
~s     _____/~~~~~ (late: inverter delay)
a&s    ~~~~~\_____
b&~s   _____/~~~~~
y      ~~~~~\_/_~~~~~ <- glitch

```

Fix (if the output really must be glitch-free): add the consensus term $a \& b$.

Figure 2.5 - A static-1 hazard in a gate-level multiplexer.

WHY SYNCHRONOUS DESIGN TOLERATES GLITCHES - AND WHERE IT DOES NOT

In a synchronous design glitches are harmless on data paths: the capturing flop only looks at its input at the clock edge, after everything has settled (that is what the setup check guarantees). Glitches are **fatal** on anything used as a clock, an asynchronous reset, or a signal crossing into another clock domain, and they **waste dynamic power** everywhere. Hence the golden rules: never generate clocks or resets from combinational logic, and always register a signal before it crosses a clock domain.

2.11 Summary

- Size every literal; unsized literals are 32-bit and silently widen expressions.
- Two's complement makes signed and unsigned add/subtract the same hardware; sign-extend by replicating the MSB; overflow wraps silently.
- Muxes, decoders and priority encoders are the vocabulary of RTL; priority logic is serial and slower than parallel selection.
- Latches are level-sensitive and appear by accident from incomplete assignments; flip-flops are the default storage element.
- Setup: $T \geq t_{cq} + t_{logic,max} + t_{su} - skew$. Hold: $t_{cq} + t_{logic,min} \geq t_h + skew$. Hold does not depend on frequency.
- Metastability is unavoidable across asynchronous boundaries and is managed with synchronizers (Chapter 11). Glitches are harmless on synchronous data paths but fatal on clocks and resets.

2.12 Exercises

- 1 Write -37 as an 8-bit two's-complement number in binary and hex. Sign-extend it to 12 bits. What unsigned value does the 8-bit pattern represent?
- 2 Show that 8-bit signed overflow on addition can be detected as `(a[7] == b[7]) && (s[7] != a[7])`, and derive the equivalent expression using the carry into and out of the MSB.
- 3 A path has $t_{cq} = 90$ ps, 14 gates of 45 ps each, $t_{su} = 40$ ps, clock uncertainty 30 ps and skew +20 ps. What is f_{max} ? Which numbers would change if the same path had a hold problem?
- 4 Modify `prio_enc` so the *lowest* index wins. How does the inferred mux chain change?
- 5 Draw the K-map for the 3-input XOR function. Why can no two 1s be grouped, and what does that tell you about the cost of parity trees?
- 6 Explain, with a timing diagram, why a glitch on the enable of a latch-based clock gate can corrupt a design, whereas the same glitch on the D input of a flop does not.

3. Verilog and SystemVerilog for Design

Verilog was created in 1984 as a simulation language; synthesis came later and adopted a *subset* of it. SystemVerilog (IEEE 1800, which absorbed Verilog in 2009) added a large verification language and a smaller but very valuable set of design features: `logic`, `always_ff/always_comb`, `typedef`, `enum`, `struct`, packages and interfaces. Modern RTL is written in SystemVerilog's design subset, and that is what this chapter teaches.

The mindset matters more than the syntax. You are not writing a program that executes line by line; you are writing a **description of hardware that exists all at once**. Every statement below should make you picture wires, gates and flops.

3.1 Modules, ports and hierarchy

The **module** is the unit of hardware: a box with ports. A design is a tree of module **instances**; the top of the tree is the chip (or the testbench). Below, a full adder is instantiated eight times by a `generate` loop to build a parameterized ripple-carry adder:

```
module full_adder (input  logic a, b, cin,
                  output logic s, cout);
    assign s    = a ^ b ^ cin;
    assign cout = (a & b) | (cin & (a ^ b));
endmodule

module ripple_add #(parameter int W = 4) (
    input  logic [W-1:0] a, b,
    input  logic          cin,
    output logic [W-1:0] sum,
    output logic          cout
);
    logic [W:0] c;           // carry chain, c[0] = cin
    assign c[0] = cin;
    for (genvar i = 0; i < W; i++) begin : g_bit
        full_adder u_fa (.a(a[i]), .b(b[i]), .cin(c[i]), .s(sum[i]), .cout(c[i+1]));
    end
    assign cout = c[W];
endmodule
```

ANSI-style port declarations, a parameter, and named port connections. Each loop iteration creates an instance with the hierarchical name `g_bit[i].u_fa`.

```
module tb;
    logic [7:0] a, b, s; logic co;
    ripple_add #(.W(8)) dut (.a(a), .b(b), .cin(1'b0), .sum(s), .cout(co));
    initial begin
        a = 8'd200; b = 8'd100;
        #1 $display("%0d + %0d = %0d carry=%b {cout,sum}=%0d", a, b, s, co, {co, s});
        a = 8'd17;  b = 8'd25;
        #1 $display("%0d + %0d = %0d carry=%b {cout,sum}=%0d", a, b, s, co, {co, s});
    end
endmodule
```

```
200 + 100 = 44 carry=1 {cout,sum}=300
17 + 25 = 42 carry=0 {cout,sum}=42
```

Connection style	Example	Verdict
Named (by port name)	<code>.a(x), .b(y)</code>	Use this. Order-independent, self-documenting, survives port reordering
Positional (by order)	<code>full_adder u (x, y, ci, s, co);</code>	Avoid except for tiny primitives: a reordered port list silently miswires

Connection style	Example	Verdict
Implicit named	<code>.clk, .rst_n</code> (same-name signal)	Convenient; still checks that the names exist
Wildcard	<code>.*</code>	Terse but hides connectivity; avoid in reviewed RTL

IMPLICIT NETS: ONE TYPO, ONE SILENTLY WRONG CHIP

An undeclared identifier in a port connection is silently declared as a **1-bit wire**. Below, `data_inn` is a typo: the 8-bit port gets a single floating bit and the design still compiles. Put the compiler directive shown in the second listing at the top of every RTL file to turn this into an error.

```
module inv8 (input logic [7:0] a, output logic [7:0] y);
    assign y = ~a;
endmodule
module top (input logic [7:0] data_in, output logic [7:0] data_out);
    inv8 u_inv (.a(data_inn), .y(data_out)); // typo -> implicit 1-bit wire!
endmodule
```

```
c3_impl.sv:5: warning: implicit definition of wire 'data_inn'.
c3_impl.sv:5: warning: Port 1 (a) of inv8 expects 8 bits, got 1.
c3_impl.sv:5:      : Padding 7 high bits of the port.
```

iverilog -Wall: only warnings - the build succeeds. Many flows never show warnings to anyone.

```
`default_nettype none // first line of every RTL file
```

```
c3_impl2.sv:6: error: Unable to bind wire/reg/memory `data_inn' in `top'
c3_impl2.sv:6: error: Failed to elaborate port expression.
2 error(s) during elaboration.
```

With the directive the typo is a hard error. (If you must link legacy code that relies on implicit nets, re-enable them at the end of the file with the same directive and the value `wire`.)

3.2 Data types: wire, reg, logic, and four-state values

Verilog had two kinds of signal: **nets** (`wire`), which are driven continuously by `assign` statements or instance outputs, and **variables** (`reg`), which hold the last value assigned to them in procedural code. The name `reg` misled generations of engineers: a `reg` is *not* necessarily a register - a `reg` assigned in a combinational block is just a wire. SystemVerilog fixed this with `logic`, a four-state variable type that can be used in both places.

Type	Kind	States	Use
<code>logic</code>	Variable (or net type, with <code>wire logic</code>)	0 1 x z	Default for all RTL signals with a single driver
<code>wire</code>	Net	0 1 x z	Multiple drivers (tri-state buses), inout ports; otherwise <code>logic</code> is better because multiple drivers become a compile error
<code>reg</code>	Variable	0 1 x z	Legacy Verilog; identical to <code>logic</code>
<code>bit</code>	Variable	0 1	Testbenches, loop counters. Hides X in RTL - avoid for hardware signals
<code>int / integer</code>	Variable, 32-bit signed	2-state / 4-state	Loop indices, parameters
<code>byte, shortint, longint</code>	8/16/64-bit signed, 2-state	0 1	Testbench modelling

Four-state logic is how simulation represents uncertainty. **x** means 'unknown - could be 0 or 1' (an uninitialized flop, a conflict between drivers); **z** means 'high impedance - nobody is driving'. Synthesis maps real hardware only to 0 and 1; x and z are simulation concepts (except z for real tri-state pads).

```

module tb;
  logic [3:0] v;           // 4-state: 0 1 x z
  bit [3:0] b2;           // 2-state: x/z become 0
  wire      bus;          // net with two tri-state drivers
  logic      en0, en1, d0, d1;
  assign bus = en0 ? d0 : 1'bz;
  assign bus = en1 ? d1 : 1'bz;
  initial begin
    $display("uninit v=%b", v);
    v = 4'b10xz; b2 = v;
    $display("v=%b b2=%b v==4'b10xz:%b v===4'b10xz:%b", v, b2, v == 4'b10xz, v === 4'b10xz);
    $display("v+4'd1=%b &4'b0x00=%b |4'b1x00=%b", v + 4'd1, &4'b0x00, |4'b1x00);
    en0 = 0; en1 = 0; d0 = 0; d1 = 1; #1 $display("no driver : bus=%b", bus);
    en0 = 1;                               #1 $display("one driver: bus=%b", bus);
    en1 = 1;                               #1 $display("contention: bus=%b", bus);
  end
endmodule

```

```

uninit v=xxxx
v=10xz b2=1000 v==4'b10xz:x v===4'b10xz:1
v+4'd1=xxxx &4'b0x00=0 |4'b1x00=1
no driver : bus=z
one driver: bus=0
contention: bus=x

```

Note: `==` returns x when either side has x/z; the case-equality operator `===` compares x and z literally (testbench only - it is not synthesizable). Arithmetic with any x bit gives all x; logic ops are smarter: `0 AND x = 0`, `1 OR x = 1`.

3.3 Vectors, part-selects and arrays

A vector is declared with a range, conventionally **[MSB:LSB]** with LSB = 0: `logic [31:0] addr`. Slices take a constant range: `addr[15:8]`. When the *position* must be variable, use the **indexed part-select base +: width** (ascending from base) or **base -: width** (descending); the width must be constant, the base may be a variable - which infers a shifter/mux.

```

module tb;
  logic      [7:0] v = 8'b1011_0110;
  logic signed [7:0] s = -8'sd96;           // 8'b1010_0000
  logic      [31:0] word = 32'hDEAD_BEEF;
  int        k = 2;
  initial begin
    $display("v[7:4]=%b v[3:0]=%b v[0]=%b", v[7:4], v[3:0], v[0]);
    $display("word[8*k +: 8] = %h word[31 -: 8] = %h", word[8*k +: 8], word[31 -: 8]);
    $display("&v=%b |v=%b ^v=%b ~&v=%b", &v, |v, ^v, ~&v);
    $display("{v[3:0],v[7:4]} = %b", {v[3:0], v[7:4]});
    $display("{4{2'b10}} = %b", {4{2'b10}});
    $display("v >> 2 = %b v >>> 2 = %b (unsigned: both logical)", v >> 2, v >>> 2);
    $display("s >> 2 = %b s >>> 2 = %b (%0d)", s >> 2, s >>> 2, s >>> 2);
    $display("!v = %b ~v = %b", !v, ~v);
  end
endmodule

```

Part-selects, reduction, concatenation, replication and shifts.

```

v[7:4]=1011 v[3:0]=0110 v[0]=0
word[8*k +: 8] = ad word[31 -: 8] = de
&v=0 |v=1 ^v=1 ~&v=1
{v[3:0],v[7:4]} = 01101011
{4{2'b10}} = 10101010
v >> 2 = 00101101 v >>> 2 = 00101101 (unsigned: both logical)
s >> 2 = 00101000 s >>> 2 = 11101000 (-24)
!v = 0 ~v = 01001001

```


Arrays add dimensions. SystemVerilog distinguishes **packed** dimensions (declared before the name; the bits are contiguous and the whole thing is a vector you can do arithmetic on) from **unpacked** dimensions (after the name; separate elements, like a memory).

Declaration	Meaning	Typical hardware
<code>logic [3:0][7:0] w;</code>	Packed 2-D: a 32-bit vector viewed as 4 bytes; <code>w[2]</code> is a byte, <code>w</code> is 32 bits	A bus with sub-fields, byte lanes
<code>logic [7:0] mem [0:255];</code>	Unpacked array of 256 bytes; one element accessed at a time	Register file or SRAM (Ch 9)
<code>logic [7:0] m2 [4][4];</code>	Unpacked 2-D, C-style sizes	Coefficient tables, small matrices
<code>logic [31:0] q [\$];</code>	Queue (dynamic)	Testbench only
<code>int aa [string];</code>	Associative array	Testbench only

3.4 Operators

Group	Operators	Result width / notes
Bitwise	<code>~ & ^ ~^</code>	Width of the widest operand; one gate per bit
Reduction	<code>&v v ^v ~&v ~ v ~^v</code>	1 bit; a tree of gates. <code>^v</code> is parity; <code> v</code> is 'any bit set'
Logical	<code>! && </code>	1 bit; operands are tested as 'non-zero'
Relational / equality	<code>< <= > >= == !=</code>	1 bit; x if any operand bit is x/z. <code>===</code> <code>!==</code> are testbench-only
Arithmetic	<code>+ - * / % **</code>	<code>+</code> <code>-</code> are cheap, <code>*</code> is large; <code>/</code> <code>%</code> only by constants or powers of two in RTL
Shift	<code><< >></code> logical, <code><<< >>></code> arithmetic	<code>>>></code> fills with the sign bit <i>only if the operand is signed</i>
Concatenation	<code>{a, b, 2'b01}</code>	Sum of widths; operands must be sized
Replication	<code>{4{x}}, {{8{s[7]}}, s}</code>	Sign extension idiom
Conditional	<code>c ? a : b</code>	A multiplexer
Streaming	<code>{<<{v}}</code>	Bit reversal; supported by most modern synthesis tools

THE >>> OPERATOR DOES NOTHING SPECIAL ON UNSIGNED VALUES

As the output above shows, `v >>> 2` on an unsigned `v` is a logical shift. Arithmetic right shift requires a **signed** operand: declare the signal `logic signed`, or cast with `$signed(v) >>> 2`. Also: `!v` is logical NOT (is v zero?), `~v` is bitwise NOT - confusing them in an `if` is a classic bug for multi-bit signals.

3.5 Expression sizing and extension rules

This is where most subtle Verilog arithmetic bugs live. The rules (IEEE 1800 clause 11.6-11.8) are:

- **Context-determined width.** For most operators the expression is evaluated at the width of the *largest operand, including the left-hand side of the assignment*. Operands are extended to that width *before* the operation.
- **Self-determined operands** are evaluated at their own width, regardless of context: concatenation operands, the shift amount, the operand of a reduction, the condition of `?:`.
- **Signedness.** An expression is signed only if *all* its operands are signed. One unsigned operand makes the whole expression unsigned, and signed operands are then zero-extended, not sign-extended.
- Extension then follows the expression's signedness: sign extension for signed, zero extension for unsigned. Truncation on assignment drops the MSBs.

```

module tb;
  logic      [7:0] a = 8'd200, b = 8'd100;
  logic      [7:0] sum8;
  logic      [8:0] sum9;
  logic      [15:0] prod;
  logic signed [7:0] sa = -8'sd4;
  logic      [3:0] u = 4'd3;
  logic signed [15:0] r1, r2;
  initial begin
    sum8 = a + b;           // context is 8 bits: carry lost
    sum9 = a + b;           // context is 9 bits: operands extended first
    $display("sum8=%0d sum9=%0d", sum8, sum9);
    $display("(a+b)>>1 = %0d {1'b0,a}+b >> 1 = %0d", (a + b) >> 1, ({1'b0, a} + b) >> 1);
    prod = a * b; $display("prod=%0d", prod);
    r1 = sa * u;             // mixed: sa treated as UNSIGNED 8-bit 252
    r2 = sa * $signed({1'b0, u}); // both signed: -4 * 3
    $display("r1=%0d r2=%0d", r1, r2);
    $display("-4 < 3u ? %b -4 < 3 ? %b", sa < u, sa < $signed({1'b0, u}));
    $display("4'hF + 1 in 4 bits = %0d, as literal expr = %0d", 4'(4'hF + 1), 4'hF + 1);
  end
endmodule

```

A gallery of sizing and signedness pitfalls.

```

sum8=44 sum9=300
(a+b)>>1 = 22 {1'b0,a}+b >> 1 = 150
prod=20000
r1=756 r2=-12
-4 < 3u ? 0 -4 < 3 ? 1
4'hF + 1 in 4 bits = 0, as literal expr = 16

```

Read the output line by line - each is a real bug pattern:

- **sum8**: the carry is lost because the context is 8 bits. **sum9** works because the 9-bit LHS widens the operands *before* the add.
- **(a+b)>>1** inside **\$display** has no LHS, so the context is 8 bits and the average of 200 and 100 comes out as 22. The fix makes one operand 9 bits wide. **Averages, midpoints and address arithmetic hit this constantly.**
- **r1**: multiplying a signed by an unsigned value makes the whole expression unsigned, so -4 is treated as 252 and zero-extended. The fix: make every operand signed. Note **\$signed(u)** alone would reinterpret 4'd3 correctly, but **\$signed(4'd12)** would become -4 - hence the **{1'b0, u}** idiom that adds an explicit zero sign bit.
- **-4 < 3u** is *false* for the same reason - a comparator bug that passes most tests.
- **4'hF + 1** is 16, not 0: the unsized **1** is 32 bits, so the expression is 32 bits wide. A size cast **4'(...)** forces 4-bit wrap-around.

RULES THAT PREVENT SIZING BUGS

(1) Size every literal. (2) Make result widths explicit: an N-bit + N-bit add needs N+1 bits; N x M needs N+M bits. (3) Never mix signed and unsigned operands in one expression - convert explicitly. (4) Use lint (Verilator **-Wall**, Spyglass, Ascent): width-mismatch warnings are real bugs more often than not (Chapter 25).

3.6 Parameters, localparams and generate

Parameters make modules reusable: widths, depths and feature switches are set per instance with **#(.W(8))**. A **localparam** is a constant derived inside the module that callers cannot override - for example **localparam int AW = \$clog2(DEPTH);**. Both are **elaboration-time** constants: they are resolved before simulation or synthesis starts, and cost no hardware.

Generate constructs build structure at elaboration time. A generate **for** replicates hardware (as in **ripple_add**); a generate **if** or **case** chooses between alternative structures. Always name generate blocks

(begin : g_name) so hierarchical paths are stable for debug, constraints and assertions.

```
module pipe_reg #(
    parameter int W      = 8,
    parameter bit HAS_RST = 1'b1          // elaboration-time choice
) (
    input  logic      clk, rst_n, en,
    input  logic [W-1:0] d,
    output logic [W-1:0] q
);
    if (HAS_RST) begin : g_rst          // generate-if: only one branch is built
        always_ff @(posedge clk or negedge rst_n)
            if (!rst_n) q <= '0;
            else if (en) q <= d;
    end else begin : g_norst            // cheaper flops for pure datapath
        always_ff @(posedge clk)
            if (en) q <= d;
    end
end
endmodule
```

A register with an elaboration-time choice of reset. Chapter 12 discusses when datapath flops may omit reset.

```
module tb;
    logic clk = 0, rst_n = 0, en = 0;
    logic [7:0] d = 8'h00, q1, q2;
    pipe_reg #(.W(8), .HAS_RST(1'b1)) u_r (.clk(clk), .rst_n(rst_n), .en(en), .d(d), .q(q1));
    pipe_reg #(.W(8), .HAS_RST(1'b0)) u_nr (.clk(clk), .rst_n(rst_n), .en(en), .d(d), .q(q2));
    always #5 clk = ~clk;
    task automatic drive(input logic [7:0] val, input logic e);
        @(negedge clk); d = val; en = e;          // change inputs away from the edge
        @(posedge clk); #1 $display("%3t en=%b d=%h | with_rst q=%h no_rst q=%h",
            $time, en, d, q1, q2);
    endtask
    initial begin
        drive(8'h11, 1'b1);                        // still in reset
        rst_n = 1;
        drive(8'h22, 1'b1);
        drive(8'h33, 1'b0);                        // en=0 -> hold
        drive(8'h44, 1'b1);
        $finish;
    end
endmodule
```

A testbench task drives stimulus on the falling edge - a simple race-avoidance discipline explained in Chapter 4.

```
16 en=1 d=11 | with_rst q=00 no_rst q=11
26 en=1 d=22 | with_rst q=22 no_rst q=22
36 en=0 d=33 | with_rst q=22 no_rst q=22
46 en=1 d=44 | with_rst q=44 no_rst q=44
```

```

d[W-1:0] ---->|1\      +-----+
              | |----->| D   Q |-----> q
+--->|0/      |   |   |   |
|      |en     |   >clk |   |
|      +---R---+   |   (R: async clear, only in g_rst)
+-----+-----+

```

Figure 3.1 - What pipe_reg infers: a flop with a recirculating enable mux (or, after synthesis, an enable flop or a clock gate - Chapter 19).

3.7 Functions and tasks

A **function** returns a value, executes in zero time and cannot contain timing controls: it describes combinational logic and is synthesizable. Declare RTL functions **automatic** so each call has its own variables (and recursion works). A **task** may consume time (**@, #, wait**) and is primarily a testbench construct, like **drive** above; a task without timing controls is synthesizable but rarely useful in RTL.

3.8 typedef, enum, struct and packages

SystemVerilog's user-defined types are the single biggest readability improvement over Verilog, and they are fully synthesizable. An **enum** gives names to FSM states and opcodes (and lets waveform viewers display them); a packed **struct** names the fields of a bus or a pipeline register while remaining a plain vector; a **package** shares types, constants and functions across modules.

```
package bus_pkg;
  typedef enum logic [1:0] {OP_READ = 2'b00, OP_WRITE = 2'b01, OP_IDLE = 2'b11} op_e;
  typedef struct packed {
    op_e      op;          // bits [41:40]
    logic [7:0] id;        // bits [39:32]
    logic [31:0] addr;     // bits [31:0]
  } req_t;                // 42 bits total, usable like a vector
  localparam int unsigned REQ_W = $bits(req_t);
  function automatic logic parity(input logic [31:0] x);
    return ^x;
  endfunction
endpackage
```

A package with an enum, a packed struct, a derived constant and a function. The first struct member occupies the MSBs.

```
module tb;
  import bus_pkg::*;
  req_t      r;
  op_e       op;
  logic [7:0] mem_unpk [0:3];    // unpacked: 4 separate bytes (a memory)
  logic [3:0][7:0] mem_pk;      // packed: one 32-bit vector made of 4 bytes
  initial begin
    r.op = OP_WRITE; r.id = 8'h2A; r.addr = 32'h4000_0010;
    op = r.op;
    $display("REQ_W=%0d op=%s raw=%h parity(addr)=%b", REQ_W, op.name(), r, parity(r.addr));
    r.op = OP_IDLE; op = r.op;
    $display("op=%s (%b) r[41:40]=%b", op.name(), op, r[41:40]);
    mem_pk = 32'hA1B2_C3D4;
    foreach (mem_unpk[i]) mem_unpk[i] = mem_pk[i];
    $display("mem_pk[0]=%h mem_pk[3]=%h mem_unpk[1]=%h mem_pk=%h", mem_pk[0], mem_pk[3],
      mem_unpk[1], mem_pk);
  end
endmodule
```

```
REQ_W=42 op=OP_WRITE raw=12a40000010 parity(addr)=0
op=OP_IDLE (11) r[41:40]=11
mem_pk[0]=d4 mem_pk[3]=a1 mem_unpk[1]=c3 mem_pk=a1b2c3d4
```

The struct is a 42-bit vector with named fields; **name()** prints an enum's label. Packed element [0] is the least significant byte.

STYLE THAT SCALES

Put interface types (request/response structs, opcode enums, address maps) in a package shared by RTL and testbench; name enums with an **_e** suffix and structs with **_t**; give enums an explicit base type (**enum logic [1:0]**) so their width is under your control. Assign a struct in RTL with an assignment pattern, e.g. **r = '{op: OP_WRITE, id: 8'h2A, addr: a};** (supported by all commercial tools; Icarus Verilog 12 lacks it, hence the field-by-field assignment above).

3.9 always_comb, always_ff and always_latch

Verilog had one procedural block, `always`, whose meaning depended on its sensitivity list and coding style. SystemVerilog adds three **intent-carrying** variants. They simulate like `always`, but tools check that the hardware inferred matches the declared intent.

Block	Intent	What the tools check / do
<code>always_comb</code>	Combinational logic	Automatic complete sensitivity (including signals read inside called functions); runs once at time zero; warns if a latch is inferred; a variable it writes may not be written elsewhere
<code>always_ff @(posedge clk ...)</code>	Flip-flops	Must have exactly one event control; warns if the logic is not a flop; single-writer rule
<code>always_latch</code>	Intentional latches	Complete sensitivity; documents that the latch is deliberate
<code>always @*</code>	Legacy combinational	Complete sensitivity but no time-zero run and no function-body sensitivity; prefer <code>always_comb</code>
<code>assign</code>	Continuous assignment	Simple combinational expressions on one net or variable

```
// The canonical two-block style: next-state logic + state register
always_comb begin
    cnt_d = cnt_q;                // default: hold (prevents latches)
    if (load) cnt_d = load_val;
    else if (inc) cnt_d = cnt_q + 1'b1;
end

always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) cnt_q <= '0;
    else cnt_q <= cnt_d;
```

Separate the combinational next-value logic (blocking `=`) from the register (non-blocking `<=`). The `_d/_q` naming mirrors the flop's D and Q pins. Chapter 4 explains why the assignment types matter.

3.10 The synthesizable subset

Synthesis tools accept only constructs that map to hardware. The boundary is not arbitrary: anything that needs a notion of absolute time, dynamic memory allocation or file I/O has no gate-level equivalent.

Construct	Synthesizable?	Notes
module, ports, parameters, localparam	Yes	
assign, always_comb, always_ff, always_latch	Yes	The core of RTL
if / case / unique / priority, ?:	Yes	Become muxes and priority logic
for loops with constant bounds	Yes	Unrolled into parallel hardware - not iteration in time
generate for / if / case	Yes	Structural replication
functions (automatic), constant functions	Yes	No timing controls
typedef, enum, packed struct/union, packages	Yes	
interfaces and modports	Yes (most tools)	Check your flow; many teams restrict them at top level
+ - * and shifts; / % by powers of two	Yes	Full / and % infer huge combinational dividers - usually a bug
initial blocks	FPGA: often (RAM/flop init). ASIC: no	ASIC state must come from reset
#delays	Ignored	Causes sim/synth mismatch; never in RTL

Construct	Synthesizable?	Notes
===, !==, casex/casez with x	No / partly	casez with ? wildcards is fine; avoid casex
while / repeat / forever with timing, wait, @ inside logic	No	Testbench only
\$display, \$finish, \$random, file I/O	No (ignored)	Testbench only
classes, dynamic arrays, queues, associative arrays, strings	No	Verification language (Part V)
real / shortreal	No	Model analog or reference behaviour only
force / release, fork / join	No	Testbench only

Table 3.1 - The synthesizable subset of SystemVerilog (IEEE 1800-2017 plus common tool support).

A FOR LOOP IS HARDWARE REPLICATION, NOT TIME

for (int i = 0; i < 64; i++) sum += x[i]; inside `always_comb` does not take 64 cycles; it builds a 64-input adder chain that completes in one cycle - and may be your critical path. If you want a result computed over many cycles, you must build the counter and the accumulator register yourself. Interviewers use this to separate software thinking from hardware thinking.

3.11 Summary

- Use named port connections, name every generate block, and start every file with the default-nettype-none directive.
- `logic` is the default type; `wire` only for multiply-driven nets. x means unknown, z means undriven; `===` is testbench-only.
- Master part-selects (`base +: width`), reduction, concatenation and replication; `>>>` needs a signed operand.
- Expressions take the width of their largest operand including the LHS, and are unsigned if any operand is unsigned. Size literals; never mix signedness.
- Parameters and generate build configurable structure at elaboration time; enums, packed structs and packages make RTL readable and safe.
- Use `always_comb` for combinational logic, `always_ff` for flops, and stay inside the synthesizable subset.

3.12 Exercises

- 1 Rewrite `ripple_add` without `generate`, using a single `always_comb` with a `for` loop. Is the hardware different?
- 2 Predict, then check in a simulator: with `logic [3:0] a = 4'd9, b = 4'd9; logic [4:0] y;` what are `y = a + b`, `y = {a + b}` and `y = (a + b) >> 1`? Explain each using the sizing rules.
- 3 Write a function `sat_add8` that adds two signed 8-bit values and saturates to -128/127. Test it on all 65,536 input pairs against a reference computed with `int`.
- 4 Using `+:`, write a module that extracts byte `sel` (0-3) from a 32-bit word. Sketch the hardware it infers.
- 5 Define a packed struct for an AXI-Lite write-address beat (`addr`, `prot`) and a package holding it. What does `$bits` return?
- 6 Classify each as synthesizable or not, and why: `#5 q <= d;`, `initial q = 0;`, `a / 3`, `a >> sh`, `$clog2(DEPTH)`, a `while` loop whose bound depends on an input.

4. Simulation Semantics: The Event Scheduler, Blocking vs Non-Blocking, and Races

Hardware is massively parallel: every gate and flop works at once. A simulator runs on a sequential CPU. The bridge between the two is the **event scheduler** defined in IEEE 1800 clause 4, and it decides what your RTL means in simulation. Understanding it turns the famous rules - 'use `<=` for flops, `=` for combinational logic' - from folklore into logic, and it is the key to races, simulation/synthesis mismatches and X bugs. It is also one of the most-asked topics in RTL interviews.

4.1 Processes, events and time steps

A SystemVerilog simulation is a set of concurrent **processes**: every `always` and `initial` block, every continuous `assign`, every gate primitive. A process runs until it hits a timing control (`@`, `#`, `wait`) and then suspends. An **update event** is a change of value on a signal; it wakes every process sensitive to that signal (an **evaluation event**). Simulation time advances only when no events remain at the current time.

THE FUNDAMENTAL NON-DETERMINISM

When several processes are ready at the same time in the same region, the standard lets the simulator run them in **any order**. Code whose result depends on that order has a **race**: it may work in one simulator, fail in another, and change behaviour when an unrelated line is added. Correct RTL is race-free by construction.

4.2 The stratified event queue

Each time step is divided into ordered **regions**. The simulator executes all events in a region, possibly adding new events to earlier regions, and loops until everything is empty before advancing time. The regions that matter to designers are:

Region	What executes there
Preponed	Sampling of values for concurrent assertions and clocking-block inputs: the values <i>before</i> anything changes in this time step
Active	Blocking assignments, continuous assignments, evaluation of the RHS of non-blocking assignments, <code>\$display</code> , resumption of processes
Inactive	Processes resumed after an explicit <code>#0</code> delay
NBA	Updates of the LHS of non-blocking assignments (<code><=</code>), in the order the assignments executed
Observed	Evaluation of concurrent assertion properties using the preponed samples
Reactive (+ Re-Inactive, Re-NBA)	Program-block code, assertion action blocks; mirrors Active/Inactive/NBA for testbench code
Postponed	<code>\$strobe</code> , <code>\$monitor</code> ; the final, settled values of the time step. No new events allowed

Table 4.1 - Scheduling regions (the standard also defines PLI callback regions such as Pre-Active, omitted here).



Figure 4.1 - The stratified event queue for one time step. Each pass Active -> Inactive -> NBA and back is one delta cycle.

The following testbench makes the regions visible:

```

module tb;
  logic [3:0] a = 4'd0, b = 4'd0;
  initial begin
    a = 4'd1; // blocking: updates now (active region)
    b <= 4'd7; // NBA: update scheduled for the NBA region
    $display("display: a=%0d b=%0d", a, b); // active region: sees old b
    $strobe ("strobe : a=%0d b=%0d", a, b); // postponed region: sees final values
    #0 $display("#0 : a=%0d b=%0d", a, b); // inactive region: still before NBA
    #1 $display("#1 : a=%0d b=%0d", a, b); // next time step
  end
endmodule

```

```

display: a=1 b=0
#0      : a=1 b=0
strobe  : a=1 b=7
#1      : a=1 b=7

```

`$display` prints immediately and sees the old `b`; `#0` defers only to the Inactive region, which is still before the NBA update; `$strobe` prints in Postponed, after all updates. Note that the strobe line appears after the `#0` line even though it was executed first.

4.3 Delta cycles

A **delta cycle** is one iteration through the Active/Inactive/NBA loop without advancing time. A chain of continuous assignments `assign b = ~a; assign c = ~b;` settles in successive deltas: `a` changes, `b` updates in the next evaluation, then `c`. On a waveform all three change at the same time stamp, but they did so in a well-defined causal order. Delta cycles are how a simulator models zero-delay logic without an ordering paradox - and a combinational loop shows up as a simulation that never leaves the time step (an infinite delta loop, which simulators report as an iteration limit).

4.4 Blocking versus non-blocking assignment

	Blocking <code>a = b;</code>	Non-blocking <code>a <= b;</code>
When the RHS is evaluated	Immediately	Immediately (Active)
When the LHS is updated	Immediately, before the next statement	Later, in the NBA region, after all Active processes have run

	Blocking <code>a = b;</code>	Non-blocking <code>a <= b;</code>
Next statement in the same block sees	The new value	The old value
Models	Wires and combinational intermediate values	Flip-flops: every flop samples its D before any Q changes
Use in	<code>always_comb</code>	<code>always_ff</code>

The key insight is that **non-blocking assignment models the physics of a clock edge**. In real hardware, all flops on a clock sample their D inputs at the same instant, and only afterwards (a clock-to-Q delay later) do their outputs change. NBA reproduces exactly this: every `always_ff` evaluates its RHS with the old values in the Active region, and all LHS updates happen together in the NBA region. The order in which the simulator runs the blocks no longer matters.

Example 1: the swap

```

module swap_bad (input logic clk, rst, output logic [3:0] a, b);
    always @(posedge clk) if (rst) a = 4'd1; else a = b;    // blocking: RACE
    always @(posedge clk) if (rst) b = 4'd2; else b = a;
endmodule

module swap_good (input logic clk, rst, output logic [3:0] a, b);
    always_ff @(posedge clk) if (rst) a <= 4'd1; else a <= b;
    always_ff @(posedge clk) if (rst) b <= 4'd2; else b <= a;
endmodule

module tb;
    logic clk = 0, rst = 1;
    logic [3:0] a1, b1, a2, b2;
    swap_bad u_bad (.clk(clk), .rst(rst), .a(a1), .b(b1));
    swap_good u_good (.clk(clk), .rst(rst), .a(a2), .b(b2));
    always #5 clk = ~clk;
    initial begin
        @(negedge clk) rst = 0;
        repeat (3) @(negedge clk)
            $display("%2t bad: a=%0d b=%0d    good: a=%0d b=%0d", $time, a1, b1, a2, b2);
        $finish;
    end
endmodule

```

```

20 bad: a=2 b=2    good: a=2 b=1
30 bad: a=2 b=2    good: a=1 b=2
40 bad: a=2 b=2    good: a=2 b=1

```

Two registers that should exchange values every cycle. With non-blocking assignments they do. With blocking assignments, whichever block runs first overwrites its register before the other reads it.

Icarus happened to run the `a` block first, so both became 2; another simulator - or the same one after an unrelated edit - may run the `b` block first and give 1 and 1. Synthesis, meanwhile, builds two cross-coupled flops for *both* versions, which do swap. The blocking version is therefore not just wrong in simulation, it is a **simulation/synthesis mismatch**: the gates you tape out behave differently from the RTL you verified.

Example 2: the shift register

```

module shift_bad (input logic clk, d, output logic q3);
  logic q1, q2;
  always @(posedge clk) begin      // blocking: each line sees the NEW value above it
    q1 = d;
    q2 = q1;
    q3 = q2;
  end
endmodule

module shift_good (input logic clk, d, output logic q3);
  logic q1, q2;
  always_ff @(posedge clk) begin  // NBA: every RHS is sampled before any update
    q1 <= d;
    q2 <= q1;
    q3 <= q2;
  end
endmodule

```

```

module tb;
  logic clk = 0, d = 0, qb, qg;
  shift_bad u_b (.clk(clk), .d(d), .q3(qb));
  shift_good u_g (.clk(clk), .d(d), .q3(qg));
  always #5 clk = ~clk;
  initial begin
    $display("cycle d | bad q3  good q3");
    for (int i = 1; i <= 7; i++) begin
      @(negedge clk) d = (i == 3);          // a single 1-cycle pulse
      @(posedge clk) #1 $display(" %0d  %b |  %b          %b", i, d, qb, qg);
    end
    $finish;
  end
endmodule

```

cycle	d	bad q3	good q3
1	0	0	x
2	0	0	0
3	1	1	0
4	0	0	0
5	0	0	1
6	0	0	0
7	0	0	0

The correct shift register delays the pulse by three clock edges (it was captured at edge 3 and reaches q3 at edge 5, printed just after). The blocking version passes d straight through: it is one flop, not three - and synthesis agrees with the simulation this time, so the bug ships silently as a missing pipeline stage.

```

intended (NBA):  d -->[FF q1]-->[FF q2]-->[FF q3]--> q3      3-cycle delay

blocking, this order:  d --->> q1 (FF)
                      +--> q2 (FF)    q1, q2 are the same flop value;
                      +--> q3 (FF)    synthesis merges them: 1-cycle delay

```

Figure 4.2 - What the two shift-register versions infer.

REVERSING THE ORDER 'FIXES' IT - DO NOT DO THAT

Writing the blocking version bottom-up ($q3 = q2$; $q2 = q1$; $q1 = d$;) happens to model a shift register, and you will see it in old code. It is fragile: it breaks when the statements are reordered or split across blocks, and it relies on reasoning that NBA makes unnecessary. Use $<=$ in every clocked block.

The guidelines

Clifford Cummings' classic SNUG 2000 paper distilled these rules, and they remain the industry standard:

- 1 Sequential logic (flops, latches): **non-blocking** `<=`.
- 2 Combinational logic in an `always` block: **blocking** `=`.
- 3 Sequential and combinational logic in the same block: non-blocking - better still, split them (the `_d/_q` style of Chapter 3).
- 4 Never mix `=` and `<=` for the same variable, or in the same block.
- 5 Never assign the same variable from more than one `always` block (`always_ff/always_comb` make this a compile error).
- 6 Use `$strobe` (or print after a small delay) to display values updated by NBAs.
- 7 Do not use `#0` assignments.

4.5 Why blocking for combinational logic

Inside `always_comb`, intermediate variables must behave like wires: a later statement must see the value computed by an earlier one *in the same evaluation*. Blocking assignment does exactly that. If you used `<=` in combinational logic, a statement reading an intermediate variable would see its stale value; the block would then re-trigger when the NBA update arrives and settle one delta later. The final value is usually right, but the simulation is slower, the waveform shows spurious transitions, and any code sampling the signal in between - for instance a `$display` or an assertion - sees garbage. And with `always @(a)`-style incomplete lists it can simply be wrong.

```
always_comb begin
    tmp = a & b;           // blocking: tmp is updated now...
    y  = tmp | c;         // ...so this sees the new tmp, like a wire
end
```

Correct: a two-gate network described with blocking assignments.

4.6 Races between always blocks and between testbench and DUT

A race exists whenever two processes triggered by the same event communicate through a variable updated with a blocking assignment. Between RTL blocks, NBA removes the race (Example 1). The more insidious case is the **testbench-to-DUT race**: the testbench drives an input with a blocking assignment on the same clock edge at which the DUT samples it.

```
module dff (input logic clk, d, output logic q);
    always_ff @(posedge clk) q <= d;
endmodule

module tb;
    logic clk = 0, d_race = 0, d_safe = 0, q_race, q_safe;
    dff u_race (.clk(clk), .d(d_race), .q(q_race));
    dff u_safe (.clk(clk), .d(d_safe), .q(q_safe));
    always #5 clk = ~clk;
    initial begin
        @(posedge clk);
        d_race = 1;           // blocking at the clock edge: races with the DUT's sampling
        d_safe <= 1;          // NBA at the edge: DUT always samples the OLD value
        @(posedge clk) #1;
        $display("after 2nd edge: q_race=%b q_safe=%b", q_race, q_safe);
        $finish;
    end
    initial begin
        @(posedge clk) #1;
        $display("after 1st edge: q_race=%b q_safe=%b", q_race, q_safe);
    end
end
```

```
endmodule
```

```
after 1st edge: q_race=1 q_safe=0
after 2nd edge: q_race=1 q_safe=1
```

Here the testbench process ran before the flop's process, so the flop captured the new value at the very edge that 'produced' it - a zero-cycle path that real hardware can never have. The NBA version is deterministic.

Robust testbenches remove the race structurally:

- Drive DUT inputs on the **opposite clock edge** (as the **drive** task in Chapter 3 does), or with NBAs at the active edge.
- Better, use a **clocking block**: it samples DUT outputs in the Preponed region (the values from just before the edge) and drives inputs with a specified skew after the edge - exactly like a real synchronous neighbour. Chapter 22 builds testbenches this way.
- Program blocks (Reactive region) were introduced for the same purpose; clocking blocks in modules are now the common practice.

#0 IS NOT A FIX

Adding **#0** moves a process to the Inactive region, which runs after the Active region - so it can make one particular race disappear. But the next engineer who adds another **#0** restores the race one level deeper. The standard's own guidance is to avoid **#0**; treat it as a code smell in any review.

4.7 Sensitivity lists and simulation/synthesis mismatch

Synthesis **ignores** sensitivity lists: it builds hardware from the statements in the block. Simulation obeys them literally: a block runs only when a listed signal changes. An incomplete list therefore makes the RTL simulate differently from the gates.

```
module and_incomplete (input logic a, b, output logic y);
    always @(a) y = a & b;          // b missing from the list: simulation-only bug
endmodule

module and_comb (input logic a, b, output logic y);
    always_comb y = a & b;          // implied complete sensitivity
endmodule

module tb;
    logic a = 0, b = 0, y1, y2;
    and_incomplete u1 (.a(a), .b(b), .y(y1));
    and_comb u2 (.a(a), .b(b), .y(y2));
    initial begin
        #1 a = 1; #1 $display("a=%b b=%b | @(a): y=%b always_comb: y=%b", a, b, y1, y2);
        #1 b = 1; #1 $display("a=%b b=%b | @(a): y=%b always_comb: y=%b", a, b, y1, y2);
        #1 a = 0; #1 $display("a=%b b=%b | @(a): y=%b always_comb: y=%b", a, b, y1, y2);
    end
endmodule
```

```
a=1 b=0 | @(a): y=0 always_comb: y=0
a=1 b=1 | @(a): y=0 always_comb: y=1
a=0 b=1 | @(a): y=0 always_comb: y=0
```

When b rises, the incomplete block does not wake up, so y stays 0 - it behaves like a latch in simulation. The synthesized AND gate outputs 1. Gate-level simulation or silicon will disagree with your RTL regressions.

Mismatch source	Simulation does	Synthesis builds	Prevention
Incomplete sensitivity list	Holds old value	Pure combinational logic	always_comb / @*
# delays in RTL	Delays events	Ignores delays	No delays in RTL
Blocking in flops (swap)	Order-dependent race	Correct flops	<= in always_ff
initial values (ASIC)	Registers start known	Registers start random	Reset every control flop

Mismatch source	Simulation does	Synthesis builds	Prevention
casex/casez with x inputs, full_case/parallel_case pragmas	Matches/ignores x or unlisted items	Different logic from the pragmas	unique/priority case; no pragmas (Ch 5)
X optimism (next section)	Picks a branch for x	Real 0/1 hardware	X-prop sim, assertions, reset

Table 4.2 - The usual sources of simulation/synthesis mismatch. Gate-level simulation and equivalence checking (Chapters 25-26) exist largely to catch these.

4.8 X-propagation: optimism and pessimism

An x in simulation stands for a value that real hardware would have as a definite 0 or 1 - we just do not know which. Ideally x would propagate exactly as far as the uncertainty does in hardware. Verilog's semantics do not achieve that; they err in both directions.

```

module tb;
  logic sel, a, b, y_if, y_tern, y_case;
  always_comb begin
    if (sel) y_if = a;           // X on sel is treated as FALSE -> optimistic
    else    y_if = b;
  end
  assign y_tern = sel ? a : b;   // X on sel merges both inputs bit by bit
  always_comb begin
    case (sel)
      1'b1:    y_case = a;
      1'b0:    y_case = b;
      default: y_case = 1'bx;    // explicit X for "impossible" selects
    endcase
  end
  initial begin
    sel = 1'bx;
    a = 1; b = 0; #1 $display("sel=x a=1 b=0 | if: %b  ?: %b  case: %b", y_if, y_tern, y_case);
    a = 1; b = 1; #1 $display("sel=x a=1 b=1 | if: %b  ?: %b  case: %b", y_if, y_tern, y_case);
  end
endmodule

```

```

sel=x a=1 b=0 | if: 0  ?: x  case: x
sel=x a=1 b=1 | if: 1  ?: 1  case: x

```

Three descriptions of the same 2:1 mux, three different X behaviours.

- **X optimism (if):** an x condition is treated as false, so the **else** branch runs and y becomes a clean 0. Real hardware could have produced 1. The x - which might be an uninitialized register or a missing reset - is **hidden**, and the bug survives RTL simulation. The same happens with **case** when no item matches and there is no default assigning x.
- **Accurate (?):** when the select is x, each output bit is x unless both inputs agree. With a=b=1 the result is 1, which is exactly what the hardware would do.
- **X pessimism (case with default x):** y is x even when a=b=1 and any hardware would output 1. Pessimism causes false failures - most painfully in gate-level simulation, where an AND-OR mux gives x for sel=x even with equal inputs, and a flop without reset that feeds back on itself ($q \leq q \wedge t$) stays x forever although silicon would settle.

HOW PROFESSIONAL FLOWS HANDLE X

Commercial simulators offer X-propagation modes (for example VCS **-xprop**, Xcelium **-xprop**) that make **if/case** merge both branches like **?:**, exposing optimism bugs in RTL. Teams combine this with: resets on all control flops, assertions such as **assert property (@(posedge clk) disable iff (!rst_n) !\$isunknown(valid))** on key interface signals, formal X-checks, and a targeted gate-level simulation run. Chapter 12 discusses which flops need reset; Chapter 26 covers gate-level X-pessimism in detail.

4.9 Summary

- Simulation is a set of concurrent processes scheduled region by region; order within a region is undefined, which is the root of all races.
- Key regions: Preponed (sampling), Active (blocking, RHS of NBA), Inactive (#0), NBA (LHS updates), Observed (assertions), Reactive, Postponed (\$strobe, \$monitor).
- NBA models a clock edge: all flops sample before any updates. Use `<=` in `always_ff`, `=` in `always_comb`, and never mix them.
- Blocking assignments in clocked blocks cause races (swap) or wrong hardware (shift register). TB/DUT races are avoided with NBAs, opposite-edge driving, or clocking blocks - never with `#0`.
- Incomplete sensitivity lists, delays, initial values and case pragmas cause simulation/synthesis mismatches; `always_comb` removes the first.
- `if/case` are X-optimistic, `?:` is accurate, gate-level simulation is X-pessimistic. Use resets, X-prop modes and `$isunknown` assertions.

4.10 Exercises

- 1 Predict the output order of `$display`, `$strobe`, `$monitor` and a `#0 $display` when all four are executed at time 0 after an NBA to a monitored variable. Verify in a simulator.
- 2 Rewrite `swap_bad` so that it is race-free using only blocking assignments in a *single* `always` block and a temporary variable. Is this good style? What does synthesis build?
- 3 A colleague's shift register passes simulation with blocking assignments written bottom-up. List three edits that would silently break it.
- 4 Write a testbench with a race that gives different results depending on the order of two `initial` blocks. Change the order of the blocks in the file - does Icarus's answer change?
- 5 For `always @(a or b) y = sel ? a : b;` describe precisely when simulation and synthesized hardware disagree.
- 6 Design a 4:1 mux whose RTL simulation is neither X-optimistic nor pessimistic for a single-bit X on either select bit. Test it with all combinations of sel containing one x.

PART II

Core RTL Building Blocks

The handful of structures from which every chip is assembled: combinational logic that does not accidentally remember, flops with the right reset, state machines that cannot get lost, arithmetic that does not silently overflow, memories and FIFOs, and the valid/ready pipelines that move data through an SoC at one word per clock.

5. Combinational Logic

Combinational logic is the part of a design whose outputs depend only on its **current** inputs. It has no memory, no clock and no state - and the single most common way beginners break a design is by writing something they believe is combinational that the synthesis tool correctly interprets as having memory. This chapter shows how to describe combinational hardware so that simulation, synthesis and your intent all agree, and what gates each coding style produces.

THE CONTRACT OF A COMBINATIONAL BLOCK

For every possible input combination, every output is assigned a value, and that value is a pure function of the inputs. If any path through the code leaves an output unassigned, the output must 'remember' its previous value - and the only hardware that remembers without a clock edge is a **latch**.

5.1 assign versus always_comb

There are two ways to write combinational logic in SystemVerilog. A continuous `assign` describes one net as an expression; it is ideal for short Boolean equations, muxes written with `?:`, and wiring. An `always_comb` block describes one or more variables procedurally, which is clearer as soon as there are `case` statements, loops or intermediate values.

```
module mux4_assign (
    input  logic [7:0] a, b, c, d,
    input  logic [1:0] sel,
    output logic [7:0] y
);
    assign y = (sel == 2'd0) ? a :
               (sel == 2'd1) ? b :
               (sel == 2'd2) ? c : d;
endmodule

module mux4_case (
    input  logic [7:0] a, b, c, d,
    input  logic [1:0] sel,
    output logic [7:0] y
);
    always_comb begin
        case (sel)
            2'd0: y = a;
            2'd1: y = b;
            2'd2: y = c;
            default: y = d;
        endcase
    end
endmodule
```

Two descriptions of the same 4:1 multiplexer. Synthesis produces identical netlists: the tool recognises that the `sel` values are mutually exclusive and builds a balanced mux, not a chain.

Property	<code>always @*</code> (Verilog-2001)	<code>always_comb</code> (SystemVerilog)
Sensitivity list	Inferred, but misses variables read inside called functions	Inferred, including function bodies
Runs at time 0	No - output is X until an input changes	Yes - evaluates once at time zero, so constants propagate
Multiple drivers	Allowed (a bug)	Illegal: a variable written in <code>always_comb</code> may not be written anywhere else
Latch check	None	Tools warn if the block infers storage

Property	<code>always @*</code> (Verilog-2001)	<code>always_comb</code> (SystemVerilog)
Blocking =	Required by convention	Required by convention

Always prefer `always_comb`; use `always_latch` in the rare case you really want a latch, so your intent is explicit.

USE BLOCKING ASSIGNMENTS IN COMBINATIONAL BLOCKS

Inside `always_comb`, write `=` (blocking). The block then reads like software: a later statement sees the value an earlier statement assigned, which is exactly how the default-then-override idiom works. Non-blocking `<=` in combinational code creates simulation/synthesis mismatches and extra delta cycles (Chapter 4).

5.2 Priority logic versus parallel logic

An `if / else if` chain has **priority** semantics: the first true condition wins, so the second condition is only evaluated when the first is false. In hardware that becomes a chain of 2:1 muxes whose depth grows with the number of branches. A `case` whose items are mutually exclusive describes **parallel** selection: an AND-OR mux whose depth grows only with log of the number of inputs.

```

if (c0) y=a; else if (c1) y=b; else if (c2) y=c; else y=d;

d ---|0\
    | |---+
c ---|1/  +--|0\
      c2      | |---+
b -----|1/  +--|0\
           c1      | |---- y    3 mux levels in series: c0 has the
a -----|1/                          shortest path to y, d and c the longest
           c0

case (sel) 0:a 1:b 2:c 3:d    ->  decode sel once, then one AND-OR level

    a & (sel==0) --+
    b & (sel==1) --+--[ OR ]---- y    every input has the same path length
    c & (sel==2) --+
    d & (sel==3) --+

```

Priority chain versus parallel mux. Use priority only where the specification really has priority (interrupts, arbiters, exception ordering).

Modern synthesis tools are good at restructuring: if they can prove the `if` conditions are mutually exclusive (for example because they all compare the same signal against different constants), they flatten the chain. They **cannot** do this when the conditions are independent signals, because then the priority is real functionality. The RTL designer's job is therefore to write priority only when it is intended - and to put the late-arriving (timing-critical) signal in the **first** `if`, closest to the output.

INTERVIEW INSIGHT: 'WHERE WOULD YOU PUT A LATE SIGNAL?'

In a priority chain, the condition tested **first** drives the final mux nearest the output, so it has the shortest path. If static timing reports that `c2` arrives late, restructure so that `c2` selects between two precomputed results at the output: `y = c2 ? y_if_c2 : y_if_not_c2;`. This 'late-select' transformation is a standard timing fix you will use in Chapter 18.

5.3 unique, priority, and the dangers of parallel_case

SystemVerilog adds two qualifiers that let you state what you know about a `case` or `if` chain. They are **checked in simulation** and used as **optimisation permission** by synthesis.

Qualifier	What you promise	Simulation check	Synthesis may
<code>unique case</code>	Exactly one item matches (items are mutually exclusive AND complete)	Warning if zero or more than one item matches	Build a parallel mux; treat unmatched values as don't-care

Qualifier	What you promise	Simulation check	Synthesis may
<code>unique0 case</code>	At most one item matches	Warning if more than one matches	Build a parallel mux
<code>priority case</code>	At least one item matches; order matters	Warning if none matches	Treat unmatched values as don't-care, keep priority
<code>// synopsys parallel_case full_case</code>	Same as <code>unique</code> , but only the synthesis tool reads it	None - simulation still has priority and latch behaviour	Optimise as told - creating a sim/synth mismatch if you were wrong

```

module alu_unique (
    input  logic [1:0] op,
    input  logic [7:0] a, b,
    output logic [7:0] y
);
always_comb begin
    y = '0;
    unique case (op)
        2'b00: y = a + b;
        2'b01: y = a - b;
        2'b10: y = a & b;
        2'b11: y = a | b;
    endcase
end
endmodule

```

`unique` documents that the four items are exhaustive and exclusive; if `op` were ever X, simulators flag a violation instead of silently keeping the default.

PITFALL - THE OLD FULL_CASE / PARALLEL_CASE PRAGMAS

These comments change what synthesis builds without changing what simulation does. If the promise is false, the gate-level netlist behaves differently from the RTL you verified - the classic 'works in RTL sim, fails in gate sim' bug. Many companies ban them in their coding guidelines (Chapter 27). `unique` and `priority` are safe replacements because the simulator checks the promise.

5.4 casez, casex and don't-cares

`casez` treats `z` and `?` in the case items as don't-care bits. It is the standard way to write a priority encoder or an instruction decoder compactly. `casex` also treats `x` in the **selector** as don't-care, which means an X on the input can silently match a branch and hide a bug; avoid `casex` in design code.

```

// 8-input priority encoder: highest set bit wins
module prienc8 (
    input  logic [7:0] req,
    output logic [2:0] idx,
    output logic      valid
);
always_comb begin
    idx  = '0;          // default assignments: no latch possible
    valid = 1'b1;
    casez (req)
        8'b1???????: idx = 3'd7;
        8'b01?????: idx = 3'd6;
        8'b001?????: idx = 3'd5;
        8'b0001????: idx = 3'd4;
        8'b00001???: idx = 3'd3;
        8'b000001?: idx = 3'd2;
        8'b0000001?: idx = 3'd1;
        8'b00000001: idx = 3'd0;
    endcasez
end

```

```

        default:      valid = 1'b0;
    endcase
end
endmodule

```

A table like this does not scale to 64 requesters. The parameterised form uses a loop in which the **last** assignment wins - so iterating upward gives the highest index priority. Synthesis unrolls the loop into exactly the same priority structure.

```

// Same function written as a parameterised loop (last assignment wins)
module prienc_loop #(parameter int N = 8) (
    input  logic [N-1:0]      req,
    output logic [$clog2(N)-1:0] idx,
    output logic              valid
);
    localparam int W = $clog2(N);
    always_comb begin
        idx = '0;
        for (int i = 0; i < N; i++)
            if (req[i]) idx = W'(i);
        valid = |req;
    end
endmodule

```

5.5 Decoders, encoders and one-hot logic

A **decoder** turns an n-bit binary code into a 2^n -bit one-hot vector; an **encoder** does the reverse. They are the glue of every SoC: address decoders select peripherals (Chapter 14), register-file write ports decode the destination register (Chapter 9), and arbiters produce one-hot grants that must be encoded back to an index.

```

module dec3to8 (
    input  logic [2:0] a,
    input  logic      en,
    output logic [7:0] y
);
    assign y = en ? (8'b1 << a) : 8'b0;
endmodule

// One-hot -> binary encoder: an OR tree, no priority
module oh2bin #(parameter int N = 8) (
    input  logic [N-1:0]      oh,
    output logic [$clog2(N)-1:0] bin
);
    localparam int W = $clog2(N);
    always_comb begin
        bin = '0;
        for (int i = 0; i < N; i++)
            bin |= oh[i] ? W'(i) : '0;
    end
endmodule

```

Notice the difference between `oh2bin` and `prienc_loop`. Because the one-hot encoder **assumes** at most one bit is set, it can simply OR together the indices of the set bits: bit k of the output is the OR of all input bits whose index has bit k set. That is a single OR level of $N/2$ inputs per output bit - much smaller and faster than a priority encoder. The cost is that it returns garbage if two bits are set. Use it when the one-hot property is guaranteed by construction (an arbiter's grant, a one-hot FSM) and protect the guarantee with an assertion (Chapter 23).

```

module tb_ch5;
    logic [7:0] req;
    logic [2:0] i1, i2, b;

```

```

logic      v1, v2;
int errors = 0;
prienc8     u1 (.req(req), .idx(i1), .valid(v1));
prienc_loop #(8) u2 (.req(req), .idx(i2), .valid(v2));
oh2bin      #(8) u3 (.oh(req), .bin(b));
initial begin
    for (int k = 0; k < 256; k++) begin
        req = k[7:0];
        #1;
        if (i1 != i2 || v1 != v2) errors++;
        if ($countones(req) == 1 && b != i1) errors++;
        if (k == 0 || k == 1 || k == 8'h12 || k == 8'h80 || k == 8'hFF)
            $display("req=%b prienc idx=%0d valid=%b loop idx=%0d", req, i1, v1, i2);
        end
        $display("exhaustive check of 256 vectors: %0d errors", errors);
        $finish;
    end
endmodule

```

An exhaustive self-checking testbench: with only 8 inputs, all 256 combinations are cheap, so there is no excuse for anything less.

```

req=00000000 prienc idx=0 valid=0 loop idx=0
req=00000001 prienc idx=0 valid=1 loop idx=0
req=00010010 prienc idx=4 valid=1 loop idx=4
req=10000000 prienc idx=7 valid=1 loop idx=7
req=11111111 prienc idx=7 valid=1 loop idx=7
exhaustive check of 256 vectors: 0 errors

```

Icarus Verilog output. The table-driven and loop-based encoders agree on every input, and the one-hot encoder agrees wherever its precondition holds.

The one-hot mux is the data-path counterpart of the one-hot encoder: AND each input with its select bit and OR the results. It is the fastest possible mux for a wide select and it appears in every crossbar and arbiter.

```

module onehot_mux #(parameter int N = 4, W = 8) (
    input  logic [N-1:0]    sel,      // must be one-hot (or zero)
    input  logic [N-1:0][W-1:0] din,
    output logic [W-1:0]    y
);
always_comb begin
    y = '0;
    for (int i = 0; i < N; i++)
        y |= din[i] & {W{sel[i]}};
    end
endmodule

```

5.6 Unintended latches, and how never to write one

A latch is inferred whenever a variable assigned in a combinational block is **not assigned on every path**. The two classic causes are an incomplete **case** and an **if** without an **else**.

```
// BUG: latch inferred on y when sel == 2'd3, on grant when req == 0
module latch_bug (
    input  logic [1:0] sel,
    input  logic [7:0] a, b, c,
    input  logic      req,
    output logic [7:0] y,
    output logic      grant
);
    always @* begin
        case (sel)
            2'd0: y = a;
            2'd1: y = b;
            2'd2: y = c;
        endcase
        if (req) grant = 1'b1;
    end
endmodule
```

Two latches: **y** keeps its value when **sel** is 3, and **grant** once set can never be cleared. Icarus compiles this without complaint - latch detection is a job for synthesis and lint.

```
module latch_fixed (
    input  logic [1:0] sel,
    input  logic [7:0] a, b, c,
    input  logic      req,
    output logic [7:0] y,
    output logic      grant
);
    always_comb begin
        y      = '0;          // default first ...
        grant = 1'b0;
        case (sel)            // ... then override
            2'd0: y = a;
            2'd1: y = b;
            2'd2: y = c;
            default: ;         // explicit: keep the default
        endcase
        if (req) grant = 1'b1;
    end
endmodule
```

The default-assignment idiom: assign every output at the top of the block, then override. It is impossible to create a latch this way, however the branches are later edited.

PITFALL - WHY LATCHES ARE BAD IN AN ASIC FLOW

A latch is not wrong in itself - high-performance custom designs use them deliberately. Unintended latches are wrong because: (1) they are **transparent** while enabled, so a combinational loop through them can oscillate; (2) their enable is a data-derived signal that glitches, so they capture garbage; (3) static timing uses time-borrowing rules for latches that no one on your team planned for; (4) scan insertion and ATPG handle them poorly (Chapter 20); (5) the behaviour differs between RTL and gate-level simulation. Treat every synthesis 'latch inferred' message as an error.

Latch-free combinational code

- ☐ Use **always_comb**, never **always @(a or b)** with a hand-written list.
- ☐ Assign a default value to every output at the top of the block.
- ☐ Give every **case** a **default**, even when all values are listed (it costs nothing and survives later edits).
- ☐ Every **if** that assigns an output either has an **else** or is covered by a default.
- ☐ Never read a variable in the same **always_comb** before it has been assigned in that block (that is a loop or a latch).

5.7 Parity, comparators and other everyday functions

```

module parity_cmp (
    input  logic [31:0] d,
    input  logic [15:0] x, y,
    output logic        even_par, // 1 when the number of ones is odd -> makes total even
    output logic        eq, lt_u, lt_s
);
    assign even_par = ^d;
    assign eq       = (x == y);
    assign lt_u     = (x < y);
    assign lt_s     = ($signed(x) < $signed(y));
endmodule

```

Function	Hardware	Depth for N bits	Notes
^d parity	XOR tree	$\log_2(N)$ XOR levels	Used in memory parity and bus protection (Chapter 9)
x == y	N XNORs + AND tree	$1 + \log_2(N)$	Cheap; equality against a constant is even cheaper
x < y	A subtractor's carry-out (or a dedicated comparator tree)	$\sim \log_2(N)$ with a fast carry	Signedness changes the logic - see Chapter 8
 v, &v	OR / AND reduction tree	$\log_2(N)$	'Any bit set' / 'all bits set'
\$countones(v)	Adder tree (population count)	$\sim \log_2(N)$ adder levels	Much larger than it looks - avoid on wide buses

5.8 Combinational loops and logic depth

A **combinational loop** is a path from a signal back to itself through gates only, with no flop in between. It is almost always a bug: the circuit either oscillates or latches in an unpredictable state, static timing cannot analyse it (the tool breaks the loop arbitrarily), and simulators may hang in an infinite delta-cycle loop. Loops usually appear across module boundaries - module A's output feeds B, whose output combinationaly feeds back into A. The valid/ready handshake is the most common place this happens; Chapter 10 gives the rules that prevent it.

```

+----- module A -----+      +----- module B -----+
| ready_out = f(valid_in) |----->| valid_out = g(ready) |---+
+-----+                  +-----+
      ^                      |
      +----- no flop anywhere -----+

```

A combinational loop created by two individually reasonable modules. Lint tools (Chapter 25) find these; code reviews should too.

Logic depth is the number of gate levels between two flops. At a given process node, one gate level (an FO4 inverter delay) costs roughly 10-20 ps in an advanced node and 50-100 ps in an older one, so a 1 GHz clock allows roughly 20-40 levels after flop overhead, and a fast CPU pipeline stage perhaps 12-20. As an RTL designer you control depth by choosing structures: a parallel mux instead of a priority chain, a one-hot encoder instead of a priority encoder, a tree instead of a chain, and - when nothing else works - an extra pipeline stage (Chapter 10).

THINK IN LEVELS, NOT LINES

Ten lines of **assign** can be two levels of logic; one line containing **a * b + c** can be forty. When you write RTL, keep a rough gate-level picture in mind for every expression. Synthesis is powerful, but it cannot change the fundamental depth of the function you asked for.

5.9 Summary

- Combinational means every output is assigned on every path and depends only on current inputs; anything else infers a latch.
- Use `always_comb` with blocking assignments and the default-then-override idiom; use `assign` for simple equations.
- `if/else if` chains imply priority (a mux chain); mutually exclusive `case` items imply a parallel mux. Put late signals first in a chain.
- `unique` and `priority` document intent and are checked in simulation; never use the `full_case/parallel_case` pragmas.
- Use `casez` for don't-cares, never `casex`. Know the hardware behind decoders, one-hot encoders, priority encoders and one-hot muxes.
- Avoid combinational loops (especially across modules) and control logic depth by choosing the right structure.

5.10 Exercises

- 1 Write a parameterised priority encoder where the **lowest** set bit wins, and verify it exhaustively for $N = 8$ against a behavioural model.
- 2 Rewrite `latch_bug` so that `grant` genuinely needs to hold state (set by `req`, cleared by `clr`). What kind of element should hold it, and where must that code live?
- 3 Draw the gate-level structure of `oh2bin` for $N = 8$. How many OR gates, and how many levels, does each output bit need?
- 4 An `if` chain tests `a`, then `b`, then `c`, all independent 1-bit signals. Timing shows `c` arrives very late. Rewrite the logic so `c` drives only the final 2:1 mux, and prove equivalence with an exhaustive testbench.
- 5 Explain precisely what goes wrong in simulation and in synthesis if `unique case` is used but two items can match at the same time.

6. Sequential Logic: Flip-Flops, Resets and Enables

Sequential logic is where a design gets its memory and its sense of time. In a synchronous ASIC almost all state lives in edge-triggered D flip-flops that update on the rising edge of a clock. How you write a flop decides which standard cell the synthesis tool picks - with or without a reset pin, with or without an enable mux - and those choices ripple into area, timing, power, testability and reset-domain safety. This chapter covers the flop itself, the families of counters and registers built from it, and the small utility circuits every SoC reuses.

6.1 Coding the D flip-flop

The rules are short. Use `always_ff`, trigger on `posedge clk` (plus the reset edge for an asynchronous reset), use non-blocking `<=` for every assignment, and assign each register in exactly one block. `always_ff` lets tools check that the block really describes flops: an assignment path without a clock edge becomes an error rather than a silent latch.

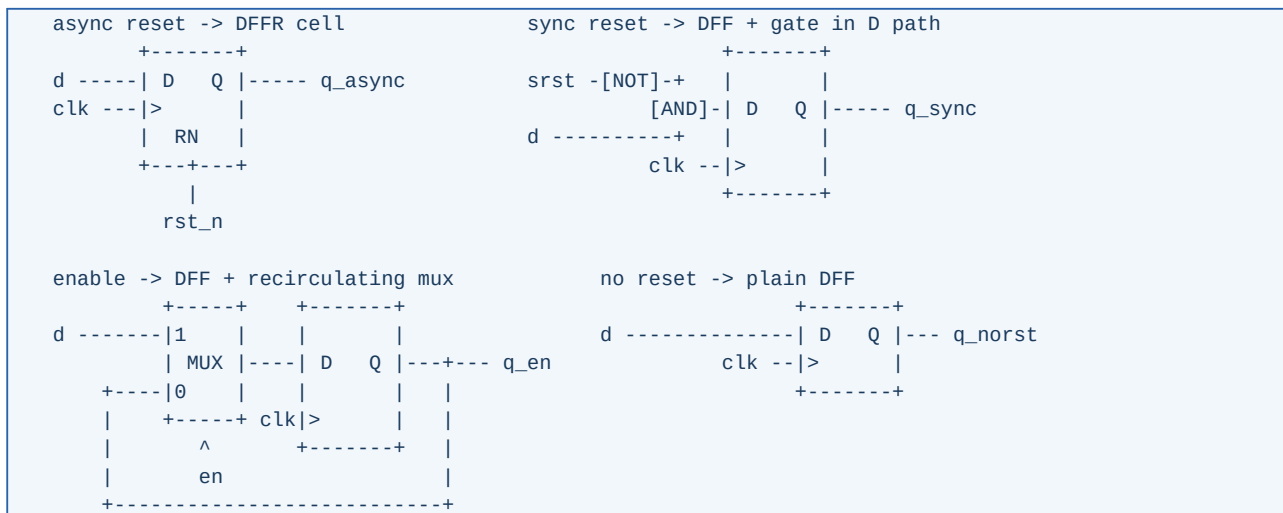
```
// Flop styles
module dff_styles (
    input logic      clk, rst_n, srst, en,
    input logic [7:0] d,
    output logic [7:0] q_async, q_sync, q_en, q_norst
);
    // async active-low reset: maps to a DFF with a reset pin (DFFR)
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) q_async <= '0;
        else       q_async <= d;

    // sync active-high reset: plain DFF + AND gate in front of D
    always_ff @(posedge clk)
        if (srst) q_sync <= '0;
        else     q_sync <= d;

    // enable: recirculating mux (or an enable-flop / clock gate)
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) q_en <= '0;
        else if (en) q_en <= d;

    // datapath flop, no reset: smallest, fastest cell
    always_ff @(posedge clk)
        q_norst <= d;
endmodule
```

Four flop flavours. Each maps to different cells; knowing which is the difference between an RTL coder and an RTL designer.



6.2 Synchronous versus asynchronous reset

Aspect	Asynchronous reset	Synchronous reset
Cell	DFF with reset/set pin (larger, slightly slower $\text{clk} \rightarrow Q$)	Plain DFF; reset is an AND gate in the D path
Needs a clock to reset?	No - resets immediately, even with the clock stopped or not yet running	Yes - reset must be held for at least one active clock edge
Timing	Reset path is timed with recovery/removal checks on deassertion	Reset is ordinary data; it adds a gate to the D path, which can hurt critical paths
Glitch sensitivity	A glitch on the reset net resets the flop - the net must be clean	Glitches are filtered by the clock edge
Reset fan-out	Large distribution tree, built like a clock tree	Also large, but treated as data by synthesis and timing
DFT	Reset must be controllable in scan mode (Chapter 20)	Transparent to scan
Typical use	Most ASIC control logic, power-on reset, anything that must be safe before clocks run	FPGA designs (flop reset pins are often synchronous), some high-speed ASIC datapaths

The deciding question in an ASIC is usually: **must this flop be in a known state before the clock is running?** Output enables of pads, power-switch controls and clock-gating controls must be, so they get asynchronous reset. The danger of an asynchronous reset is not its assertion but its **deassertion**: if reset releases too close to a clock edge, some flops leave reset in this cycle and some in the next, or a flop goes metastable. The universal solution is **asynchronous assert, synchronous deassert** - reset asserts immediately, but the release is passed through a two-flop synchroniser in each clock domain. Chapter 12 builds this reset synchroniser and the complete reset architecture of an SoC.

PITFALL - MIXING RESET AND NON-RESET FLOPS IN ONE BLOCK

If a single `always_ff` resets `ctrl` but not `data`, synthesis must keep `data` unchanged during reset - so it adds an enable mux on `data` driven by `rst_n`, turning the reset into a data-path signal. Put flops with reset and flops without reset in separate `always_ff` blocks.

6.3 Flops without reset

Not every flop needs a reset. A pipeline data register, a memory array, or a shift register whose contents are always qualified by a separately reset **valid** bit can start with any value. Removing the reset from these flops saves the reset pin (area), removes a load from the reset tree (power and routing), and removes a gate from the data path (timing). In a large accelerator, datapath flops outnumber control flops by ten to one, so this is a significant saving.

THE RULE

Reset the control, not the data. Every flop whose value can influence control flow or be observed before it is written (FSM state, valid bits, counters, configuration registers, anything that drives an output pin) must be reset. Pure data that is always qualified by a reset valid bit need not be. The price is X in simulation until the first write - which is a feature: X propagation reveals any place where unqualified data leaks into control.

6.4 Enables, and a first look at clock gating

`else if (en) q <= d;` describes a flop that holds its value when `en` is low. Synthesis implements this as a 2:1 recirculating mux in front of D or as a library enable-flop. Both still clock the flop every cycle and so still burn clock power. For a bank of flops sharing one enable, the power tool can instead insert an **integrated clock-gating cell** (ICG) that stops the clock altogether - often the single largest dynamic-power saving in a chip. Writing clean, shared enables is what makes this automatic insertion possible; Chapter 19 covers clock gating in depth.

6.5 Shift registers

```
module shreg #(parameter int W = 8, D = 4) (
    input logic      clk, en, sin,
    input logic [W-1:0] din,
    output logic [W-1:0] dout,
    output logic [D-1:0] sr
);
    logic [W-1:0] pipe [D];
    always_ff @(posedge clk) begin
        if (en) sr <= {sr[D-2:0], sin};    // serial-in, parallel-out
        pipe[0] <= din;                    // D-deep delay line (maps to SRL on FPGA)
        for (int i = 1; i < D; i++) pipe[i] <= pipe[i-1];
    end
    assign dout = pipe[D-1];
endmodule
```

Non-blocking assignments make the order of statements irrelevant: every stage samples its predecessor's **old** value, exactly as real flops do. With blocking = the loop would collapse into a single stage.

6.6 Counters

Counters are the most common sequential structure after plain registers: timers, address generators, FIFO pointers, baud-rate dividers and watchdogs are all counters. The encoding of the count is a design choice with real consequences.

Counter	States with N flops	Next-state logic	Property that matters
Binary	2^N	An incrementer (carry chain)	Dense; several bits toggle at once (e.g. 0111 -> 1000)
Gray	2^N	Binary increment + XOR	One bit changes per step - safe to sample in another clock domain (Chapter 11)

Counter	States with N flops	Next-state logic	Property that matters
Johnson (twisted ring)	2N	One inverter	Glitch-free decode with 2-input gates; no carry chain
Ring (one-hot)	N	None (a rotate)	Output is already decoded; fastest possible
LFSR	$2^N - 1$	A few XORs	Pseudo-random sequence; tiny and fast; used for BIST, scramblers, CRCs, test patterns

```

module up_down_counter #(parameter int W = 4) (
    input  logic      clk, rst_n, en, up,
    output logic [W-1:0] cnt
);
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) cnt <= '0;
        else if (en) cnt <= up ? cnt + 1'b1 : cnt - 1'b1;
endmodule

// Gray counter: keep a binary register, register its Gray code
module gray_counter #(parameter int W = 4) (
    input  logic      clk, rst_n, en,
    output logic [W-1:0] gray
);
    logic [W-1:0] bin, bin_nxt;
    assign bin_nxt = bin + 1'b1;
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) begin
            bin <= '0;
            gray <= '0;
        end else if (en) begin
            bin <= bin_nxt;
            gray <= bin_nxt ^ (bin_nxt >> 1); // registered: glitch-free output
        end
end
endmodule

```

```

module johnson_counter #(parameter int W = 4) (
    input  logic      clk, rst_n,
    output logic [W-1:0] q
);
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) q <= '0;
        else q <= {q[W-2:0], ~q[W-1]};
endmodule

// 8-bit maximal-length Fibonacci LFSR, taps x^8 + x^6 + x^5 + x^4 + 1
module lfsr8 (
    input  logic      clk, rst_n, en,
    output logic [7:0] q
);
    logic fb;
    assign fb = q[7] ^ q[5] ^ q[4] ^ q[3];
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) q <= 8'h01; // any non-zero seed; all-zero locks up
        else if (en) q <= {q[6:0], fb};
endmodule

```

PITFALL - THE GRAY OUTPUT MUST COME FROM A FLOP

Computing `gray = bin ^ (bin >> 1)` combinatorially from a binary register gives the right **value**, but when several `bin` bits toggle, the XOR outputs can glitch through intermediate codes before settling. A synchroniser in another domain can sample a glitch. The counter above registers the Gray code itself - this is exactly how the asynchronous FIFO pointers of Chapter 11 are built.

Ring and Johnson counters have the opposite weakness: most of their 2^N possible states are **illegal**. An SEU (a particle strike) or a glitch that puts a ring counter into 0110 leaves it cycling two hot bits forever. Production designs add self-correction - for example, force the state back to the reset value whenever the `$onehot` property is violated - the same 'safe FSM' idea as Chapter 7.

6.7 Edge detectors, pulse stretchers and debouncers

```
module edge_detect (
    input  logic clk, rst_n, sig,
    output logic rise, fall, any
);
    logic sig_d;
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) sig_d <= 1'b0;
        else      sig_d <= sig;
    assign rise = sig & ~sig_d;
    assign fall = ~sig & sig_d;
    assign any  = sig ^ sig_d;
endmodule

// Stretch a 1-cycle pulse to N cycles (retriggerable)
module pulse_stretch #(parameter int N = 4) (
    input  logic clk, rst_n, pin,
    output logic pout
);
    logic [$clog2(N+1)-1:0] cnt;
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) cnt <= '0;
        else if (pin) cnt <= N[$clog2(N+1)-1:0];
        else if (cnt != '0) cnt <= cnt - 1'b1;
    assign pout = (cnt != '0);
endmodule
```

The edge detector compares the signal with its value one clock ago. `sig` must already be synchronous to `clk` - an asynchronous input needs a synchroniser first (Chapter 11).

```
// Debouncer: output follows input only after it is stable for STABLE cycles.
// 'din' must already be synchronised to clk (Chapter 11).
module debounce #(parameter int STABLE = 4) (
    input  logic clk, rst_n, din,
    output logic dout
);
    logic [$clog2(STABLE)-1:0] cnt;
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) begin
            cnt <= '0;
            dout <= 1'b0;
        end else if (din == dout) begin
            cnt <= '0; // no change pending
        end else if (cnt == $clog2(STABLE)'(STABLE-1)) begin
            dout <= din; // stable long enough
            cnt <= '0;
        end else begin
            cnt <= cnt + 1'b1;
        end
```

```

    end
endmodule

```

A real button debouncer uses STABLE of about 10-20 ms worth of clock cycles, i.e. a counter of 18-22 bits at tens of MHz; the logic is identical.

The testbench below instantiates all of these blocks side by side and prints them cycle by cycle, measures the LFSR period, and feeds the debouncer a glitchy input followed by a clean one.

```

module tb_ch6;
    logic clk = 0, rst_n = 0, en = 1, up = 1;
    logic [3:0] cnt, gray, john;
    logic [7:0] lf;
    logic sig = 0, rise, fall, any, pout, db_in = 0, db_out;
    always #5 clk = ~clk;
    up_down_counter #(4) u_cnt (.clk, .rst_n, .en, .up, .cnt);
    gray_counter      #(4) u_gry (.clk, .rst_n, .en, .gray);
    johnson_counter   #(4) u_jon (.clk, .rst_n, .q(john));
    lfsr8             u_lf  (.clk, .rst_n, .en, .q(lf));
    edge_detect       u_ed  (.clk, .rst_n, .sig, .rise, .fall, .any);
    pulse_stretch     #(3) u_ps (.clk, .rst_n, .pin(rise), .pout);
    debounce          #(4) u_db (.clk, .rst_n, .din(db_in), .dout(db_out));

    int period = 0;
    logic [7:0] seed;
    initial begin
        repeat (2) @(posedge clk);
        rst_n <= 1;
        $display("cyc bin gray john lfsr | sig rise fall pout");
        for (int c = 0; c < 9; c++) begin
            @(posedge clk);
            sig <= (c == 2 || c == 3 || c == 4);
            #1 $display("%3d %b %b %b %h | %b %b %b %b",
                c, cnt, gray, john, lf, sig, rise, fall, pout);
        end
        // LFSR period
        seed = lf;
        do begin @(posedge clk); #1 period++; end while (lf != seed);
        $display("LFSR period = %0d", period);
        // debouncer: glitchy press then a clean one
        db_in <= 1; @(posedge clk); db_in <= 0; @(posedge clk);
        db_in <= 1; @(posedge clk); db_in <= 0; @(posedge clk);
        $display("after glitches : dout=%b", db_out);
        db_in <= 1; repeat (6) @(posedge clk);
        $display("after 6 stable 1s: dout=%b", db_out);
        $finish;
    end
endmodule

```

```

cyc bin gray john lfsr | sig rise fall pout
0 0001 0001 0001 02 | 0 0 0 0
1 0010 0011 0011 04 | 0 0 0 0
2 0011 0010 0111 08 | 1 1 0 0
3 0100 0110 1111 11 | 1 0 0 1
4 0101 0111 1110 23 | 1 0 0 1
5 0110 0101 1100 47 | 0 0 1 1
6 0111 0100 1000 8e | 0 0 0 0
7 1000 1100 0000 1c | 0 0 0 0
8 1001 1101 0001 38 | 0 0 0 0
LFSR period = 255
after glitches : dout=0
after 6 stable 1s: dout=1

```

Icarus Verilog output. Check the Gray column: exactly one bit changes per row. The Johnson counter walks 0001 -> 0011 -> 0111 -> 1111 -> 1110 -> ... with period $2N = 8$. The LFSR visits all 255 non-zero states. **rise** is a one-cycle pulse; **pout** stretches it to 3 cycles, starting one cycle later because the stretcher is registered.

WHY THE TESTBENCH WAITS #1 BEFORE PRINTING

At `@(posedge clk)` the non-blocking updates of that edge have not happened yet, so a `$display` in the same time step would print the **old** values - a race by design (Chapter 4). A small delay after the edge (or sampling at the opposite edge) makes the output unambiguous. An early version of this very testbench measured an 'LFSR period of 1' because the loop compared `lf` before it updated.

6.8 Register retiming

Retiming moves flops backward or forward across combinational logic without changing the cycle-level behaviour seen at the module's outputs. If a path has 30 levels of logic before a register and 5 after, moving the register back by 12 levels balances the two stages. Synthesis tools can retime automatically (Chapter 17), but only across logic they are allowed to touch: flops with asynchronous reset, flops with `dont_touch`, and flops that are observed by other modules or debug logic often cannot move. A practical RTL technique is to add one or two extra register stages at the end of a deep function and let the tool redistribute them.

```

before: [FF] --- 30 levels --- [FF] --- 5 levels --- [FF]    max stage = 30
after :  [FF] --- 18 levels --- [FF] --- 17 levels -- [FF]    max stage = 18
         same latency (2 cycles), same outputs, ~1.6x higher clock

```

Retiming changes where registers sit, never how many sit on each path.

6.9 Summary

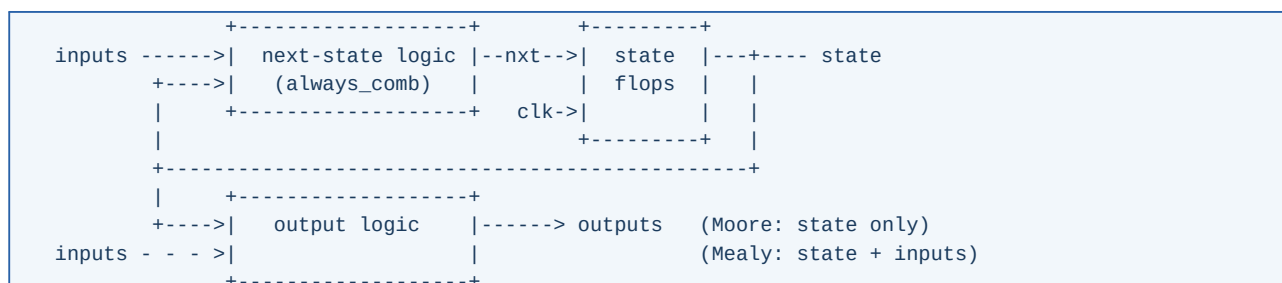
- Code flops with `always_ff`, non-blocking `<=`, one block per register group; separate reset and non-reset flops.
- Asynchronous reset uses a reset pin and works without a clock; its deassertion must be synchronised (Chapter 12). Synchronous reset is data-path logic that needs a clock edge.
- Reset the control, not the data: datapath flops qualified by a valid bit need no reset.
- Enables become recirculating muxes or clock gates (Chapter 19).
- Choose counter encodings deliberately: binary (dense), Gray (one bit changes, CDC-safe when registered), Johnson/ring (fast decode, need self-correction), LFSR (tiny pseudo-random).
- Edge detectors, pulse stretchers and debouncers are small, reusable blocks - but all require an already-synchronised input.

6.10 Exercises

- 1 Write a modulo-10 (BCD) counter with a carry output and chain two of them into a 00-99 counter. Verify every transition in a testbench.
- 2 Convert `lfsr8` to the Galois form (XORs inside the shift path) and show by simulation that it also has period 255. Which form has the shorter critical path, and why?
- 3 Add self-correction to `johnson_counter`: detect any of the 8 illegal 4-bit states and return to 0000. Inject an illegal state with `force` in the testbench to prove recovery.
- 4 Explain, with a gate-level sketch, what synthesis builds for an `always_ff` that resets `state` but not `data` in the same block.
- 5 A 32-bit datapath register loads every cycle when `valid` is high. Estimate the area saving of removing its reset in a library where a DFFR is 20% larger than a DFF, and list what must be true for the change to be safe.

7. Finite State Machines

A finite state machine (FSM) is the control brain of almost every block: bus interfaces, DMA engines, protocol controllers, power sequencers and arbiters are all FSMs steering a datapath. An FSM is a state register plus two pieces of combinational logic - one computing the **next state** from the current state and inputs, the other computing the **outputs**. The art lies in coding it so that it is readable, glitch-free where it matters, fast, and impossible to lock up.



The canonical FSM structure. Moore outputs depend on the state only; Mealy outputs also depend on the current inputs (dashed path).

7.1 Moore versus Mealy

	Moore	Mealy
Outputs depend on	Current state only	Current state and current inputs
Reaction to an input	One cycle later (after the state changes)	Same cycle - combinational path from input to output
Number of states	Often more	Often fewer
Glitches on outputs	Only from state decoding; none if outputs are registered or one-hot	Input glitches pass straight through
Timing across modules	Clean: output starts at a flop (or near one)	Creates input-to-output combinational paths that chain across modules and may form loops
Typical use	Most SoC control; anything leaving the block	Fast internal handshakes, where one cycle of latency matters

7.2 Coding styles: one, two or three always blocks

There are three common ways to code an FSM. All are legal and synthesise well; the differences are readability and where the outputs come from.

Style	Structure	Strengths	Weaknesses
Three-block	<code>always_ff</code> state register; <code>always_comb</code> next state; separate <code>always_comb/assign</code> outputs	Clearest mapping to hardware; easy to review; Moore/Mealy obvious	Outputs are combinational (decoded from state) unless registered separately
Two-block	<code>always_ff</code> for state (and registered outputs); one <code>always_comb</code> for next state (and any combinational outputs)	Compact; the most common industrial style	Mixing outputs into the next-state block can hide latches without defaults
One-block	Everything in one <code>always_ff</code>	Short; every output automatically registered	Outputs lag the state by one cycle unless you assign them on the transition; harder to read and review

The first example is a serial pattern detector for the bit sequence **1011**, allowing overlaps (so 1011011 contains two matches). It uses the three-block style and an `enum` state type.


```
// Moore detector for serial pattern 1011 (overlapping), 3-block style
module seq1011_moore (
    input  logic clk, rst_n, din,
    output logic hit
);
    typedef enum logic [2:0] {S0, S1, S10, S101, S1011} state_t;
    state_t state, nxt;

    // 1) state register
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) state <= S0;
        else      state <= nxt;

    // 2) next-state logic
    always_comb begin
        nxt = state;                // default: stay
        case (state)
            S0:   if (din) nxt = S1; else nxt = S0;
            S1:   if (din) nxt = S1; else nxt = S10;
            S10:  if (din) nxt = S101; else nxt = S0;
            S101: if (din) nxt = S1011; else nxt = S10;
            S1011: if (din) nxt = S1; else nxt = S10; // overlap: "1" or "10" already seen
            default: nxt = S0;                // illegal codes 5..7 recover
        endcase
    end

    // 3) output logic: Moore -> function of state only
    assign hit = (state == S1011);
endmodule
```

State names describe the history seen so far. The **default** item sends the three unused 3-bit codes back to S0.

State (history seen)	Next if din = 0	Next if din = 1	hit
S0 (nothing useful)	S0	S1	0
S1 ("1")	S10	S1	0
S10 ("10")	S0	S101	0
S101 ("101")	S10	S1011	0
S1011 (match)	S10	S1	1

State-transition table of the Moore 1011 detector. After a match, the final '1' can start a new pattern (-> S1), and a following '0' already gives '10' (-> S10): that is what 'overlapping' means.

The Mealy version needs one state fewer: instead of entering a 'found it' state, it raises **hit** during the cycle in which the final '1' is on the input.

```
// Mealy detector for the same pattern: one fewer state, one cycle earlier
module seq1011_mealy (
    input  logic clk, rst_n, din,
    output logic hit
);
    typedef enum logic [1:0] {S0, S1, S10, S101} state_t;
    state_t state, nxt;

    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) state <= S0;
        else      state <= nxt;

    always_comb begin
        nxt = state;
        hit = 1'b0;
    end
endmodule
```

```

case (state)
  S0:   if (din) nxt = S1; else nxt = S0;
  S1:   if (din) nxt = S1; else nxt = S10;
  S10:  if (din) nxt = S101; else nxt = S0;
  S101: begin
        if (din) nxt = S1; else nxt = S10;
        hit = din;           // Mealy: depends on the input
      end
  default: nxt = S0;
endcase
end
endmodule

```

```

module tb_seq;
  logic clk = 0, rst_n = 0, din = 0, hit_moore, hit_mealy;
  logic [15:0] stim = 16'b1011_0110_1101_1000; // sent MSB first
  logic [3:0] hist = '0;
  int exp_hits = 0, moore_hits = 0, mealy_hits = 0;
  always #5 clk = ~clk;
  seq1011_moore u_moore (.clk, .rst_n, .din, .hit(hit_moore));
  seq1011_mealy u_mealy (.clk, .rst_n, .din, .hit(hit_mealy));
  initial begin
    @(negedge clk) rst_n = 1;
    $display("bit din | mealy moore");
    for (int i = 15; i >= 0; i--) begin
      din = stim[i];
      hist = {hist[2:0], din};
      exp_hits += (hist == 4'b1011); // reference model
      #1 $display("%3d %b | %b %b", 15 - i, din, hit_mealy, hit_moore);
      mealy_hits += hit_mealy;
      @(negedge clk);
      moore_hits += hit_moore;
    end
    $display("expected %0d, Mealy saw %0d, Moore saw %0d",
      exp_hits, mealy_hits, moore_hits);
    $finish;
  end
endmodule

```

Inputs change on the falling edge, so the rising edge always samples stable data. A 4-bit history register is the reference model.

```

bit din | mealy moore
0 1 | 0 0
1 0 | 0 0
2 1 | 0 0
3 1 | 1 0
4 0 | 0 1
5 1 | 0 0
6 1 | 1 0
7 0 | 0 1
8 1 | 0 0
9 1 | 1 0
10 0 | 0 1
11 1 | 0 0
12 1 | 1 0
13 0 | 0 1
14 0 | 0 0
15 0 | 0 0
expected 4, Mealy saw 4, Moore saw 4

```

Icarus Verilog output. Four overlapping matches (bits 0-3, 3-6, 6-9, 9-12). The Mealy output fires in the same cycle as the last '1'; the Moore output fires one cycle later, after the state register has moved to S1011.

7.3 State encoding

An **enum** lets you write the FSM with names and leave the encoding to a later decision - either by giving explicit values or by letting the synthesis tool re-encode (FSM extraction). The choice matters:

Encoding	Flops for S states	Next-state / output logic	Best for
Binary	$\text{ceil}(\log_2 S)$	Denser decode; outputs need a comparator on several bits	Area-critical ASIC FSMs with many states
One-hot	S	Very simple: each state flop's input is an OR of its incoming transitions; outputs are ORs of state bits	Speed; FPGAs (flops are free); FSMs with many transitions
Gray	$\text{ceil}(\log_2 S)$	Like binary; one bit changes on each sequential transition	Linear FSMs, lower toggle power, state observed across a CDC
Johnson	S/2	Two-bit decode per state	Small sequencers
Output-encoded	Varies	Outputs are state bits directly - no decode at all	Glitch-free registered outputs at zero cost

```
// One-hot FSM coded explicitly: one flop per state, "reverse case" on 1'b1
module arb_onehot (
    input  logic clk, rst_n, req, gnt_ack, err,
    output logic busy
);
    localparam int IDLE = 0, WAIT = 1, XFER = 2, ERR = 3;
    logic [3:0] st, nx;
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) st <= 4'b0001;
        else      st <= nx;
    always_comb begin
        nx = '0;
        unique case (1'b1)
            st[IDLE]: if (req)      nx[WAIT] = 1'b1; else nx[IDLE] = 1'b1;
            st[WAIT]: if (gnt_ack) nx[XFER] = 1'b1; else nx[WAIT] = 1'b1;
            st[XFER]: if (err)      nx[ERR]  = 1'b1; else nx[IDLE] = 1'b1;
            st[ERR]:              nx[IDLE] = 1'b1;
            default:              nx[IDLE] = 1'b1;    // 0 or >1 hot bits: recover
        endcase
    end
    assign busy = st[WAIT] | st[XFER];           // one OR gate: fast outputs
endmodule
```

The 'reverse case' idiom compares the constant **1'b1** against each state bit. With **unique**, synthesis knows only one bit is hot and builds per-bit logic without priority.

WHAT SYNTHESIS REALLY DOES WITH YOUR ENUM

Tools such as Design Compiler and Genus recognise FSMs (a register whose next value is a function of itself through a **case**) and may re-encode them - to one-hot for speed, or Gray/binary for area - unless told not to. Re-encoding is safe for function but has two side effects: the state bits you probe in gate-level simulation no longer match your enum values, and the 'safe' recovery logic for illegal states can be optimised away because, from the tool's point of view, unreachable states do not exist. The next section shows how to keep it.

7.4 Safe FSMs and illegal-state recovery

With binary encoding and S states that are not a power of two, some codes are unused; with one-hot encoding, all codes with zero or more than one hot bit are illegal - 12 of the 16 codes in **arb_onehot**. In normal operation these states are unreachable. But a single-event upset from a particle strike, a supply glitch, or a reset released asynchronously without synchronisation can put the register into one. A **safe** FSM guarantees it

returns to a known state within a bounded number of cycles.

- **Default branch.** Always code `default: nxt = IDLE;` (and `nx = '0` plus a recovery branch for one-hot). This is necessary but not sufficient, because synthesis may remove logic for unreachable states.
- **Tell the tool.** Use the tool's safe-FSM option (for example a `fsm_encoding/safe_implementation` attribute or a `set_fsm_...` command, depending on the vendor) or disable FSM re-encoding for that register so your recovery logic survives.
- **Detect and report.** For safety-critical designs (ISO 26262 automotive, IEC 61508), add an explicit illegal-state detector (`!$onehot(st)`) that raises an error flag or interrupt, and consider parity or triple-modular redundancy (TMR) on the state register.
- **Verify it.** Use `force` in simulation or a formal check (Chapter 25) to prove every illegal code recovers.

7.5 Registered outputs

Outputs decoded combinationally from state bits can glitch when several state bits change at once - harmless inside a synchronous block, fatal if the output is a clock enable, an asynchronous reset, a pad or a signal crossing a clock domain. There are three standard cures: (1) one-hot encoding, where a Moore output that depends on a single state is just a flop output; (2) **register the outputs**, computing them from the **next** state so they are not delayed a cycle; (3) output encoding, where the state code is chosen so that output bits are state bits. The handshake controller below uses technique (2).

```
// 4-phase req/ack initiator with registered outputs (2-block style:
// one always_ff for state AND outputs, one always_comb for next state)
module hs4_master (
    input  logic clk, rst_n,
    input  logic start,      // request a transfer (sampled in IDLE)
    input  logic ack,        // from the responder (already synchronised)
    output logic req,         // registered -> glitch-free
    output logic done        // 1-cycle pulse, registered
);
typedef enum logic [1:0] {IDLE, REQ_HI, REQ_LO} state_t;
state_t state, nxt;

always_comb begin
    nxt = state;
    case (state)
        IDLE:    if (start) nxt = REQ_HI;
        REQ_HI:  if (ack)   nxt = REQ_LO; // phase 2 seen: drop req
        REQ_LO:  if (!ack)  nxt = IDLE;   // phase 4 seen: finished
        default:                nxt = IDLE;
    endcase
end

always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) begin
        state <= IDLE;
        req   <= 1'b0;
        done  <= 1'b0;
    end else begin
        state <= nxt;
        req   <= (nxt == REQ_HI);           // output decoded from NEXT state
        done  <= (state == REQ_LO) && !ack; // registered: aligned with IDLE
    end
endmodule
```

A four-phase handshake: req rises, ack rises, req falls, ack falls. Because `req` is computed from `nxt`, it rises on the same edge that the state enters `REQ_HI` - registered, yet with no extra latency.

```
// slow responder in the testbench: ack follows req after 2 clocks
always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) {ack, req_d} <= '0;
    else        {ack, req_d} <= {req_d, req};
always @(posedge clk) if (mon)
    $display("t=%3t start=%b req=%b ack=%b done=%b state=%s",
            $time, start, req, ack, done, u_hs.state.name());
```

Excerpt of the testbench (the full file also drives **start** and tests the GCD FSM below). **.name()** prints the enum label.

```
t= 25 start=0 req=0 ack=0 done=0 state=IDLE
t= 35 start=1 req=0 ack=0 done=0 state=IDLE
t= 45 start=0 req=1 ack=0 done=0 state=REQ_HI
t= 55 start=0 req=1 ack=0 done=0 state=REQ_HI
t= 65 start=0 req=1 ack=0 done=0 state=REQ_HI
t= 75 start=0 req=1 ack=1 done=0 state=REQ_HI
t= 85 start=0 req=0 ack=1 done=0 state=REQ_LO
t= 95 start=0 req=0 ack=1 done=0 state=REQ_LO
t=105 start=0 req=0 ack=1 done=0 state=REQ_LO
t=115 start=0 req=0 ack=0 done=0 state=REQ_LO
t=125 start=0 req=0 ack=0 done=1 state=IDLE
```

Icarus Verilog output, sampled at each rising edge. **req** and the state change on the same edge; **done** pulses as the FSM returns to IDLE.

7.6 FSM: a state machine with a datapath

Real blocks are rarely a bare FSM. An **FSMD** (FSM with datapath) combines a controller with registers and arithmetic that it sequences. The classic example is Euclid's GCD algorithm, which also introduces the valid/ready interface used throughout the rest of the book.

```
// FSM: Euclid's GCD by subtraction. Controller + datapath in one module.
module gcd_fsmd #(parameter int W = 16) (
    input  logic      clk, rst_n,
    input  logic      in_valid,
    output logic      in_ready,
    input  logic [W-1:0] a_in, b_in,
    output logic      out_valid,
    output logic [W-1:0] result
);
typedef enum logic [1:0] {IDLE, CALC, DONE} state_t;
state_t state;
logic [W-1:0] a, b;

assign in_ready  = (state == IDLE);
assign out_valid = (state == DONE);
assign result    = a;

always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) begin
        state <= IDLE;
        a <= '0;
        b <= '0;
    end else begin
        case (state)
            IDLE: if (in_valid) begin
                    a <= a_in; b <= b_in; state <= CALC;
                end
            CALC: if (b == '0) state <= DONE;
                  else if (a >= b) a <= a - b;
                  else begin a <= b; b <= a; end // swap
            DONE: state <= IDLE;
        endcase
    end
endmodule
```

```

        default: state <= IDLE;
    endcase
end
endmodule

```

One-block style suits an FSM: each state describes what the datapath does in that cycle. `in_ready` and `out_valid` are decoded from state only (Moore).

```

gcd(48,18) = 6   after 9 cycles
gcd(1071,462) = 21 after 16 cycles

```

Icarus Verilog output. The latency depends on the data - typical of iterative FSMs, and the reason they need a handshake rather than a fixed pipeline delay.

CONTROLLER + DATAPATH IS HOW YOU SHOULD SEE EVERY BLOCK

When you read a specification, separate **what is computed** (datapath: registers, adders, muxes) from **when it is computed** (control: an FSM that drives mux selects and register enables). Draw the datapath first, list its control signals, then write the FSM that drives them. Chapter 16's DMA engine and Chapter 28's UART are built exactly this way.

7.7 FSM pitfalls

Pitfall	Symptom	Fix
No default next state	Latch on <code>nxt</code> , or lock-up in an illegal state	<code>nxt = state;</code> at the top and a <code>default</code> item
Outputs assigned in only some states of a comb block	Latches on outputs	Default all outputs at the top of the block
Mealy outputs leaving the block	Long input-to-output paths, combinational loops between modules	Register outputs or use Moore outputs at module boundaries
Asynchronous input used directly in next-state logic	Different state bits see different values -> illegal state	Synchronise every asynchronous input first (Chapter 11)
Enum ternary like <code>nxt = din ? S1 : S0;</code>	Some tools reject it without a cast; readability suffers	Use <code>if/else</code> or a cast <code>state_t'(...)</code>
Relying on enum value order (e.g. <code>state > S2</code>)	Breaks when states are added or the tool re-encodes	Compare against named states only
Huge monolithic FSM	Unreviewable, slow next-state logic	Split into hierarchical or communicating FSMs, or use counters for repetitive sequences

7.8 Summary

- An FSM is a state register, next-state logic and output logic. Moore outputs depend on state only; Mealy outputs also on inputs and react a cycle earlier.
- Use an `enum` state type, `nxt = state` defaults, and a `default` item. Three-block style is clearest; two-block with registered outputs is the most common in industry.
- Binary encoding minimises flops; one-hot minimises logic depth; Gray minimises toggles; output encoding gives free registered outputs.
- Make FSMs safe: illegal states must recover, and the recovery logic must survive synthesis.
- Register outputs that leave the block by computing them from the next state.
- Think of every block as an FSM: a controller sequencing a datapath.

7.9 Exercises

- 1 Design a Moore detector for the pattern 1101 **without** overlap and verify it against a reference model with 10000 random bits.
- 2 Re-code `seq1011_moore` with explicit one-hot encoding. Count the flops and write the Boolean equation for each next-state bit.
- 3 Add an `error` output to `arb_onehot` that is high when the state register is not one-hot, and show in simulation (using `force`) that the FSM recovers from 4'b0110.
- 4 Modify `gcd_fsmd` so that it accepts a new input in the same cycle that it presents a result (i.e. no idle cycle between jobs). What must be true of the downstream consumer?
- 5 Explain why a Mealy output of one module feeding the Mealy input of another can create a combinational loop, and give a concrete example using valid/ready signals.

8. Datapath and Arithmetic

Datapaths are where the area and the critical paths of most chips live: adders, multipliers, shifters and the muxes that connect them. You will rarely hand-build an adder in production RTL - you write `+` and the synthesis tool picks an architecture from its library of arithmetic generators. But you must know what those architectures are, because they explain why `+` costs what it costs, why a 64-bit compare can fail timing, and why a single missing `signed` keyword can corrupt an entire ML accelerator's output.

8.1 Adders: from ripple-carry to parallel prefix

Every adder computes, for each bit, a **generate** $g = a \& b$ (this bit creates a carry) and a **propagate** $p = a \wedge b$ (this bit passes an incoming carry on). The sum bit is $p \oplus c$. The whole problem of fast addition is computing all the carries quickly.

```
module rca #(parameter int W = 8) (
    input  logic [W-1:0] a, b,
    input  logic          cin,
    output logic [W-1:0] s,
    output logic          cout
);
    logic [W:0] c;
    assign c[0] = cin;
    for (genvar i = 0; i < W; i++) begin : g_fa
        assign s[i]  = a[i] ^ b[i] ^ c[i];
        assign c[i+1] = (a[i] & b[i]) | (c[i] & (a[i] ^ b[i]));
    end
    assign cout = c[W];
endmodule
```

Ripple-carry adder: W full adders in a chain. Smallest possible adder, but the carry ripples through all W bits - $O(W)$ delay.

```
// 4-bit carry-lookahead block: every carry is a 2-level function of g, p, cin
module cla4 (
    input  logic [3:0] a, b,
    input  logic          cin,
    output logic [3:0] s,
    output logic          cout
);
    logic [3:0] g, p;
    logic [4:0] c;
    assign g = a & b;           // generate
    assign p = a ^ b;           // propagate
    assign c[0] = cin;
    assign c[1] = g[0] | (p[0] & cin);
    assign c[2] = g[1] | (p[1] & g[0]) | (p[1] & p[0] & cin);
    assign c[3] = g[2] | (p[2] & g[1]) | (p[2] & p[1] & g[0]) | (p[2] & p[1] & p[0] & cin);
    assign c[4] = g[3] | (p[3] & g[2]) | (p[3] & p[2] & g[1]) | (p[3] & p[2] & p[1] & g[0])
                | (p[3] & p[2] & p[1] & p[0] & cin);
    assign s  = p ^ c[3:0];
    assign cout = c[4];
endmodule
```

Carry-lookahead: carries are flattened into sum-of-products. The gates grow quickly with width, so real CLAs use 4-bit groups and a second level of lookahead between groups.


```
// Carry-select: compute the upper half twice, pick with the real carry
module csel16 (
    input  logic [15:0] a, b,
    output logic [15:0] s,
    output logic      cout
);
    logic      c8, c_hi0, c_hi1;
    logic [7:0] s_hi0, s_hi1;
    assign {c8, s[7:0]} = a[7:0] + b[7:0];
    assign {c_hi0, s_hi0} = a[15:8] + b[15:8]; // assumes carry-in 0
    assign {c_hi1, s_hi1} = a[15:8] + b[15:8] + 8'd1; // assumes carry-in 1
    assign {cout, s[15:8]} = c8 ? {c_hi1, s_hi1} : {c_hi0, s_hi0};
endmodule
```

Carry-select trades area (the upper half is duplicated) for delay (the upper half no longer waits for the lower carry). It is the same speculation idea as the late-select trick of Chapter 5.

```
rca8 + cla4 exhaustive, csel16 random: 0 errors
```

Icarus Verilog output: the ripple adder and CLA block were checked against + for all 2^{17} input combinations, the carry-select adder for 131072 random pairs.

Parallel-prefix adders (Kogge-Stone, Brent-Kung, Sklansky, Han-Carlson) generalise lookahead. They combine (g, p) pairs with an associative operator $(g1, p1) \circ (g0, p0) = (g1 \mid p1 \& g0, p1 \& p0)$ in a tree, giving all carries in $\log_2(W)$ levels. The variants differ in how many operator cells they use and how much wiring and fan-out they tolerate.

Architecture	Delay	Area	Where it is used
Ripple-carry	$O(W)$	Smallest	Slow paths, small widths, counters with relaxed timing
Carry-lookahead (grouped)	$O(\log W)$	Medium	Textbook; basis of prefix adders
Carry-select	$O(\sqrt{W})$ or better	~1.5-2x ripple	Mid-speed designs; conditional-sum variants
Brent-Kung prefix	$2 \log_2 W$	Small (~2W cells)	Low-power, area-constrained paths
Kogge-Stone prefix	$\log_2 W$	Large ($W \log W$ cells), heavy wiring	The fastest adders: CPU ALUs, address generation
Carry-save (3:2 compressors)	$O(1)$ per stage	One FA per bit	Inside multipliers and multi-operand sums: defers carries

WHAT SYNTHESIS DOES WITH THE + OPERATOR

The tool replaces $a + b$ with an arithmetic generator (DesignWare in Synopsys tools, similar libraries elsewhere) and then chooses an architecture per instance **based on timing**: a ripple adder where there is slack, a prefix adder where there is not. It also merges chains like $a + b + c + d$ into a carry-save tree with one final adder. So write arithmetic at the highest level you can - +, -, * - and let the tool optimise; hand-built adders defeat these optimisations. Hand-structure only to express something the tool cannot infer, such as sharing, pipelining or a known-late input.

8.2 Subtraction, overflow and saturation

In two's complement, $a - b = a + \sim b + 1$: an adder with inverted b and carry-in 1, so subtractors cost the same as adders and an add/subtract unit costs one XOR per bit extra. The carry-out of an unsigned subtraction is the inverted borrow, so $a < b$ (unsigned) is just $\sim \text{cout}$ of $a - b$.

unsigned overflow (add): carry out of the MSB = 1

signed overflow (add) : operands have the same sign AND the sum's sign differs
 $= c_in(MSB) \text{ XOR } c_out(MSB)$

result width : N-bit + N-bit needs N+1 bits; N x M needs N+M bits

Overflow rules. Growing the result by one bit and checking whether the top two bits differ is the easiest way to detect signed overflow in RTL.

When a result overflows, a datapath can **wrap** (modular arithmetic - what **+** does by default, correct for addresses, counters and checksums) or **saturate** (clamp to the most positive or negative value - required for audio, video, control loops and quantised neural networks, where a wrap from +max to -max is a catastrophic error).

8.3 Signed and unsigned: the rules that cause real bugs

Verilog's expression rules are the source of more silent datapath bugs than anything else. The two rules to memorise: (1) **if any operand is unsigned, the whole expression is evaluated unsigned**; (2) operands are extended to the width of the **context** (including the left-hand side) before the operation, and signedness decides whether that extension is sign- or zero-extension.

```
module tb_signed_pitfalls;
  logic      [7:0] u8 = 8'd200;
  logic signed [7:0] s8 = -8'sd3;
  logic signed [15:0] r1, r2, r3;
  logic      [3:0] a4 = 4'd9, b4 = 4'd8;
  logic      [3:0] sum4;
  logic      [4:0] sum5;
  initial begin
    // 1. One unsigned operand makes the whole expression unsigned
    r1 = s8 + u8;           // s8 zero-extended: 253 + 200
    r2 = s8 + $signed({1'b0, u8}); // correct: -3 + 200
    $display("s8 + u8      = %0d    (expected 197)", r1);
    $display("s8 + signed(u8) = %0d", r2);
    // 2. Comparisons follow the same rule
    $display("s8 < 8'd1      = %b    (-3 < 1 ?)", s8 < 8'd1);
    $display("s8 < 8'sd1     = %b", s8 < 8'sd1);
    // 3. Width of the result is set by the LHS context, not the operands
    sum4 = a4 + b4;
    sum5 = a4 + b4;
    $display("4-bit sum = %0d, 5-bit sum = %0d", sum4, sum5);
    // 4. Part-selects are always unsigned, even of a signed vector
    r3 = s8[7:0];
    $display("s8[7:0] into 16b = %0d", r3);
    // 5. >>> is only arithmetic on a signed operand
    $display("s8 >>> 1 = %0d, u8 >>> 1 = %0d", s8 >>> 1, u8 >>> 1);
  end
endmodule
```

```
s8 + u8      = 453    (expected 197)
s8 + signed(u8) = 197
s8 < 8'd1     = 0      (-3 < 1 ?)
s8 < 8'sd1    = 1
4-bit sum = 1, 5-bit sum = 17
s8[7:0] into 16b = 253
s8 >>> 1 = -2, u8 >>> 1 = 100
```

Icarus Verilog output - every line is standard-conforming behaviour, and every line has shipped in a real chip as a bug.

PITFALL - UNSIZED AND UNSIGNED LITERALS

`s8 < 1` is fine (the integer literal 1 is signed), but `s8 < 8'd1` is unsigned and `s8 < 1'b1` too. Likewise `x + 1'b1` is unsigned. In signed datapaths use `'sd` literals or `$signed()`, and when you convert unsigned to signed, prepend a 0 bit first (`$signed({1'b0, u})`), otherwise values with the MSB set become negative. Lint tools (Chapter 25) flag mixed-signedness expressions - enable that rule.

8.4 Multipliers

An $N \times M$ multiplier forms $N \times M$ partial-product bits (AND gates for unsigned) and sums them. The architectures differ in how the summing is done:

- **Array multiplier** - rows of adders, one per partial product. Regular and easy to lay out, but $O(N)$ adder delays.
- **Wallace / Dadda tree** - reduce the partial products with layers of carry-save adders (3:2 compressors, or 4:2) until two rows remain, then one fast final adder. $O(\log N)$ delay; the standard for high-speed multipliers.
- **Radix-4 (modified) Booth encoding** - recode the multiplier in digits $\{-2, -1, 0, +1, +2\}$, halving the number of partial products; handles signed operands naturally. Almost every production multiplier combines Booth encoding with a Wallace/Dadda tree.
- **Sequential (shift-and-add)** - one partial product per cycle: tiny, N cycles per result. Used when throughput is irrelevant.

<code>a[3:0] x b[3:0]</code> <code> a3b0 a2b0 a1b0 a0b0</code> <code> a3b1 a2b1 a1b1 a0b1</code> <code>a3b2 a2b2 a1b2 a0b2</code> <code>...</code> <code>sum the columns -> 8-bit product</code>	Booth radix-4 + Dadda tree (16 x 16): <code>16 PPs -> Booth -> 9 PPs</code> <code>9 -> 6 -> 4 -> 3 -> 2 rows (3:2 / 4:2 CSAs)</code> <code>2 rows -> one 32-bit prefix adder</code> <code>levels ~ log base 1.5 of #PPs, + 1 fast add</code>
--	--

Partial products and the reduction tree. Carry-save reduction is why a multiplier is only a few times slower than an adder despite being an order of magnitude larger.

MULTIPLIER SIZING IN PRACTICE

A 16×16 multiplier is roughly 15-25 times the area of a 16-bit adder. In ML accelerators the multipliers dominate: an INT8 MAC array of 1024 units is mostly partial-product logic. That is why quantisation from FP32 to INT8 or INT4 saves so much silicon - multiplier area scales roughly with the **product** of operand widths. Chapter 29 returns to this.

8.5 Shifters and the barrel shifter

A shift by a constant is free - it is just wiring. A shift by a variable amount is a mux network. The **barrel shifter** (strictly, a logarithmic shifter) uses $\log_2(W)$ stages; stage k shifts by 2^k if bit k of the amount is set.

```

module barrel_rotl #(parameter int W = 32) (
    input  logic [W-1:0]    din,
    input  logic [$clog2(W)-1:0] amt,
    output logic [W-1:0]    dout
);
    // log2(W) stages of 2:1 muxes: stage k rotates by 2^k if amt[k]
    logic [W-1:0] stage [0:$clog2(W)];
    assign stage[0] = din;
    for (genvar k = 0; k < $clog2(W); k++) begin : g_stage
        assign stage[k+1] = amt[k] ? {stage[k][W-1-(1<<k):0], stage[k][W-1 -: (1<<k)]}
                                   : stage[k];
    end
    assign dout = stage[$clog2(W)];
endmodule

```

A 32-bit rotator: 5 stages x 32 two-input muxes = 160 muxes. Writing `din << amt` gives the tool the same structure for a plain shift.

Arithmetic right shift (`>>>` on a signed operand) fills with the sign bit; logical shifts fill with zeros; rotates wrap around. A general-purpose ALU shifter supports all of them with a little extra fill logic around the same log-depth network.

8.6 Division

Division has no fast, cheap combinational implementation. A combinational `/` by a non-constant synthesises into W stages of subtract-and-select - large and very slow - so ASIC RTL either divides by a constant (which the tool turns into a multiply by the reciprocal and shifts), divides by a power of two (a shift), or uses an **iterative** divider that produces one (or, with radix-4 SRT, two) quotient bits per cycle.

```

// Unsigned restoring divider: one quotient bit per clock, W cycles
module div_iter #(parameter int W = 8) (
    input  logic      clk, rst_n,
    input  logic      start,
    input  logic [W-1:0] dividend, divisor,
    output logic      busy, done,
    output logic [W-1:0] quot, rem
);
    logic [W-1:0] d;          // divisor copy
    logic [W:0]    r;          // partial remainder, one extra bit
    logic [W-1:0] q;          // shifts in quotient, shifts out dividend
    logic [$clog2(W):0] n;
    logic [W:0]    r_sh, diff;

    assign r_sh = {r[W-1:0], q[W-1]}; // shift remainder left, bring down bit
    assign diff = r_sh - {1'b0, d};

    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) begin
            busy <= 1'b0; done <= 1'b0;
            r <= '0; q <= '0; d <= '0; n <= '0;
        end else begin
            done <= 1'b0;
            if (start && !busy) begin
                busy <= 1'b1;
                r <= '0; q <= dividend; d <= divisor;
                n <= ($clog2(W)+1)'(W);
            end else if (busy) begin
                if (diff[W]) begin // negative: restore
                    r <= r_sh; q <= {q[W-2:0], 1'b0};
                end else begin
                    r <= diff; q <= {q[W-2:0], 1'b1};
                end
            end
        end

```

```

        end
        n <= n - 1'b1;
        if (n == 1) begin busy <= 1'b0; done <= 1'b1; end
    end
end

assign quot = q;
assign rem  = r[W-1:0];
endmodule

```

The dividend shifts out of **q** as quotient bits shift in, so one register serves both. A single W+1-bit subtractor is reused W times.

```

200 / 7 = 28 rem 4 (9 cycles)
divider: 500 random divisions, 0 errors
barrel rotator: 1000 random vectors, 0 errors

```

Icarus Verilog output. The testbench compares against the simulator's **/** and **%** and a reference rotate. Division by zero returns quotient all-ones and remainder = dividend; a real design must define and document that case.

8.7 Fixed-point arithmetic and Q formats

Most signal-processing and ML inference hardware uses **fixed-point** numbers: integers with an implied binary point. The notation **Qm.n** means m integer bits (including the sign) and n fraction bits; the value is the integer divided by 2^n . Q1.15 (16-bit, range $[-1, 1 - 2^{-15}]$) is the classic DSP format; INT8 with a per-tensor scale is its modern ML cousin.

Operation	Rule	Example
Add / subtract	Align binary points first; result Q(m+1).n	Q1.15 + Q1.15 -> Q2.15 (17 bits)
Multiply	Integer bits add, fraction bits add	Q1.15 x Q1.15 -> Q2.30 (32 bits)
Back to Q1.15	Drop n fraction bits (with rounding), then saturate the extra integer bit	Q2.30 >> 15 -> Q2.15 -> sat -> Q1.15
Accumulate K products	Add ceil(log2 K) guard bits	1024 Q2.30 products need a 42-bit accumulator

Rounding mode	RTL	Bias	Cost
Truncate (floor)	drop the bits	-0.5 LSB on average (accumulates!)	Free
Round half up	add 2^{k-1} , then drop k bits	Tiny positive bias at exact halves	One adder
Round half to even (convergent)	as above, but ties go to the even value	Unbiased	Adder + tie detection
Round toward zero	truncate magnitude	Toward zero	Sign-dependent add

```
// Q1.15 x Q1.15 -> Q1.15 with round-half-up and saturation
module q15_mul (
    input  logic signed [15:0] a, b,
    output logic signed [15:0] y
);
    logic signed [31:0] p;          // Q2.30
    logic signed [32:0] pr;         // + rounding constant, one guard bit
    logic signed [17:0] t;          // Q2.30 >> 15 -> Q3.15 candidate
    assign p = a * b;
    assign pr = p + 33'sd16384;      // add half an LSB of the result (2^14)
    assign t = pr[32:15];           // drop 15 fraction bits
    always_comb begin
        if (t > 18'sd32767) y = 16'sh7FFF; // only -1 x -1 overflows
        else if (t < -18'sd32768) y = 16'sh8000;
        else
            y = 16'(t);
    end
endmodule
```

```
0.50000 x 0.50000 = 0.25000 (0x2000)
-0.50000 x 0.50000 = -0.25000 (0xe000)
-1.00000 x -1.00000 = 0.99997 (0x7fff)
0.00003 x 0.50000 = 0.00003 (0x0001)
0.99997 x 0.99997 = 0.99994 (0x7ffe)
```

Icarus Verilog output. The only Q1.15 product that overflows is $(-1) \times (-1) = +1$, which saturates to 0x7FFF. 1 LSB x 0.5 = 0.5 LSB rounds up to 1 LSB rather than truncating to zero.

PITFALL - TRUNCATION BIAS IN ACCUMULATORS

Truncation always rounds toward minus infinity, so the error of each operation averages -0.5 LSB. In a filter or a neural network layer that sums thousands of truncated products, the bias adds up to a visible DC offset or an accuracy drop. Keep full precision inside the accumulator and round **once**, at the output.

8.8 Resource sharing

If two operations are never needed in the same cycle, one operator can serve both behind muxes. Synthesis does this automatically within a single `always` block when the operations are in mutually exclusive branches; you can also do it explicitly.

```
// Unshared: two adders feeding one mux
assign y_unshared = sel ? a + b : c + d;
// Shared: two muxes feeding one adder
assign y_shared = (sel ? a : c) + (sel ? b : d);
```

Sharing saves an adder but adds muxes before it, lengthening the path through the select. Worth it for multipliers and dividers, rarely for small adders.

8.9 A verified saturating MAC

The multiply-accumulate (MAC) is the atom of DSP and of neural-network inference: $acc = acc + a \times b$, repeated over a dot product. The version below uses signed operands, a guard bit to detect overflow, saturation, and a sticky flag that software can read (Chapter 15) to learn that clipping occurred.

```
// Signed saturating multiply-accumulate: acc <= sat(acc + a*b)
module sat_mac #(
    parameter int DW = 8,          // operand width (signed)
    parameter int AW = 20         // accumulator width (signed)
) (
    input  logic          clk, rst_n,
    input  logic          clr,      // synchronous clear of the accumulator
    input  logic          en,       // accumulate this cycle
    input  logic signed [DW-1:0] a, b,
```

```

output logic signed [AW-1:0] acc,
output logic          sat      // sticky: saturation has occurred
);
localparam logic signed [AW-1:0] MAXV = {1'b0, {(AW-1){1'b1}}};
localparam logic signed [AW-1:0] MINV = {1'b1, {(AW-1){1'b0}}};

logic signed [2*DW-1:0] prod;
logic signed [AW:0]    sum;      // one guard bit catches overflow

assign prod = a * b;              // both operands signed -> signed multiply
assign sum  = acc + prod;         // sign-extended to AW+1 bits

always_ff @(posedge clk or negedge rst_n)
  if (!rst_n) begin
    acc <= '0;
    sat <= 1'b0;
  end else if (clr) begin
    acc <= '0;
    sat <= 1'b0;
  end else if (en) begin
    // overflow iff the two top bits of the (AW+1)-bit sum differ
    if (sum[AW] != sum[AW-1]) begin
      acc <= sum[AW] ? MINV : MAXV;
      sat <= 1'b1;
    end else begin
      acc <= sum[AW-1:0];
    end
  end
end
endmodule

```

One guard bit is enough because $|prod| < 2^{AW-1}$ whenever $AW \geq 2 DW$, so a single addition can overshoot the range by less than a factor of two.

```

module tb_sat_mac;
  localparam int DW = 8, AW = 16;
  logic clk = 0, rst_n = 0, clr = 0, en = 0, sat;
  logic signed [DW-1:0] a, b;
  logic signed [AW-1:0] acc;
  longint model;
  int errors = 0;
  always #5 clk = ~clk;
  sat_mac #(DW, AW) dut (.*);

  task automatic step(input logic signed [DW-1:0] x, y);
    @(negedge clk) begin a = x; b = y; en = 1; end
    model = model + x * y;
    if (model > 32767) model = 32767;
    if (model < -32768) model = -32768;
    @(negedge clk) en = 0;
    if (acc != model) errors++;
  endtask

  initial begin
    model = 0;
    repeat (2) @(negedge clk);
    rst_n = 1;
    step(100, 100); $display("acc=%6d sat=%b", acc, sat);
    step(127, 127); $display("acc=%6d sat=%b", acc, sat);
    step(127, 127); $display("acc=%6d sat=%b <- clipped at +max", acc, sat);
    step(-128, 127); $display("acc=%6d sat=%b <- recovers from the rail", acc, sat);
    @(negedge clk) clr = 1; @(negedge clk) clr = 0; model = 0;
    for (int i = 0; i < 2000; i++) step($urandom, $urandom);
  end
endmodule

```

```

    $display("2000 random MACs vs model: %0d errors, final acc=%0d sat=%b",
            errors, acc, sat);
    $finish;
end
endmodule

```

A 16-bit accumulator is deliberately small so that saturation happens often. The model saturates after every step, exactly like the hardware.

```

acc= 10000 sat=0
acc= 26129 sat=0
acc= 32767 sat=1 <- clipped at +max
acc= 16511 sat=1 <- recovers from the rail
2000 random MACs vs model: 0 errors, final acc=-23003 sat=1

```

Icarus Verilog output. $26129 + 16129 = 42258$ exceeds 32767 and clips; the next product (-16256) is applied to the clipped value, giving 16511. The random run exercises both rails.

INTERVIEW INSIGHT: SATURATE EVERY STEP OR ONLY AT THE END?

Saturating arithmetic is **not associative**: $(MAX + 1) - 1$ gives $MAX - 1$ with per-step saturation but MAX with a wide accumulator. Most ML accelerators therefore accumulate in a wide register (e.g. 32 bits for INT8 x INT8 over thousands of terms) that cannot overflow, and saturate only once when requantising the result. Per-step saturation, as above, matches DSP instruction semantics (e.g. saturating MAC instructions) where bit-exactness with a software model is required.

8.10 Summary

- Adders compute generate/propagate and then carries; ripple is small and slow, prefix adders are fast and large. Synthesis picks the architecture per instance from timing - write **+**.
- Signed overflow = top two bits of a one-bit-wider result differ. Choose wrap or saturate deliberately.
- One unsigned operand makes the whole expression unsigned; context width and signedness determine extension. Part-selects are unsigned.
- Multipliers = partial products + carry-save tree + final adder; Booth halves the partial products. Area scales with the product of widths.
- Variable shifts are log-depth mux networks; division is iterative.
- Track Q formats through every operation, round once, and size accumulators with guard bits.

8.11 Exercises

- 1 Write a parameterised 16-bit Kogge-Stone adder using a generate loop for the prefix tree and verify it against **+** with random vectors.
- 2 Extend `barrel_rotl` into a full shifter supporting logical left, logical right and arithmetic right shifts, with a 2-bit `op` input.
- 3 Modify `sat_mac` to use round-half-to-even when producing a Q8 output from the accumulator. Verify ties explicitly.
- 4 Make `div_iter` handle signed operands (divide magnitudes, then fix the signs, with the remainder taking the sign of the dividend). Test all four sign combinations.
- 5 An expression `y = a * b + c` has 8-bit unsigned `a`, `b`, a signed 16-bit `c` and a signed 17-bit `y`. What does Verilog actually compute? Rewrite it so that the result is correct and prove it in simulation.

9. Memories, Register Files and Synchronous FIFOs

On a modern SoC, embedded memory often occupies half of the die area. Caches, scratchpads, frame buffers, packet buffers, weight and activation buffers in ML accelerators - all are SRAM. Small, many-ported storage is built from flops as a register file. Between producers and consumers sit FIFOs. This chapter covers how each is described in RTL, what physical structure it becomes, and how to build a FIFO that is correct at the full and empty boundaries - the corner cases where most FIFO bugs live.

9.1 Flop-based register files

A register file is an array of registers with a small number of read and write ports. Built from flops, each **write port** costs a decoder plus an enable per word, and each **read port** costs a full N:1 multiplexer of the whole array. Read ports are therefore the expensive part: a 32 x 32 register file with two read ports contains two 32-bit 32:1 muxes - about $2 \times 31 \times 32 = 1984$ 2:1 mux equivalents.

```
// 2-read / 1-write flop-based register file, x0 hard-wired to zero (RISC-V style)
module regfile_2r1w #(parameter int W = 32, N = 32, localparam int AW = $clog2(N)) (
    input  logic          clk,
    input  logic          we,
    input  logic [AW-1:0] waddr, raddr1, raddr2,
    input  logic [W-1:0]  wdata,
    output logic [W-1:0]  rdata1, rdata2
);
    logic [W-1:0] mem [N];
    always_ff @(posedge clk)
        if (we && waddr != '0) mem[waddr] <= wdata;
    // asynchronous (combinational) reads: a big mux per port
    assign rdata1 = (raddr1 == '0) ? '0 : mem[raddr1];
    assign rdata2 = (raddr2 == '0) ? '0 : mem[raddr2];
endmodule
```

No reset on the array: software never reads a register it has not written (and x0 is constant). Resetting 1024 flops would cost area and reset-tree load for nothing.

```
waddr --[decoder]--> we[0..31]    (one load enable per register)
                        |
wdata -----+-----> [reg 0 ]--+
               +-----> [reg 1 ]--+== all 32 words ==+==> [32:1 mux] --> rdata1
               +-----> [ ...  ]--+                    |      ^ raddr1
               +-----> [reg 31]--+                    +==> [32:1 mux] --> rdata2
                                   ^ raddr2

cost ~ N x W flops + (write ports x N enables) + (read ports x W x (N-1) mux2)
```

Structure of a flop-based register file. Adding a read port adds a whole mux tree; adding a write port adds a mux in front of every flop.

Multi-port register files are needed by superscalar CPUs (e.g. 6 read, 3 write ports) and by some accelerators. Beyond two or three ports, flop-based implementations explode in area and wiring; designers then use custom multi-port SRAM bit-cells, **banking** (split registers into banks and arbitrate conflicts), or **replication** (two copies of a 1-read file give 2 read ports, at the price of writing both).

WRITE-BEFORE-READ BYPASS

If a pipeline writes register *r* and reads it in the same cycle, the combinational-read file above returns the **old** value. CPUs add a bypass mux: `rdata1 = (we && waddr == raddr1) ? wdata : mem[raddr1]`. Whether you need it is an architectural decision, not a coding one - write it down in the specification.

9.2 SRAM macros versus inferred memory

Above a few hundred bits, flops become far too large: a 6-transistor SRAM bit-cell is roughly 5-10 times smaller than a flop-based bit with its mux share. ASIC memories are therefore **hard macros** produced by a **memory compiler** from the foundry or an IP vendor. You give the compiler words, bits, number of ports, mux factor, and options (byte write, redundancy, power gating, BIST interface), and it generates the layout, a timing model (.lib), an abstract (LEF), and a Verilog behavioural model.

	FPGA	ASIC
How RAM is obtained	Inferred from RTL templates into block RAM (BRAM/URAM) or LUT RAM	Instantiated hard macros from a memory compiler; synthesis does not infer SRAM
Portability trick	Coding templates the tool recognises	A wrapper module: behavioural model for simulation and FPGA, macro instance for the ASIC, chosen by a define or generate
Read latency	Synchronous read (1 cycle), optional output register	Synchronous read; clock-to-Q of the macro is a large part of the cycle
Test	Not required	Memory BIST, repair with redundant rows or columns (Chapter 20)
Low power	Little control	Light sleep, deep sleep, shut-down pins; retention (Chapter 19)

ALWAYS WRAP YOUR MEMORIES

Never instantiate a vendor SRAM macro directly throughout your RTL. Create one wrapper per memory type (e.g. `sp_ram_1024x32`) with a clean, technology-neutral interface. Inside it, select the macro for the ASIC, an inferred template for the FPGA prototype (Chapter 26), and a behavioural array for simulation. The wrapper is also where BIST muxes, ECC, and power-control pins are hidden from the functional logic.

9.3 Single-port and dual-port RAMs, and read-during-write

Type	Ports	Typical use
Single-port (1RW)	One address; read OR write each cycle	Scratchpads, buffers that are filled then drained
Simple dual-port (1R1W)	One write port + one read port	FIFOs, line buffers - the most common in datapaths
True dual-port (2RW)	Two independent ports, each read or write	Shared memories, ping-pong buffers; larger bit-cell
Two-port register file (macro)	1R1W, small and fast	Small high-bandwidth buffers

When a read and a write hit the same address in the same cycle, the memory returns one of three things: the **old** data (read-first), the **new** data (write-first / write-through), or undefined data. Which one is a property of the macro and must match your RTL model exactly - otherwise RTL simulation and silicon disagree.

```
// Single-port RAM, synchronous read, read-first (old data on a write)
module spram_rf #(parameter int W = 32, D = 1024, localparam int AW = $clog2(D)) (
    input  logic      clk, en, we,
    input  logic [AW-1:0] addr,
    input  logic [W-1:0] wdata,
    output logic [W-1:0] rdata
);
    logic [W-1:0] mem [D];
    always_ff @(posedge clk)
        if (en) begin
            if (we) mem[addr] <= wdata;
            rdata <= mem[addr];           // reads the value BEFORE this write
        end
endmodule
```

Because both assignments are non-blocking, **rdata** samples **mem[addr]** before the write lands: read-first behaviour.

```
// Simple dual-port RAM with byte enables: one write port, one read port
module sdpram_be #(parameter int W = 32, D = 256, localparam int AW = $clog2(D)) (
    input  logic      clk,
    input  logic      we,
    input  logic [W/8-1:0] be,
    input  logic [AW-1:0] waddr, raddr,
    input  logic [W-1:0] wdata,
    input  logic      re,
    output logic [W-1:0] rdata
);
    logic [W-1:0] mem [D];
    always_ff @(posedge clk) begin
        if (we)
            for (int i = 0; i < W/8; i++)
                if (be[i]) mem[waddr][8*i +: 8] <= wdata[8*i +: 8];
        if (re) rdata <= mem[raddr];
    end
endmodule
```

Byte enables let a 32-bit bus perform 8- and 16-bit stores without a read-modify-write - required behind AXI **WSTRB** and AHB **HSIZE** (Chapter 13).

```
read during write of BBBB0002: rdata = aaaa0001 (old data)
next plain read                : rdata = bbbb0002
byte enables 0101 over 11223344 -> 11ff33ff
```

Icarus Verilog output: a second write to the same address returns the old word; byte enables 0101 overwrite only bytes 0 and 2.

9.4 Parity and ECC basics

SRAM bits can flip - from alpha particles, neutrons or marginal voltage. Large memories are protected with codes stored alongside the data:

Scheme	Extra bits (64-bit word)	Capability	Where
Parity	1 (or 1 per byte = 8)	Detects any odd number of flipped bits; no correction	Caches that can refetch clean data, instruction RAMs, buses
SECDED Hamming	8	Corrects 1-bit errors, detects 2-bit errors	Main SoC SRAMs, DRAM controllers, automotive
DECTED / BCH	~15	Corrects 2, detects 3	High-reliability and advanced-node memories

RTL impact: the encoder is an XOR tree on the write path, the decoder an XOR tree (syndrome) plus a correction mux on the read path - often a full pipeline stage. Byte writes into an ECC-protected word require a **read-modify-write**, because the check bits cover the whole word. Corrected errors are counted and reported through status registers and interrupts (Chapter 15); a background **scrubber** reads and rewrites memory

periodically so single errors are fixed before a second one accumulates in the same word.

9.5 The synchronous FIFO

A FIFO decouples a producer from a consumer that run on the same clock but not in lock-step. It is a circular buffer: a write pointer, a read pointer and a dual-port memory. The only hard part is telling **full** from **empty**, because in both cases the pointers are equal.

```
DEPTH = 8, pointers have 3 index bits + 1 lap bit

empty:  wptr = 0_101  rptr = 0_101  same index, same lap      -> empty
full :  wptr = 1_101  rptr = 0_101  same index, different lap -> full
count = wptr - rptr (mod 16) -> 0..8, no ambiguity

+---+---+---+---+---+---+---+---+
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | write at mem[wptr[2:0]]
+---+---+---+---+---+---+---+---+
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | read at mem[rptr[2:0]]
+---+---+---+---+---+---+---+---+
          ^rptr          ^wptr      count = 3
```

The extra-bit trick: one more pointer bit than the address needs distinguishes 'writer has lapped the reader' (full) from 'caught up' (empty).

Alternative schemes exist - a separate occupancy counter, or a registered **full/empty** flag pair updated from the next-state pointers - and are sometimes preferred for timing or for non-power-of-two depths. The extra-bit scheme is the most common and is also the basis of the **asynchronous** FIFO in Chapter 11, where the pointers are Gray-coded before crossing clock domains.

```
// Synchronous FIFO, standard (non-FWFT) read: rdata valid the cycle AFTER rd_en.
// DEPTH must be a power of two. Pointers carry one extra wrap bit.
module sync_fifo #(
    parameter int W = 8,
    parameter int DEPTH = 8,
    parameter int AF_LEVEL = DEPTH - 2,          // almost-full threshold
    localparam int AW = $clog2(DEPTH)
) (
    input  logic      clk, rst_n,
    input  logic      wr_en,
    input  logic [W-1:0] wdata,
    input  logic      rd_en,
    output logic [W-1:0] rdata,
    output logic      full, empty, almost_full,
    output logic [AW:0] count
);
    logic [W-1:0] mem [DEPTH];
    logic [AW:0]  wptr, rptr;          // AW+1 bits: MSB is the lap bit
    logic        do_wr, do_rd;

    assign do_wr = wr_en && !full;      // overflow-protected
    assign do_rd = rd_en && !empty;     // underflow-protected

    always_ff @(posedge clk)
        if (do_wr) mem[wptr[AW-1:0]] <= wdata; // storage: no reset needed

    always_ff @(posedge clk)
        if (do_rd) rdata <= mem[rptr[AW-1:0]];

    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) begin
            wptr <= '0;
            rptr <= '0;
        end else begin
            if (do_wr) wptr <= wptr + 1'b1;
            if (do_rd) rptr <= rptr + 1'b1;
```

```

end

// same index + same lap -> empty; same index + different lap -> full
assign count      = wptr - rptr;
assign empty      = (wptr == rptr);
assign full       = (wptr == {~rptr[AW], rptr[AW-1:0]});
assign almost_full = (count >= (AW+1)'(AF_LEVEL));
endmodule

```

Flags are combinational decodes of registered pointers - glitch-free enough for synchronous logic and only a comparator deep.

Note the design decisions: writes when full and reads when empty are **ignored** rather than corrupting the pointers (many designs instead flag them as errors or assert on them); the memory has no reset; and **almost_full** gives an upstream block with a pipeline of latency L enough warning to stop in time - set **AF_LEVEL = DEPTH - L**.

```

module tb_fifo;
  localparam int W = 8, DEPTH = 8;
  logic clk = 0, rst_n = 0, wr_en = 0, rd_en = 0;
  logic [W-1:0] wdata, rdata;
  logic full, empty, almost_full;
  logic [3:0] count;
  logic [W-1:0] q[$]; // reference model
  logic [W-1:0] exp_data;
  logic check = 0;
  int errors = 0, n_rd = 0;
  always #5 clk = ~clk;
  sync_fifo #(.W(W), .DEPTH(DEPTH)) dut (.*);

  // scoreboard: compare rdata one cycle after an accepted read
  always @(posedge clk) begin
    if (check) begin
      if (rdata !== exp_data) errors++;
      n_rd++;
    end
    check = 0;
    if (rd_en && !empty) begin exp_data = q.pop_front(); check = 1; end
    if (wr_en && !full) q.push_back(wdata);
  end

  initial begin
    repeat (2) @(negedge clk);
    rst_n = 1;
    // fill completely, then try one extra write
    for (int i = 0; i < DEPTH + 1; i++) begin
      wr_en = 1; wdata = 8'hA0 + i[7:0];
      @(negedge clk);
      $display("wr %h  count=%0d full=%b afull=%b empty=%b",
        wdata, count, full, almost_full, empty);
    end
    wr_en = 0;
    // one read from the full FIFO: data appears the cycle after rd_en
    rd_en = 1; @(negedge clk); rd_en = 0;
    $display("rd -> rdata=%h  count=%0d full=%b", rdata, count, full);
    // random traffic
    for (int i = 0; i < 5000; i++) begin
      wr_en = $urandom_range(0, 1); rd_en = $urandom_range(0, 1);
      wdata = $urandom;
      @(negedge clk);
      if (count !== q.size()) errors++;
    end
    wr_en = 0; rd_en = 0; @(negedge clk);
  end
endmodule

```

```

    $display("random: %0d reads checked, %0d errors", n_rd, errors);
    $finish;
end
endmodule

```

A queue (`q[$]`) is the reference model; the scoreboard checks every read's data and every cycle's `count`.

```

wr a0  count=1 full=0 afull=0 empty=0
wr a1  count=2 full=0 afull=0 empty=0
wr a2  count=3 full=0 afull=0 empty=0
wr a3  count=4 full=0 afull=0 empty=0
wr a4  count=5 full=0 afull=0 empty=0
wr a5  count=6 full=0 afull=1 empty=0
wr a6  count=7 full=0 afull=1 empty=0
wr a7  count=8 full=1 afull=1 empty=0
wr a8  count=8 full=1 afull=1 empty=0
rd -> rdata=a0  count=7 full=0
random: 2344 reads checked, 0 errors

```

Icarus Verilog output. The ninth write (a8) is dropped because the FIFO is full; the first read returns a0; 5000 cycles of random traffic produce no mismatches.

PITFALL - FIFO BUGS LIVE AT THE BOUNDARIES

Classic FIFO bugs: simultaneous read and write when **empty** (a standard FIFO cannot return the word being written this cycle) or when **full** (is the write allowed because a slot is freed? in this design, no - `full` blocks it); pointer width off by one so full and empty alias; non-power-of-two depth with the extra-bit scheme (the pointers must then wrap explicitly at DEPTH, and the lap bit toggles on wrap); and flags derived from `count` with a different latency from the data. A random testbench with a queue model, run long enough to hit full and empty many times, finds all of them.

9.6 Standard versus first-word-fall-through (FWFT)

In a **standard** FIFO, `rd_en` is a request and the data appears one cycle later. In a **first-word-fall-through** (show-ahead) FIFO, the head word is already on `rdata` whenever the FIFO is not empty, and a read (`pop`) acknowledges it. FWFT maps directly onto a valid/ready interface (`valid = !empty, pop = valid && ready`), which is why streaming designs prefer it (Chapter 10).

```

// FWFT (show-ahead) FIFO: the head word is visible on rdata whenever !empty.
// Uses an asynchronous-read flop array; a 'pop' consumes the current word.
module fwft_fifo #(parameter int W = 8, DEPTH = 4, localparam int AW = $clog2(DEPTH)) (
    input  logic          clk, rst_n,
    input  logic          push,
    input  logic [W-1:0] wdata,
    input  logic          pop,
    output logic [W-1:0] rdata,
    output logic          full, empty
);
    logic [W-1:0] mem [DEPTH];
    logic [AW:0]  wptr, rptr;
    always_ff @(posedge clk)
        if (push && !full) mem[wptr[AW-1:0]] <= wdata;
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) begin
            wptr <= '0; rptr <= '0;
        end else begin
            if (push && !full) wptr <= wptr + 1'b1;
            if (pop && !empty) rptr <= rptr + 1'b1;
        end
    assign rdata = mem[rptr[AW-1:0]]; // head of queue, no read latency
    assign empty = (wptr == rptr);
    assign full  = (wptr == {~rptr[AW], rptr[AW-1:0]});

```

```
endmodule
```

```
empty=0 head=11    (visible before any pop)
empty=0 head=22    (after one pop)
empty=1
```

Icarus Verilog output: two pushes, then the head is visible without a read request.

With a flop array, FWFT is free. With an SRAM macro (synchronous read), FWFT needs a small **prefetch** stage: one or two output registers that are refilled from the SRAM whenever they are empty - effectively a standard FIFO followed by a skid buffer, which is the subject of the next chapter.

9.7 Content-addressable memory (CAM), briefly

A CAM is searched by content rather than address: present a key, and every entry compares it in parallel, returning a hit and the matching index. CAMs implement TLBs, cache tag lookups in fully associative caches, network routing tables (TCAMs, with don't-care bits) and outstanding-transaction tables in bus interfaces. A small CAM in RTL is an array of registers, one equality comparator per entry, and a priority or one-hot encoder (Chapter 5) on the match vector. The parallel compare makes CAMs power-hungry and large; above a few dozen entries they are custom macros.

9.8 Summary

- Flop register files cost a mux tree per read port and an enable per word per write port; beyond a few ports use banking, replication or custom macros.
- ASIC SRAMs are compiled macros, not inferred - always hide them behind a technology-neutral wrapper.
- Know and model the read-during-write behaviour of each memory; use byte enables for sub-word writes.
- Parity detects, SECDED corrects single-bit errors; ECC forces read-modify-write for partial writes.
- A synchronous FIFO uses pointers with one extra lap bit: equal = empty, equal except the MSB = full, difference = count.
- Standard FIFOs return data a cycle after `rd_en`; FWFT FIFOs present the head word and fit valid/ready directly.

9.9 Exercises

- 1 Add a write-to-read bypass to `regfile_2r1w` and write a test that reads and writes the same register in one cycle.
- 2 Rewrite `sync_fifo` for `DEPTH = 6` (not a power of two). Keep the lap-bit idea but wrap the index explicitly. Rerun the random testbench.
- 3 Add registered `full` and `empty` flags computed from the next-state pointers, and prove with the testbench that they match the combinational versions every cycle.
- 4 Design an 8-entry CAM with 16-bit keys, a write port and a search port returning `hit` and a 3-bit index (lowest matching entry wins).
- 5 A memory compiler offers a 1024 x 64 macro with read-first or write-first behaviour. Write the behavioural model for write-first and a testbench that distinguishes the two.

10. Pipelining, Valid/Ready Handshakes and Flow Control

A modern SoC is a network of blocks streaming data to each other: a camera interface feeding an ISP, a DMA feeding an accelerator, an accelerator feeding a memory controller. Each block is pipelined to reach its clock frequency, and each link between blocks must cope with the receiver sometimes being unable to accept data. This chapter develops the two ideas that make this work - **pipelining** and **flow control** - and the precise rules of the valid/ready handshake that AXI-Stream, AXI and nearly every internal interface are built on.

10.1 Latency versus throughput

Latency is how many cycles a single item takes to pass through a block. **Throughput** is how many items the block completes per cycle (or per second). Pipelining cuts a long combinational path into S stages separated by registers: each stage is shorter, so the clock can be faster, and S items are in flight at once.

$$f_{\text{max}} \sim 1 / (t_{\text{logic}} / S + t_{\text{clk} \rightarrow q} + t_{\text{setup}} + t_{\text{skew}})$$

$$\text{latency} = S \text{ cycles} = S / f_{\text{max}} \text{ seconds (slightly MORE time than unpipelined)}$$

$$\text{throughput} = 1 \text{ item per cycle when full (S x more items per second, ideally)}$$

$$\text{time for N items} = (S - 1 + N) \text{ cycles}$$

Pipelining buys throughput with latency and flops. The fixed per-stage overhead (flop delay, setup, clock skew) is why doubling the stages never quite doubles the frequency.

THE LAUNDRY ANALOGY IS EXACTLY RIGHT

Washer, dryer, folding: one load takes three hours regardless. But if you start the next load in the washer as soon as the first moves to the dryer, you finish a load every hour. The slowest stage sets the rate - which is why balancing pipeline stages (Chapter 6's retiming) matters.

10.2 Pipeline registers and valid bits

The simplest pipeline has no backpressure: data enters every cycle it is valid, and a **valid bit** travels alongside the data through each stage. Only the valid bits need a reset; the data registers are enabled by valid, which prevents useless toggling and lets the power tool gate their clocks (Chapter 19).

```
// y = a*b + c, 3-stage pipeline, valid travels with the data (no backpressure)
module mac_pipe (
    input  logic          clk, rst_n,
    input  logic          in_valid,
    input  logic signed [15:0] a, b,
    input  logic signed [31:0] c,
    output logic          out_valid,
    output logic signed [31:0] y
);
    logic          v1, v2;
    logic signed [15:0] a1, b1;
    logic signed [31:0] c1, c2, p2;
    // valid pipeline: the only part that needs a reset
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) {v1, v2, out_valid} <= '0;
        else      {v1, v2, out_valid} <= {in_valid, v1, v2};
    // data pipeline: no reset, enabled by valid to save power
    always_ff @(posedge clk) begin
        if (in_valid) begin a1 <= a;      b1 <= b;  c1 <= c;  end // S1: register inputs
        if (v1)      begin p2 <= a1 * b1; c2 <= c1;  end // S2: multiply
        if (v2)      y <= p2 + c2;                // S3: add
    end
end
```



```
endmodule
```

Note that **c** must be delayed through the pipeline too (**c1**, **c2**) so that it meets the product of the **same** item - forgetting to delay a side input is the most common pipelining bug.

```
issue i=1 at t=30
issue i=2 at t=40
issue i=3 at t=50
  result at t=60 y=-99
  result at t=70 y=-198
  result at t=80 y=-297
```

Icarus Verilog output (a = i, b = -100, c = i). Three items issued on consecutive cycles emerge three cycles later on consecutive cycles: latency 3, throughput 1 per cycle.

10.3 Hazards in datapath pipelines

A pipeline is correct only if every item sees the right operands. Three kinds of hazard break this:

Hazard	Cause	Datapath example	Remedies
Data (RAW)	An item needs a result that a later stage has not produced yet	Accumulator feedback: $acc + x$ needs last cycle's acc , but the adder is 2 stages deep	Forwarding/bypass muxes; stall; interleave independent streams (e.g. 2 accumulators)
Structural	Two items need one resource in the same cycle	Single-port SRAM read and write in the same cycle	Duplicate or bank the resource; arbitrate and stall
Control	The next item depends on a decision not yet made	A branch; a packet whose length is in its header	Speculate and flush; stall until resolved

THE FEEDBACK-LOOP LIMIT

Pipelining cannot speed up a loop. If a result must feed back into the next computation (an accumulator, an IIR filter, a CRC, an arbiter's pointer), the entire loop must complete in one cycle, no matter how many registers you add elsewhere. Classic fixes are algebraic: carry-save accumulation (keep the accumulator in redundant form), look-ahead transformation of IIR filters, or processing K interleaved independent streams so that each loop has K cycles. Recognising the loop is the first step - in design reviews, ask 'where are the feedback paths?'

10.4 The valid/ready handshake

When a receiver can stall, the link needs **backpressure**. The valid/ready protocol (AXI's **VALID/READY**, AXI-Stream's **TVALID/TREADY**) uses two signals: the source drives **valid** together with the data; the sink drives **ready**. A **transfer** happens on every rising clock edge where both are high.

clock cycle	1	2	3	4	5	6	7
valid	0	1	1	1	0	1	1
data	-	D0	D0	D1	-	D2	D2
ready	1	0	1	1	1	0	1
transfer		stall	D0	D1		stall	D2

cycle 2: valid without ready -> source holds D0 (rule 2)

cycle 5: ready without valid -> allowed, nothing happens (rule 4)

A transfer happens at the end of every cycle in which valid AND ready are both high. While stalled, the source holds valid high and the data stable.

Rule	Why
1. A transfer occurs when valid & ready at a clock edge - and only then	Both sides count transfers identically; no separate acknowledge is needed
2. Once valid is asserted, the source must keep it asserted, and keep the data stable, until the transfer happens	The sink may sample the data at any edge; a source that 'changes its mind' loses or corrupts data

Rule	Why
3. valid must not depend combinationally on ready	Prevents deadlock (a source waiting for ready before raising valid, facing a sink that waits for valid) and combinational loops
4. ready may depend combinationally on valid , and may be asserted before valid	Allowed by AXI, but every such dependency lengthens the path and risks loops; registered ready is better
5. After reset, valid must be low	Ensures no phantom transfer during reset release

The valid/ready rules (as in the AMBA AXI and AXI-Stream specifications). Chapter 13 applies them to all five AXI channels.

PITFALL - COMBINATIONAL LOOPS THROUGH VALID AND READY

If block A computes `valid_out = f(ready_in)` and block B computes `ready_out = g(valid_in)`, connecting them closes a loop through two modules that each looked fine alone. Rule 3 forbids the first half. More subtly, long chains where every stage passes **ready** combinationaly upstream create a timing path from the last sink to the first source across the whole pipeline - legal, but often the critical path of the design. The skid buffer below cuts it.

10.5 A pipeline register with ready propagation

To add a register stage to a valid/ready link, the stage must stall when downstream stalls. The simplest correct version accepts a new word when it is empty **or** when its current word is leaving this cycle:

```
// Forward-registered pipeline stage: data/valid registered, ready is combinational
// (s_ready = !m_valid || m_ready). Full throughput, but a ready path runs through.
module pipe_reg #(parameter int W = 8) (
    input  logic      clk, rst_n,
    input  logic      s_valid,
    output logic      s_ready,
    input  logic [W-1:0] s_data,
    output logic      m_valid,
    input  logic      m_ready,
    output logic [W-1:0] m_data
);
    assign s_ready = !m_valid || m_ready; // empty, or emptying this cycle
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) m_valid <= 1'b0;
        else if (s_ready) m_valid <= s_valid;
    always_ff @(posedge clk)
        if (s_ready && s_valid) m_data <= s_data; // datapath: no reset
endmodule
```

s_ = slave (sink) side, **m_** = master (source) side, as in AXI naming. The stage sustains one word per cycle, but **m_ready** passes straight through to **s_ready**.

Two tempting simplifications are wrong. `s_ready = !m_valid` (accept only when empty) is correct but halves throughput: the stage alternates between full and empty. `s_ready = m_ready` (just forward ready) loses data when the stage holds a word and the source sends another while downstream is stalled - unless the stage is always emptied, which it is not.

10.6 The skid buffer

To cut the ready path as well, **s_ready** must come from a flop. But a registered ready is one cycle late: when downstream stalls, upstream has already been told 'ready' for this cycle and sends one more word. The **skid buffer** has a second register to catch exactly that word - the data 'skids' into it like a car braking. It is the standard building block for timing-closing long valid/ready paths, and appears as the 'register slice' in commercial AXI interconnects.

```
// Skid buffer: full throughput AND s_ready comes straight from a flop, so the
```

```

// ready path is cut. When downstream stalls, the word already accepted
// (because s_ready was promised a cycle earlier) lands in the skid register.
module skid_buffer #(parameter int W = 8) (
    input  logic      clk, rst_n,
    input  logic      s_valid,
    output logic      s_ready,
    input  logic [W-1:0] s_data,
    output logic      m_valid,
    input  logic      m_ready,
    output logic [W-1:0] m_data
);
    logic [W-1:0] skid_data;
    logic      skid_valid;                // skid register occupied

    assign s_ready = !skid_valid;        // a flop output (plus inverter)

    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) begin
            m_valid    <= 1'b0;
            skid_valid <= 1'b0;
        end else if (m_ready || !m_valid) begin // output register free or draining
            m_valid    <= skid_valid || s_valid; // skid first, else the input
            skid_valid <= 1'b0;
        end else if (s_valid && s_ready) begin // output stalled: catch the word
            skid_valid <= 1'b1;
        end

    always_ff @(posedge clk) begin // datapath: no reset
        if (m_ready || !m_valid)
            m_data <= skid_valid ? skid_data : s_data;
        if (s_valid && s_ready && m_valid && !m_ready)
            skid_data <= s_data;
    end
endmodule

```

Both outputs are registered: **m_valid/m_data** directly, **s_ready** as the inverse of the **skid_valid** flop. No combinational path exists from **m_ready** to **s_ready**.

Output reg	Skid reg	s_ready	What happens at the next edge
empty	empty	1	Input word (if valid) goes to the output register
full, draining	empty	1	Output takes the new input word; full throughput
full, stalled	empty	1	Input word is caught in the skid register; s_ready drops
full, stalled	full	0	Nothing moves; upstream sees ready low and holds
full, draining	full	0	Skid word moves to the output; s_ready returns next cycle

The skid buffer's four reachable situations. The skid register can only fill when the output is stalled, and it always empties first when the output drains, so ordering is preserved.

The testbench chains a skid buffer and a pipeline register between a random source and a random sink. The source obeys rule 2 (it holds valid and data until accepted), a checker enforces the same rule on the skid buffer's output, and the sink verifies that the sequence 0, 1, 2, ... arrives in order with nothing lost or duplicated.

```

// AXI-style source: once valid is raised, hold it and the data until accepted
logic [W-1:0] nxt_word;
assign nxt_word = next_in + (s_valid && s_ready);
always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) begin
        s_valid <= 0; s_data <= 0;
    end else if (!s_valid || s_ready) begin
        s_valid <= ($urandom_range(1, 100) <= p_valid) && nxt_word < limit;
        s_data  <= nxt_word;
    end

```

```

    end
    always_ff @(posedge clk) next_in <= rst_n ? nxt_word : '0;
    // protocol check on the skid output: a stalled word must hold valid and data
    logic v1_q = 0, r1_q; logic [W-1:0] d1_q;
    always_ff @(posedge clk) begin
        if (v1_q && !r1_q && (!v1 || d1 !== d1_q)) errors++;
        v1_q <= v1; r1_q <= r1; d1_q <= d1;
    end
    // sink: random ready; check in-order, no loss, no duplication
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) m_ready <= 0;
        else m_ready <= ($urandom_range(1, 100) <= p_ready);
    always_ff @(posedge clk) if (rst_n) begin
        cycles <= cycles + 1;
        if (m_valid && m_ready) begin
            if (m_data !== next_out) errors++;
            next_out <= next_out + 1'b1;
        end
    end
end

```

The core of the testbench (source -> skid_buffer -> pipe_reg -> sink). A `run(p_valid, p_ready)` task sets the probabilities, sends 3000 words and reports the cycles taken.

```

valid 100% ready 100% : 3000 words in 3003 cycles, errors=0
valid 100% ready 50%  : 3000 words in 5978 cycles, errors=0
valid 50%  ready 100% : 3000 words in 5895 cycles, errors=0
valid 70% ready 70%  : 3000 words in 4958 cycles, errors=0

```

Icarus Verilog output. With no stalls the chain sustains one word per cycle (3000 words + 3 cycles of fill latency). With random stalls the throughput is set by the less willing side, and no word is ever lost, duplicated or reordered.

HOW WE KNOW THE TESTBENCH CAN FAIL

A testbench that always passes proves nothing. Mutating the skid buffer so that it never captures the skid word (`skid_valid <= 1'b0` in the stall branch) still passes the stall-free run - but in the first run with stalls the in-order check reported 1497 errors, and because words were lost the run never reached its word count (a watchdog timeout ended it). Always try a mutation on every checker you write, and always give testbenches a timeout (Chapter 22).

10.7 Credit-based flow control

Valid/ready needs a round trip within one cycle: the sink's `ready` must reach the source before the next edge. Across long wires, chip partitions, clock-domain crossings or network-on-chip links (Chapter 14) that is impossible. **Credit-based** flow control replaces ready with a counter at the sender: it starts with as many credits as the receiver has buffer slots, spends one per word sent, and receives one back each time the receiver frees a slot. The sender can never overflow the receiver, however long the link latency.

```

// Credit-based sender: may only send while it holds credits.
// The receiver returns one credit per buffer slot it frees.
module credit_tx #(parameter int CREDITS = 4, localparam int CW = $clog2(CREDITS+1)) (
    input  logic clk, rst_n,
    input  logic want_send,      // producer has a word
    input  logic credit_ret,     // pulse: receiver freed one slot
    output logic send,           // word goes on the link this cycle
    output logic [CW-1:0] credits
);
    assign send = want_send && (credits != '0);
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) credits <= CW'(CREDITS); // = receiver buffer depth
        else credits <= credits - CW'(send) + CW'(credit_ret);
endmodule

```

```
sent 4 words with 4 credits and no returns (credits=0)
one credit returned -> send=1
```

Icarus Verilog output: the sender stops by itself after four words and resumes as soon as one credit comes back.

credits needed for full throughput $\geq$ round-trip latency in cycles

RTT = forward link latency + receiver processing + credit return latency

with fewer credits: throughput = credits / RTT words per cycle

Credit sizing. The receiver buffer must be as deep as the credit count, so long links cost buffering - the same bandwidth-delay product that governs network protocols.

10.8 Elastic buffers and bubble collapsing

A pipeline built from `pipe_reg` or skid stages is **elastic**: each stage holds a word or a bubble (an empty slot), and stalls propagate backward only as far as necessary. Two properties follow:

- **Bubble collapsing.** In `pipe_reg`, a stage accepts new data when it is empty even if downstream is stalled (`s_ready = !m_valid || m_ready`). Bubbles are squeezed out while the output is blocked, so the pipeline fills up completely and can restart at full rate. A naive pipeline that stalls **every** stage on a global stall signal keeps its bubbles forever and wastes throughput.
- **Buffering absorbs burstiness.** An elastic pipeline of S stages holds up to S words ($2S$ with skid buffers). When a producer is bursty and a consumer is steady (or vice versa), the required depth is set by the longest burst minus what the consumer drains during it - if that exceeds a few stages, insert a FIFO (Chapter 9).
- **Global stall versus local handshakes.** A single global enable (`if (!stall) all_regs <= next`) is small but creates a high-fanout, timing-critical stall net. Local valid/ready handshakes cost more logic but scale; large designs mix both: global enables inside a tightly timed core, handshakes between blocks.

10.9 Throughput math for handshaked pipelines

Situation	Throughput (words / cycle)
No stalls, fully pipelined	1
Stage accepts only when empty (<code>s_ready = !m_valid</code>)	1/2
Source valid with probability p_v , sink ready with p_r , independent, deep buffering	about $\min(p_v, p_r)$
Shallow buffering between random source and sink	below $\min(p_v, p_r)$ - both must be willing in the same cycle more often
Iterative block needing k cycles per item (e.g. <code>div_iter</code>)	$1/k$ - replicate k copies to get back to 1
Credit link with C credits and round trip R cycles	$\min(1, C/R)$

The simulation above agrees: with valid at 100% and ready at 50%, 3000 words took 5978 cycles, i.e. 0.50 words per cycle; with both at 70%, 0.61 words per cycle - a little below 0.70 because the chain holds only three words, so it cannot fully absorb runs where the two sides are unwilling at different times.

Valid/ready design review checklist

- ☐ `valid` never depends combinational on `ready` in any module.
- ☐ Once asserted, `valid` and data are held until the transfer.
- ☐ Every register stage either sustains full throughput or documents why not.
- ☐ Long `ready` chains are cut with skid buffers or register slices at block boundaries.
- ☐ Data and sideband signals (last, keep, user, id) are pipelined with the same enables as the data.
- ☐ `valid` flops are reset; data flops normally are not.
- ☐ The testbench includes random stalls on both sides and a protocol checker (or SVA properties - Chapter 23).

10.10 Summary

- Pipelining trades latency and flops for throughput; the slowest stage and the per-stage overhead limit the gain; feedback loops cannot be pipelined away.
- Valid/ready transfers when both are high at a clock edge; valid must be held until accepted and must not depend on ready.
- `pipe_reg` gives full throughput with a combinational ready path; the skid buffer gives full throughput with every output registered.
- Credit-based flow control handles long-latency links; credits needed equal the round-trip latency.
- Elastic pipelines collapse bubbles; FIFOs absorb longer bursts.
- Always verify handshake logic with random stalls on both sides and an in-order, no-loss scoreboard.

10.11 Exercises

- 1 Prove, by listing cases, that `s_ready = m_ready` (without the `!m_valid` term) in `pipe_reg` can lose data. Then demonstrate the loss with the testbench.
- 2 Build a 4-stage pipeline of skid buffers and measure throughput with valid = 100% and ready = 70%. Compare with four `pipe_reg` stages.
- 3 Convert `mac_pipe` to a valid/ready interface with backpressure, using a single stall-enable for the three stages. Why must `in_ready` then depend on `out_ready`?
- 4 Design the receiver side of the credit link: a 4-entry FIFO that pulses `credit_ret` when a word is popped. Connect it to `credit_tx` through a 3-cycle delay line in each direction and find the minimum credits for full throughput.
- 5 A source produces bursts of 64 words at one per cycle every 256 cycles; the sink accepts one word every 3 cycles. What FIFO depth prevents the source from ever stalling?

PART III

SoC Architecture in RTL

Part II built blocks that live inside one clock domain. A real system-on-chip has dozens of clocks, several reset domains, a hierarchy of buses and an interconnect joining processors, accelerators, memories and peripherals. This part is about the glue: crossing clock and reset domains safely, speaking the AMBA protocols (APB, AHB, AXI), building arbiters, memory maps and networks-on-chip, specifying registers and interrupts, and integrating peripherals, DMA and a RISC-V core into a working SoC.

11. Clocks and Clock-Domain Crossing

Every flop in Part II shared one clock, so every path had a full, known period to settle and static timing analysis (Chapter 18) could prove it correct. Put two clocks on a chip whose edges bear no fixed relationship and that guarantee disappears for every signal that passes between them. Sooner or later such a signal changes inside a receiving flop's setup/hold window, and the flop goes **metastable**. Clock-domain crossing (CDC) design is the discipline of making that event harmless. CDC bugs are the classic silicon escape: they pass every RTL simulation, show up as a one-in-a-week field failure, and cost a respin. That is why CDC sign-off (Chapter 25) is mandatory on every SoC tapeout.

11.1 Where clocks come from

A modern SoC typically has one or two crystal references (for example 24 MHz or 38.4 MHz) and derives everything else on chip:

Source	What it does	RTL designer's concern
PLL / FLL	Multiplies a reference: $f_{out} = f_{ref} \times N / (M \times P)$. Low jitter, needs lock time (tens of us)	Nothing is clocked until pll_lock ; the reset controller waits for it (Chapter 12)
Integer divider	Counter/toggle flop producing $f/2$, $f/4$, ...; usually related (synchronous) to its source	Must be a generated clock in SDC; never build it from arbitrary logic
Clock mux	Selects between sources (PLL vs. reference, low-power clock)	A plain mux glitches when switching; use a glitch-free mux (Section 11.10)
Clock gate (ICG)	Latch + AND cell that stops the clock to idle logic	Instantiate the library ICG; never AND a clock with a flop output (Chapter 19)
External / recovered clocks	JTAG TCK, SPI SCLK, PHY RX clocks, audio MCLK	Fully asynchronous to everything on chip

PITFALL: CLOCKS MADE IN RTL

Writing `assign clk_div2 = cnt[0];` and using it as a clock, or `assign gclk = clk & en;`, creates a clock that STA cannot reason about, glitches on enable changes, and skews against its source. Rule: clocks are produced only by dedicated, instantiated cells (PLL, ICG, clock mux, divider flop marked as a generated clock) in a small, reviewed clock-generation module, and everything else uses **clock enables** (Chapter 6).

11.2 How two clocks can be related

Relationship	Definition	Example	Crossing technique
Synchronous	Same source, fixed integer ratio and phase; STA times the paths	clk and clk/2 from one PLL	None needed - normal timing, maybe multicycle constraints
Mesochronous	Same frequency, unknown but constant phase	Same PLL distributed over a large die without balancing	Small FIFO or phase-detect retiming
Plesiochronous	Nominally equal frequency, slightly different (ppm)	Two Ethernet ports on separate crystals	Async FIFO with elastic buffering / rate adaptation
Asynchronous	No relationship at all	CPU PLL vs. USB PHY clock	Synchronizers, handshakes, async FIFOs

In practice, unless two clocks come from the same source with a guaranteed ratio **and** the paths between them are timed by STA, treat them as asynchronous. The SDC then declares them in different clock groups (`set_clock_groups -asynchronous`) and STA stops timing those paths - which is exactly why the RTL must make them safe by construction.

11.3 Metastability and MTBF

A flop is a bistable loop. If its input changes inside the setup/hold aperture, the loop can be left balanced near the midpoint voltage, and it resolves to 0 or 1 only after a random, exponentially-distributed time. Metastability cannot be prevented; it can only be given enough time to resolve before anyone looks at the value. The standard model gives the mean time between failures:

$$\text{MTBF} = \exp(t_r / \tau) / (T_0 \times f_{\text{clk}} \times f_{\text{data}})$$

t_r = resolution time available = $T_{\text{clk}} - t_{\text{setup}} - t_{\text{clk} \rightarrow q}$ (- logic delay)
 τ = regeneration time constant of the flop (process/cell dependent, ~10 ps)
 T_0 = effective metastability window (~tens of ps, from the library)
 f_{clk} = receiving clock frequency; f_{data} = toggle rate of the async input

Resolution time is in the exponent. Every extra picosecond of slack multiplies MTBF; every extra gate after the first flop divides it.

Put numbers in: $\tau = 12$ ps, $T_0 = 20$ ps, a 1 GHz receiving clock and data toggling at 100 MHz, so the denominator is 2×10^6 per second.

Design	t_r	MTBF
One flop, then 800 ps of logic	200 ps	about 8.7 seconds
Two flops back-to-back (2FF synchronizer)	900 ps	about 1.9×10^{26} s (6×10^{18} years)
Three flops	1800 ps	about 7×10^{58} s

WHY THE SECOND FLOP IS ENOUGH (USUALLY)

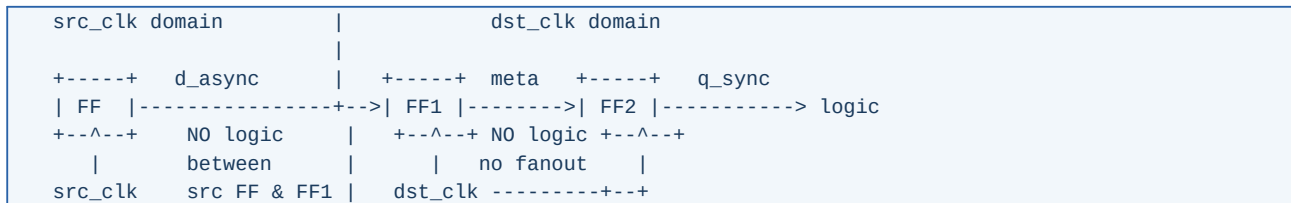
The first flop may go metastable; the second flop samples it one full period later, by which time the probability that it has not resolved is $\exp(-t_r/\tau)$ - astronomically small. Note the system MTBF is the per-synchronizer MTBF divided by the number of synchronizers: 10 000 synchronizers on a chip shipped in 10 million units need each one to be very, very good. At high frequencies (>1 GHz) or on older/low-voltage processes where τ grows, designs use 3 stages. The library team, not the RTL designer, provides τ and T_0 .

11.4 The two-flop synchronizer

The workhorse for a **single, slowly changing control bit**: an enable, a mode bit, a level interrupt, a status flag.

```
module sync_2ff #(parameter int W = 1, parameter logic [W-1:0] RST_VAL = '0) (
    input  logic      clk,
    input  logic      rst_n,
    input  logic [W-1:0] d_async,    // from another clock domain (quasi-static bits only!)
    output logic [W-1:0] q_sync
);
    (* ASYNC_REG = "TRUE" *) logic [W-1:0] meta;
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) begin meta <= RST_VAL; q_sync <= RST_VAL; end
        else      begin meta <= d_async; q_sync <= meta; end
endmodule
```

sync_2ff: the basic synchronizer. The **ASYNC_REG** attribute (Vivado) keeps the flops adjacent and excluded from retiming; ASIC flows use a dedicated synchronizer cell instead.



Rules encoded in this picture: the source is a **flop** (no combinational glitches can be caught), there is **no logic** between domains, FF1 feeds only FF2, and the two flops sit next to each other.

```
module tb_sync;
  logic clk = 0, rst_n = 0, a = 0;
  logic q;
  always #5 clk = ~clk; // 100 MHz destination clock
  sync_2ff u (.clk, .rst_n, .d_async(a), .q_sync(q));
  initial begin
    #12 rst_n = 1;
    #21 a = 1; // changes at t=33, between edges
    repeat (4) @(posedge clk) #1 $display("t=%0t a=%b meta=%b q=%b", $time, a, u.meta, q);
    $finish;
  end
endmodule
```

```
t=36 a=1 meta=1 q=0
t=46 a=1 meta=1 q=1
t=56 a=1 meta=1 q=1
t=66 a=1 meta=1 q=1
```

The async input rises at t=33; **meta** captures it at the edge at t=35 and **q_sync** follows one cycle later at t=45 - a latency of 1 to 2 destination cycles depending on where the change lands.

PITFALL: LOGIC IN FRONT OF THE SYNCHRONIZER

`sync_2ff u(.d_async(a & b), ...)` with **a** and **b** from the source domain looks harmless, but the AND can **glitch** when both inputs change, and the synchronizer can catch the glitch - a pulse that never existed in the source domain. Always register the value in the source domain and send the flop output across.

11.5 What must never cross unsynchronized

- **Any single-bit control signal** feeding logic in another domain (enables, validates, requests, interrupts, FSM inputs).
- **Multi-bit buses**, even through individual 2FF synchronizers: each bit resolves independently, so a counter going 0111 -> 1000 can be seen as 1111 or 0000 for a cycle (Section 11.8).
- **Short pulses** from a faster domain: a 1-cycle pulse at 400 MHz may fall entirely between two 100 MHz edges and be missed.
- **Reset de-assertion**: the edge that releases reset is an asynchronous event for the flops it reaches (Chapter 12).
- **Clock-gating enables** and **clock-mux selects**: these must be synchronized to the clock they gate or select.
- The exception: **quasi-static** configuration registers written once while the destination is idle or in reset may be marked static - but that is a documented CDC waiver, not a default.

11.6 Pulse (toggle) synchronizer

To send an **event** rather than a level, convert it to a level change in the source domain, synchronize the level, and detect the change in the destination:

```

module pulse_sync (
    input  logic src_clk, src_rst_n, src_pulse,    // 1-cycle pulse in source domain
    input  logic dst_clk, dst_rst_n,
    output logic dst_pulse                        // 1-cycle pulse in destination domain
);
    logic src_tgl;                                // flips once per event
    always_ff @(posedge src_clk or negedge src_rst_n)
        if (!src_rst_n) src_tgl <= 1'b0;
        else if (src_pulse) src_tgl <= ~src_tgl;

    logic [2:0] dst_sr;                            // 2 sync flops + 1 history flop
    always_ff @(posedge dst_clk or negedge dst_rst_n)
        if (!dst_rst_n) dst_sr <= '0;
        else            dst_sr <= {dst_sr[1:0], src_tgl};

    assign dst_pulse = dst_sr[2] ^ dst_sr[1];      // edge detect on synchronized toggle
endmodule

```

Toggle-based pulse synchronizer: each source pulse flips `src_tgl`; the destination edge-detects the synchronized toggle.

```

module tb_psync;
    logic sclk = 0, dclk = 0, rst_n = 0, sp = 0;
    logic dp;
    int    sent = 0, got = 0;
    always #2 sclk = ~sclk;    // 250 MHz source
    always #7 dclk = ~dclk;    // ~71 MHz destination
    pulse_sync u (.src_clk(sclk), .src_rst_n(rst_n), .src_pulse(sp),
                 .dst_clk(dclk), .dst_rst_n(rst_n), .dst_pulse(dp));
    always @(posedge dclk) if (dp) got++;
    initial begin
        #20 rst_n = 1;
        repeat (5) begin            // events spaced >= 3 dst cycles apart
            @(posedge sclk) sp <= 1; sent++;
            @(posedge sclk) sp <= 0;
            repeat (12) @(posedge sclk);
        end
        repeat (10) @(posedge dclk);
        $display("sent=%0d received=%0d", sent, got);
        $finish;
    end
endmodule

```

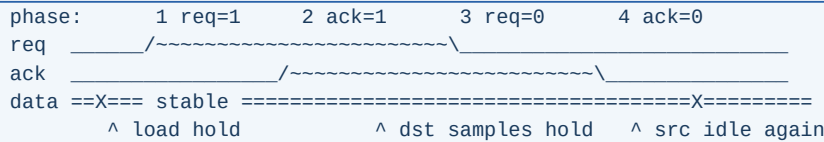
```
sent=5 received=5
```

THE PULSE SYNCHRONIZER'S CONTRACT

Events must be spaced at least ~2-3 **destination** clock cycles apart; two pulses closer than that flip the toggle twice and the destination sees nothing. If the source can burst faster, either add feedback (a busy/ack back to the source) or use an async FIFO. Unlike a direct pulse, the toggle works from fast to slow domains because a level cannot be missed.

11.7 Request/acknowledge handshake

The four-phase handshake moves a **multi-bit word** with only two synchronized control bits. The data sits in a source-domain register that is held stable for the whole transfer, so the destination can sample it directly - it was stable long before `req` got through the synchronizer.



Four-phase (return-to-zero) handshake. Each phase costs two synchronizer delays, so a transfer takes ~4-6 cycles of the slower clock: great for configuration and status, too slow for streaming.

```
// 4-phase req/ack handshake: moves a W-bit word safely between unrelated clocks.
module hs_cdc #(parameter int W = 8) (
    input  logic      s_clk, s_rst_n,
    input  logic      s_valid,          // request to send s_data
    input  logic [W-1:0] s_data,
    output logic      s_ready,          // idle: may accept a new word
    input  logic      d_clk, d_rst_n,
    output logic      d_valid,          // 1-cycle strobe with d_data
    output logic [W-1:0] d_data
);
    logic      req, ack;                // control: each crosses through 2 flops
    // ---- source domain ----
    logic [W-1:0] hold;                 // data: crosses UNSynchronized, stable while req=1
    logic [1:0]  ack_s;                 // ack synchronized into s_clk
    assign s_ready = !req && !ack_s[1];
    always_ff @(posedge s_clk or negedge s_rst_n)
        if (!s_rst_n) begin req <= 0; hold <= '0; ack_s <= '0; end
        else begin
            ack_s <= {ack_s[0], ack};
            if (s_valid && s_ready) begin hold <= s_data; req <= 1; end
            else if (ack_s[1]) req <= 0; // phase 3: drop req
        end
    // ---- destination domain ----
    logic [1:0] req_d;                  // req synchronized into d_clk
    always_ff @(posedge d_clk or negedge d_rst_n)
        if (!d_rst_n) begin req_d <= '0; ack <= 0; d_valid <= 0; d_data <= '0; end
        else begin
            req_d <= {req_d[0], req};
            d_valid <= 0;
            if (req_d[1] && !ack) begin d_data <= hold; d_valid <= 1; ack <= 1; end
            else if (!req_d[1]) ack <= 0; // phase 4: drop ack
        end
endmodule
```

hs_cdc: **req** and **ack** are the only synchronized signals; **hold** crosses unsynchronized but is guaranteed stable when sampled. CDC tools recognise this as a 'data held by control' scheme.

```
module tb_hs;
    logic sclk = 0, dclk = 0, rst_n = 0, sv = 0, sr, dv;
    logic [7:0] sd = 0, dd;
    int errs = 0, n = 0;
    always #3 sclk = ~sclk;
    always #8 dclk = ~dclk;
    hs_cdc u (.s_clk(sclk), .s_rst_n(rst_n), .s_valid(sv), .s_data(sd), .s_ready(sr),
              .d_clk(dclk), .d_rst_n(rst_n), .d_valid(dv), .d_data(dd));
    always @(posedge dclk) if (dv) begin
        if (dd != 8'h10 + n) errs++;
        n++;
    end
    initial begin
        #30 rst_n = 1;
        for (int i = 0; i < 6; i++) begin
            @(posedge sclk); while (!sr) @(posedge sclk);
        end
    end
endmodule
```

```

        sv <= 1; sd <= 8'h10 + i;
        @(posedge sclk); sv <= 0;
    end
    repeat (20) @(posedge dclk);
    $display("words received=%0d errors=%0d last=%h at t=%0t", n, errs, dd, $time);
    $finish;
end
endmodule

```

```
words received=6 errors=0 last=15 at t=936
```

A **two-phase** (non-return-to-zero) variant treats every toggle of **req** as a new request and every toggle of **ack** as its completion; it halves the latency at the cost of XOR-based edge detection on both sides - the same trick as the toggle synchronizer.

11.8 Multi-bit crossings and reconvergence

Passing a bus through per-bit synchronizers fails because the bits resolve independently: with skew and metastability, the destination may see a mixture of old and new bits for a cycle - a value that never existed. The same hazard appears in a subtler form called **reconvergence**: two separately synchronized signals that are combined later (for example **valid** and **mode**, each through its own 2FF) can arrive a cycle apart, so logic that relies on their relationship sees an impossible combination.

Technique	Use when	How it keeps the value coherent
Gray code + 2FF per bit	Counters / pointers that change by +/-1	Only one bit changes per step, so a mis-sample yields either the old or the new value, both legal
Handshake (req/ack)	Occasional words, config, status	Data held stable; only one control bit crosses
MUX recirculation (data-enable)	A bus with a qualifying valid	Synchronize only the valid; use it to load the bus into a destination register (below)
Async FIFO	Streaming data, bursts	Gray-coded pointers + dual-port memory
Single synchronized control, derived locally	Several related control bits	Encode them in the source into one bit or a Gray-coded state, then decode after synchronizing

```

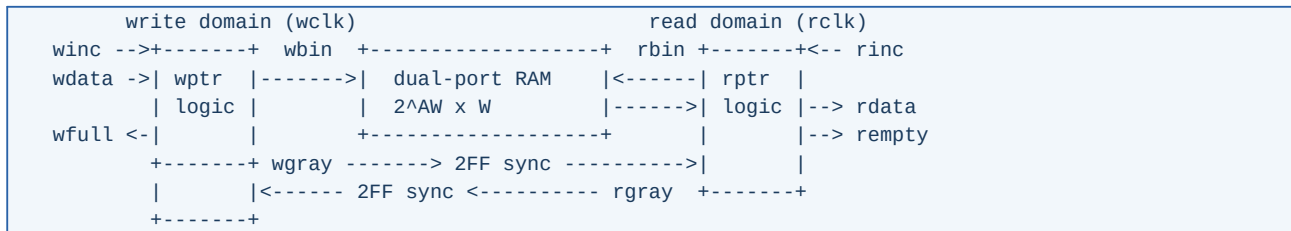
// MUX-recirculation (enable-based) synchronizer: only 'load' is synchronized.
// 'bus' must be held stable by the source from before load toggles until
// the destination has captured it (typically guaranteed by a handshake).
always_ff @(posedge dst_clk)
    if (load_pulse_dst) bus_dst <= bus_src; // else recirculate (hold)

```

The mux in front of **bus_dst** recirculates its value until the synchronized enable arrives; all bus bits are sampled on the same edge long after they settled.

11.9 The asynchronous FIFO

The async FIFO is the standard answer for streaming data between unrelated clocks. The memory is a dual-port RAM written in the write domain and read in the read domain. The only things that cross are the **pointers**, and they cross in **Gray code**, so each synchronized pointer is always either current or slightly stale - never garbage. A stale pointer is always **conservative**: the writer may think the FIFO is fuller than it is, and the reader may think it is emptier, but neither can overflow or underflow.



Pointers are N+1 bits (AW address bits plus a wrap bit). Each side compares its own pointer with the synchronized Gray pointer of the other side.

Flag	Computed in	Condition (Gray pointers, AW+1 bits)
empty	read domain	<code>rgray_next == wgray_synced</code> - identical, including the wrap bit
full	write domain	<code>wgray_next == {~rgray_synced[AW:AW-1], rgray_synced[AW-2:0]}</code> - top two bits inverted, rest equal (the binary pointers differ by exactly the depth)

```

module async_fifo #(parameter int W = 8, parameter int AW = 4) (    // depth = 2**AW
    input  logic          wclk, wrst_n, winc,
    input  logic [W-1:0] wdata,
    output logic          wfull,
    input  logic          rclk, rrst_n, rinc,
    output logic [W-1:0] rdata,
    output logic          rempty
);
    logic [W-1:0] mem [2**AW];
    logic [AW:0] wbin, wgray, rbin, rgray;           // one extra bit: wrap indicator
    logic [AW:0] rgray_w1, rgray_w2;                 // rgray synchronized into wclk
    logic [AW:0] wgray_r1, wgray_r2;                 // wgray synchronized into rclk

    function automatic logic [AW:0] b2g(input logic [AW:0] b);
        return b ^ (b >> 1);
    endfunction

    // ----- write domain -----
    logic [AW:0] wbin_nxt, wgray_nxt;
    assign wbin_nxt = wbin + (winc & ~wfull);
    assign wgray_nxt = b2g(wbin_nxt);
    always_ff @(posedge wclk or negedge wrst_n)
        if (!wrst_n) begin
            wbin <= '0; wgray <= '0; wfull <= 1'b0; {rgray_w2, rgray_w1} <= '0;
        end else begin
            {rgray_w2, rgray_w1} <= {rgray_w1, rgray};
            wbin <= wbin_nxt;
            wgray <= wgray_nxt;           // registered: glitch-free when sampled
            // full: next wgray equals rgray with the two MSBs inverted
            wfull <= (wgray_nxt == {~rgray_w2[AW:AW-1], rgray_w2[AW-2:0]});
        end
    always_ff @(posedge wclk)
        if (winc && !wfull) mem[wbin[AW-1:0]] <= wdata;

    // ----- read domain -----
    logic [AW:0] rbin_nxt, rgray_nxt;
    assign rbin_nxt = rbin + (rinc & ~rempty);
    assign rgray_nxt = b2g(rbin_nxt);
    always_ff @(posedge rclk or negedge rrst_n)
        if (!rrst_n) begin
            rbin <= '0; rgray <= '0; rempty <= 1'b1; {wgray_r2, wgray_r1} <= '0;
        end else begin
            {wgray_r2, wgray_r1} <= {wgray_r1, wgray};
            rbin <= rbin_nxt;

```

```

    rgray <= rgray_nxt;
    rempty <= (rgray_nxt == wgray_r2);          // empty: pointers identical
end
assign rdata = mem[rbin[AW-1:0]];              // FWFT-style combinational read
endmodule

```

A Cummings-style asynchronous FIFO. Flags are registered from the **next** pointer so they are glitch-free and assert in the same cycle as the pointer update.

```

module tb_afifo;
    logic wclk = 0, rclk = 0, rst_n = 0, winc = 0, rinc = 0, wfull, rempty;
    logic [7:0] wdata = 0, rdata, expect_v = 0;
    int nw = 0, nr = 0, errs = 0, full_seen = 0;
    always #5 wclk = ~wclk;                    // 100 MHz writer
    always #13 rclk = ~rclk;                   // ~38 MHz reader (unrelated)
    async_fifo #(.W(8), .AW(3)) dut (.*, .wrst_n(rst_n), .rrst_n(rst_n));
    // writer: tries to write every cycle, honours wfull
    always @(posedge wclk) if (rst_n) begin
        if (winc && !wfull) begin nw++; wdata <= wdata + 1; end
        if (wfull) full_seen++;
        winc <= (nw < 200);
    end
    // reader: random-ish throttling, checks order
    always @(posedge rclk) if (rst_n) begin
        if (rinc && !rempty) begin
            if (rdata !== expect_v) errs++;
            expect_v <= expect_v + 1; nr++;
        end
        rinc <= ($urandom % 4) != 0;
    end
    initial begin
        #40 rst_n = 1;
        wait (nr == 200);
        $display("written=%0d read=%0d order_errors=%0d wclk_cycles_full=%0d",
            nw, nr, errs, full_seen);
        $finish;
    end
endmodule

```

The testbench runs a 100 MHz writer against a ~38 MHz reader that randomly stalls, so the FIFO fills and throttles the writer hundreds of times.

```
written=200 read=200 order_errors=0 wclk_cycles_full=429
```

Real run: 200 words through an 8-deep FIFO across two unrelated clocks, in order, with the full flag exercised 429 times.

PITFALLS IN ASYNC FIFOS

(1) Synchronizing the **binary** pointer: several bits change at once, so the far side can see a wildly wrong count. (2) Gray-coding a pointer through **combinational** logic and sending that across - the encoder output can glitch; the Gray pointer must come straight from a flop (**wgray** here). (3) Non-power-of-two depths break the single-bit-change property unless you use a special Gray sequence. (4) Reading **rdata** from the RAM is a timing path from the write clock's memory to the read clock - it is safe because the reader only looks at entries whose write completed two synchronizer delays ago; constrain it with **set_max_delay -datapath_only**, not a false path.

SIZING AN ASYNC FIFO

Depth must cover the **round-trip latency** of the flags (about 2 synchronizer delays + 1-2 cycles each way) plus the largest burst that can arrive while the reader is slower: for a burst of B words written at f_w and read at $f_r < f_w$, at least $B \times (1 - f_r/f_w) + \text{the latency slack}$. Interviewers love this calculation.

11.10 Glitch-free clock multiplexing

Switching a clock with an ordinary mux can produce a runt pulse shorter than either clock's high or low phase, which violates the minimum pulse width of every flop downstream. The glitch-free mux disables the old clock on its falling edge, waits until that is known in the new clock's domain, and only then enables the new clock, also on a falling edge.

```
// Glitch-free 2:1 clock mux for unrelated clocks (behavioural model; in silicon the
// AND/OR gates are hand-instantiated clock-gating/clock-mux cells, never inferred).
module clk_mux_gf (
    input  logic clk0, clk1, rst_n,
    input  logic sel,           // asynchronous select: 0 -> clk0, 1 -> clk1
    output logic clk_out
);
    logic [1:0] s0, s1;          // per-domain 2-flop synchronizers
    logic      en0, en1;         // enable, changed only while its clock is low
    always_ff @(posedge clk0 or negedge rst_n)
        if (!rst_n) s0 <= 2'b11; else s0 <= {s0[0], ~sel & ~en1};
    always_ff @(negedge clk0 or negedge rst_n)
        if (!rst_n) en0 <= 1'b1; else en0 <= s0[1];
    always_ff @(posedge clk1 or negedge rst_n)
        if (!rst_n) s1 <= 2'b00; else s1 <= {s1[0], sel & ~en0};
    always_ff @(negedge clk1 or negedge rst_n)
        if (!rst_n) en1 <= 1'b0; else en1 <= s1[1];
    assign clk_out = (clk0 & en0) | (clk1 & en1);
endmodule
```

Cross-coupled glitch-free clock mux. Each enable is synchronized in its own domain and updated on the falling edge, so the AND gates only open or close while their clock is low.

```
module tb_cmux;
    logic c0 = 0, c1 = 0, rst_n = 0, sel = 0, co;
    realtime t_edge = 0, min_hi = 1e9, min_lo = 1e9;
    always #5 c0 = ~c0;           // 10 ns period: phases of 5 ns
    always #7 c1 = ~c1;           // 14 ns period: phases of 7 ns
    clk_mux_gf u (.clk0(c0), .clk1(c1), .rst_n, .sel, .clk_out(co));
    always @(co) if (rst_n && $realtime > 30) begin
        if (co == 0 && $realtime - t_edge < min_hi) min_hi = $realtime - t_edge;
        if (co == 1 && $realtime - t_edge < min_lo) min_lo = $realtime - t_edge;
        t_edge = $realtime;
    end
    initial begin
        #23 rst_n = 1;
        repeat (20) begin
            #($urandom_range(80, 200)) sel = ~sel; // switch at random times
        end
        #200 $display("switches=20  shortest high=%0.1f ns  shortest low=%0.1f ns",
            min_hi, min_lo);
        $finish;
    end
endmodule
```

The testbench measures the shortest high and low phase at the output while toggling `sel` at random times.

```
switches=20  shortest high=5.0 ns  shortest low=5.0 ns
```

With `sel` held for at least 80 ns between switches, no phase is shorter than the source clocks' own 5 ns: no glitches.

Now shorten the hold time to 5-30 ns by changing the range to `$urandom_range(5, 30)` and run again:

```
switches=20  shortest high=5.0 ns  shortest low=1.0 ns
```

A 1 ns runt pulse. The select changed back before the previous switch completed, so both enables were briefly on.

CLOCK-MUX CONTRACT

A glitch-free mux needs **both clocks running** while it switches (a dead clock can never release its enable - real designs add a clock-fail override), and **sel** must be held stable until the switch completes (about 3 cycles of each clock). In silicon the gating is done by library clock-mux/ICG cells, and the RTL module is written once, reviewed and reused everywhere.

11.11 Summary

- Treat any clocks not timed together by STA as asynchronous; STA will ignore those paths, so correctness must be by construction.
- Metastability is unavoidable; MTBF grows exponentially with resolution time. A 2FF synchronizer with no logic between stages is the baseline; 3 stages at very high frequencies.
- Single-bit levels: 2FF. Events: toggle/pulse synchronizer. Occasional words: req/ack handshake or MUX recirculation. Streams: async FIFO with Gray pointers.
- Never synchronize a multi-bit binary value bit-by-bit, never put combinational logic in front of a synchronizer, and watch for reconvergence of separately synchronized signals.
- Clock generation, gating and muxing use instantiated cells in a dedicated module; the glitch-free mux needs both clocks alive and a stable select.

11.12 Exercises

- 1 Recompute the MTBF table for a 2 GHz receiving clock with the same **tau** and **T₀**. How many synchronizer stages do you need for an MTBF above 10^9 years across 5 000 synchronizers?
- 2 Modify **pulse_sync** to return a **busy** signal to the source that stays high until the event has been seen in the destination. Simulate back-to-back source pulses.
- 3 Prove (by enumerating all 16 values) that a 4-bit Gray counter changes exactly one bit per increment including the wrap, and show why a 6-entry Gray sequence built from the first 6 codes does not.
- 4 Add **wr_count** (write-domain fill level) to **async_fifo**. Which pointer must be converted from Gray back to binary, and where?
- 5 In **hs_cdc**, what happens if the source changes **s_data** while **req** is high? Why is **hold** needed instead of using **s_data** directly?
- 6 A 1 GHz source sends 200-cycle bursts into a 600 MHz reader. Estimate the minimum async FIFO depth.

12. Reset Architecture and RDC

Chapter 6 covered the flop-level choice between synchronous and asynchronous reset. At the SoC level, reset is a **system**: several sources, a controller that sequences them, a distribution network as large as a clock tree, and domains that can be reset independently while their neighbours keep running. That last property creates a failure class of its own - **reset-domain crossing (RDC)** - which is invisible to both functional simulation and ordinary CDC analysis.

12.1 Kinds of reset in an SoC

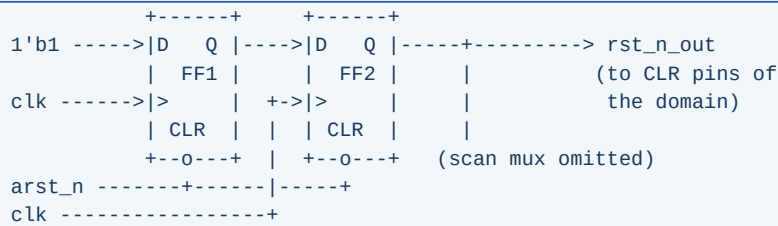
Reset	Trigger	Scope	Notes
Power-on reset (POR)	Analog POR cell detects the supply ramp / brown-out	Everything, including always-on logic	Only reset that may clear sticky 'reset reason' registers
External / pad reset	RESET_N pin, debugger	Usually everything except the POR-only state	Debounced and synchronized before use
Warm reset	Software request, fatal error	Cores and peripherals; keeps PLL, always-on domain, reset-reason, some memory	Fast recovery without re-locking PLLs
Watchdog reset	Watchdog timer expiry (Chapter 16)	Usually a warm reset	Latches the cause so boot code can report it
Software / block reset	A CSR bit (e.g. <code>PERIPH_RST[3]</code>)	One IP block or subsystem	The main source of RDC hazards
Domain reset	Power-domain turn-on (Chapter 19), clock domain start-up	Everything in one power or clock domain	Must be asserted while isolation is on and released after power is good
Debug reset	Debug module <code>ndmreset</code> / <code>hartreset</code>	Cores and system, but not the debug module itself	The debugger must stay connected across it

12.2 Asynchronous assert, synchronous de-assert

Most ASIC SoCs use flops with asynchronous reset pins (so reset works with the clock stopped and does not lengthen data paths) but release reset **synchronously**. The reason is the reset equivalent of setup and hold: **recovery** and **removal** time. If the reset de-asserts too close to a clock edge, some flops leave reset in this cycle and some in the next, or go metastable - a state machine can wake up in an illegal state. The reset synchronizer asserts immediately and releases only on a clock edge of the domain it serves.

```
// Asynchronous assert, synchronous de-assert reset synchronizer.
module rst_sync #(parameter int STAGES = 2) (
    input  logic clk,
    input  logic arst_n,          // raw asynchronous reset (pad, POR, reset controller)
    input  logic scan_mode,      // DFT: bypass so the tester controls reset directly
    input  logic scan_rst_n,
    output logic rst_n_out       // drives the async reset pins of this domain's flops
);
    (* ASYNC_REG = "TRUE" *) logic [STAGES-1:0] sr;
    always_ff @(posedge clk or negedge arst_n)
        if (!arst_n) sr <= '0;                // assert immediately, no clock needed
        else        sr <= {sr[STAGES-2:0], 1'b1}; // release after STAGES clock edges
    assign rst_n_out = scan_mode ? scan_rst_n : sr[STAGES-1];
endmodule
```

Reset synchronizer, one per clock domain. The `scan_mode` mux gives the tester direct control of reset for DFT (Chapter 20).



Assertion reaches both flops' CLR pins at once (no clock needed). De-assertion lets a 1 ripple through two flops, so the domain leaves reset on a clock edge, and any metastability of FF1 is resolved by FF2 - exactly a 2FF synchronizer with a constant input.

```
module tb_rsync;
  logic clk = 0, arst_n = 1, rn;
  always #5 clk = ~clk;
  rst_sync u (.clk, .arst_n, .scan_mode(1'b0), .scan_rst_n(1'b1), .rst_n_out(rn));
  initial begin
    $monitor("t=%3t arst_n=%b sr=%b rst_n_out=%b", $time, arst_n, u.sr, rn);
    #13 arst_n = 0;          // assert between edges -> output drops at once
    #24 arst_n = 1;          // release at t=37, between edges
    #40 $finish;
  end
endmodule
```

```
t=  0 arst_n=1 sr=xx rst_n_out=x
t=  5 arst_n=1 sr=x1 rst_n_out=x
t= 13 arst_n=0 sr=00 rst_n_out=0
t= 37 arst_n=1 sr=00 rst_n_out=0
t= 45 arst_n=1 sr=01 rst_n_out=0
t= 55 arst_n=1 sr=11 rst_n_out=1
```

Before reset the flops are X (a real hazard in simulation - see Section 12.8). Assertion at t=13 is immediate; the release at t=37 reaches **rst_n_out** two clock edges later, at t=55.

PITFALLS WITH RESET SYNCHRONIZERS

(1) One synchronizer shared by two clock domains - the release is synchronous to only one of them. (2) Glitches on **arst_n**: a combinational reset source (**assign arst_n = a & b;**) can glitch and reset a domain spuriously; generate reset sources from flops. (3) Forgetting the clock must be **running** for de-assertion - a gated or not-yet-locked clock holds its domain in reset forever. (4) Synchronous reset in the reset synchronizer itself - it would need a clock to assert.

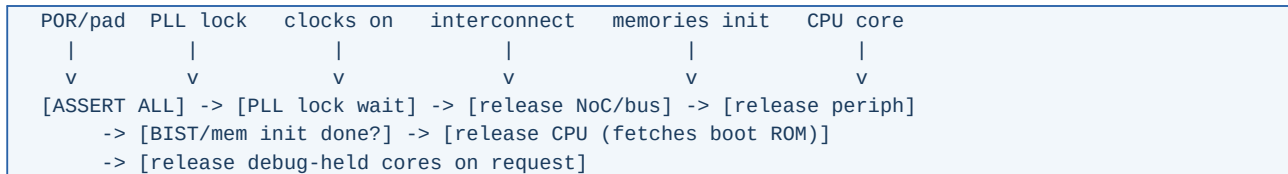
12.3 Reset distribution trees

The synchronized reset of a large domain fans out to hundreds of thousands of flops. Physical design builds a **reset tree** with buffers, like a clock tree but with relaxed skew requirements. The de-assertion edge must still meet recovery/removal at every flop in a single cycle, which STA checks automatically for synchronous release. When the tree is too big to meet recovery in one cycle, the usual fixes are:

- **Pipeline the reset**: add further flops after the synchronizer (and per sub-block) so each segment drives a smaller load. Reset latency grows by a few cycles, which nobody notices.
- **Hierarchical synchronizers**: one synchronizer per major block, fed from a central reset controller.
- **Multicycle recovery**: only when the design guarantees that no flop acts on the first cycles after release, and documented in SDC.

12.4 Reset sequencing

A reset controller is a small FSM in the always-on domain that releases resets in a defined order, each step waiting for its precondition:

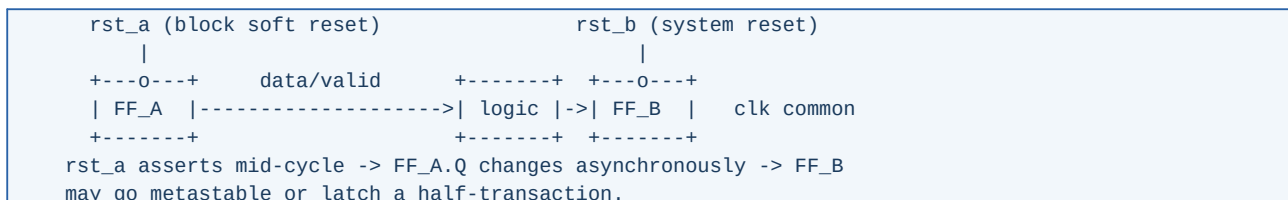


A typical boot sequence. The CPU comes last so that everything it can reach answers when the first instruction is fetched.

Rule	Why
Release a slave before its masters	A master that issues a transaction into a block still in reset gets no response - the bus hangs
Keep clocks running for N cycles with reset asserted	Synchronous-reset flops and memories with sync reset need edges to be cleared
Record the reset cause in a POR-only register	Boot code must distinguish watchdog, brown-out, software and pad resets
Stretch short reset requests	A 1-cycle software reset pulse may be shorter than a slow domain's period

12.5 Reset-domain crossing (RDC)

An RDC path goes from a flop reset by reset A to a flop reset by a different reset B, **in the same clock domain**. CDC analysis says it is fine - one clock. The danger appears when A is asserted while B is not: the source flop's output changes **asynchronously** (the reset pin acts immediately, not on a clock edge), and the destination flop, still running, can sample the change inside its setup/hold window and go metastable, or capture a value that corrupts its state.



The RDC hazard. Classic real case: resetting a UART while the interconnect is mid-transaction - the bus sees a response channel valid drop without a handshake.

RDC fix	How it works
Isolate before reset	Gate the crossing signals with an enable (e.g. <code>valid & ~iso</code>) driven from the destination domain; assert isolation, wait, then assert the source reset
Quiesce first	Stop traffic to the block (bus 'drain' or clock gating) before resetting it; the software reset sequence becomes: stop -> wait idle -> reset -> release -> re-enable
Reset the destination too	If B is always asserted whenever A is (A is a superset), the crossing is safe
Synchronize the crossing	Treat the signal as asynchronous and put a synchronizer on it in the destination

RDC IN SIGN-OFF

Dedicated RDC checkers (Chapter 25) enumerate reset domains from the reset tree, find every flop-to-flop path whose resets differ, and require one of the fixes above or an explicit waiver citing the reset ordering constraint (for example 'rst_b always asserted when rst_a is'). Many silicon bugs surfaced by the first software-reset test on a real board are RDC bugs.

12.6 Which flops need a reset?

Reset is not free: every reset pin costs area, reset-tree buffers and routing, and forces the async-reset flop variant, which is larger and slower. Good designers reset **control**, not **data**.

Needs reset	Usually does not
FSM state registers	Datapath pipeline registers qualified by a reset valid bit
Valid bits, request/grant flags, counters that control behaviour	Data payload registers, FIFO/RAM contents
CSRs (software expects documented reset values)	Shift registers whose contents are fully overwritten before use
Interrupt status and enables	Multiplier / MAC internal registers in accelerators
Anything that drives an output port or another block's control	Large register-file arrays (use a valid or init sequence)

MIXING RESET AND NON-RESET FLOPS IN ONE ALWAYS BLOCK

If one always block resets **valid** but not **data**, synthesis must implement 'hold **data** during reset' - it adds an enable mux on every data bit driven by **rst_n**. Write reset and non-reset flops in **separate** always blocks (as **async_fifo** does for its memory in Chapter 11).

12.7 Reset and DFT

During scan test (Chapter 20) the tester must control every asynchronous reset directly, or flops would be reset unpredictably while shifting. That is what **scan_mode/scan_rst_n** do in **rst_sync**. Additional rules: no reset may be generated from a flop's output in functional logic without a scan-mode bypass; reset synchronizer flops are usually excluded from scan chains or controlled; and the ATPG tool may test the reset network itself by pulsing **scan_rst_n** in capture.

12.8 X-propagation and reset

In RTL simulation, flops without reset start as **X**. Verilog's optimistic semantics can hide the problem: an **if (x_signal)** takes the else branch, and a **case** on an **X** matches **default**, so the simulation looks clean while silicon may start in any state. The reverse also happens: pessimistic X propagation in gate-level simulation flags **X** on non-reset datapath flops that silicon never actually uses.

- Enable an X-propagation mode (commercial simulators offer T-merge/X-prop modes) for RTL regressions.
- Assert that key control outputs are never X after reset: **assert property (@(posedge clk) disable iff (!rst_n) !\$isunknown(state));** (Chapter 23).
- Do not 'fix' X by adding resets everywhere - fix the logic that consumes an uninitialized value, or qualify it with a valid.
- Randomize initial values of non-reset flops in simulation (tools offer +initreg-style options) to catch reset-dependent bugs.

12.9 Summary

- An SoC has many resets (POR, pad, warm, watchdog, software, domain, debug); a reset controller in the always-on domain sequences them.
- Assert asynchronously, release synchronously: one reset synchronizer per clock domain, with a DFT bypass.
- Release slaves before masters; keep clocks running during reset; record the reset cause.
- RDC: an asynchronous reset in one domain can corrupt flops in a same-clock domain with a different reset. Isolate or quiesce first.
- Reset control, not data; separate reset and non-reset flops; use X-propagation checks to find reset-dependent bugs.

12.10 Exercises

- 1 Extend `rst_sync` with a `STAGES` parameter of 3 and a `rst_n_out` that is additionally stretched to at least 16 cycles after release of `arst_n`. Simulate it.
- 2 Draw the reset sequence (as a waveform) for a chip with a PLL, an AXI interconnect, a DDR controller that needs 200 us of stable clock before its own reset release, and a CPU.
- 3 A DMA engine has a software reset bit and talks to an AXI interconnect under system reset. List the RDC paths and design the reset sequence that makes the software reset safe.
- 4 Why is `always_ff @(posedge clk or negedge rst_n) if (!rst_n) q <= 0; else q <= d;` combined with a `data` register in the same block a synthesis-quality problem? Show the inferred hardware.
- 5 Write a SystemVerilog assertion that `rst_n_out` never de-asserts except in the cycle after a rising clock edge sampled `arst_n` high twice.

13. On-Chip Buses: APB, AHB, AXI4, AXI4-Lite, AXI4-Stream

Blocks in an SoC talk to each other through standard bus protocols, so that a CPU from one vendor, an interconnect generator from another and a hundred in-house peripherals plug together. By far the most common family is Arm's **AMBA** (Advanced Microcontroller Bus Architecture), whose specifications are freely available. Knowing APB and AXI signal-by-signal is a baseline expectation for any RTL role - interviewers routinely ask you to draw a handshake or write a slave on a whiteboard.

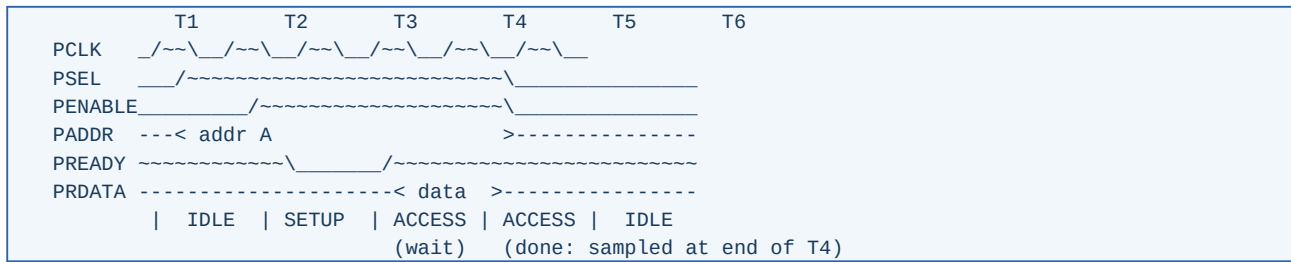
13.1 The AMBA family at a glance

Protocol	Typical use	Key properties
APB (APB3/APB4/APB5)	Low-bandwidth peripherals and CSRs: UART, timers, GPIO, config registers	Unpipelined, 2+ cycles per transfer, no bursts, tiny slave logic
AHB / AHB-Lite (AHB5)	Older/MCU-class system bus, on-chip SRAM, flash controllers	Pipelined address/data, single master (Lite), bursts, one outstanding transfer
AXI4 (AXI5)	High-performance memory-mapped: CPU, DMA, DDR, accelerators	5 independent channels, bursts up to 256 beats, IDs, many outstanding transactions, out-of-order completion
AXI4-Lite	Register access for high-speed IP	AXI4 handshakes but single beat, no IDs; simple slave
AXI4-Stream	Point-to-point data streams: video, DSP, networking, ML accelerators	No addresses; TVALID/TREADY handshake, TLAST framing, optional side-band
ACE / CHI	Cache-coherent interconnect for multi-core	Snoop channels / packetized coherent protocol (Chapter 14)

13.2 APB: the peripheral bus

APB is the simplest AMBA protocol and the one you will implement most often, because every block needs control/status registers. There is one requester (the **APB bridge**, usually converting from AXI or AHB) and many completers, each selected by its own **PSEL**.

Signal	Driver	Meaning
PCLK , PRESETn	system	Clock; active-low reset
PADDR[31:0]	requester	Byte address
PSELx	requester (decoder)	This completer is selected
PENABLE	requester	0 in the SETUP cycle, 1 in ACCESS cycles
PWRITE	requester	1 = write, 0 = read
PWDATA[31:0]	requester	Write data
PSTRB[3:0]	requester (APB4)	Byte-lane write strobes
PPROT[2:0]	requester (APB4)	Privileged / secure / instruction attributes
PREADY	completer	Extends the ACCESS phase (wait states) while low
PRDATA[31:0]	completer	Read data, valid when PENABLE & PREADY on a read
PSLVERR	completer	Error response, valid only in the last ACCESS cycle



APB read with one wait state. The transfer completes at the rising edge where PSEL, PENABLE and PREADY are all high.

- **SETUP** (one cycle): **PSEL=1**, **PENABLE=0**, address/control/write data valid.
- **ACCESS** (one or more cycles): **PENABLE=1**; all signals from SETUP must stay stable. The transfer ends on the edge where **PREADY=1**.
- Back-to-back transfers go ACCESS -> SETUP directly (PSEL stays high, PENABLE drops) - so an APB transfer takes at least 2 cycles.

```

module apb_regs (
    input  logic      pclk, presetn,
    input  logic      psel, penable, pwrite,
    input  logic [11:0] paddr,
    input  logic [31:0] pwrdata,
    input  logic [3:0] pstrb,          // APB4 byte strobes
    output logic [31:0] prdata,
    output logic      pready, pslverr,
    input  logic [7:0] hw_status      // read-only status from the hardware
);
    localparam logic [11:0] A_CTRL = 12'h000, A_STAT = 12'h004,
                          A_DATA = 12'h008, A_ID  = 12'h00C;
    logic [31:0] ctrl, data;
    logic      wait_done;          // DATA accesses take one wait state
    wire access = psel & penable;
    wire ro     = (paddr == A_STAT) || (paddr == A_ID);
    wire hit    = ro || (paddr == A_CTRL) || (paddr == A_DATA);
    wire slow   = (paddr == A_DATA);

    // PREADY: low for exactly one ACCESS cycle when the slow register is addressed
    always_ff @(posedge pclk or negedge presetn)
        if (!presetn) wait_done <= 1'b0;
        else if (access && slow && !wait_done) wait_done <= 1'b1;
        else wait_done <= 1'b0;
    assign pready = access ? (!slow || wait_done) : 1'b1;
    assign pslverr = access && pready && (!hit || (pwrite && ro));

    // writes commit in the last ACCESS cycle (PREADY high), honouring PSTRB
    wire wr_en = access && pready && pwrite && !pslverr;
    always_ff @(posedge pclk or negedge presetn)
        if (!presetn) begin ctrl <= '0; data <= '0; end
        else if (wr_en)
            for (int b = 0; b < 4; b++) if (pstrb[b]) begin
                if (paddr == A_CTRL) ctrl[8*b +: 8] <= pwrdata[8*b +: 8];
                if (paddr == A_DATA) data[8*b +: 8] <= pwrdata[8*b +: 8];
            end
        end

    always_comb
        case (paddr)
            A_CTRL: prdata = ctrl;
            A_STAT: prdata = {24'h0, hw_status};
            A_DATA: prdata = data;
            A_ID:   prdata = 32'hA9B0_0101;
            default: prdata = '0;
        end

```



```
endcase
endmodule
```

An APB4 completer with four registers: CTRL (RW), STATUS (RO, from hardware), DATA (RW, one wait state to demonstrate PREADY) and ID (RO constant). Unmapped addresses and writes to read-only registers return PSLVERR.

```
module tb_apb;
  logic pclk = 0, presetn = 0, psel = 0, penable = 0, pwrite = 0, pready, pslverr;
  logic [11:0] paddr = 0; logic [31:0] pwrdata = 0, prdata, rd; logic [3:0] pstrb = 0;
  int cyc = 0;
  always #5 pclk = ~pclk;
  apb_regs dut (.*, .hw_status(8'h5A));
  always @(posedge pclk) begin
    cyc++;
    if (psel) begin
      string wd, rdat;
      wd = "-----"; rdat = "-----";
      if (pwrite) wd = $sformatf("%h", pwrdata);
      if (penable && pready && !pwrite) rdat = $sformatf("%h", prdata);
      $display("%2d  %b  %b  %b  %h  %s  %b  %s  %b",
        cyc, psel, penable, pwrite, paddr, wd, pready, rdat, pslverr);
    end
  end
  task automatic apb(input bit wr, input [11:0] a, input [31:0] d, output [31:0] q);
    @(negedge pclk) begin psel = 1; penable = 0; pwrite = wr; paddr = a; pwrdata = d;
      pstrb = wr ? 4'hF : 4'h0; end // SETUP phase
    @(negedge pclk) penable = 1; // ACCESS phase
    @(posedge pclk); while (!pready) @(posedge pclk); // wait states
    q = prdata;
    @(negedge pclk) begin psel = 0; penable = 0; end
  endtask
  initial begin
    $display("cyc SEL EN WR ADDR PWDATA RDY PRDATA ERR");
    #12 presetn = 1;
    apb(1, 12'h000, 32'h0000_0003, rd); // write CTRL
    apb(0, 12'h000, 0, rd); // read CTRL
    apb(1, 12'h008, 32'hCAFE_F00D, rd); // write DATA (1 wait state)
    apb(0, 12'h004, 0, rd); // read STATUS
    apb(1, 12'h00C, 32'h1234_5678, rd); // write to read-only ID -> PSLVERR
    apb(0, 12'h010, 0, rd); // unmapped -> PSLVERR
    $finish;
  end
endmodule
```

A requester task that drives SETUP and ACCESS on the falling edge and prints one line per cycle in which PSEL is high.

cyc	SEL	EN	WR	ADDR	PWDATA	RDY	PRDATA	ERR
3	1	0	1	000	00000003	1	-----	0
4	1	1	1	000	00000003	1	-----	0
6	1	0	0	000	-----	1	-----	0
7	1	1	0	000	-----	1	00000003	0
9	1	0	1	008	cafef00d	1	-----	0
10	1	1	1	008	cafef00d	0	-----	0
11	1	1	1	008	cafef00d	1	-----	0
13	1	0	0	004	-----	1	-----	0
14	1	1	0	004	-----	1	0000005a	0
16	1	0	1	00c	12345678	1	-----	0
17	1	1	1	00c	12345678	1	-----	1
19	1	0	0	010	-----	1	-----	0
20	1	1	0	010	-----	1	00000000	1

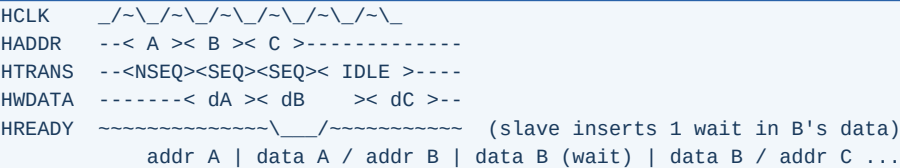
A waveform in table form, from the real run. Note the SETUP/ACCESS pairs, the extra ACCESS cycle 10 (RDY=0) for the DATA register, and PSLVERR in the final ACCESS cycle of the two bad transfers.

PITFALLS IN APB COMPLETERS

(1) Writing on **every** ACCESS cycle instead of only when **PREADY=1** - harmless with zero wait states, a double write with side effects (FIFO push, W1C) otherwise. (2) Acting in the SETUP cycle - the requester may still change the address in a later SETUP. (3) Driving PSLVERR outside the last ACCESS cycle. (4) A read with side effects (read-to-clear, FIFO pop) must also fire exactly once, in the completing cycle.

13.3 AHB-Lite: pipelined address and data

AHB overlaps the **address phase** of one transfer with the **data phase** of the previous one, so a zero-wait-state slave sustains one transfer per cycle. **HREADY** low stretches the current data phase **and** holds the next address phase.



Pipelined AHB. The slave must register address and control in the address phase (when **HREADY** is high) because they change in the next cycle.

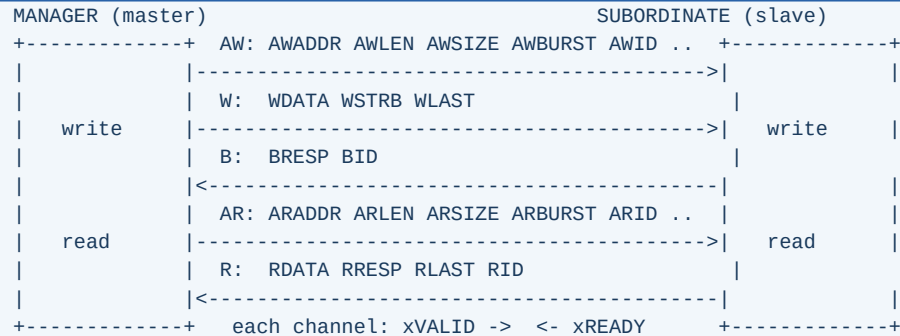
Signal	Meaning
HTRANS[1:0]	IDLE, BUSY, NONSEQ (first/single), SEQ (subsequent beats of a burst)
HBURST[2:0]	SINGLE, INCR, WRAP4/INCR4, WRAP8/INCR8, WRAP16/INCR16
HSIZE[2:0]	Bytes per beat (1 to 128)
HREADY / HREADYOUT	Global ready (from the mux) / this slave's ready
HRESP	OKAY or ERROR; ERROR is a two-cycle response

WHERE AHB STILL LIVES

Microcontrollers (Cortex-M buses are AHB-Lite), flash and SRAM controllers, and legacy IP. New high-performance designs choose AXI, but you will meet AHB bridges in almost every SoC.

13.4 AXI4: five independent channels

AXI4 separates a transaction into five channels, each a **valid/ready** stream exactly like those in Chapter 10. Because channels are independent, reads and writes proceed in parallel, address issue is decoupled from data, and many transactions can be in flight.



The five AXI channels. Write = AW + W + B; read = AR + R.

Handshake rules (the ones interviews ask about)

- A transfer occurs on a rising edge where **VALID** and **READY** are both high.
- The source **must not wait for READY before asserting VALID** - otherwise two blocks that each wait for the other deadlock.
- Once VALID is asserted it must **stay asserted**, with payload stable, until the handshake completes.
- The destination **may** wait for VALID before asserting READY, and may drop READY freely while VALID is low.
- Dependencies: the slave may wait for AW **and** W before giving B (it must wait for WLAST); it may wait for AR before R; a manager must not make W depend on AWREADY (write data may even arrive before the address).
- No combinational path from input VALID to output READY is required, and good IP registers both (Chapter 10's skid buffer).

Bursts

Field	Meaning
AxLEN[7:0]	Beats minus one: 1-256 beats for INCR, 1-16 for FIXED, exactly 2, 4, 8 or 16 for WRAP
AxSIZE[2:0]	Bytes per beat = 2^{AxSIZE} , up to the data bus width
AxBURST	FIXED (same address every beat - FIFOs), INCR (incrementing - normal memory), WRAP (incrementing, wraps at a boundary of len x size - cache-line fills, critical word first)
Rule	An INCR burst must not cross a 4 KB boundary, so a burst never spans two slaves or pages
WSTRB	One bit per byte lane; unaligned and narrow writes
xRESP	OKAY, EXOKAY (exclusive success), SLVERR (slave error), DECERR (no slave at this address - from the interconnect)

INCR : $\text{addr}_n = \text{start} + n \times 2^{\text{size}}$ (aligned after the first beat)

WRAP : $\text{lower} = \text{floor}(\text{start} / (\text{len} \times 2^{\text{size}})) \times (\text{len} \times 2^{\text{size}})$

addr_n wraps from $\text{lower} + \text{len} \times 2^{\text{size}}$ back to lower

e.g. WRAP4, 4-byte beats, start 0x38 -> 0x38, 0x3C, 0x30, 0x34

Burst address generation (len = AxLEN+1 beats).

IDs, ordering and outstanding transactions

Each address carries an ID (**AWID/ARID**). Responses with the **same ID** must return in order; responses with **different IDs** may return out of order. A DMA engine can thus issue reads to a fast SRAM and a slow DDR and use whichever answers first. Write data has no ID in AXI4 (the AXI3 **WID** signal was removed), so write data must follow the order of write addresses. The number of transactions a manager may have outstanding is a design parameter; latency x bandwidth determines how many you need:

outstanding reads needed ~ (latency_cycles x bytes_per_cycle) / bytes_per_burst

e.g. 200-cycle DDR latency, 16 B/cycle, 64 B bursts -> ~50 outstanding

Little's law applied to a bus: without enough outstanding transactions, a high-latency memory can never be kept busy.

OTHER AXI ATTRIBUTES

AXCACHE (bufferable, modifiable, allocate hints), **AXPROT** (privileged/secure/instruction), **AXLOCK** (exclusive access for atomics - EXOKAY), **AXQOS** (priority hint for the interconnect, Chapter 14), **AXREGION** and **xUSER** side-band. AXI5 adds atomics, poison, parity/ECC signalling and more.

13.5 An AXI4-Lite slave

AXI4-Lite keeps the five channels and the handshake rules but removes bursts (every transaction is one beat), IDs, and most attributes. It is the standard register interface for high-performance IP, and the classic place to make one mistake: assuming AW and W arrive together.

```
module axil_regs #(parameter int N = 4) (           // N 32-bit RW registers
    input logic      aclk, aresetn,
    // write address / write data / write response
    input logic      awvalid, output logic awready, input logic [7:0] awaddr,
    input logic      wvalid, output logic wready, input logic [31:0] wdata,
    input logic [3:0] wstrb,
    output logic      bvalid, input logic bready, output logic [1:0] bresp,
    // read address / read data
    input logic      arvalid, output logic arready, input logic [7:0] araddr,
    output logic      rvalid, input logic rready, output logic [31:0] rdata,
    output logic [1:0] rresp
);
    localparam logic [1:0] OKAY = 2'b00, SLVERR = 2'b10;
    logic [31:0] regs [N];
    // ----- write path: AW and W may arrive in any order -----
    logic      aw_full, w_full;           // one-entry skid holders
    logic [7:0] aw_q; logic [31:0] w_q; logic [3:0] s_q;
    assign awready = !aw_full;
    assign wready = !w_full;
    wire do_write = aw_full && w_full && (!bvalid || bready); // B slot free
    wire aw_ok = (aw_q[7:2] < N) && (aw_q[1:0] == 2'b00);
    always_ff @(posedge aclk or negedge aresetn)
        if (!aresetn) begin
            aw_full <= 0; w_full <= 0; bvalid <= 0; bresp <= OKAY;
            aw_q <= '0; w_q <= '0; s_q <= '0;
            for (int i = 0; i < N; i++) regs[i] <= '0;
        end else begin
            if (awvalid && awready) begin aw_q <= awaddr; aw_full <= 1; end
            if (wvalid && wready) begin w_q <= wdata; s_q <= wstrb; w_full <= 1; end
            if (bvalid && bready) bvalid <= 0;
            if (do_write) begin
                if (aw_ok) for (int b = 0; b < 4; b++)
                    if (s_q[b]) regs[aw_q[7:2]][8*b +: 8] <= w_q[8*b +: 8];
                bresp <= aw_ok ? OKAY : SLVERR;
                bvalid <= 1; aw_full <= 0; w_full <= 0;
            end
        end
    // ----- read path: one outstanding read -----
    assign arready = !rvalid || rready; // accept when R slot is (becoming) free
    always_ff @(posedge aclk or negedge aresetn)
        if (!aresetn) begin rvalid <= 0; rdata <= '0; rresp <= OKAY; end
        else if (arvalid && arready) begin
            rvalid <= 1;
            if ((araddr[7:2] < N) && (araddr[1:0] == 2'b00)) begin
                rdata <= regs[araddr[7:2]]; rresp <= OKAY;
            end else begin
                rdata <= 32'hDEAD_BEEF; rresp <= SLVERR;
            end
        end else if (rvalid && rready) rvalid <= 0;
endmodule
```

An AXI4-Lite register slave. AW and W are captured independently into one-entry holders; the write happens only when both are present and the B channel can take the response. Reads support back-to-back accesses at one per cycle.

```

module tb_axil;
  logic aclk = 0, aresetn = 0;
  logic awvalid = 0, awready, wvalid = 0, wready, bvalid, bready = 0;
  logic arvalid = 0, arready, rvalid, rready = 0;
  logic [7:0] awaddr = 0, araddr = 0; logic [31:0] wdata = 0, rdata; logic [3:0] wstrb = 0;
  logic [1:0] bresp, rresp;
  always #5 aclk = ~aclk;
  axil_regs dut (.*);
  // bounded "valid stays high until ready" protocol drivers
  task automatic send_aw(input [7:0] a);
    awaddr <= a; awvalid <= 1; do @(posedge aclk); while (!awready); awvalid <= 0;
  endtask
  task automatic send_w(input [31:0] d, input [3:0] s);
    wdata <= d; wstrb <= s; wvalid <= 1; do @(posedge aclk); while (!wready); wvalid <= 0;
  endtask
  task automatic get_b(input int stall);
    repeat (stall) @(posedge aclk); // master back-pressure on B
    bready <= 1; do @(posedge aclk); while (!bvalid); bready <= 0;
    $display("t=%0t B bresp=%b", $time, bresp);
  endtask
  task automatic read(input [7:0] a);
    araddr <= a; arvalid <= 1; do @(posedge aclk); while (!arready); arvalid <= 0;
    rready <= 1; do @(posedge aclk); while (!rvalid); rready <= 0;
    $display("t=%0t R addr=%h rdata=%h rresp=%b", $time, a, rdata, rresp);
  endtask
  initial begin
    repeat (2) @(posedge aclk); aresetn <= 1; @(posedge aclk);
    fork send_aw(8'h04); send_w(32'h1122_3344, 4'hF); get_b(0); join // together
    fork send_w(32'hAABB_CCDD, 4'b0011); get_b(3); join // W first,
    begin repeat (4) @(posedge aclk); send_aw(8'h04); end join // AW late
    fork send_aw(8'h40); send_w(32'h0, 4'hF); get_b(0); join // bad address
    read(8'h04);
    read(8'h41);
    $finish;
  end
endmodule

```

The testbench sends AW and W together, then W four cycles before AW with B back-pressure, then an unmapped address, and reads back.

```

t=55 B bresp=00
t=125 B bresp=00
t=155 B bresp=10
t=175 R addr=04 rdata=1122ccdd rresp=00
t=195 R addr=41 rdata=deadbeef rresp=10

```

The second write used WSTRB=0011, so only the low half changed: 0x11223344 -> 0x1122CCDD. Out-of-range accesses return SLVERR (10).

PITFALLS IN AXI SLAVES (FREQUENT PROTOCOL-CHECKER HITS)

(1) `assign awready = awvalid && wvalid;` - legal for a slave, but it builds combinational VALID -> READY paths through the interconnect and deadlocks against a (non-compliant, yet common) master that waits for AWREADY before driving WVALID; accept each channel independently, as `axil_regs` does. (2) Dropping BVALID or RVALID before READY. (3) Accepting a second AW while the previous B is still pending, then losing it. (4) Ignoring WSTRB. (5) Forgetting that the response ordering with IDs is per-ID, not global. Run an AXI protocol checker (assertion IP, Chapter 23) on every AXI port.

13.6 AXI4-Stream

AXI4-Stream has no addresses and no responses - just a single valid/ready channel carrying data from a source to a sink. It is the natural interface between pipeline stages of a DSP chain, a video pipeline or an ML accelerator (the Chapter 29 project uses it).

Signal	Meaning
TVALID / TREADY	Handshake, with exactly the AXI rules
TDATA[8n-1:0]	Payload
TLAST	Last transfer of a packet/frame/row
TKEEP[n-1:0]	Byte is part of the stream (0 = null byte removed, e.g. at the end of a packet)
TSTRB[n-1:0]	Byte is data (1) or position byte (0)
TID, TDEST	Stream identifier and routing destination for interleaved streams
TUSER	User side-band (e.g. start-of-frame for video)

```
// AXI4-Stream pass-through register stage with packet counter.
// Data advances when the output is empty or being drained (Chapter 10).
assign s_tready = !m_tvalid || m_tready;
always_ff @(posedge aclk or negedge aresetn)
  if (!aresetn) begin m_tvalid <= 1'b0; pkt_cnt <= '0; end
  else if (s_tready) begin
    m_tvalid <= s_tvalid;
    if (s_tvalid && s_tlast) pkt_cnt <= pkt_cnt + 1'b1;
  end
always_ff @(posedge aclk) // payload: no reset needed
  if (s_tready && s_tvalid) {m_tdata, m_tkeep, m_tlast} <= {s_tdata, s_tkeep, s_tlast};
```

The same register slice from Chapter 10 with AXI-Stream names; the handshake logic is identical.

13.7 Choosing a protocol

Criterion	APB	AHB-Lite	AXI4-Lite	AXI4	AXI4-Stream
Addressed	yes	yes	yes	yes	no
Pipelined	no	yes	yes (channels)	yes	yes
Bursts	no	up to 16 / INCR	no	up to 256	unbounded packets
Outstanding	1	1	1+ (impl.)	many, with IDs	n/a
Out-of-order	no	no	no	yes (by ID)	no (TID interleave)
Slave cost	tiny	small	small	large	tiny
Typical use	CSRs, slow periph.	MCU system	IP registers	memory, DMA, CPU	datapaths

13.8 Bridges

Real SoCs mix all of these, joined by bridges generated by the interconnect tool or taken from an IP library:

- **AXI -> APB bridge:** accepts AXI(-Lite) transactions, serializes them into APB SETUP/ACCESS pairs, maps PSLVERR to SLVERR, and splits bursts into single transfers. It also decodes PSEL for each peripheral.
- **AXI <-> AHB bridges:** convert burst types (AHB WRAP4 vs. AXI WRAP) and handle AHB's single outstanding transfer.
- **Protocol converters** AXI4 -> AXI4-Lite: split bursts, drop IDs (while returning the right ID to the manager).
- **Width converters, clock converters (async FIFOs per channel) and register slices** are covered in Chapter 14.

13.9 Summary

- APB: SETUP then ACCESS, PREADY for wait states, PSLVERR for errors; commit writes and read side effects only in the completing cycle.
- AHB-Lite overlaps address and data phases; a slave samples address and control when HREADY is high.
- AXI4: five independent valid/ready channels; VALID never waits for READY; bursts FIXED/INCR/WRAP, 4 KB rule; per-ID ordering; enough outstanding transactions to cover latency.
- AXI4-Lite slaves must accept AW and W in either order; AXI4-Stream is a bare handshake plus TLAST/TKEEP framing.
- Bridges and converters glue protocols together; understand their latency and ordering behaviour.

13.10 Exercises

- 1 Extend `apb_regs` so that a write to a new address 0x010 pushes into a 4-entry FIFO and a read from 0x014 pops it. Make sure a wait state cannot cause a double push.
- 2 List the addresses of an AXI WRAP8 burst with 4-byte beats starting at 0x1014, and of an INCR8 burst starting at 0xFF8. Is the INCR burst legal?
- 3 Modify `axil_regs` to accept a new write while the previous B response is still waiting for BREADY (a two-deep response queue). What is the throughput gain?
- 4 Explain with a timing diagram why an AXI master that waits for AWREADY before asserting WVALID can deadlock against a legal slave.
- 5 Design (block diagram and FSM) an AXI4-Lite to APB bridge. Where does the latency go, and how are errors mapped?

14. Interconnect, Arbitration, Memory Maps and NoCs

Chapter 13 defined how two blocks talk. The **interconnect** decides who may talk to whom, when, and through which wires: it decodes addresses, arbitrates between masters, converts widths, clocks and protocols, and - in large chips - becomes a packet-switched network-on-chip. Interconnect is usually generated by a tool (Arm CoreLink, Arteris FlexNoC, open-source generators such as the PULP AXI library), but every RTL designer must understand its building blocks, because performance problems and deadlocks are diagnosed at this level.

14.1 Memory maps and address decoding

The memory map assigns every slave a range of the physical address space. It is a contract between hardware, boot code, drivers and the device tree, and changing it late is expensive.

Region	Base	Size	Slave / notes
Debug module	0x0000_0000	4 KB	RISC-V debug ROM/RAM; accessible in debug mode
Boot ROM	0x0001_0000	64 KB	Reset vector; read-only
On-chip SRAM	0x1000_0000	512 KB	Tightly coupled or via AXI
Peripherals (APB)	0x4000_0000	64 KB	4 KB per peripheral: UART0 0x4000_0000, UART1 0x4000_1000, SPI 0x4000_2000 ...
PLIC / interrupt ctrl	0x0C00_0000	64 MB	Standard RISC-V location
CLINT / timer	0x0200_0000	64 KB	mtime, mtimecmp, msip
Accelerator CSRs	0x5000_0000	64 KB	AXI4-Lite
DDR	0x8000_0000	2 GB	Through the memory controller

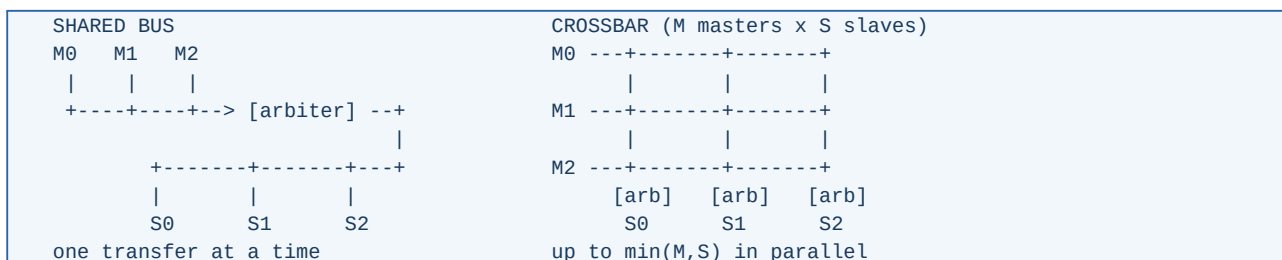
MEMORY-MAP DESIGN RULES

Align every region to its size (a power of two) so decoding is a compare on the upper address bits. Give each peripheral at least 4 KB - the MMU page size - so an OS can map it to one driver with its own permissions. Leave holes for growth. Accesses to holes must get an **error response** (DECERR from a default slave) rather than hang the bus. Generate the C header, device tree and the RTL decoder from one source (Chapter 15).

```
// One-hot address decoder for an APB bridge: 16 peripherals, 4 KB each.
localparam logic [31:0] PERIPH_BASE = 32'h4000_0000;
wire in_periph = (paddr[31:16] == PERIPH_BASE[31:16]); // 64 KB window
always_comb begin
    psel_vec = '0;
    if (psel && in_periph) psel_vec[paddr[15:12]] = 1'b1; // 4 KB slots
end
wire decode_err = psel && !in_periph; // -> PSLVERR
```

Decoding with aligned, power-of-two regions: no adders, no comparators beyond equality on the upper bits.

14.2 Shared bus versus crossbar



A shared bus serializes everything behind one arbiter; a crossbar has an arbiter per slave port and a decoder per master port.

Topology	Bandwidth	Area / wiring	Used for
Shared bus	One transfer at a time	Smallest	APB peripheral segments, tiny MCUs
Full crossbar	Parallel transfers to different slaves	Grows as $M \times S \times \text{width}$; wire-dominated	CPU cluster <-> memories, up to ~8x8
Partial crossbar / sparse	Only needed paths	Moderate	Most AXI interconnects (not every master reaches every slave)
Hierarchical	Local traffic stays local	Moderate	High-speed AXI + bridge to low-speed APB
Network-on-chip	Scales to tens-hundreds of endpoints	Routers + narrow links	Large SoCs, many-core, AI chips

14.3 Arbiters

Whenever several requesters share one resource - a slave port, a memory bank, a bus - an arbiter picks one per cycle (or per transaction). The key properties are **fairness**, **latency bound**, **starvation freedom** and **timing** (arbiters sit on critical paths).

Arbiter	Rule	Pros / cons
Fixed priority	Lowest index wins: $\text{gnt} = \text{req} \ \& \ \sim \text{req}$	Minimal logic; low-priority requesters can starve
Round-robin	Priority rotates to just after the last winner	Fair, worst-case wait $N-1$ grants; slightly more logic
Weighted round-robin	Each requester gets up to W_i consecutive grants or credits per round	Bandwidth shares (e.g. 50/25/25); needs counters
Matrix (least-recently-granted)	$N \times N$ bit matrix: $W[i][j]=1$ means i beats j ; winner's row cleared, column set	True LRG fairness, $O(N^2)$ flops, fast
TDMA / slot-based	Fixed time slots	Deterministic latency (real-time), wastes idle slots
QoS / priority classes	Priority from AxQoS or urgency, round-robin within a class	Serves real-time masters (display) first; needs starvation guards

A round-robin arbiter

The compact implementation uses a **mask** holding ones above the last winner. The fixed-priority picker $x \ \& \ (\sim x + 1)$ isolates the lowest set bit; it is applied first to the masked requests (those 'after' the last winner) and, if none, to all requests.

```

module rr_arb #(parameter int N = 4) (
    input logic      clk, rst_n,
    input logic [N-1:0] req,
    input logic      advance,           // e.g. end of the granted transfer
    output logic [N-1:0] gnt           // one-hot (or zero)
);
    logic [N-1:0] mask;                // 1s at positions ABOVE the last winner
    // fixed-priority (LSB wins) picker: x & ~x isolates the lowest set bit
    function automatic logic [N-1:0] lsb1(input logic [N-1:0] x);
        return x & (~x + 1'b1);
    endfunction
    wire [N-1:0] req_m = req & mask;
    assign gnt = (req_m != 0) ? lsb1(req_m) : lsb1(req);
    always_ff @(posedge clk or negedge rst_n)

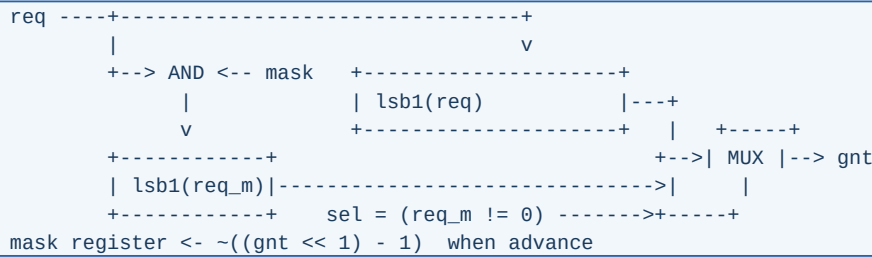
```

```

    if (!rst_n) mask <= '1;
    else if (advance && gnt != 0)
        mask <= ~((gnt << 1) - 1'b1);    // all bits strictly above the one-hot winner
endmodule

```

Mask-based round-robin arbiter. **advance** lets the grant be held for a multi-cycle transaction (for example until WLAST).



Two fixed-priority pickers and a mux. The mask update $\sim((gnt \ll 1) - 1)$ produces ones strictly above the one-hot winner.

```

module tb_rr;
    logic clk = 0, rst_n = 0; logic [3:0] req = 0, gnt, g;
    int cnt [4], wait_c [4], worst = 0, bad = 0;
    always #5 clk = ~clk;
    rr_arb #(N(4)) dut (.clk, .rst_n, .req, .advance(1'b1), .gnt);
    initial begin
        foreach (cnt[i]) begin cnt[i] = 0; wait_c[i] = 0; end
        #12 rst_n = 1;
        req = 4'b1111;
        repeat (6) @(posedge clk) #1 $display("req=%b gnt=%b", req, gnt);
        // random traffic: a requester keeps its request until granted
        repeat (2000) begin
            @(posedge clk) g = gnt;           // the grant this edge commits
            #1;
            if ($countones(g) > 1 || (g & ~req) != 0) bad++;
            for (int i = 0; i < 4; i++) begin
                if (g[i]) begin cnt[i]++; wait_c[i] = 0; req[i] = $urandom % 2; end
                else if (req[i]) begin wait_c[i]++; if (wait_c[i] > worst) worst = wait_c[i]; end
                else req[i] = $urandom % 2;
            end
        end
        $display("grants=%0d/%0d/%0d/%0d  worst wait=%0d cycles  protocol errors=%0d",
            cnt[0], cnt[1], cnt[2], cnt[3], worst, bad);
        $finish;
    end
endmodule

```

The testbench shows the rotation with all four requesting, then runs 2000 cycles of random traffic in which each requester holds its request until granted, checking one-hot grants and the worst wait.

```

req=1111 gnt=0010
req=1111 gnt=0100
req=1111 gnt=1000
req=1111 gnt=0001
req=1111 gnt=0010
req=1111 gnt=0100
grants=504/501/507/485  worst wait=3 cycles  protocol errors=0

```

Grants rotate 1 -> 2 -> 3 -> 0 (requester 0 won the first edge). Under random load the shares are even and the worst-case wait is exactly $N-1 = 3$ cycles - the round-robin bound.

PITFALLS WITH ARBITERS

(1) Changing the grant in the middle of a burst or packet - hold it until the last beat (**advance** = handshake of the last beat). (2) A requester that drops its request before being granted (violating 'hold until served') breaks fairness assumptions and may cause lost transactions. (3) Updating the pointer on every cycle instead of only when a grant is consumed - the arbiter 'skips' requesters. (4) Fixed priority on a path that carries sustained traffic: the lowest priority master starves.

ARBITER TIMING

The masked-priority structure is two N-bit carry chains (the **-x**) plus a mux: fast enough for $N \leq 16$ at GHz rates. For larger N, use a parallel-prefix priority encoder, a tree of 4-input arbiters, or the matrix arbiter, whose grant is a single AND-OR level per requester.

14.4 Starvation, QoS and deadlock at the transaction level

Round-robin is fair per **grant**, not per **byte**: a master issuing 256-beat bursts gets 256x the bandwidth of one issuing single beats. Interconnects therefore add burst-length limits, bandwidth regulators (token buckets per master) and QoS priority driven by **AxQoS**, with aging so that low priority traffic is eventually promoted. Display controllers and audio are the classic latency-critical masters - an underflow is visible or audible.

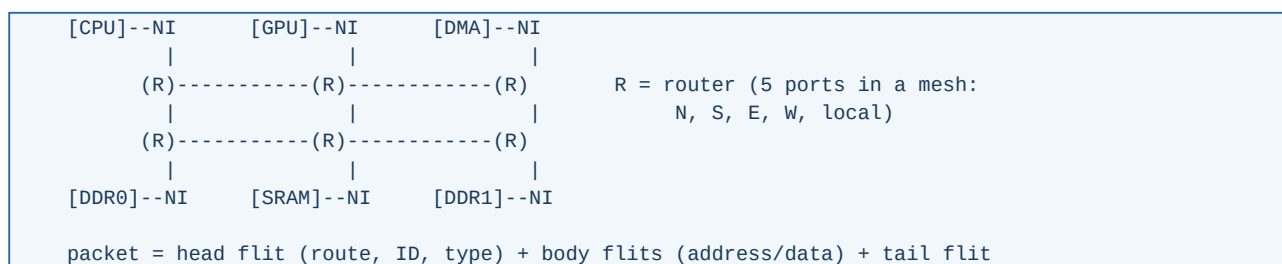
Transaction-level deadlock appears when resources are acquired in different orders, for example: a master holds the W channel of slave A while waiting for AW acceptance at slave B, and another master does the reverse. AXI interconnects avoid this by forwarding write data only in the order write addresses were accepted, and by limiting the number of outstanding transactions per ID to prevent ordering dependencies across slaves.

14.5 Bridges and converters

Component	Function	Design notes
Register slice	Breaks long timing paths on all channels	Full (2-entry skid) slices keep throughput 1/cycle (Chapter 10)
Width converter (upsizer)	e.g. 32-bit master -> 128-bit slave	Packs beats; adjusts AxSIZE/AxLEN; merges WSTRB
Width converter (downsizer)	128-bit master -> 32-bit slave	Splits each beat; may split bursts to respect AxLEN limits
Clock converter	Crosses AXI between clock domains	One async FIFO per channel (AW, W, B, AR, R) - Chapter 11
Protocol converter	AXI4 <-> AXI4-Lite, AXI <-> AHB/APB	Burst splitting, ID reflection, response mapping
Default slave	Responds to unmapped addresses	Returns DECERR; absorbs write data; returns RLAST correctly
Firewall / MPU	Checks AxPROT / master ID against rules	Security: TrustZone-style isolation of secure peripherals

14.6 Networks-on-chip

Beyond roughly a dozen masters, crossbar wiring explodes and a centralized arbiter cannot meet timing across a large die. A **network-on-chip** (NoC) packetizes transactions at **network interfaces** (NIs) that speak AXI to the IP, and routes the packets through small routers connected by point-to-point links.



A 2 x 3 mesh NoC. Each NI converts AXI transactions to packets and back.

Concept	Meaning
Flit	Flow-control unit - the link width worth of a packet; head/body/tail flits
Topology	Ring (simple, long latency), 2-D mesh (regular, scalable), torus, tree, crossbar-of-rings, custom
Routing	Deterministic XY (route X first then Y - deadlock-free on a mesh) or adaptive
Switching	Wormhole: head flit reserves the path, flits follow; small buffers, but a blocked packet holds several links
Flow control	Credit-based: the sender tracks free buffer slots at the receiver and never sends without a credit
Virtual channels (VC)	Several logical queues share one physical link; break routing cycles and separate traffic classes
Deadlock	Cyclic buffer dependency; avoided by routing restrictions (XY / turn model) or VC ordering
Protocol deadlock	Requests blocking responses; avoided by separate request and response networks or VCs

WHY NOCS MATTER TO RTL DESIGNERS

Even if the NoC is generated, you design the IP that sits at its edges. Your block must tolerate high and variable latency, must not create dependencies between its own requests and responses (for example refusing to accept R data until an unrelated AW is accepted), and should issue enough outstanding transactions to hide NoC latency (Chapter 13's Little's-law estimate).

14.7 Cache coherency in one page

When several cores (or a core and a DMA-capable accelerator) cache the same memory, a write by one must invalidate or update the copies of the others. Coherency protocols (MESI, MOESI) keep a state per cache line; interconnect protocols carry the messages:

- **ACE** (AXI Coherency Extensions): adds snoop channels (AC, CR, CD) to AXI so the interconnect can query and invalidate caches. **ACE-Lite** is for IO-coherent masters without caches (a DMA reading coherent data).
- **CHI** (Coherent Hub Interface): a layered, packet-based protocol (request, response, snoop and data channels as flits) designed for NoCs and many-core servers, with a home node managing a snoop filter or directory.
- **TileLink** (RISC-V ecosystem, e.g. Rocket/Chipyard) offers uncached (TL-UL/UH) and cached (TL-C) variants.
- Accelerators that are **not** coherent must rely on software cache maintenance (clean/invalidate) around DMA buffers - a classic source of driver bugs (Chapter 16).

14.8 Summary

- Design memory maps with aligned power-of-two regions, 4 KB granularity for peripherals, and a default slave for holes; generate all views from one source.
- Shared buses are cheap but serial; crossbars give parallelism at M x S wiring cost; NoCs scale to large SoCs.
- Round-robin arbitration bounds wait to N-1 grants; hold grants for whole bursts; add QoS and bandwidth regulation for real-time masters.
- Register slices, width/clock/protocol converters and default slaves are standard interconnect components.

- NoCs move flits through routers with credit flow control; deadlock is prevented by routing rules, virtual channels and separate request/response classes. ACE and CHI add coherency.

14.9 Exercises

- 1 Write a matrix (least-recently-granted) arbiter for $N=4$ and compare its grant sequence with `rr_arb` under the same random traffic.
- 2 Add weights to `rr_arb`: requester i may win up to $W[i]$ consecutive grants while it keeps requesting. Measure the bandwidth shares for weights 4/2/1/1.
- 3 Design the memory map for an SoC with 2 CPUs, 256 KB SRAM, 12 APB peripherals, a 64 KB accelerator register space and 1 GB of DDR. Write the decoder for the APB segment.
- 4 Show a four-router ring with wormhole switching that deadlocks, and explain how adding a second virtual channel with a 'dateline' removes the deadlock.
- 5 A camera writes 1080p60 frames (2 bytes/pixel) into DDR through an interconnect shared with a CPU. What bandwidth does it need, and what QoS mechanism protects it?

15. Control/Status Registers, Interrupts and Register Generation

Control and status registers (CSRs) are the hardware/software interface of every IP block: software configures the block through them, reads its state, and services its interrupts. A typical SoC has tens of thousands of register fields. They are simple individually, but their **access semantics** and **side effects** are where hardware and firmware disagree, so the industry specifies them formally and generates the RTL, the C headers, the documentation and the verification model from one source.

15.1 Register access types

Type	Software write	Software read	Typical use
RW	Stores value	Returns stored value	Configuration, enables, thresholds
RO	Ignored (or error)	Returns hardware value	Status, counters, version/ID
WO	Triggers action / stores	Returns 0	Command registers, FIFO push, 'kick' registers
W1C	1 clears the bit, 0 no effect	Returns value	Interrupt status, sticky error flags
W1S	1 sets the bit, 0 no effect	Returns value	Set half of a set/clear pair; software-triggered interrupts
W0C / W1T	0 clears / 1 toggles	Returns value	Rare; legacy or GPIO toggles
RC (read-clear)	Ignored	Returns value, then clears	Event counters, legacy status
RS / RW1S	Various	Read sets	Semaphores / locks
RW, lock	Writable until a LOCK bit is set	Returns value	Security configuration frozen after boot

WHY W1C INSTEAD OF RW FOR INTERRUPT STATUS

With an RW status register, software clearing bit 2 has to do a read-modify-write: read 0b0110, write back 0b0010. If hardware sets bit 3 between the read and the write, the write of 0b0010 **clears the new event** before anyone saw it. With W1C, software writes 0b0100 - only the bit it means to clear - and cannot destroy events it has not seen. Similarly, set/clear register pairs (W1S at one address, W1C at another) avoid read-modify-write races between CPUs.

PITFALL: READ SIDE EFFECTS

RC registers and FIFO-pop-on-read registers change state when **read**. A debugger memory window, a speculative or prefetching CPU read, or a `printf` of the register in a debug build silently consumes data. Modern guidelines avoid read side effects; if you must have them, map them to non-cacheable, non-speculative device memory, and document them loudly. The same applies to APB/AXI read logic: the side effect must fire once, in the completing cycle.

15.2 Documenting a register map

A register specification lists, for every register: name, offset, width, reset value, and for every field its bit range, access type, reset value, description and side effects. A typical entry:

Bits	Field	Access	Reset	Description
31:4	-	RO	0	Reserved; reads 0, writes ignored
3	OVF	W1C	0	RX FIFO overflowed. Write 1 to clear
2	TXE	W1C	0	TX FIFO became empty

Bits	Field	Access	Reset	Description
1	RXF	W1C	0	RX FIFO reached the threshold in <code>CTRL.RXTH</code>
0	ERR	W1C	0	Framing or parity error

`INT_STATUS` @ offset 0x04 - an interrupt status register.

- **Reserved bits** read as 0 and ignore writes, so software can safely write 0 and future versions can add fields.
- **Reset values** are part of the spec; firmware relies on them.
- **Atomicity**: fields that software updates together must live in the same register; wider-than-bus values (64-bit counters) need a defined read order with a shadow/latch.
- Separate **configuration**, **status** and **interrupt** registers; never mix W1C fields with RW fields in one register if you can avoid it.

15.3 A CSR block with interrupts

The block below implements a common pattern: a raw interrupt status register (W1C) set by hardware events, an enable register (RW), a software set register (WO, for testing interrupt handlers), a read-to-clear event counter, and an interrupt output that is the OR of enabled pending bits. The host interface is a simple one-cycle request, as produced by an APB or AXI-Lite adapter (Chapter 13).

```
// Simple register interface (e.g. behind an APB/AXI-Lite adapter): 1-cycle access.
module irq_csr (
    input  logic      clk, rst_n,
    input  logic      req, we,
    input  logic [4:0] addr,
    input  logic [31:0] wdata,
    output logic [31:0] rdata,          // valid in the cycle after req (registered)
    input  logic [3:0] hw_event,       // 1-cycle event pulses from the hardware
    output logic [3:0] ctrl,
    output logic      irq              // level interrupt to the interrupt controller
);
    localparam logic [4:0] CTRL = 5'h00, STATUS = 5'h04, ENABLE = 5'h08,
                       SET   = 5'h0C, EVTCNT = 5'h10;

    logic [3:0] status, enable;
    logic [7:0] evtcnt;
    wire wr = req & we, rd = req & ~we;

    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) begin
            ctrl <= '0; status <= '0; enable <= '0; evtcnt <= '0; rdata <= '0;
        end else begin
            if (wr && addr == CTRL) ctrl <= wdata[3:0];          // RW
            if (wr && addr == ENABLE) enable <= wdata[3:0];      // RW
            // STATUS: W1C from software, W1S via SET (test hook), set by hardware.
            // Hardware set has priority so an event in the clear cycle is never lost.
            status <= (status & ~(wr && addr == STATUS) ? wdata[3:0] : 4'h0)
                | ((wr && addr == SET) ? wdata[3:0] : 4'h0)
                | hw_event;
            // EVTCNT: RC - counts events, cleared by a read (side effect!)
            if (rd && addr == EVTCNT) evtcnt <= 8'(|hw_event);
            else if (|hw_event) evtcnt <= evtcnt + 1'b1;
            // registered read mux
            if (rd)
                case (addr)
                    CTRL: rdata <= {28'h0, ctrl};
                    STATUS: rdata <= {28'h0, status};
                    ENABLE: rdata <= {28'h0, enable};
                    EVTCNT: rdata <= {24'h0, evtcnt};
                end
        end
endmodule
```

```

        default: rdata <= '0; // SET is WO
    endcase
end
assign irq = |(status & enable);
endmodule

```

A CSR block with RW, W1C, WO(W1S), RC and RO semantics. Hardware set wins over a simultaneous software clear, so no event is ever lost.

```

hw_event[i] -----+
                    v
sw W1C (wdata[i] & wr STATUS) --> NOT --> AND --> OR --> [status[i]] --+
sw W1S (wdata[i] & wr SET) -----> OR          ^          |
                    status[i] -----+ (hold)          |
enable[i] -----> AND <-----+
                    |
                    irq = OR over all i of (status & enable)

```

One bit of the interrupt status register: next = (status & ~clear) | set | hw_event.

```

module tb_csr;
    logic clk = 0, rst_n = 0, req = 0, we = 0, irq;
    logic [4:0] addr = 0; logic [31:0] wdata = 0, rdata; logic [3:0] ev = 0, ctrl;
    always #5 clk = ~clk;
    irq_csr dut (.clk, .rst_n, .req, .we, .addr, .wdata, .rdata, .hw_event(ev), .ctrl, .irq);
    task automatic wr(input [4:0] a, input [31:0] d);
        @(negedge clk) begin req = 1; we = 1; addr = a; wdata = d; end
        @(negedge clk) req = 0;
    endtask
    task automatic rd(input [4:0] a, input string name);
        @(negedge clk) begin req = 1; we = 0; addr = a; end
        @(negedge clk) begin req = 0; $display("%-7s = %h   irq=%b", name, rdata, irq); end
    endtask
    task automatic pulse(input [3:0] e);
        @(negedge clk) ev = e; @(negedge clk) ev = 0;
    endtask
    initial begin
        #12 rst_n = 1;
        wr(5'h08, 32'h5); // enable sources 0 and 2
        pulse(4'b0010); // source 1 fires (masked)
        rd(5'h04, "STATUS");
        pulse(4'b0100); // source 2 fires (enabled) -> irq
        rd(5'h04, "STATUS");
        wr(5'h04, 32'h4); // W1C: clear bit 2 only
        rd(5'h04, "STATUS");
        // race: software clears bit 1 in the same cycle hardware sets it again
        @(negedge clk) begin req = 1; we = 1; addr = 5'h04; wdata = 32'h2; ev = 4'b0010; end
        @(negedge clk) begin req = 0; ev = 0; end
        rd(5'h04, "STATUS");
        rd(5'h10, "EVCNT");
        rd(5'h10, "EVCNT"); // read-to-clear: second read returns 0
        $finish;
    end
endmodule

```

```

STATUS = 00000002   irq=0
STATUS = 00000006   irq=1
STATUS = 00000002   irq=0
STATUS = 00000002   irq=0
EVCNT  = 00000003   irq=0
EVCNT  = 00000000   irq=0

```

Source 1 is recorded but masked (irq=0); source 2 raises irq; the W1C write clears only bit 2; in the race cycle the hardware set of bit 1 wins over the software clear; EVCNT counts 3 events and clears on read.

RAW STATUS, MASKED STATUS

Many IPs expose both **raw** status (`INT_RAW`, events regardless of enable) and **masked** status (`INT_STATUS = RAW & ENABLE`). Drivers read the masked view in the handler; diagnostics and polling-mode drivers read the raw view. Keep the `irq` output a registered signal when it leaves the block, so it is glitch-free and can be synchronized if the interrupt controller is in another clock domain (Chapter 11).

15.4 Interrupts and interrupt controllers

Concept	Meaning
Level-sensitive	The line stays high while the condition is pending; it drops only when software clears the source. Robust; shareable; the norm for SoC peripherals
Edge-triggered	A rising edge (or pulse) records a pending interrupt in the controller. Short pulses must be captured by a flop; lost if two events merge before capture
Mask / enable	Per-source and per-target gating
Pending	Latched request waiting for service
Priority / threshold	Only sources above a target's threshold interrupt it; highest priority is delivered first
Claim / complete	The handler claims the highest-priority pending source (atomically clearing pending) and signals completion when done
Vectored / non-vectored	Jump to a per-source handler vs. one handler that reads the source ID

```

UART.irq --+                                     +-----+
SPI.irq  --+--> [gateway: level/ ]--> pending[i] -> priority[i] |
DMA.irq  --+   [ edge capture  ] | enable[i][target]         |--> meip (hart 0)
GPIO.irq --+                                     | threshold[target]         |--> meip (hart 1)
                                     | claim/complete regs      |
                                     +-----+

RISC-V PLIC: gateways turn each source into at most one pending request;
a target reads CLAIM to get the ID, services the device, writes COMPLETE.

```

Platform-level interrupt controller (PLIC). Arm's **GIC** is conceptually similar: distributor (sources, priority, routing) plus CPU interfaces (acknowledge = claim, end-of-interrupt = complete).

- RISC-V also has the **CLINT/ACLINT** for timer (`mtip`) and software (`msip`) interrupts, and the newer **AIA** (APLIC + IMSIC) with message-signalled interrupts.
- Message-signalled interrupts (MSI)**: a device writes to a doorbell address instead of driving a wire - no wires across the chip, naturally ordered after the data it announces (PCIe uses MSI).
- Interrupt service sequence** for a level interrupt: claim -> read device status -> handle -> W1C the device status -> complete. Completing before clearing the device causes an immediate spurious re-entry.
- Interrupts crossing clock domains: level interrupts go through a 2FF synchronizer; pulses need a toggle synchronizer (Chapter 11).

15.5 Register generation: SystemRDL and IP-XACT

Hand-writing thousands of registers in RTL, C headers, documentation and testbench models guarantees that the four drift apart. Instead, registers are described once in a machine-readable format and everything is generated:

```
// SystemRDL 2.0 (Accellera) description of the CSR block above
addrmap irq_csr {
    default regwidth = 32;
    reg { field { sw = rw; hw = r; } CTRL[3:0] = 0; } ctrl @ 0x00;
    reg {
        field { sw = rw; hw = w; onwrite = woclr; intr; } EV[3:0] = 0; // intr: sticky
    } status @ 0x04;
    reg { field { sw = rw; hw = r; } EN[3:0] = 0; } enable @ 0x08;
    reg { field { sw = w; hw = r; singlepulse; } SET[3:0] = 0; } set @ 0x0C;
    reg { field { sw = r; onread = rclr; counter; } CNT[7:0] = 0; } evtcnt @ 0x10;
};
```

The same register map in SystemRDL. Properties such as **onwrite = woclr** (W1C), **onread = rclr** (RC), **counter** and **intr** (a sticky field set by hardware) define the semantics.

Format / tool	What it is
SystemRDL 2.0	Accellera register description language; compact, human-writable; open-source compiler systemrdl-compiler and generators (PeakRDL: RTL, C headers, HTML, UVM)
IP-XACT (IEEE 1685)	XML standard for IP metadata: ports, bus interfaces, memory maps, registers; used for SoC assembly and tool exchange
Spreadsheets / YAML / JSON	Common in-house front ends (e.g. OpenTitan's reggen uses Hjson), converted to the formats above
Commercial tools	Agnisys IDesignSpec, Magillem, Semifore CSRCompiler

```
+--> RTL register block (SV, with bus adapter)
SystemRDL / IP-XACT +--> C header: #define UART_INT_STATUS_OFFSET 0x04 ...
(single source) -----> HTML/PDF register documentation
+--> UVM register model (RAL) for the testbench
+--> device tree / Rust / Python access layers
```

Single source of truth for registers.

15.6 Verification tie-in: the UVM register layer

The generated **UVM Register Abstraction Layer (RAL)** model mirrors every register and field, with access policies ("RW", "W1C", "RC" ...) and reset values. Tests then read and write registers by name, independent of the bus protocol, and built-in sequences check the whole map automatically: reset values (**uvm_reg_hw_reset_seq**), access policies and bit-bashing (**uvm_reg_bit_bash_seq**), and address aliasing. A predictor keeps the model's mirror in sync with observed bus traffic, so every read can be checked. Chapter 24 builds this in detail.

INTERVIEW FAVOURITE: W1C + HARDWARE SET IN THE SAME CYCLE

State the policy explicitly in the spec (normally **set wins**, as in **irq_csr**), and make sure the RAL predictor knows it - otherwise the model predicts 0, the hardware returns 1, and the testbench reports a false mismatch. Such cases are marked 'volatile' in the model.

15.7 Summary

- Access types (RW, RO, WO, W1C, W1S, RC ...) define both the software view and the hardware logic; W1C and set/clear pairs avoid read-modify-write races.
- Avoid read side effects; commit any side effect exactly once, in the completing bus cycle.
- Interrupt status = sticky W1C bits set by hardware (set wins), gated by enables, ORed into a registered level interrupt.
- Interrupt controllers (PLIC, GIC) add gateways, priorities, thresholds and claim/complete; MSIs replace wires with writes.
- Describe registers once in SystemRDL or IP-XACT and generate RTL, headers, documentation and the UVM RAL model.

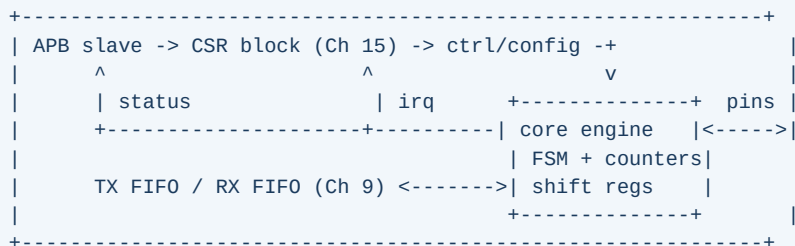
15.8 Exercises

- 1 Add a 64-bit free-running counter to `irq_csr`, readable over a 32-bit bus. Design the latch-on-read-of-low-word mechanism so software always gets a coherent value.
- 2 Change `irq_csr` so that STATUS has edge-capture semantics for level inputs: a source that stays high must not re-set the bit after software clears it until it goes low and high again.
- 3 Write the C interrupt handler for `irq_csr` and a PLIC, in the correct claim/clear/complete order. What goes wrong if the order is changed?
- 4 Write SystemRDL for the APB register block of Chapter 13 (`apb_regs`) and run it through an open-source generator if you have one available.
- 5 List three ways a debugger can corrupt device state through read side effects, and how the register design can prevent each.

16. Peripherals, DMA and Processor Integration

This chapter assembles everything in Part III into a working system. Peripherals are where most RTL engineers start their careers - they are small, self-contained, and exercise every skill: FSMs, counters, CDC, CSRs, interrupts and bus interfaces. Then we move data without the CPU (DMA), integrate a RISC-V core, and look at caches and the memory hierarchy from the RTL designer's side.

Common peripheral skeleton



Almost every peripheral is a bus slave, a register block, FIFOs, and a protocol engine with an interrupt output.

16.1 GPIO

General-purpose I/O is the simplest peripheral but has real subtleties at the pin boundary:

- Registers: **DATA_OUT** (RW), **DIR/OUT_EN** (RW), **DATA_IN** (RO), plus **set/clear/toggle** aliases so different drivers can change different pins without read-modify-write races (Chapter 15).
- Inputs are **asynchronous**: every input passes through a 2FF synchronizer before use (Chapter 11), and optionally a debounce filter (counter-based: accept a new level only after it is stable for N samples).
- Interrupts per pin: rising, falling, both edges, or level - edge detection on the **synchronized** value, W1C status.
- The pad itself (output driver, input buffer, pull-ups, drive strength, pin mux) is an instantiated I/O cell in the chip top level, controlled by pad-control registers.

16.2 Timers, PWM and watchdog

A timer is a counter plus comparators. The same structure gives periodic interrupts, input capture (latch the count on an external edge), output compare and **pulse-width modulation**, used for motor control, LED dimming and switching power converters.

```

module pwm #(parameter int W = 8) (
    input  logic      clk, rst_n, en,
    input  logic [W-1:0] period,    // counter wraps at period (PWM period = period+1 clks)
    input  logic [W-1:0] duty,      // output high while cnt < duty
    output logic      pwm_out,
    output logic      wrap          // 1-cycle pulse per period (e.g. timer interrupt)
);
    logic [W-1:0] cnt, duty_sh;    // duty is shadowed: updates only at period boundary
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) begin cnt <= '0; duty_sh <= '0; pwm_out <= 0; wrap <= 0; end
        else if (en) begin
            wrap <= (cnt == period);
            if (cnt == period) begin cnt <= '0; duty_sh <= duty; end
            else
                cnt <= cnt + 1'b1;
            pwm_out <= (cnt == period) ? (duty != 0) : (cnt + 1'b1 < duty_sh);
        end else begin
            cnt <= '0; pwm_out <= 0; wrap <= 0;
        end
endmodule

```

A PWM generator. The duty cycle is **shadowed**: a new value written mid-period takes effect only at the next period boundary, so the output never produces a truncated or double pulse.

```

module tb_pwm;
    logic clk = 0, rst_n = 0, en = 0, o, wrap;
    logic [7:0] duty = 3;
    string s = "";
    always #5 clk = ~clk;
    pwm #(.W(8)) dut (.clk, .rst_n, .en, .period(8'd9), .duty, .pwm_out(o), .wrap);
    initial begin
        #12 rst_n = 1; en = 1;
        @(posedge wrap) #1; // first cycle of a period
        for (int i = 1; i <= 30; i++) begin
            s = {s, o ? "#" : "_"};
            if (i % 10 == 0) s = {s, " "}; // one period per group
            if (i == 15) duty = 7; // change duty mid-period
            @(posedge clk) #1;
        end
        $display("pwm_out: %s", s);
        $finish;
    end
endmodule

```

The testbench prints one character per clock, grouped by 10-cycle periods, and changes the duty from 3 to 7 in the middle of the second period.

```
pwm_out: ###_____###_____#####_____
```

Period 1 and 2: 3 of 10 cycles high. The change made in the middle of period 2 only appears in period 3 (7 of 10) thanks to the shadow register.

PITFALL: UNSHADOWED COMPARE REGISTERS

Without `duty_sh`, lowering the duty from 7 to 3 while the counter is at 5 would end the pulse early and raising it would extend it - a glitch that a motor driver or DC/DC converter may not tolerate. The same applies to timer reload values: load them at the wrap.

The **watchdog timer** is a down-counter that resets the system when it reaches zero unless software 'kicks' (reloads) it in time. RTL guidelines for watchdogs: clock it from an always-on, independent oscillator so a stuck PLL cannot freeze it; require a **key sequence** (e.g. write 0x5555 then 0xAAAA) to kick or reconfigure it so runaway code cannot kick it accidentally; lock its configuration after boot; support a **window** mode where kicking too early is also a fault; and record 'watchdog' as the reset cause (Chapter 12). A pre-timeout interrupt gives software a chance to log state.

16.3 UART

A UART sends asynchronous serial frames: idle high, one start bit (0), 5-9 data bits LSB first, optional parity, and 1-2 stop bits (1). There is no clock wire; both ends agree on a baud rate within ~2-3%.

```
idle ~~~~\_ D0 _ D1 _ D2 _ D3 _ D4 _ D5 _ D6 _ D7 _ (P) _/~~~ stop ~~~ idle
      start
RX samples: detect falling edge, wait 8 ticks (x16 oversampling) to
mid-start, check still 0, then sample every 16 ticks (majority of 3)
```

UART frame, 8N1: 10 bit-times per byte.

- **Baud generator:** fractional divider producing a 16x (or 8x) oversampling tick: $\text{div} = f_{\text{clk}} / (16 \times \text{baud})$, with a fractional part to keep error below 1%.
- **TX:** FSM IDLE -> START -> DATA(8) -> PARITY -> STOP, a shift register, and a TX FIFO.
- **RX:** the RX pin is asynchronous - 2FF synchronize it first; false-start rejection; majority vote of 3 samples around mid-bit; framing error if the stop bit is 0; overrun if the RX FIFO is full; break detection (line low for a whole frame).

- **CSRs and interrupts:** TX empty / below threshold, RX above threshold, RX timeout (data waiting, line idle for N characters), errors - all W1C. Chapter 28 builds a complete APB UART.

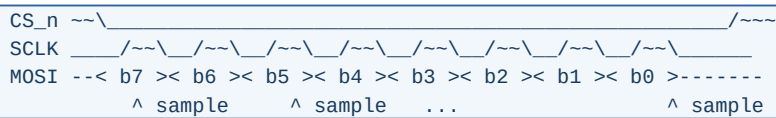
16.4 SPI

SPI is a synchronous, full-duplex serial bus: the master drives **SCLK**, **MOSI** and a chip select per slave (**CS_n**); the slave drives **MISO**. Each clock shifts one bit each way, so a transfer is really an exchange between two shift registers. Four modes define clock polarity and phase:

Mode	CPOL (idle SCLK)	CPHA	Data sampled on	Data changes on
0	0	0	rising edge	falling edge (first bit at CS fall)
1	0	1	falling edge	rising edge
2	1	0	falling edge	rising edge (first bit at CS fall)
3	1	1	rising edge	falling edge

```
// SPI master, mode 0 (CPOL=0, CPHA=0), MSB first, 8-bit frames.
// SCLK = clk / (2*(DIV+1)). MOSI changes on falling SCLK, MISO sampled on rising SCLK.
module spi_master #(parameter int DIV = 1) (
    input logic      clk, rst_n,
    input logic      start,           // pulse: begin a frame with tx_byte
    input logic [7:0] tx_byte,
    output logic [7:0] rx_byte,
    output logic      busy, done,     // done: 1-cycle pulse at end of frame
    output logic      sclk, mosi, cs_n,
    input logic      miso
);
    logic [$clog2(DIV+1)-1:0] div_cnt;
    logic [7:0] sh;                  // one shift register for TX and RX
    logic [3:0] bits;                // rising edges still to come
    logic      miso_q;               // sample taken on the rising edge
    wire       tick = (div_cnt == DIV); // half-period elapsed
    assign mosi = sh[7];
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) begin
            div_cnt <= '0; sh <= '0; bits <= '0; busy <= 0; done <= 0;
            sclk <= 0; cs_n <= 1; rx_byte <= '0; miso_q <= 0;
        end else begin
            done <= 0;
            if (!busy) begin
                div_cnt <= '0;
                if (start) begin sh <= tx_byte; bits <= 4'd8; busy <= 1; cs_n <= 0; end
            end else begin
                div_cnt <= tick ? '0 : div_cnt + 1'b1;
                if (tick) begin
                    if (!sclk && bits != 0) begin           // rising edge: sample MISO
                        sclk <= 1; miso_q <= miso; bits <= bits - 1'b1;
                    end else if (sclk) begin                // falling edge: next MOSI bit out,
                        sclk <= 0;                          // captured MISO bit in
                        sh <= {sh[6:0], miso_q};
                    end
                    if (!sclk && bits == 0) begin           // all 8 bits shifted: end frame
                        busy <= 0; cs_n <= 1; done <= 1; rx_byte <= sh;
                    end
                end
            end
        end
    end
endmodule
```

A mode-0 SPI master shift engine. One 8-bit shift register serves both directions: MOSI is its MSB; MISO is captured on the rising SCLK edge and shifted in on the falling edge.



Mode 0: the first bit is valid when CS_n falls; the slave samples on rising edges, and both sides change data on falling edges.

```

module tb_spi;
  logic clk = 0, rst_n = 0, start = 0, busy, done, sclk, mosi, cs_n, miso;
  logic [7:0] tx = 0, rx, slave_tx, slave_rx;
  always #5 clk = ~clk;
  spi_master #(.DIV(1)) dut (.clk, .rst_n, .start, .tx_byte(tx), .rx_byte(rx),
                           .busy, .done, .sclk, .mosi, .cs_n, .miso);
  // behavioural mode-0 SPI slave: drive on falling edge / CS fall, sample on rising
  logic [7:0] sreg;
  assign miso = cs_n ? 1'bz : sreg[7];
  always @(negedge cs_n) sreg = slave_tx;
  always @(posedge sclk) if (!cs_n) slave_rx = {slave_rx[6:0], mosi};
  always @(negedge sclk) if (!cs_n) sreg = {sreg[6:0], 1'b0};
  task automatic xfer(input [7:0] m, input [7:0] s);
    slave_tx = s;
    @(negedge clk) begin tx = m; start = 1; end
    @(negedge clk) start = 0;
    @(posedge done);
    #1 $display("master sent %h, slave got %h | slave sent %h, master got %h",
               m, slave_rx, s, rx);
  endtask
  initial begin
    #12 rst_n = 1;
    xfer(8'h3C, 8'hA5);
    xfer(8'h81, 8'h7E);
    $finish;
  end
endmodule

```

A behavioural mode-0 slave exchanges a byte with the master; both directions are checked.

```

master sent 3c, slave got 3c | slave sent a5, master got a5
master sent 81, slave got 81 | slave sent 7e, master got 7e

```

WHY SPI MASTERS GET HARD AT HIGH SPEED

At tens of MHz the MISO round trip (SCLK out through the pad, board trace, slave clock-to-out, trace, pad back in) can exceed half an SCLK period. Fast controllers (QSPI flash, 100+ MHz) therefore sample MISO with a delayed or feedback clock, add programmable sampling delay, or use DDR sampling - and the I/O timing is constrained in SDC with generated clocks on the SCLK pin (Chapter 18).

16.5 I2C

I2C uses two open-drain wires, **SCL** and **SDA**, with pull-up resistors: any device can pull a line low, nobody drives it high. That makes multi-master arbitration and clock stretching possible.

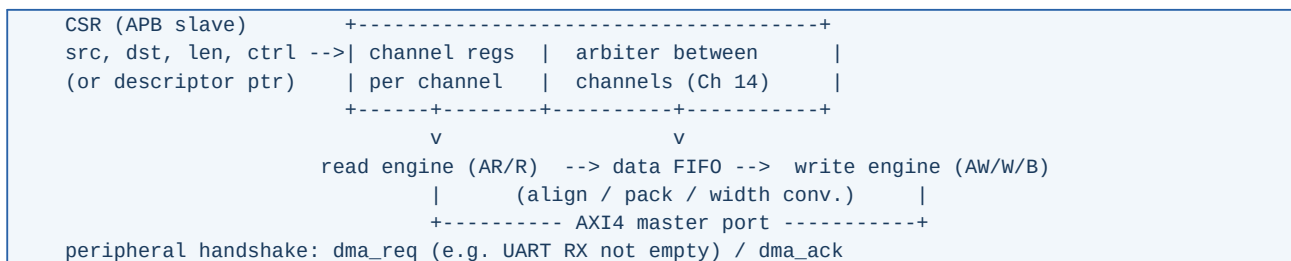
- **START**: SDA falls while SCL is high. **STOP**: SDA rises while SCL is high. Otherwise SDA changes only while SCL is low.
- **Frame**: START, 7-bit address + R/W bit, ACK (receiver pulls SDA low in the 9th clock), data bytes each followed by ACK/NACK, STOP (or repeated START).
- **Clock stretching**: a slave holds SCL low to pause the master; the master must monitor SCL, not assume its own timing.
- **Arbitration**: a master that drives 1 but reads 0 on SDA has lost and backs off - no data is corrupted.

- RTL: pins are implemented as **oe** controls of open-drain pads (`assign sda_oe = ~sda_out;` - drive low only); inputs are 2FF synchronized and glitch-filtered (50 ns spike suppression in fast mode); a bit-level FSM generates SCL phases from a divider and a byte-level FSM sequences address, data and ACK.
- Speeds: 100 kHz standard, 400 kHz fast, 1 MHz fast-plus; I3C is the faster, push-pull-capable successor.

	UART	SPI	I2C
Wires	2 (TX, RX)	3 + 1 CS per slave	2 (shared)
Clocking	Asynchronous, agreed baud	Master clock	Master clock, slave can stretch
Duplex	Full	Full	Half
Addressing	Point-to-point	Chip select	7/10-bit address
Typical speed	115.2 kbit/s - few Mbit/s	1-100+ MHz	100 kHz - 1 MHz
RTL difficulty	RX sampling / oversampling	I/O timing at speed	Open-drain, stretching, arbitration

16.6 DMA controllers

A DMA (direct memory access) controller is a bus **master** that moves data so the CPU does not have to. The CPU programs a transfer, the DMA executes it with long bursts and many outstanding transactions, and interrupts the CPU on completion.



Block diagram of a multi-channel DMA engine.

Feature	What it means for the RTL
Memory-to-memory	Read bursts fill the FIFO, write bursts drain it; keep both engines busy with several outstanding bursts
Peripheral flow control	Transfers paced by <code>dma_req</code> from a peripheral FIFO level; FIXED-address bursts to the FIFO register
Descriptors	The transfer description (src, dst, length, control, next pointer) lives in memory; the DMA fetches it
Scatter-gather	A linked list of descriptors - e.g. one network packet in several buffers - processed without CPU intervention
Alignment	Unaligned src/dst need a byte-shifting barrel and WSTRB generation; the 4 KB boundary splits bursts
Completion	Per-descriptor interrupt flag, write-back of status into the descriptor, error reporting (SLVERR/DECERR)

```

// A typical scatter-gather descriptor (in memory, 32-byte aligned)
typedef struct packed {
    logic [63:0] next;      // address of next descriptor, 0 = end of list
    logic [63:0] src;       // source address
    logic [63:0] dst;       // destination address
    logic [31:0] len;       // bytes
    logic [31:0] ctrl;      // [0] irq_on_done [1] src_fixed [2] dst_fixed
} dma_desc_t;             // ... status written back here on completion
  
```

Descriptor layout. The DMA reads it with one burst, executes it, optionally writes status back, and follows `next`.

PITFALL: DMA AND CACHES

If the CPU has a write-back data cache and the DMA is not coherent (Chapter 14), a buffer the CPU just wrote may still sit in the cache when the DMA reads DDR, and a buffer the DMA just wrote may be shadowed by stale cache lines. Drivers must clean the cache before a DMA read of memory and invalidate after a DMA write - or the hardware must be IO-coherent (ACE-Lite). A second classic: raising the completion interrupt before the last write response (B) has returned - the CPU may read data that has not landed yet.

16.7 Integrating a RISC-V core

Open RISC-V cores (Ibex, CV32E40P, CVA6, Rocket, VexRiscv, PicoRV32) make it practical to build a complete SoC in RTL. Integration is mostly about the environment around the core:

Item	What to provide
Memory interfaces	Instruction and data ports (AXI, AHB, OBI or TileLink depending on the core), connected through the interconnect to ROM, SRAM and peripherals
Boot ROM and reset vector	The core fetches its first instruction from a parameter/strap address (<code>boot_addr</code>); the ROM initializes the stack, clocks and memory and loads or jumps to the application
Memory map	Must match the linker script and device tree; mark device regions non-cacheable / non-idempotent (PMA)
Interrupts	Machine timer (<code>mtip</code>) and software (<code>msip</code>) from the CLINT; external (<code>meip</code>) from the PLIC (Chapter 15)
Debug	RISC-V Debug Spec: a Debug Module (DM) on the bus with abstract commands, program buffer and system bus access, reached from JTAG through a Debug Transport Module (DTM); <code>debug_req</code> into the core; works with OpenOCD + GDB
Clocks/resets	Core clock gating in WFI, reset released last (Chapter 12), <code>ndmreset</code> from the DM
Other	Hart ID strap, PMP configuration, performance counters, optional FPU / custom instruction interface

```
// Integration sketch (port names follow a typical small core; check yours)
riscv_core #(.BOOT_ADDR(32'h0001_0000)) u_cpu (
    .clk_i (clk_cpu), .rst_ni (rst_cpu_n),
    .hart_id_i (32'd0),
    // instruction and data ports -> interconnect masters 0 and 1
    .instr_req_o (ireq), .instr_addr_o (iaddr), .instr_rdata_i (irdata), ...
    .data_req_o (dreq), .data_we_o (dwe), .data_addr_o (daddr), ...
    // interrupts
    .irq_timer_i (mtip), .irq_software_i (msip), .irq_external_i (meip),
    .debug_req_i (dm_debug_req)
);
```

Not a runnable listing - each core has its own port list - but the connections every integration needs.

16.8 Caches and the memory hierarchy, from the RTL side

A cache keeps recently used lines of memory in fast SRAM next to the core. For an RTL designer it is a datapath (tag RAM, data RAM, comparators, way mux) plus a controller FSM (lookup, miss, refill, write-back, eviction).

```

address = | tag | index | offset |
          |         (selects byte in line)
          v
+-----+-----+-----+-----+
| way0: V D tag | way1: V D tag | | data RAM way0 | way1 |
+-----+-----+-----+-----+
| == addr.tag ? | ==          | |          |
+--> hit0      +--> hit1 -----> way mux ----> rdata
miss -> FSM: pick victim (LRU) -> write back if D=1 -> refill (WRAP burst)

```

A 2-way set-associative cache. The critical path is tag RAM read -> compare -> way select.

Level	Typical size / latency	RTL notes
Registers	32 x 64 bit / 0 cycles	Register file, Chapter 9
L1 I/D cache	16-64 KB / 1-4 cycles	SRAM macros, often virtually indexed physically tagged; WRAP bursts for refills
L2 / LLC	256 KB - tens of MB / 10-40 cycles	Banked, shared, often inclusive with a snoop filter
On-chip SRAM / scratchpad	KB-MB / few cycles	Software-managed; common in accelerators and MCUs
DDR / LPDDR	GB / 100+ ns	Memory controller + PHY IP; long latency needs many outstanding transactions

- **Write policies:** write-back (dirty bit, evictions write whole lines) vs. write-through (simpler, more bus traffic); write-allocate or not.
- **Non-cacheable regions:** device registers must never be cached - the PMA/MPU marks them, and the bus carries it in **AxCACHE**.
- **Accelerators** (Chapter 29) usually prefer software-managed scratchpads with DMA over caches: deterministic latency, no tag overhead, and double-buffering hides DDR latency.

16.9 Putting it together: an example SoC

```

+----- always-on domain -----+
XTAL 24MHz ->| POR | reset ctrl (Ch 12) | PLL | clk gen/mux (Ch 11) |
+-----+-----+-----+-----+
JTAG --> [DTM]--[Debug Module]          | clocks / resets
          |                               v
+-----+-----+-----+-----+
| RISC-V | I/D |          AXI4 crossbar / NoC (Ch 14)          |
| core   |=====>| M: cpu-I, cpu-D, DMA, debug   S: ROM SRAM DDR APB |
| +L1$   | +---+-----+-----+-----+-----+
+-----+-----+-----+-----+
^ irq      [Boot ROM] [SRAM] [DDR ctrl] [AXI->APB] [ML accel]
|          +-----+-----+-----+-----+
[PLIC]<--- irq s ---+          + PHY          | AXI-Lite CSR
[CLINT]          |          |          |          | AXI-S datapath
                  +-----+-----+-----+-----+ (Ch 29)
                  +-----+-----+-----+-----+
                  | UART | SPI | I2C | GPIO | TIMER/PWM | WDT
[DMA] AXI master  (Ch 13-16) +-----+-----+-----+

```

A small but complete SoC. Every box is a topic of Part III; Parts IV and V turn it into silicon and prove it works.

SoC integration checklist

- ☐ Every clock-domain crossing identified and synchronized; CDC tool clean
- ☐ One reset synchronizer per clock domain; reset sequence documented
- ☐ Memory map single-sourced; default slave returns DECERR for holes
- ☐ All bus ports pass protocol-checker assertions
- ☐ Every interrupt: W1C status, enable, routed to PLIC, synchronized if needed
- ☐ Boot ROM address = core boot vector; debug module reachable over JTAG
- ☐ Device regions non-cacheable; DMA buffers coherent or cache-managed

- ☐ Register map documentation, C headers and RAL model generated from RDL

16.10 Summary

- Peripherals share one skeleton: APB slave + CSRs + FIFOs + a protocol engine + interrupts; asynchronous pins are synchronized.
- Timers/PWM need shadowed compare values; watchdogs need an independent clock, key sequences and a recorded reset cause.
- UART: oversampled asynchronous frames; SPI: shift-register exchange with CPOL/CPHA modes; I2C: open-drain with START/STOP, ACK, clock stretching and arbitration.
- DMA engines are AXI masters with descriptors and scatter-gather; watch cache coherency and completion ordering.
- Integrating a RISC-V core means memory map, boot ROM, CLINT/PLIC interrupts, debug module and reset order; caches trade tag logic and complexity for latency.

16.11 Exercises

- 1 Extend `spi_master` with CPOL and CPHA parameters supporting all four modes, and verify each mode against a matching slave model.
- 2 Design the RX half of a UART with 16x oversampling, false-start rejection and majority voting. Test it with a transmitter whose baud rate is 2% fast and 2% slow.
- 3 Add a `capture` input to `pwm` that latches the counter value on a rising edge of an asynchronous external signal. What CDC measures are needed?
- 4 Write the FSM for a scatter-gather DMA channel: fetch descriptor, issue read bursts, issue write bursts, write back status, follow `next`. Where do errors abort the chain?
- 5 Draw the address decode for the example SoC and write the linker script MEMORY section that matches it.
- 6 For a 32 KB, 4-way cache with 64-byte lines and 32-bit addresses, compute the tag, index and offset widths and the tag RAM size.

PART IV

Implementation-Aware RTL

What happens to your RTL after you commit it: synthesis into standard cells, static timing analysis and the SDC that drives it, low-power techniques and UPF power intent, design for test from scan chains to JTAG, and the power-performance-area trade-offs and hand-off package that carry a design to tapeout.

17. Logic Synthesis: What Your RTL Becomes

Up to now the book has treated RTL as a description of behaviour that a simulator executes. From this chapter on we look at the other consumer of your code - the **synthesis tool** - which reads exactly the same text and turns it into a netlist of standard cells: NAND gates, flip-flops, multiplexers and adders drawn from a library characterised by the foundry. The simulator and the synthesis tool do not interpret your code in the same way, and almost every expensive RTL bug in industry lives in the gap between them.

A good RTL designer can look at any **always** block and sketch the gates it will become, estimate their delay to within a factor of two, and predict which lines will produce warnings. This chapter builds that skill: the steps inside a synthesis tool, the library it maps to, what each coding idiom infers, what synthesis silently ignores, and how to read the reports that tell you whether your design is any good.

17.1 The synthesis pipeline

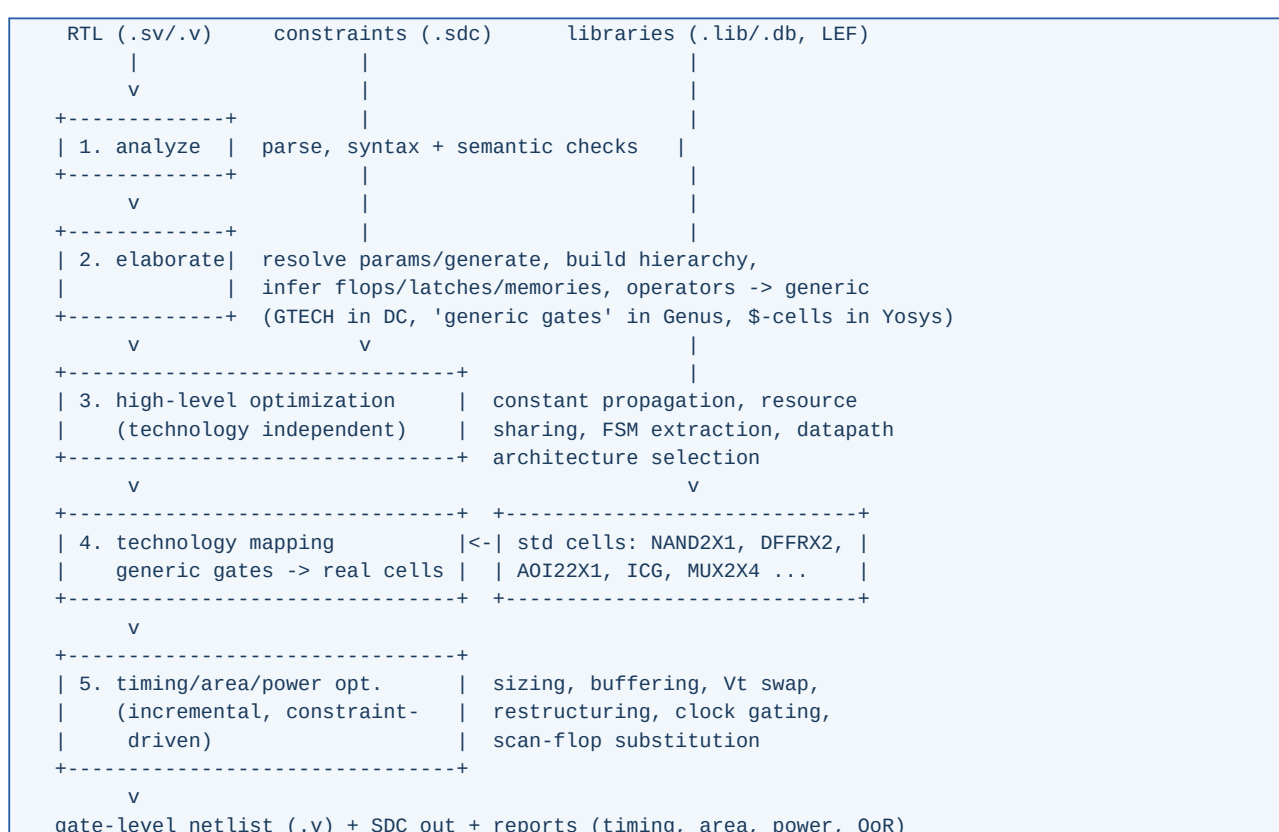


Figure 17.1 - The stages every synthesis tool goes through. Steps 3-5 are iterated and interleaved in modern tools, but the conceptual split is the same in Design Compiler, Genus and Yosys.

Step	What happens	What you see if your RTL is wrong
Analyze	Lexing and parsing of each file; per-file semantic checks; results stored in a work library	Syntax errors, undeclared identifiers (with <code>default_nettype none</code>), illegal SV constructs for that tool
Elaborate	Top-down instantiation: parameters resolved, <code>generate</code> expanded, every <code>always</code> block analysed for its storage type; arithmetic becomes generic operators	'Latch inferred', 'multiple drivers', 'width mismatch', 'unresolved module', 'combinational loop'
High-level optimization	Constants propagated, unused logic removed, common subexpressions shared, adder/multiplier architectures chosen (ripple, carry-lookahead, Booth/Wallace)	Registers 'removed because constant' or 'unloaded' - often a sign of a missing connection

Step	What happens	What you see if your RTL is wrong
Technology mapping	Generic logic covered by cells from the target library, minimising a cost function (delay, area)	Unmapped cells (a <code>dont_use</code> list too strict); huge area from a <code>/</code> or <code>%</code> by a variable
Optimization	Constraint-driven: upsizing cells on critical paths, buffering high fanout, restructuring, swapping Vt flavours, inserting ICGs, replacing flops with scan flops	Negative slack that the tool cannot fix - the fix is in the RTL (Chapter 18)

SYNTHESIS IS CONSTRAINT-DRIVEN

Without an SDC file a synthesis tool has no notion of 'fast enough' and will produce a small, slow netlist. With a clock period it trades area for speed only on paths that need it. This is why the same RTL can produce a 20k-gate block at 200 MHz and a 35k-gate block at 800 MHz, and why area numbers quoted without a frequency are meaningless.

17.2 Standard-cell libraries: what the tool maps to

The target of mapping is a **standard-cell library**: a few hundred to a few thousand pre-designed, pre-characterised cells of equal height that abut in rows. The foundry or an IP vendor provides each cell in several views; synthesis reads the **Liberty** (`.lib`, compiled to `.db` for Synopsys tools) timing and power model, and physically-aware synthesis additionally reads **LEF** (abstract layout) and technology files.

```
cell (NAND2X1) {
  area : 0.532 ;
  pin (A) { direction : input ; capacitance : 0.0012 ; }
  pin (B) { direction : input ; capacitance : 0.0013 ; }
  pin (Y) {
    direction : output ;
    function : "!(A & B)" ;
    timing () {
      related_pin : "A" ;
      timing_sense : negative_unate ;
      cell_rise (delay_template_5x5) { /* NLDM table */
        index_1 ("0.01, 0.03, 0.08, 0.20, 0.50"); /* input slew, ns */
        index_2 ("0.001, 0.004, 0.01, 0.03, 0.08"); /* load, pF */
        values ("...", "...") ; /* 5 x 5 delays */
      }
      rise_transition (delay_template_5x5) { ... }
    }
  }
  leakage_power () { when : "!A!B" ; value : 1.8 ; }
}
```

An abridged Liberty cell. Delay is a lookup table indexed by input slew and output load - which is why the same gate is faster when it drives less. Advanced nodes add CCS/ECSM current-source models and LVF statistical tables, but the idea is unchanged.

Library dimension	Typical choices	Why an RTL designer cares
PVT corner	Process ss/tt/ff (plus ssg/ffg), voltage e.g. 0.72/0.80/0.88 V, temperature -40/25/125 degC; file names like <code>ss_0p72v_125c</code>	Setup is signed off at the slow corner, hold at the fast corner; at advanced nodes temperature inversion can make the cold corner the slow one
Threshold voltage (Vt)	ULVT, LVT, SVT/RVT, HVT (and sometimes eLVT)	Lower Vt is faster but leaks exponentially more; tools use HVT by default and swap LVT onto critical paths
Drive strength	X1, X2, X4 ... X16 (or D1, D2 ...)	Sizing fixes load-dominated delay; very high fanout needs a buffer tree, not a bigger gate
Track height	e.g. 6T, 7.5T, 9T, 12T libraries	Short cells are dense and low power; tall cells are fast

Library dimension	Typical choices	Why an RTL designer cares
Channel length	C-poly variants (e.g. +4 nm) in some nodes	Another leakage/speed knob, used like Vt swapping
Special cells	ICG (clock gate), retention flops, level shifters, isolation, always-on buffers, tie-hi/lo, spare and ECO cells, synchronizer flops	Chapters 11, 19, 20 and 21 instantiate or rely on them

THE NAND2 GATE EQUIVALENT (GE)

Area is often quoted in **gate equivalents**: total cell area divided by the area of a NAND2X1 in the same library. It lets you compare designs across nodes. Rough rules of thumb: a D flip-flop is 4.5-6 GE, a scan flop with reset 6-8 GE, a full adder about 6-8 GE, and an N x N array multiplier roughly $6-8 \times N^2$ GE before optimisation.

17.3 What infers what

Synthesis tools recognise a small set of **templates**. Code that matches a template maps cleanly; code that does not either fails or, worse, maps to something you did not intend. The following module exercises the most common templates in one place:

```
module infer_zoo (
    input wire      clk, rst_n, en, sel,
    input wire [7:0] a, b,
    output reg [7:0] q_rst,    // DFF with async reset
    output reg [7:0] q_en,     // DFF + enable (mux feedback or ICG)
    output wire [7:0] m,      // 2:1 mux
    output wire [8:0] s,      // adder (carry out)
    output wire      lt      // comparator
);
    assign m = sel ? a : b;
    assign s = a + b;
    assign lt = a < b;
    always @(posedge clk or negedge rst_n)
        if (!rst_n) q_rst <= 8'h00;
        else      q_rst <= m;
    always @(posedge clk)
        if (en) q_en <= s[7:0];
endmodule
```

```
module tb_zoo;
    reg clk = 0, rst_n = 0, en = 0, sel = 1;
    reg [7:0] a = 8'd200, b = 8'd100;
    wire [7:0] q_rst, q_en, m; wire [8:0] s; wire lt;
    infer_zoo dut (.);
    always #5 clk = ~clk;
    initial begin
        #12 rst_n = 1; en = 1;
        @(posedge clk); #1;
        $display("m=%0d s=%0d lt=%b q_rst=%0d q_en=%0d", m, s, lt, q_rst, q_en);
        $finish;
    end
endmodule
```

```
m=200 s=300 lt=0 q_rst=200 q_en=44
```

Simulated with iverilog. **q_en** holds the low 8 bits of 300 (= 44): the 9-bit sum was truncated by the part-select exactly as the hardware will truncate it.

RTL idiom	Inferred hardware	Notes
<code>assign y = s ? a : b;</code>	MUX2 (or AND-OR / AOI)	Wide or nested ?: chains become priority muxes

RTL idiom	Inferred hardware	Notes
<code>always @* case (s) ... fully assigned</code>	Parallel mux / decoder	If items overlap, a priority chain
<code>always @*</code> with a missing assignment on some path	Latch (level-sensitive storage)	Almost always a bug in ASIC RTL - Chapter 5
<code>always @(posedge clk) q <= d;</code>	D flip-flop	
<code>... or negedge rst_n) if (!rst_n)</code>	DFF with async clear/preset	Reset value picks DFFR vs DFFS
<code>if (!rst_n) q <= 0;</code> in a posedge-only block	DFF + AND gate on D (sync reset)	Some libraries have sync-reset flops; <code>sync_set_reset</code> pragma keeps the reset next to the flop
<code>if (en) q <= d;</code>	DFF + MUX2 feedback, or an ICG	Tool-selected: clock gating replaces the mux for wide banks (Chapter 19)
<code>+, -</code>	Adder / subtractor	Architecture picked by timing: ripple, carry-select, Kogge-Stone...
<code>*</code>	Multiplier (Booth + Wallace/Dadda tree + final adder)	Large; constants multiply into shift-add networks
<code>/, %</code> by a variable	Array divider - huge and slow	Use a sequential divider (Chapter 8); powers of two are free
<code><<</code> by a variable	Barrel shifter (log2(N) mux levels)	Constant shifts are just wiring
<code>==, <</code>	Comparator (XNOR tree / subtractor carry)	
<code>reg [W-1:0] mem [0:D-1]</code> with sync write	Flop array + read mux, or a memory macro if a compiler is used	Large arrays belong in SRAM macros (Chapter 9)
<code>for</code> loop with constant bounds	Unrolled replicated logic	A loop is space , not time
Tri-state <code>assign y = oe ? d : 'z;</code>	Tri-state buffer	Only at pads; internal tri-states are banned in most flows

17.4 Constructs synthesis ignores or rejects

The simulator executes the full IEEE 1800 language; synthesis implements a subset defined (loosely) by IEEE 1800 plus each vendor's manual. The dangerous category is not the constructs that are rejected - you will find those on day one - but those that are **accepted and silently ignored**, because the simulation you trusted no longer describes the chip.

Construct	Simulator	Synthesis (ASIC)
<code>#5 delays</code>	Waits 5 time units	Ignored (warning at most)
<code>initial blocks</code>	Run once at time 0	Ignored for ASIC; FPGA tools use them for power-up values
Register initialiser <code>reg q = 1'b0;</code>	Initial value	Ignored for ASIC - the flop powers up random
Incomplete sensitivity list <code>@(a)</code>	Evaluates only on <code>a</code>	Builds full combinational logic - mismatch
<code>x assignments (y = 'x;)</code>	Propagates X	Don't-care: the tool picks whatever value is cheapest
<code>==, !=, casex on X</code>	4-state compare	Treated as <code>==/!=</code> or rejected
<code>// synopsys full_case / parallel_case</code>	Ignored (comment)	Changes the logic - mismatch
<code>\$display, \$random, force, fork/join, wait</code>	Executed	Rejected or ignored

Construct	Simulator	Synthesis (ASIC)
while with data-dependent bound	Executes	Rejected
real, time, classes, dynamic arrays	Supported	Rejected

Example: the incomplete sensitivity list

```

module and_bad (input wire a, b, output reg y);
    always @(a) y = a & b;      // b missing from the sensitivity list
endmodule

module and_good (input wire a, b, output reg y);
    always @* y = a & b;        // complete by construction
endmodule

module tb_sens;
    reg a = 0, b = 0;
    wire y_bad, y_good;
    and_bad u_bad (.a(a), .b(b), .y(y_bad));
    and_good u_good (.a(a), .b(b), .y(y_good));
    initial begin
        #1 a = 1; #1 $display("a=1 b=0 : y_bad=%b y_good=%b", y_bad, y_good);
        #1 b = 1; #1 $display("a=1 b=1 : y_bad=%b y_good=%b <- mismatch", y_bad, y_good);
        #1 a = 0; #1 a = 1;
        #1 $display("a toggled: y_bad=%b y_good=%b", y_bad, y_good);
        $finish;
    end
endmodule

```

```

a=1 b=0 : y_bad=0 y_good=0
a=1 b=1 : y_bad=0 y_good=1 <- mismatch
a toggled: y_bad=1 y_good=1

```

The RTL simulation of **and_bad** misses the change on **b**. The synthesised AND gate does not - so the netlist behaves like **and_good**, and RTL simulation was lying. Note that **iverilog -Wall** did not warn; lint tools (Chapter 25) do.

PITFALL: 'IT PASSED SIMULATION' IS NOT THE SAME AS 'IT WORKS'

Incomplete sensitivity lists, **full_case** pragmas, **x**-assignments and **initial** values all produce **RTL-vs-gate mismatches**. They are caught by three independent nets: lint (rules such as 'incomplete sensitivity list', 'synthesis pragma used'), **logical equivalence checking** of RTL against the netlist, and gate-level simulation (Chapters 25-26). Use **always_comb** and **always_ff** and most of this category disappears: **always_comb** infers its own sensitivity and tools must flag latches in it.

17.5 full_case and parallel_case: pragmas that lie

The two most notorious synthesis directives are comments that change the hardware. **full_case** tells synthesis 'the listed items cover every value that will ever occur, so unlisted values are don't-care'. **parallel_case** tells it 'the items never overlap, so build a parallel mux instead of a priority chain'. The simulator reads both as comments.

```

module dec_fc (input wire [1:0] sel, output reg [2:0] y);
    always @* begin
        case (sel) // synopsys full_case <- a promise the RTL does not keep
            2'b00: y = 3'b001;
            2'b01: y = 3'b010;
            2'b10: y = 3'b100;
        endcase
    end
endmodule

```

```

module dec_ok (input wire [1:0] sel, output reg [2:0] y);
  always @* begin
    y = 3'b000;           // default assignment first: no latch, no pragma
    case (sel)
      2'b00: y = 3'b001;
      2'b01: y = 3'b010;
      2'b10: y = 3'b100;
      default: ;
    endcase
  end
endmodule

```

```

module tb_fc;
  reg [1:0] sel;
  wire [2:0] y_fc, y_ok;
  dec_fc u_fc (.sel(sel), .y(y_fc));
  dec_ok u_ok (.sel(sel), .y(y_ok));
  initial begin
    sel = 2'b10; #1 $display("sel=10 : y_fc=%b y_ok=%b", y_fc, y_ok);
    sel = 2'b11; #1 $display("sel=11 : y_fc=%b y_ok=%b (RTL sim holds old y)", y_fc, y_ok);
    $finish;
  end
endmodule

```

```

sel=10 : y_fc=100 y_ok=100
sel=11 : y_fc=100 y_ok=000 (RTL sim holds old y)

```

In simulation **dec_fc** behaves like a latch for **sel=11**. Synthesis, told the case is full, builds no latch and treats **sel=11** as don't-care: the gates may output any value. Neither matches the other, and neither is probably what the designer meant.

Mechanism	Effect on synthesis	Effect on simulation	Verdict
// synopsys full_case	Unlisted values = don't-care; no latch	None	Avoid - mismatch
// synopsys parallel_case	Parallel mux, even if items overlap	None (priority order kept)	Avoid - mismatch
Default assignment before case	Defined value, no latch	Same	Preferred
SV unique case	Like full + parallel	Run-time warning if no item or >1 item matches	Good - the promise is checked
SV unique0 case	Parallel only	Warning if >1 matches	Good for one-hot decoders
SV priority case	Like full_case (priority kept)	Warning if no item matches	Use when you mean it

INTERVIEW INSIGHT

'What is the difference between **unique case** and **// synopsys parallel_case full_case**?' The synthesis result is the same; the difference is that SystemVerilog makes the simulator **check** the assumption and report a violation, so the optimisation is backed by verification instead of hope. Formal equivalence checkers also honour the **unique** semantics.

17.6 Hierarchy, uniquification and boundary optimization

Synthesis can keep your module hierarchy or **flatten** (ungroup) it. Flattening lets the optimiser move logic across module boundaries - merging a comparator in one block with a mux in the next - and typically improves timing and area by a few percent. Keeping hierarchy preserves debuggability, lets you apply per-block constraints, and is **required** where the hierarchy carries meaning: power domains (UPF refers to instance names), DFT wrappers, hard macros, clock-gating cells you instantiated on purpose, and blocks that will be ECOed later.

- **Uniquification.** If `fifo` is instantiated four times, each instance may need different optimisation (one drives a long wire, one is on a critical path). Tools create unique copies (`fifo_0`, `fifo_1`, ...) automatically during compile. This is why netlist module names differ from your RTL.
- **Boundary optimization.** Constants and unused outputs are propagated across ports: a port tied to 0 by the parent simplifies the child's logic. It is a big win - and the reason a `dont_touch` or `set_boundary_optimization false` is needed on blocks you intend to ECO or to connect differently later.
- **Auto-ungrouping.** Small modules (a few hundred gates) are often dissolved into their parent by default; keep a hierarchy only if you need it.
- **Naming.** Registers in the netlist are named from the RTL (`u_core/state_reg_2_`), which is why meaningful signal names pay off at gate-level debug and in timing reports.

17.7 A synthesis script in three dialects

Every commercial flow is a Tcl script (see the companion Tcl book for the language). The command names differ between Synopsys Design Compiler, Cadence Genus and open-source Yosys, but the structure is identical: set up libraries, read RTL, elaborate, apply constraints, compile, write outputs, report.

```
# ----- Synopsys Design Compiler (dc_shell / dcnxt_shell) -----
set_app_var search_path      ". ./lib ./rtl"
set_app_var target_library   "stdcell_ss_0p72v_125c.db"
set_app_var link_library     "* stdcell_ss_0p72v_125c.db sram_ss_0p72v_125c.db"
define_design_lib WORK -path ./work

analyze -format sverilog [list pkg.sv fifo.sv core.sv top.sv]
elaborate top -parameters "DATA_W=32"
current_design top
link
check_design                ;# unresolved refs, multiple drivers, loops
source constraints/top.sdc   ;# clocks, IO delays, exceptions (Ch. 18)

set_dont_use [get_lib_cells */*ULVT*] ;# e.g. keep leakage in check
compile_ultra -gate_clock -scan ;# clock gating + scan-ready flops

change_names -rules verilog -hierarchy
write -format verilog -hierarchy -output out/top.syn.v
write_sdc out/top.syn.sdc
report_qor                > rpt/qor.rpt
report_timing -max_paths 20 -nworst 1 > rpt/timing.rpt
report_area -hierarchy    > rpt/area.rpt
report_power              > rpt/power.rpt
report_clock_gating       > rpt/cg.rpt
```

A minimal Design Compiler flow. Production scripts add multi-corner setup, SAIF-driven power optimisation, physical guidance (DC-Topographical/NXT with a floorplan) and many checks.

```
# ----- Cadence Genus -----
set_db init_lib_search_path ./lib
set_db init_hdl_search_path ./rtl
read_libs stdcell_ss_0p72v_125c.lib
read_hdl -sv {pkg.sv fifo.sv core.sv top.sv}
elaborate top
check_design -unresolved
read_sdc constraints/top.sdc
set_db lp_insert_clock_gating true
syn_generic           ;# RTL -> generic gates
syn_map               ;# generic -> library cells
syn_opt               ;# incremental timing/area optimisation
write_hdl > out/top.syn.v
write_sdc > out/top.syn.sdc
report_timing > rpt/timing.rpt
report_area  > rpt/area.rpt
report_gates > rpt/gates.rpt
```

```
# ----- Yosys (open source) -----
read_verilog -sv rtl/top.sv rtl/core.sv
hierarchy -check -top top
synth -top top           ;# generic synthesis + internal cell lib
dfflibmap -liberty lib/cells.lib ;# map flops to library flops
abc -liberty lib/cells.lib -D 1250 ;# map logic, delay target 1250 ps
opt_clean
stat -liberty lib/cells.lib ;# cell counts and area
write_verilog -noattr out/top.syn.v
```

Yosys runs the same conceptual flow. Yosys has limited SystemVerilog support on its own (plugins such as yosys-slang or a Surelog/UHDM front end extend it).

TRY IT WITH OPEN-SOURCE TOOLS

You do not need a commercial licence to learn implementation. **Yosys** (synthesis) + **OpenROAD** (floorplan, placement, CTS, routing, STA with OpenSTA) are packaged as **OpenLane 2** and the **OpenROAD-flow-scripts** (ORFS), with open PDKs such as SkyWater sky130, GF180MCU, IHP SG13G2 and the predictive ASAP7. Taking one of your own blocks from RTL to GDS - and reading every report on the way - teaches more about this Part than any book. (Yosys was not installed in the environment used to build this book, so no Yosys output is shown.)

17.8 Reading QoR reports

After compile, the first report to read is the **quality of results** (QoR) summary. The excerpt below is illustrative - the format follows Design Compiler's `report_qor`, the numbers are made up - but every field in it is one you must be able to interpret.

```
Timing Path Group 'core_clk'
-----
Levels of Logic:           23.00
Critical Path Length:      1.21
Critical Path Slack:       -0.08
Critical Path Clk Period:  1.25
Total Negative Slack:      -3.42
No. of Violating Paths:    117.00

Cell Count
-----
Hierarchical Cell Count:   41
Leaf Cell Count:           58233
Buf/Inv Cell Count:        9210
Sequential Cell Count:     7412
Macro Count:               4
```

Area

```

-----
Combinational Area:      31022.51
Noncombinational Area:   28140.88
Macro/Black Box Area:    112400.00
Total Cell Area:         171563.39

```

Illustrative QoR excerpt (synthetic numbers). Units follow the library: ns and μm^2 here.

Field	What it tells you	Red flags
WNS (critical path slack)	Worst negative slack: how far the worst path misses	Beyond ~10% of the period at synthesis: an RTL/architecture problem, not a tool problem
TNS, violating paths	Sum of all negative slacks: breadth of the problem	Small WNS but huge TNS means a whole bus or datapath is slow
Levels of logic	Gates on the critical path	Far above your budget for the node and frequency: pipeline or restructure
Buf/Inv count	Buffering for fanout and wire load	A large fraction (>20-25%) suggests high-fanout nets or long wires
Sequential count / area	Flops and latches	Unexpected latches; flop count far from your estimate (logic removed as unloaded, or memories built from flops)
Combinational vs non-combinational area	Balance between logic and state	Wide muxes and big arithmetic dominate combinational area

TRIAGE ORDER FOR A FRESH NETLIST

1) `check_design` / elaboration warnings: latches, multiple drivers, unconnected ports, constant registers. 2) Sequential cell count versus your expectation. 3) Timing: WNS/TNS per path group, then the **top 10 critical paths** - their start/end points tell you which RTL lines to change. 4) Area by hierarchy. 5) Clock-gating coverage and power. Never tune constraints to hide a path you have not understood.

17.9 Summary

- Synthesis = analyze -> elaborate -> technology-independent optimisation -> technology mapping -> constraint-driven optimisation, producing a netlist of library cells.
- Liberty libraries define cell function, NLDM/CCS timing and power per PVT corner; Vt flavour, drive strength and track height are the knobs the tool turns.
- Each RTL idiom maps to a template; `/` and `%` by variables, variable shifts and wide multipliers are expensive.
- Delays, `initial` blocks and register initialisers are ignored for ASIC; incomplete sensitivity lists and `full_case` / `parallel_case` cause RTL-vs-gate mismatches. Prefer `always_comb`, default assignments and `unique case`.
- Hierarchy is a trade between optimisation across boundaries and debuggability, UPF/DFT/ECO requirements.
- Read QoR in order: warnings, flop count, WNS/TNS and critical paths, area by hierarchy, power.

17.10 Exercises

- 1 Sketch the gate-level hardware for `infer_zoo` including the enable mux and the reset tree, and estimate its area in GE using the rules of thumb from this chapter.
- 2 Write a 4:1 mux three ways (`case`, nested `?:`, AND-OR of one-hot selects). Which one is a priority structure? Which one would synthesise to the same gates under `unique case`?
- 3 Explain precisely why `dec_fc` can produce a different output value in gate-level simulation than in RTL simulation for `sel=2'b11`, and how an equivalence checker would report it.

- 4 Install Yosys (or use an online Yosys playground), synthesise a 16-bit adder with `synth` and then with `abc -D` at two delay targets, and compare the `stat` cell counts.
- 5 A block has 12,000 flops and a critical path of 31 logic levels at 1 GHz, with 30% of cells being buffers. List the three most likely RTL causes and what you would inspect first.

18. Timing: STA, SDC and Closing Timing from RTL

A synchronous design works if, and only if, every flip-flop sees a stable input for a short window around each active clock edge. **Static timing analysis** (STA) proves this for every path in the design without a single simulation vector, by adding up worst-case and best-case delays from the library. Its inputs are the netlist, the libraries (one per corner), parasitics once the design is placed and routed, and the constraints you write in **SDC** (Synopsys Design Constraints, a Tcl-based de facto standard read by every synthesis, place-and-route and STA tool).

As an RTL designer you meet timing three times: when you choose an architecture (how many logic levels fit in a cycle), when you write the SDC that tells the tools your intent, and when the implementation team hands back a list of failing paths that can only be fixed in RTL. This chapter covers all three.

18.1 Timing paths

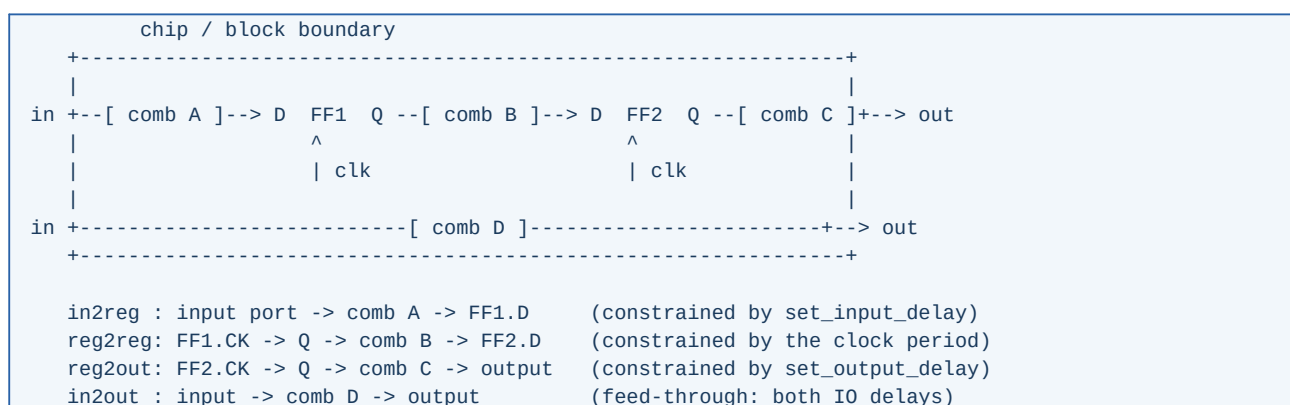


Figure 18.1 - The four path types. Every path starts at a clock pin or an input port and ends at a data pin of a sequential element or an output port. Clock paths (source -> tree -> CK pins) are analysed separately and feed the launch and capture times.

WHY GOOD BLOCKS REGISTER THEIR OUTPUTS

reg2out and in2out paths split the cycle between two blocks - and the two designers - so both sides must agree on a budget. If every block registers its outputs (and ideally its inputs), inter-block timing is almost free and each team can close timing alone. In2out combinational feed-throughs are the most fragile paths on a chip; avoid them across block boundaries.

18.2 Setup and hold, with skew and uncertainty

Consider a launch flop FF1 and a capture flop FF2 on the same clock of period T . Let $L1$ and $L2$ be the clock arrival times (insertion delay) at their clock pins, and define the **skew** as $skew = L2 - L1$ (positive when the capture clock arrives late). Uncertainty models jitter and margin the tools cannot compute.

$$\text{Setup: } L1 + t_{cq} + t_{comb,max} \leq T + L2 - t_{setup} - t_{unc,setup}$$

$$\text{slack}_{setup} = (T + \text{skew} - t_{setup} - t_{unc}) - (t_{cq} + t_{comb,max})$$

$$\text{Hold: } L1 + t_{cq,min} + t_{comb,min} \geq L2 + t_{hold} + t_{unc,hold}$$

$$\text{slack}_{hold} = (t_{cq,min} + t_{comb,min}) - (\text{skew} + t_{hold} + t_{unc,hold})$$

$$f_{max} = 1 / (t_{cq} + t_{comb,max} + t_{setup} + t_{unc} - \text{skew})$$

Positive skew helps setup and hurts hold. Setup depends on the clock period; **hold does not** - a hold violation cannot be fixed by slowing the clock, which is why it is fatal in silicon.

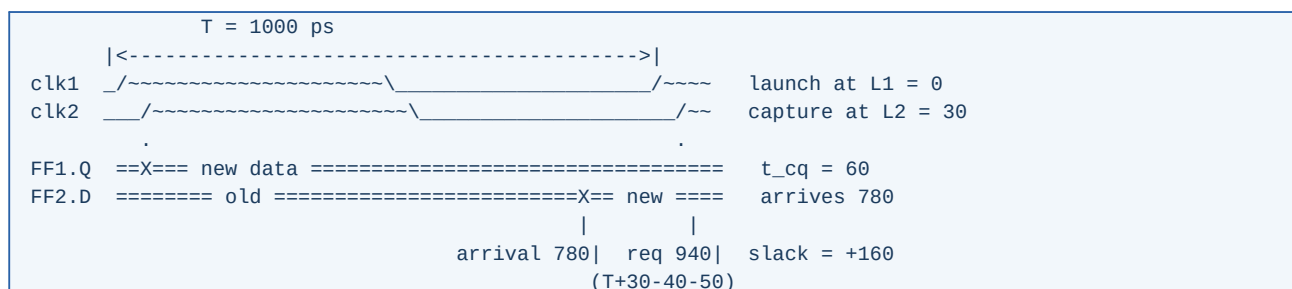


Figure 18.2 - The setup check of the worked example below on a timing diagram.

Worked example

A 1 GHz clock (T = 1000 ps). Library and extraction give: $t_{cq} = 60$ ps (max) / 45 ps (min), setup 40 ps, hold 25 ps. The comb logic between the flops is 720 ps on its longest path and 20 ps on its shortest. The clock tree delivers the capture clock 30 ps late (skew = +30). Uncertainty is 50 ps for setup and 20 ps for hold.

Setup arrival	= 60 + 720	= 780 ps	
Setup required	= 1000 + 30 - 40 - 50	= 940 ps	
Setup slack	= 940 - 780	= +160 ps	(met)
Hold arrival	= 45 + 20	= 65 ps	
Hold required	= 30 + 25 + 20	= 75 ps	
Hold slack	= 65 - 75	= -10 ps	(VIOLATED)
$f_{max} = 1 / (60 + 720 + 40 + 50 - 30) \text{ ps} = 1 / 840 \text{ ps} = 1.19 \text{ GHz}$			

The same skew that gave setup 30 ps of help causes the hold violation.

Physical design fixes the hold violation by inserting delay cells on the short path (cheap, automatic). But if the shortest path is a **direct flop-to-flop connection** - a shift register, or a synchronizer - between flops with large skew, hold fixing adds many buffers. RTL can help by avoiding long direct chains across distant clock-tree branches.

On-chip variation and derating

The equations above assume each cell has one delay per corner. Real silicon varies across the die, so signoff STA is pessimistic in a controlled way: for setup, the launch clock and data paths are slowed (late derate, e.g. x1.05) and the capture clock is sped up (early derate, e.g. x0.95); hold is the reverse. Modern flows use **AOCV**

(derate depends on path depth and distance) or **POCV / LVF** (each cell carries a statistical sigma). **CRPR** (clock reconvergence pessimism removal) gives back the pessimism on the common part of the launch and capture clock paths - which is why flops that talk to each other should share as much clock tree as possible.

Term	Meaning
Slack	Required time minus arrival time; negative = violation
WNS / TNS	Worst / total negative slack over a path group
MCMM	Multi-corner multi-mode: every mode (functional, scan shift, scan capture, low-power) at every corner
Recovery / removal	Setup/hold-like checks for the de-assertion of asynchronous resets (Chapter 12)
Clock gating check	Setup/hold of the enable at an ICG or gating cell (Chapter 19)
Max transition / capacitance / fanout	Design-rule checks: slow edges burn power, degrade delay models and hurt reliability

18.3 SDC: telling the tools what you mean

SDC describes the environment and the intent: the clocks, what happens outside the block, and which paths are **exceptions** to the default single-cycle rule. It is written in Tcl, so `get_ports`, `get_pins`, `get_clocks` and `get_cells` return object collections that the constraint commands consume.

Command	Purpose
<code>create_clock</code>	Defines a clock on a port or pin: name, period, waveform
<code>create_generated_clock</code>	A clock derived from another (divider, mux output) - keeps the phase relationship known
<code>set_clock_uncertainty</code>	Jitter + margin (larger before CTS)
<code>set_clock_latency</code> / <code>set_clock_transition</code>	Pre-CTS estimates of insertion delay and slew (ideal clocks)
<code>set_input_delay</code> / <code>set_output_delay</code>	How much of the cycle the outside world uses before/after this block
<code>set_driving_cell</code> / <code>set_load</code>	Electrical environment of IO
<code>set_false_path</code>	Path is never timed (static config, asynchronous resets into synchronizers, test-only)
<code>set_multicycle_path</code>	Path has N cycles to settle by design
<code>set_clock_groups</code>	Clocks that are asynchronous, or exclusive (never active together)
<code>set_max_delay</code> / <code>set_min_delay</code>	Point-to-point limits, e.g. on CDC data buses
<code>set_case_analysis</code>	Tie a pin to a constant for a mode (e.g. <code>test_mode = 0</code>)

Multicycle paths: the -setup / -hold rule

Suppose a register updates only every second cycle (it has a clock-enable that toggles every other cycle) and its consumer samples it only on the following enable. The path has two cycles. `set_multicycle_path 2 -setup` moves the **capture** edge from 1T to 2T. But the hold check is performed, by default, one edge **before** the setup capture edge - which is now at 1T, making the hold requirement a whole period larger and impossible to meet sensibly. So you must also move the hold edge back to 0.

```
# Two-cycle path: ALWAYS pair the setup multiplier with a hold multiplier of N-1
set_multicycle_path 2 -setup -from [get_cells u_ctl/slow_reg*] -to [get_cells u_dp/acc_reg*]
set_multicycle_path 1 -hold -from [get_cells u_ctl/slow_reg*] -to [get_cells u_dp/acc_reg*]
```

```

edge:      0T      1T      2T
clk  _____/-----\_____ /-----\_____ /-----\_____
default:   L----->S              (setup at 1T, hold at 0T)
MCP 2 setup: L----->S          (setup at 2T, hold moves to 1T !)
+ MCP 1 hold: H<- hold back at 0T      S (correct)

```

Figure 18.3 - Why **-setup N** must be accompanied by **-hold N-1** when launch and capture use the same clock.**PITFALL: EXCEPTIONS ARE PROMISES NOBODY CHECKS**

STA believes every false path and multicycle path you write. A wrong exception hides a real failure until silicon. Rules: (1) every exception must be justified by an RTL mechanism (an enable, a synchronizer, a static configuration) and documented next to the RTL; (2) prefer **-from/-to** on specific registers, never on whole clocks, unless the clocks are truly asynchronous; (3) verify multicycle paths with assertions or formal (Chapter 23) and review them at every hand-off.

Asynchronous clocks and CDC constraints

For clocks with no fixed phase relationship, the crossing must be handled by a synchronizer in RTL (Chapter 11), and timing between the domains should not be analysed as if it were synchronous. There are two tools:

- **set_clock_groups -asynchronous** - declares that no path between the groups is timed. Simple, but it removes **all** checks, including ones you want.
- **set_max_delay** on the crossing paths - keeps a bound on the data bus skew of a gray-coded pointer or a mux-recirculation synchronizer, so that all bits arrive within one destination period. In Synopsys tools use **-ignore_clock_latency** (Xilinx XDC uses **-datapath_only**).

PRECEDENCE MATTERS

Exceptions have a priority order: **set_clock_groups** and **set_false_path** override **set_max_delay**, which overrides **set_multicycle_path**. If you declare two clocks asynchronous with **set_clock_groups**, a **set_max_delay** you later write between them is silently ignored. For crossings that need a skew bound, constrain them with **set_max_delay** and do not put the clocks in asynchronous groups (or use false paths only on the paths that really need none).

A complete block-level SDC file

```

# ===== top.sdc - block 'dsp_top', 1 GHz core, 100 MHz APB, 50 MHz JTAG
set CORE_T 1.000
set APB_T 10.000

# ---- clocks -----
create_clock -name core_clk -period $CORE_T -waveform {0 0.5} [get_ports core_clk]
create_clock -name apb_clk -period $APB_T [get_ports pclk]
create_clock -name tck -period 20.0 [get_ports tck]

# divide-by-2 clock made by a flop in u_clkdiv (a generated clock, not a new one)
create_generated_clock -name core_clk_div2 -source [get_ports core_clk] \
    -divide_by 2 [get_pins u_clkdiv/div2_reg/Q]

# pre-CTS: ideal clocks with estimated latency, slew and margin
set_clock_latency -source 0.20 [get_clocks core_clk]
set_clock_transition 0.05 [all_clocks]
set_clock_uncertainty -setup 0.060 [get_clocks core_clk]
set_clock_uncertainty -hold 0.020 [get_clocks core_clk]

# ---- IO: 60% of the cycle is used outside the block (APB ports p* done below) --
set core_in [remove_from_collection [all_inputs] \
    [get_ports {core_clk pclk tck rst_n p*}]]
set_input_delay -max [expr 0.6*$CORE_T] -clock core_clk $core_in
set_input_delay -min 0.050 -clock core_clk $core_in

```

```

set_output_delay -max [expr 0.6*$CORE_T] -clock core_clk [get_ports m_axis_*]
set_output_delay -min 0.000 -clock core_clk [get_ports m_axis_*]
set_input_delay -max 4.0 -clock apb_clk [get_ports {psel penable pwrite paddr* pwrdata*}]
set_output_delay -max 4.0 -clock apb_clk [get_ports {prdata* pready pslverr}]
set_driving_cell -lib_cell BUFX4 [all_inputs]
set_load 0.010 [all_outputs]

# ---- clock-domain crossings -----
# tck is fully asynchronous and only reaches 2-flop synchronizers
set_clock_groups -asynchronous -group {tck} -group {core_clk core_clk_div2 apb_clk}
# APB <-> core go through gray-coded FIFOs: bound the pointer skew instead
set_max_delay 1.0 -ignore_clock_latency -from [get_clocks apb_clk] -to [get_clocks core_clk]
set_max_delay 1.0 -ignore_clock_latency -from [get_clocks core_clk] -to [get_clocks apb_clk]
set_min_delay 0.0 -from [get_clocks apb_clk] -to [get_clocks core_clk]
set_min_delay 0.0 -from [get_clocks core_clk] -to [get_clocks apb_clk]

# ---- exceptions -----
# asynchronous reset input goes to a reset synchronizer (Chapter 12)
set_false_path -from [get_ports rst_n]
# coefficients are static while the filter runs (documented in dsp_regs.sv)
set_false_path -from [get_cells u_regs/coef_q_reg*]
# accumulator is enabled every 2nd cycle by acc_en (see u_dp/acc_en_q)
set_multicycle_path 2 -setup -from [get_cells u_dp/prod_reg*] -to [get_cells u_dp/acc_reg*]
set_multicycle_path 1 -hold -from [get_cells u_dp/prod_reg*] -to [get_cells u_dp/acc_reg*]

# ---- modes and design rules -----
set_case_analysis 0 [get_ports test_mode]
set_max_transition 0.150 [current_design]
set_max_fanout 32 [current_design]

```

A complete functional-mode SDC. A separate scan-mode SDC sets **test_mode = 1**, defines the scan clock and constrains shift and capture (Chapter 20).

CONSTRAINING THE RESET

set_false_path -from rst_n is correct only because the port feeds a reset synchronizer: the **synchronized** reset that drives the flops must still be timed (recovery/removal checks). A false path on the internal reset net itself is a classic silicon bug.

18.4 Closing timing from RTL

When a path fails by more than the tools can recover (roughly 5-10% of the period after synthesis), the fix is architectural. The levers, from the most to the least invasive:

Technique	What it does	Cost
Pipelining	Insert registers to split the path; latency +1 per stage	Flops, control changes (valid bits, stalls, forwarding)
Retiming	Move existing registers across logic to balance stages (tool option: compile_ultra -retime , Genus retime)	Registers change names; reset values and equivalence checking get harder
Logic restructuring	Balanced trees instead of chains; carry-save instead of carry-propagate; precompute	Design effort
Late-arriving signal last	Shannon expansion: compute both outcomes, select with the late signal	Duplicated logic
One-hot encoding	State decode becomes a single bit; next-state logic shallower	More flops
Register duplication	Split a high-fanout register into copies near each load	Flops; must stop tools merging them
Precompute / look-ahead	Compute next cycle's condition one cycle early, register it	Flops, reasoning

Technique	What it does	Cost
Narrow the problem	Smaller counters, fewer compare bits, split wide muxes	Architecture

Pipelining a multiply-accumulate datapath

```
// Before: multiply and add in one cycle -> critical path = MULT + ADD
module dot2_1c (
    input wire      clk,
    input wire [15:0] a, b, c, d,
    output reg [32:0] y
);
    always @(posedge clk) y <= a*b + c*d;
endmodule

// After: two stages -> critical path = MULT (stage 1) or ADD (stage 2)
module dot2_2c (
    input wire      clk, rst_n, in_vld,
    input wire [15:0] a, b, c, d,
    output reg [32:0] y,
    output reg      y_vld
);
    reg [31:0] p0, p1;
    reg      s1_vld;
    always @(posedge clk) begin
        p0 <= a * b;           // stage 1: multipliers only
        p1 <= c * d;
        y  <= p0 + p1;         // stage 2: adder only
    end
    always @(posedge clk or negedge rst_n) // valid bits travel with the data
        if (!rst_n) {y_vld, s1_vld} <= 2'b00;
        else      {y_vld, s1_vld} <= {s1_vld, in_vld};
endmodule
```

before: a,b --[MULT 16x16]--+ c,d --[MULT 16x16]---+--[ADD 33]--> [y]	path = t_mult + t_add
after: a,b --[MULT]--> [p0] --+ c,d --[MULT]--> [p1] --+--[ADD]--> [y] in_vld -----> [s1_vld] -----> [y_vld]	path = max(t_mult, t_add)

Figure 18.4 - The pipeline register cuts the path. Data registers need no reset; the valid bits do (Chapter 10).

Pipelining changes latency, so the check must compare against the one-cycle version **delayed by one cycle**. The testbench also checks a Shannon-expanded adder (below) against its original form:

```
module tb_timing;
    reg clk = 0, rst_n = 0, in_vld = 0;
    reg [15:0] a, b, c, d;
    wire [32:0] y1, y2; wire y2_vld;
    dot2_1c u1 (.clk(clk), .a(a), .b(b), .c(c), .d(d), .y(y1));
    dot2_2c u2 (.clk(clk), .rst_n(rst_n), .in_vld(in_vld),
               .a(a), .b(b), .c(c), .d(d), .y(y2), .y_vld(y2_vld));
    reg [32:0] y1_q; // 1-cycle model output, delayed to align
    always @(posedge clk) y1_q <= y1;
    always #5 clk = ~clk;
    integer n = 0, err = 0, i;
    always @(posedge clk) if (y2_vld) begin
        n = n + 1;
        if (y2 !== y1_q) err = err + 1;
    end
    // Shannon check
    reg [15:0] base, x, z; reg late; wire [15:0] sb, sa; integer serr = 0;
```

```

late_before lb (.base(base), .x(x), .z(z), .late(late), .sum(sb));
late_after la (.base(base), .x(x), .z(z), .late(late), .sum(sa));
initial begin
    {a, b, c, d} = 0;
    #12 rst_n = 1;
    for (i = 0; i < 200; i = i + 1) begin
        @(negedge clk);
        in_vld = 1; a = $random; b = $random; c = $random; d = $random;
    end
    @(negedge clk) in_vld = 0;
    repeat (3) @(posedge clk);
    $display("dot2: %0d results compared, %0d mismatches, latency 1 -> 2 cycles",
        n, err);
    for (i = 0; i < 1000; i = i + 1) begin
        {base, x, z, late} = {$random, $random};
        #1 if (sa !== sb) serr = serr + 1;
    end
    $display("shannon: 1000 random vectors, %0d mismatches", serr);
    $finish;
end
endmodule

```

```

dot2: 200 results compared, 0 mismatches, latency 1 -> 2 cycles
shannon: 1000 random vectors, 0 mismatches

```

Moving a late-arriving signal to the end of the cone

If one input arrives late - it comes from another block's output, or from a long wire - restructure the logic so that the late signal passes through as few gates as possible. **Shannon expansion** $f(\text{late}, x) = \text{late} ? f(1, x) : f(0, x)$ computes both cofactors in parallel with the late signal's journey and uses it only as a final mux select:

```

// Late-arriving select: Shannon expansion moves 'late' to the last mux
module late_before (input wire [15:0] base, x, z, input wire late,
                    output wire [15:0] sum);
    assign sum = base + (late ? x : z); // late -> mux -> adder -> out
endmodule

module late_after (input wire [15:0] base, x, z, input wire late,
                   output wire [15:0] sum);
    wire [15:0] s_x = base + x; // both adders start early
    wire [15:0] s_z = base + z;
    assign sum = late ? s_x : s_z; // late -> one mux -> out
endmodule

```

Functionally identical (checked above on 1000 random vectors); the second costs one extra adder and removes a 16-bit carry chain from the late signal's path.

SYNTHESIS WILL NOT ALWAYS DO THIS FOR YOU

Tools restructure logic within their timing view, but they do not know that an input will be late unless the SDC says so (input delays) or the block is synthesised in context. They also will not duplicate a whole adder speculatively. When you know a signal is late, write the RTL so that the structure is already right.

Register duplication for high fanout

```
// High fanout: register duplication (keep the copies from being merged)
module fanout_dup (
    input wire      clk, en_d,
    input wire [63:0] d0, d1,
    output reg  [63:0] q0, q1
);
    (* keep = "true" *) reg en_a;    // drives the q0 bank
    (* keep = "true" *) reg en_b;    // drives the q1 bank
    always @(posedge clk) begin
        en_a <= en_d;
        en_b <= en_d;
        if (en_a) q0 <= d0;
        if (en_b) q1 <= d1;
    end
endmodule
```

Compiled with iverilog -Wall. Synthesis merges equivalent registers by default; the attribute (tool-specific: `keep`, `dont_touch`, or `set_register_merging false` in the script) keeps both. Physical synthesis can also duplicate registers itself.

A single enable flop driving 128 flops needs a buffer tree of $\log_4(128) \sim 3\text{-}4$ levels, plus the wire to reach loads spread across the block. Two copies placed near each bank halve the load and the distance. The same idea applies to pipeline stall signals, FSM state bits used as datapath selects, and reset trees.

One-hot state and precomputed flags

- **One-hot FSMs** make 'are we in state S' a single flop output rather than a decode of the binary state; next-state logic for each bit is a small OR of AND terms. For control FSMs on critical paths the extra flops are cheap (Chapter 7).
- **Precompute flags.** Instead of `if (count == LIMIT-1)` on a critical path, register `almost_done <= (count == LIMIT-2)` one cycle earlier. FIFOs precompute `full/empty` the same way.
- **Carry-save arithmetic** keeps sums in redundant form through a chain of additions and resolves the carry once (Chapter 8).
- **Balanced trees.** `a+b+c+d` written as `(a+b)+(c+d)` is two adder levels instead of three; tools usually find this, but a long `for` loop accumulation written as a chain may not be rebalanced across a generate hierarchy.

LOGIC DEPTH BUDGETS

Designers express the cycle in **FO4 delays** (the delay of an inverter driving four copies of itself), which roughly cancels out the process node. High-frequency CPU pipelines target on the order of 15-25 FO4 per cycle including flop overhead; typical SoC control logic has far more slack. Knowing your budget in logic levels lets you judge an RTL change before running synthesis.

18.5 Summary

- STA checks every path statically: in2reg, reg2reg, reg2out and in2out, for setup at slow corners and hold at fast corners, in every mode.
- Setup slack = $(T + \text{skew} - t_{\text{setup}} - t_{\text{unc}}) - (t_{\text{cq}} + t_{\text{comb}}, \text{max})$; hold slack is independent of the period and is fatal in silicon.
- OCV derates, AOCV/POCV and CRPR make signoff realistic; flops that talk should share clock tree.
- SDC defines clocks, IO budgets and exceptions; pair `-setup N` with `-hold N-1`; remember that clock groups override `set_max_delay`.
- Fix timing in RTL by pipelining, retiming, restructuring, putting late signals last, one-hot encoding, register duplication and precomputation - and prove the change with a self-checking testbench.

18.6 Exercises

- 1 For the worked example, how much skew can the design tolerate before hold fails with zero comb delay? What is the maximum comb delay for setup?
- 2 Write the SDC for a path from a register updated every 4th cycle to a register sampled on the same enable, and draw the setup and hold edges with and without the hold multiplier.
- 3 Pipeline `dot2_2c` further so that each 16x16 multiplier is split into two 8-bit partial-product stages. Extend the testbench to check the new latency.
- 4 Apply Shannon expansion to `y = ((a == b) & late) ? p : q` where `late` arrives 70% into the cycle. Verify your version against the original with random vectors.
- 5 Explain why `set_false_path -to [get_pins */rst_n_sync_reg*/D]` is dangerous, and write the correct constraints for a reset synchronizer.

19. Low-Power RTL and UPF

For a phone SoC, a wearable or an always-listening ML accelerator, power is the first-class constraint: it sets battery life, the thermal envelope that limits sustained performance, and the cost of the package and power-delivery network. Most power savings are decided **before** synthesis - by the architecture and the RTL - because tools can only optimise the switching you describe. This chapter covers where power goes, the RTL techniques that reduce it, and how multi-voltage and power-gated designs are described with the Unified Power Format (UPF, IEEE 1801).

19.1 Where the power goes

$$P_{total} = P_{dynamic} + P_{short-circuit} + P_{leakage}$$

$$P_{dynamic} = \alpha \times C_L \times V_{DD}^2 \times f \quad (\text{per node, summed over the design})$$

$$P_{leakage} = V_{DD} \times I_{leak}, \quad I_{leak} \sim \exp(-V_t / (n \times kT/q)) \quad (\text{subthreshold})$$

α = activity factor (transitions per clock that charge the node, 0..1 for data, 1 for a clock net); C_L = switched capacitance; V_{DD} = supply; f = clock frequency.

Term	Depends on	Knobs at RTL / architecture level
Dynamic (switching)	α , C , V^2 , f	Clock gating (α of clock nets), data gating, operand isolation, avoiding glitches, memory access reduction, lower f and V (DVFS)
Short-circuit	Input slew, V	Mostly physical (slew limits); ~10% of dynamic
Leakage	V_t , V , temperature, total transistor width	Power gating, HVT cells (tool), fewer/smaller memories, retention only where needed, body bias

THREE FACTS THAT DRIVE EVERY LOW-POWER DECISION

(1) V is squared: dropping supply 10% saves ~19% dynamic power, and because f must also drop, DVFS gives roughly cubic savings. (2) The **clock network** has $\alpha = 1$ and large capacitance: it is often a third or more of dynamic power, so clock gating is the single most effective RTL technique. (3) Leakage does not care whether you are busy: in always-on products that are mostly idle, leakage dominates energy and only **power gating** removes it.

19.2 Clock gating

A register with an enable keeps its value when $en = 0$, but its clock pin still toggles every cycle, burning clock-tree and internal flop power. **Clock gating** stops the clock to the whole bank when the enable is low. The gating element must not create glitches on the clock - which rules out a plain AND gate.

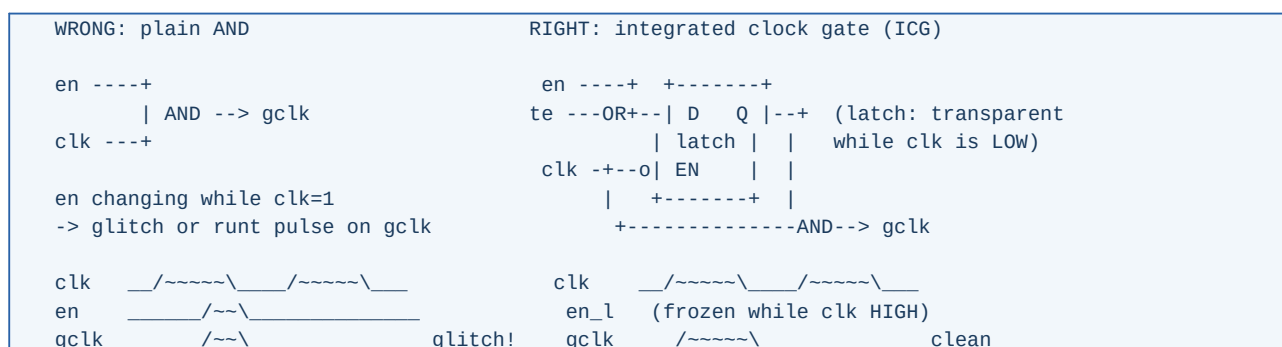


Figure 19.1 - The latch in the ICG only passes the enable while the clock is low, so the enable seen by the AND gate is stable for the whole high phase. **te** (test enable) forces the clock on during scan shift (Chapter 20).


```

// Behavioural model of an integrated clock-gating cell (latch + AND).
// In a real design you instantiate the library ICG (e.g. a CKLNQD-type cell);
// this model is for simulation only.
module icg (
    input wire clk,
    input wire en,          // functional enable
    input wire test_en,     // scan: force the clock on during shift
    output wire gclk
);
    reg en_l;
    always @(clk or en or test_en)
        if (!clk) en_l = en | test_en; // transparent while clk is LOW
    assign gclk = clk & en_l;           // en_l is stable while clk is HIGH
endmodule

// RTL style that synthesis turns into an ICG automatically
module reg_en (input wire clk, en, input wire [31:0] d, output reg [31:0] q);
    always @(posedge clk)
        if (en) q <= d;
endmodule

// The same register with an explicit (hand-instantiated) gate
module reg_gated (input wire clk, en, test_en, input wire [31:0] d,
                  output reg [31:0] q);

    wire gclk;
    icg u_icg (.clk(clk), .en(en), .test_en(test_en), .gclk(gclk));
    always @(posedge gclk) q <= d;
endmodule

// The WRONG way: a plain AND gate on the clock
module reg_and_gate (input wire clk, en, input wire [31:0] d,
                    output reg [31:0] q);
    wire gclk = clk & en; // en changing while clk=1 -> glitch
    always @(posedge gclk) q <= d;
endmodule

```

```

module tb_cg;
    reg clk = 0, en = 0;
    reg [31:0] d = 0;
    wire [31:0] q_en, q_g, q_and;
    integer err = 0, i, g_edges = 0, and_edges = 0;
    reg_en      r0 (.clk(clk), .en(en), .d(d), .q(q_en));
    reg_gated   r1 (.clk(clk), .en(en), .test_en(1'b0), .d(d), .q(q_g));
    reg_and_gate r2 (.clk(clk), .en(en), .d(d), .q(q_and));
    always #5 clk = ~clk;
    always @(posedge r1.gclk) g_edges = g_edges + 1;
    always @(posedge r2.gclk) and_edges = and_edges + 1;
    initial begin
        for (i = 0; i < 100; i = i + 1) begin
            @(negedge clk); en = $random; d = $random; // en changes while clk low
            @(posedge clk); #1 if (q_g != q_en) err = err + 1;
        end
        $display("ICG vs enable-flop: 100 cycles, %0d mismatches", err);
        // now let en change while clk is HIGH (as a late glitchy enable would)
        g_edges = 0; and_edges = 0;
        @(negedge clk) en = 0;
        @(posedge clk) #2 en = 1; #1 en = 0; // pulse inside the high phase
        @(negedge clk);
        $display("en pulse during clk high: ICG edges=%0d, AND-gate edges=%0d",
            g_edges, and_edges);
    $finish;
endmodule

```

```
end
endmodule
```

ICG vs enable-flop: 100 cycles, 0 mismatches
 en pulse during clk high: ICG edges=0, AND-gate edges=1

The ICG-gated register is cycle-equivalent to the enable flop. A short enable pulse during the high phase creates a spurious clock edge through the AND gate - a real flop would capture garbage or go metastable on the runt pulse - while the ICG ignores it.

Let synthesis do it: the enable style

You rarely instantiate ICGs by hand. Write registers with a clean enable (`if (en) q <= d;` with no `else`) and ask the tool for clock gating (`compile_ultra -gate_clock` with `set_clock_gating_style`, or Genus `set_db lp_insert_clock_gating true`). The tool groups flops that share an enable into banks and replaces the feedback muxes with one ICG per bank - but only above a minimum bank width (typically 3-8 bits), because an ICG costs area and its own power.

- **Share enables.** Forty 32-bit registers each with a different enable give forty ICGs; the same registers updated by one `load` signal give one. Architect enables at the granularity of real activity (a whole pipeline stage, a whole register slice).
- **Make enables real.** `if (en) q <= d;` where `en` is almost always 1 saves nothing. Use valid bits: a pipeline stage whose `valid` is 0 need not clock its data registers (Chapter 10).
- **Watch enable timing.** The enable must meet setup to the ICG latch, which closes at the rising edge **earlier than** the flops it drives (the gated clock has less insertion delay). The **clock-gating check** in STA reports this; late enables need restructuring, not a bigger ICG.
- **Coarse-grain gating** by hand: a block-level clock-control unit with instantiated ICGs switches whole subsystems off. Keep it in a dedicated module so that CDC, DFT and STA tools recognise it.
- **Test enable.** The ICG `te` pin is tied to scan enable (or test mode) so that every flop is clocked during scan shift. An ICG without a controllable `te` makes its flops untestable.

PITFALL: GATING A CLOCK WITH LOGIC IN RTL

`assign gclk = clk & en;` in RTL is wrong (as shown) and so is `always @(posedge (clk & en))`. Gated clocks must come from library ICG cells, instantiated in a clearly named wrapper (`cg_cell`) that has a behavioural model for simulation and FPGA and maps to the real cell in the ASIC netlist.

19.3 Operand isolation and data gating

A combinational block whose output is not used this cycle still switches whenever its inputs change. **Operand isolation** freezes its inputs (AND-gating, or holding them in registers that are not enabled) so it does not toggle. It pays off for large arithmetic units - multipliers, dividers, wide comparators - fed by busy buses.

```
// Operand isolation: freeze the multiplier inputs when its result is unused
module mac_iso (
    input wire      clk, use_mul,
    input wire [15:0] a, b, c,
    output reg  [31:0] y
);
    wire [15:0] a_i = a & {16{use_mul}}; // AND-isolation: inputs held at 0
    wire [15:0] b_i = b & {16{use_mul}};
    wire [31:0] prod = a_i * b_i;
    always @(posedge clk)
        y <= use_mul ? prod : {16'd0, c};
endmodule

module tb_iso;
    reg clk = 0, use_mul = 0; reg [15:0] a, b, c;
    wire [31:0] y; integer i, err = 0, toggles = 0;
    reg [31:0] exp_y; reg [31:0] prod_q;
    mac_iso dut (.*);
```

```

always #5 clk = ~clk;
always @(dut.prod) toggles = toggles + 1;
initial begin
    for (i = 0; i < 400; i = i + 1) begin
        @(negedge clk);
        use_mul = (i < 100); a = $random; b = $random; c = $random;
        exp_y = use_mul ? a * b : {16'd0, c};
        @(posedge clk); #1 if (y !== exp_y) err = err + 1;
        if (i == 99) begin
            $display("after 100 mul cycles : product-bus changes = %0d", toggles);
            toggles = 0;
        end
    end
    end
    $display("after 300 idle cycles: product-bus changes = %0d, errors = %0d",
            toggles, err);
    $finish;
end
endmodule

```

```

after 100 mul cycles : product-bus changes = 101
after 300 idle cycles: product-bus changes = 1, errors = 0

```

While the multiplier is idle its output bus changes once (to zero) and then stays quiet, although **a** and **b** keep changing. The count is a proxy for switching activity; a power tool would weight every internal node of the multiplier the same way.

ISOLATION HAS A COST

The AND gates add area and a gate of delay on the operand path, and **use_mul** must arrive early. Tools can insert operand isolation automatically (**set_operand_isolation_style** in DC), but the decision whether a unit is idle often long enough is architectural.

Other RTL power techniques

Technique	Idea	Where
Memory access reduction	Assert SRAM chip-enable only when reading/writing; never read 'just in case'; cache the last word read	Every SRAM: an access costs far more than a flop toggle
Memory banking	Split a big SRAM into banks and enable one	Buffers in ML accelerators, frame stores
SRAM low-power modes	Light sleep (periphery off), deep sleep (array at retention voltage), shutdown	Driven by the power controller; check wake-up latency
Data gating on buses	Hold bus values when valid = 0 instead of driving don't-care data	Wide datapaths, AXI data
Glitch reduction	Balance paths, register outputs of deep XOR/arithmetic cones before long wires	Multipliers, CRCs
Encoding	Gray code for counters on buses, bus-invert coding, one-hot where it lowers toggles	Address buses, pointers
Reduce precision	INT8/INT4 instead of FP32 - capacitance and toggles scale with width	ML datapaths (Chapter 29)

19.4 Multi-voltage design and power gating

Beyond gating the clock, a design can run different blocks at different supply voltages (**multi-voltage**, MV) and switch off the supply of idle blocks entirely (**power gating**, or power shut-off, PSO). These techniques need special cells and a precise description of the power architecture that every tool shares.

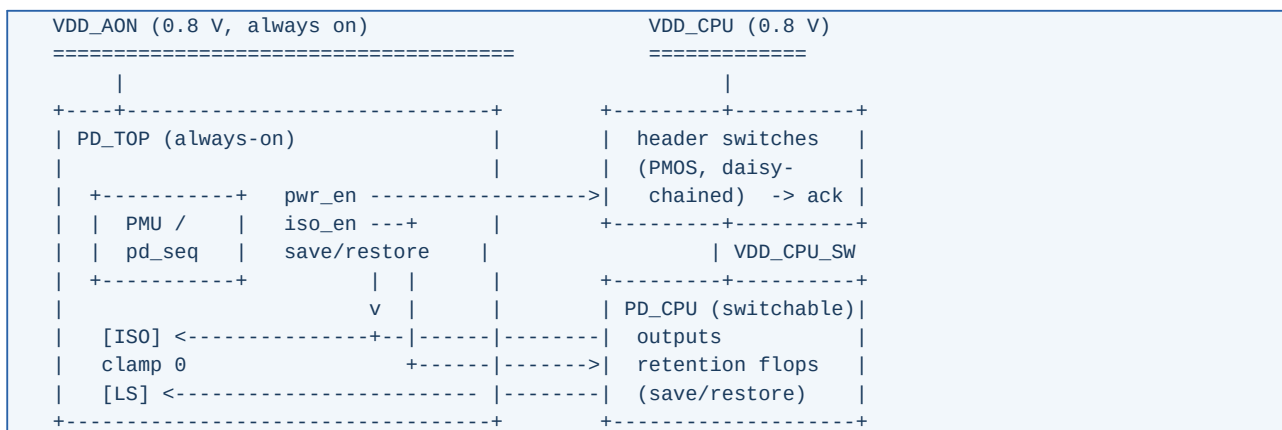


Figure 19.2 - A switchable CPU domain. Isolation cells clamp its outputs while it is off, retention flops keep selected state on the always-on supply, and level shifters (LS) are needed if the two domains run at different voltages.

Special cell	Why it is needed	RTL designer's responsibility
Power switch (header/footer)	Connects the domain's virtual supply to the real one; many small switches are daisy-chained to limit inrush current	Generate pwr_en , wait for pwr_ack before continuing
Isolation cell	An off domain's outputs float to X and can cause crowbar current and wrong values downstream; isolation clamps them to 0/1/latched	Decide safe clamp values (a request must clamp to 'no request'); drive iso_en from always-on logic
Retention flop	A flop with a shadow (balloon) latch on the always-on supply: save copies state out, restore copies it back	Choose what must be retained (costs area and leakage); reinitialise the rest after power-up
Level shifter	Signals from low to high voltage domains do not switch the receiving PMOS fully off	Know every crossing; avoid needless crossings in the architecture
Always-on buffers	Buffers inside a switchable domain that carry always-on signals	Minimise such feed-throughs

The power-down / power-up sequence

The sequence is the heart of a power-gated design, and it is ordinary RTL: an FSM in the always-on domain. Power down: **stop clocks -> isolate -> save -> switch off**. Power up: **switch on and wait for ack -> restore -> de-isolate -> start clocks**. (Some flows also assert reset to the non-retained flops during power-up.)

```
// Power-gating sequencer for one switchable domain (lives in always-on logic)
module pd_seq (
    input wire clk, rst_n,           // always-on clock and reset
    input wire sleep_req, wake_req,
    input wire pwr_ack,             // from the last switch in the daisy chain
    output reg clk_en,              // 1 = domain clock running
    output reg iso_en,              // 1 = outputs clamped
    output reg save,                // pulse: retention flops save state
    output reg restore,            // pulse: retention flops restore state
    output reg pwr_en,              // 1 = power switches on
    output reg [2:0] st
);
    localparam [2:0] ON = 3'd0, STOP = 3'd1, ISO = 3'd2, SAVE = 3'd3,
        OFF = 3'd4, PWRUP = 3'd5, RESTORE = 3'd6, UNISO = 3'd7;
    always @(posedge clk or negedge rst_n)
        if (!rst_n) begin
            st <= ON; clk_en <= 1; iso_en <= 0; save <= 0; restore <= 0; pwr_en <= 1;
        end else begin
            save <= 0; restore <= 0;
            case (st)
                ON:      if (sleep_req) begin clk_en <= 0; st <= STOP; end

```

```

        STOP:    begin iso_en <= 1; st <= ISO; end        // 1. clocks off
        ISO:     begin save  <= 1; st <= SAVE; end       // 2. isolate
        SAVE:    begin pwr_en <= 0; st <= OFF; end       // 3. save, 4. off
        OFF:     if (wake_req) begin pwr_en <= 1; st <= PWRUP; end
        PWRUP:   if (pwr_ack) begin restore <= 1; st <= RESTORE; end
        RESTORE: begin iso_en <= 0; st <= UNISO; end
        UNISO:   begin clk_en <= 1; st <= ON; end
    endcase
end
endmodule

```

```

module tb_pmu;
    reg clk = 0, rst_n = 0, sleep_req = 0, wake_req = 0, pwr_ack = 1;
    wire clk_en, iso_en, save, restore, pwr_en; wire [2:0] st;
    pd_seq dut (.*);
    always #5 clk = ~clk;
    always @(posedge pwr_en) #30 pwr_ack = 1;           // switches take 3 cycles
    always @(negedge pwr_en) pwr_ack = 0;
    reg [5*8-1:0] nm [0:7];
    initial begin
        nm[0]="ON"; nm[1]="STOP"; nm[2]="ISO"; nm[3]="SAVE";
        nm[4]="OFF"; nm[5]="PWRUP"; nm[6]="REST"; nm[7]="UNISO";
    end
    always @(negedge clk) if (rst_n)
        $display("%-5s clk_en=%b iso=%b save=%b restore=%b pwr_en=%b ack=%b",
            nm[st], clk_en, iso_en, save, restore, pwr_en, pwr_ack);
    initial begin
        #12 rst_n = 1;
        @(negedge clk) sleep_req = 1; @(negedge clk) sleep_req = 0;
        repeat (4) @(negedge clk);
        wake_req = 1; @(negedge clk) wake_req = 0;
        repeat (6) @(negedge clk);
        $finish;
    end
end
endmodule

```

```

ON    clk_en=1 iso=0 save=0 restore=0 pwr_en=1 ack=1
STOP  clk_en=0 iso=0 save=0 restore=0 pwr_en=1 ack=1
ISO   clk_en=0 iso=1 save=0 restore=0 pwr_en=1 ack=1
SAVE  clk_en=0 iso=1 save=1 restore=0 pwr_en=1 ack=1
OFF   clk_en=0 iso=1 save=0 restore=0 pwr_en=0 ack=0
OFF   clk_en=0 iso=1 save=0 restore=0 pwr_en=0 ack=0
PWRUP clk_en=0 iso=1 save=0 restore=0 pwr_en=1 ack=0
PWRUP clk_en=0 iso=1 save=0 restore=0 pwr_en=1 ack=0
PWRUP clk_en=0 iso=1 save=0 restore=0 pwr_en=1 ack=0
REST  clk_en=0 iso=1 save=0 restore=1 pwr_en=1 ack=1
UNISO clk_en=0 iso=0 save=0 restore=0 pwr_en=1 ack=1
ON    clk_en=1 iso=0 save=0 restore=0 pwr_en=1 ack=1

```

The sequencer waits in PWRUP until the switch chain acknowledges. Isolation is asserted before **save** and power-off, and released only after **restore** - exactly the order UPF-aware simulation and static checkers (Synopsys VC LP, Cadence Conformal Low Power) verify.

PITFALL: A PLAIN RTL SIMULATION DOES NOT MODEL POWER

In the simulation above nothing is actually switched off. Only a **power-aware simulation** (the simulator reads the UPF and corrupts the state of an off domain to X) shows that a missing isolation cell lets X leak into the always-on logic, or that a register you forgot to retain comes back as garbage. Run power-aware simulation for every power-state transition.

19.5 UPF: describing power intent

RTL describes function; the **UPF** file (IEEE 1801, a Tcl-based format) describes power intent: supplies, domains, switches, isolation, retention, level shifting and legal power states. The same UPF drives power-aware simulation, synthesis (which inserts the special cells), place-and-route, static low-power checks and equivalence checking. Keeping the intent out of the RTL means the same RTL can be reused with a different power architecture.

```
# ===== soc_top.upf (IEEE 1801-2013 / UPF 2.1 style) =====
upf_version 2.1
set_scope /

# ---- supply ports, nets and sets -----
create_supply_port VDD_AON
create_supply_port VDD_CPU
create_supply_port VSS
create_supply_net VDD_AON
create_supply_net VDD_CPU
create_supply_net VDD_CPU_SW ;# virtual supply after the switch
create_supply_net VSS
connect_supply_net VDD_AON -ports VDD_AON
connect_supply_net VDD_CPU -ports VDD_CPU
connect_supply_net VSS -ports VSS
create_supply_set SS_AON -function {power VDD_AON} -function {ground VSS}
create_supply_set SS_CPU -function {power VDD_CPU_SW} -function {ground VSS}

# ---- power domains -----
create_power_domain PD_TOP -include_scope -supply {primary SS_AON}
create_power_domain PD_CPU -elements {u_cpu} -supply {primary SS_CPU} \
    -supply {default_isolation SS_AON} \
    -supply {default_retention SS_AON}

# ---- power switch (controlled by the always-on sequencer) -----
create_power_switch SW_CPU -domain PD_CPU \
    -input_supply_port {vin VDD_CPU} \
    -output_supply_port {vout VDD_CPU_SW} \
    -control_port {sw_en u_pmu/u_seq/pwr_en} \
    -ack_port {sw_ack u_pmu/cpu_pwr_ack} \
    -on_state {on_s vin {sw_en}} \
    -off_state {off_s {!sw_en}}

# ---- isolation: clamp CPU outputs to 0 while it is off -----
set_isolation iso_cpu_out -domain PD_CPU -applies_to outputs \
    -isolation_supply_set SS_AON -clamp_value 0 \
    -isolation_signal u_pmu/u_seq/iso_en -isolation_sense high \
    -location parent
# an active-low output must clamp to its INACTIVE value, 1
set_isolation iso_cpu_err -domain PD_CPU -elements {u_cpu/err_n} \
    -isolation_supply_set SS_AON -clamp_value 1 \
    -isolation_signal u_pmu/u_seq/iso_en -isolation_sense high \
    -location parent

# ---- retention of the architectural state only -----
set_retention ret_cpu -domain PD_CPU -retention_supply_set SS_AON \
    -elements {u_cpu/u_regfile u_cpu/u_csr} \
    -save_signal {u_pmu/u_seq/save high} \
    -restore_signal {u_pmu/u_seq/restore high}

# ---- power states -----
add_power_state SS_AON -state ON {-supply_expr {power == `{FULL_ON, 0.80}}}
add_power_state SS_CPU -state ON {-supply_expr {power == `{FULL_ON, 0.80}}}
```

```
add_power_state SS_CPU -state OFF {-supply_expr {power == `{OFF}} -simstate CORRUPT}
add_power_state PD_CPU -state RUN {-logic_expr {SS_CPU == ON}}
add_power_state PD_CPU -state SLEEP {-logic_expr {SS_CPU == OFF}}
```

A single-voltage, power-gated CPU domain. With a second voltage (e.g. VDD_CPU at 0.65 V for low-performance modes) you would add **set_level_shifter** rules. UPF 3.0 renames some options (**-isolation_supply**, **-retention_supply**); check the version your tools accept. Older flows use a power-state table (**create_pst** / **add_pst_state**).

UPF command	Describes
create_supply_port/net , connect_supply_net	The supply network: pins of the block and nets inside it
create_supply_set	A named bundle (power, ground, optional wells) that domains and strategies refer to
create_power_domain	Which instances share a primary supply and power behaviour
create_power_switch	Switch cells, their control and ack signals, and on/off conditions
set_isolation	Which ports get isolation cells, clamp value, control signal, where the cells go
set_retention	Which registers are retention flops and their save/restore controls
set_level_shifter	Crossings between different voltages
add_power_state	Legal supply combinations - the basis for checking that isolation and shifting cover every state

19.6 DVFS: dynamic voltage and frequency scaling

DVFS runs a domain at a set of **operating performance points** (OPPs), e.g. 1.2 GHz @ 0.90 V, 800 MHz @ 0.75 V, 400 MHz @ 0.60 V. Software (the OS governor) picks the OPP; the hardware changes the PLL/divider and asks the PMIC or an on-chip regulator for the new voltage - **raise voltage before frequency, lower frequency before voltage**. RTL implications:

- Glitch-free clock switching between sources (Chapter 11) and clean handshakes with the voltage regulator (with timeouts).
- Every OPP is a separate timing-signoff scenario: the slowest corner at the lowest voltage often sets the design, and hold must be met at every voltage.
- Asynchronous interfaces to fixed-voltage domains need level shifters and CDC treatment, because their relative frequencies change at run time.
- **AVS** (adaptive voltage scaling) closes the loop with on-die delay monitors (ring oscillators, critical-path replicas) to trim the margin for each chip.

19.7 Power estimation flow

Power numbers are only as good as the activity that drives them. Tools default to **vectorless** estimation (a guessed toggle rate such as 10-20% on inputs, propagated through the logic), which is fine for early comparison and useless for signoff.

```
testbench running a REAL workload window (boot, inference, idle ...)
      |
      v
VCD / FSDB (every transition,          SAIF (toggle counts + time at 0/1
time-based, huge)                      per net, compact, averaged)
      |
      v
time-based power: peak / di-dt,        average power: synthesis power
IR-drop vectors (PrimePower,          optimisation, PrimePower/Joules/
Voltus, Joules)                       PowerArtist averages
```

Figure 19.3 - Activity files. RTL activity can be mapped onto the netlist by name, which is another reason to keep register names stable through synthesis.

```
# Averaged power with SAIF activity (Design Compiler / PrimePower style)
read_saif -input sim/inference.saif -instance_name tb_top/u_dut
report_power -hierarchy > rpt/power_hier.rpt

# Time-based power from a VCD window in PrimePower
set_app_var power_analysis_mode time_based
read_vcd -strip_path tb_top/u_dut sim/inference.vcd
update_power
report_power
```

Tool-specific commands differ; the concepts are the same everywhere. Simulators (including Icarus via `$dumpfile` / `$dumpvars`) write VCD; commercial simulators also write SAIF directly.

POWER IS AN RTL REVIEW ITEM

Early RTL power tools (PowerArtist, Joules, PrimePower RTL) report clock-gating efficiency, wasted memory reads and redundant toggles per register - with pointers to RTL lines. Run them at each milestone, not after tapeout.

19.8 Summary

- $P_{\text{dyn}} = \alpha \times C \times V^2 \times f$ and leakage grows exponentially as V_t drops and temperature rises; clocks are the largest single dynamic consumer.
- Clock gating uses latch-based ICG cells, never AND gates; write clean shared enables and let synthesis insert ICGs; tie `te` for scan.
- Operand isolation, data gating, memory-access reduction and narrower datapaths cut switching at the source.
- Power gating needs switches, isolation, retention and a sequencer: stop clocks, isolate, save, off - and the reverse.
- UPF describes supplies, domains, switches, isolation, retention, level shifters and power states for every tool in the flow.
- DVFS trades voltage and frequency at run time; each OPP is a signoff scenario. Estimate power with SAIF/VCD from real workloads.

19.9 Exercises

- 1 A block has $C = 2 \text{ nF}$ of switched capacitance with $\alpha = 0.15$ at 1 GHz and 0.8 V. Compute its dynamic power, then at 700 MHz and 0.7 V. What fraction is saved?
- 2 Extend `icg` into a negative-edge version (for flops clocked on the falling edge). Which gate and which latch phase do you need?
- 3 Add a `pwr_ack` timeout to `pd_seq` that raises an error flag if the switches do not acknowledge within 64 cycles, and test it.
- 4 Choose clamp values for these outputs of a powered-off DMA: `req`, `busy`, `irq`, `err_n`, `axi_awvalid`. Justify each.
- 5 Write the UPF lines needed if `PD_CPU` runs at 0.65 V while `PD_TOP` stays at 0.8 V. Which direction of crossing strictly requires a level shifter, and why?

20. Design for Test: Scan, BIST, JTAG

Verification asks 'is the design correct?'. **Manufacturing test** asks a different question about every single die that comes off the wafer: 'was this copy of the correct design built correctly?'. A particle, a thin via or a bridged metal line turns a perfect design into a broken chip, and a broken chip that reaches a customer costs orders of magnitude more than one caught at wafer sort. Automotive customers ask for defect rates in the low single-digit **DPPM** (defective parts per million).

Test is only possible if the design lets a tester **control** every internal node and **observe** its effect. Design for test (DFT) is the set of structures - scan chains, BIST engines, JTAG - and RTL rules that provide this. Much of it is inserted by tools, but the tools can only insert it into RTL that follows the rules, and the RTL designer owns the clocking, reset and memory decisions that make or break test coverage.

20.1 Fault models

Physical defects are infinitely varied, so test generation targets abstract **fault models** that correlate well with real defects.

Model	Definition	How it is tested
Stuck-at (SA0/SA1)	A pin or net is permanently 0 or 1	Static patterns at slow speed; the baseline, targets > 99%
Transition (STR/STF)	A node is slow-to-rise or slow-to-fall by more than a cycle	At-speed: launch a transition, capture one functional period later (launch-on-capture or launch-on-shift)
Path delay	Accumulated small delays along a specific path	At-speed patterns on selected critical paths
Bridging	Two nets shorted (wired-AND/OR or dominant)	Patterns using layout-extracted neighbour pairs
Cell-aware	Defects inside a standard cell (from transistor-level simulation of the layout)	Extra patterns per cell type; significant DPPM improvement at advanced nodes
IDDQ	Abnormal quiescent current from a defect	Current measurement; weak at advanced nodes because of leakage

$$\text{Fault coverage} = \text{detected faults} / \text{total faults}$$

$$\text{Test coverage} = \text{detected faults} / (\text{total faults} - \text{untestable faults})$$

Test coverage excludes faults that are provably untestable (redundant logic, tied nets). Coverage lost to **uncontrollable** or **unobservable** logic - the DFT-unfriendly RTL of Section 20.3 - counts against you in both.

20.2 Scan: turning sequential test into combinational test

Testing a sequential circuit directly requires sequences of inputs to steer it into each state - intractable for millions of flops. **Scan** replaces every flop with a **scan flop** - a flop with a 2:1 mux on its D input - and links them into shift registers (**scan chains**). In **shift** mode (scan enable = 1) the tester loads any state into every flop and unloads the previous contents; in **capture** mode (scan enable = 0) one functional clock captures the response of the combinational logic. Sequential test becomes combinational test between flop boundaries, which ATPG can solve.

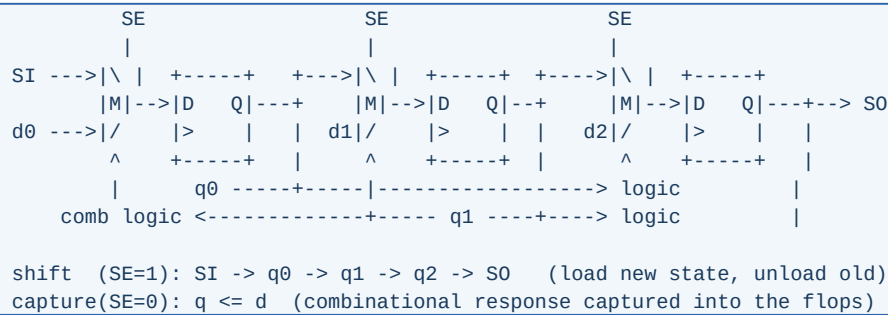


Figure 20.1 - Mux-D scan flops stitched into a chain. The scan path is a direct Q-to-SI connection, which is why scan chains are a classic source of hold violations.

The following model shows exactly what scan insertion produces for a 4-bit incrementer, and the three-phase test procedure the tester runs: shift in a stimulus, capture once, shift out the response.

```
// A mux-D scan flop: SE selects the scan input instead of functional D
module sdfp (input wire clk, rst_n, d, si, se, output reg q);
  always @(posedge clk or negedge rst_n)
    if (!rst_n) q <= 1'b0;
    else      q <= se ? si : d;
endmodule

// A 4-bit incrementer register after scan insertion (what DFT tools produce)
module inc4_scan (
  input  wire clk, rst_n, se, si,
  output wire so,
  output wire [3:0] q
);
  wire [3:0] d = q + 4'd1; // functional logic under test
  sdfp f0 (.clk(clk), .rst_n(rst_n), .d(d[0]), .si(si), .se(se), .q(q[0]));
  sdfp f1 (.clk(clk), .rst_n(rst_n), .d(d[1]), .si(q[0]), .se(se), .q(q[1]));
  sdfp f2 (.clk(clk), .rst_n(rst_n), .d(d[2]), .si(q[1]), .se(se), .q(q[2]));
  sdfp f3 (.clk(clk), .rst_n(rst_n), .d(d[3]), .si(q[2]), .se(se), .q(q[3]));
  assign so = q[3];
endmodule

module tb_scan;
  reg clk = 0, rst_n = 0, se = 0, si = 0;
  wire so; wire [3:0] q;
  inc4_scan dut (.*);
  always #5 clk = ~clk;
  integer i; reg [3:0] pat = 4'b0111, resp;
  initial begin
    #12 rst_n = 1;
    se = 1; // 1) shift in: last bit in = q[0]
    for (i = 3; i >= 0; i = i - 1) begin
      @(negedge clk) si = pat[i];
    end
    @(negedge clk) se = 0; // 2) capture: one functional clock
    $display("loaded   q = %b", q);
    @(negedge clk) se = 1;
    $display("captured q = %b (expect %b)", q, pat + 4'd1);
    for (i = 3; i >= 0; i = i - 1) begin // 3) shift out through so
      resp[i] = so;
      @(negedge clk);
    end
    $display("unloaded   = %b -> %s", resp, (resp == pat + 4'd1) ? "PASS" : "FAIL");
    $finish;
  end
endmodule
```

```
loaded    q = 0111
captured q = 1000 (expect 1000)
unloaded  = 1000 -> PASS
```

A stuck-at-0 on the carry into bit 3 would have unloaded 0000 - detected. The pattern 0111 was chosen to exercise the full carry chain; ATPG chooses patterns the same way, for every fault.

Scan compression, ATPG and test time

A modern SoC has millions of flops but a tester offers only a handful of scan pins. **Scan compression** (Synopsys DFTMAX, Siemens Tessent EDT, Cadence Modus) places a decompressor between a few scan-in channels and hundreds of short internal chains, and a compactor (XOR network, with X-masking) on the way out. Compression ratios of 50-200x are common.

$$\text{test time} \sim \text{patterns} \times (\text{flops per chain} + \text{capture cycles}) / f_{\text{shift}}$$

example: 2,000,000 flops / 400 internal chains = 5,000 flops per chain
 4,000 patterns x 5,000 / 50 MHz = 0.4 s per die

Without compression, the same design on 8 external chains would need 250,000-flop chains and 50x the test time. Shift is usually slow (25-100 MHz) to limit IR drop from millions of flops toggling together.

ATPG (automatic test pattern generation) uses algorithms in the D-algorithm / PODEM / FAN family plus SAT solvers: for each fault it finds an input assignment that **activates** the fault (drives the opposite value onto the node) and **propagates** the difference to an observable flop. Compaction then merges patterns. The RTL designer's job is to make sure nothing blocks controllability or observability.

20.3 DFT-friendly RTL rules

Rule	Why	How
Clocks controllable from a pin in test mode	ATPG must pulse every flop's clock exactly when it wants; PLLs, dividers and gated clocks are not controllable	Test-clock mux at the root of each clock (OCC - on-chip clock controllers - for at-speed); ICG <code>te</code> tied to scan enable
Async resets/sets controllable	An internally generated reset firing during shift corrupts the chain	Mux the reset with a test pin; synchronizer outputs bypassed in test mode
No latches (or transparent in test)	Latches are not scannable and block propagation	Use flops; force latches transparent with <code>test_mode</code>
No combinational loops	ATPG cannot model them	Lint; break loops with a flop
No clocks used as data / data used as clocks	Unpredictable capture	Synchronise instead
Handle memories and analog	SRAM outputs are X in scan unless modelled; macros are black boxes to ATPG	MBIST plus bypass/observe logic around each macro; wrappers for analog
No X sources on scan paths	Uninitialised data corrupts compressed responses	Initialise or mask; avoid floating tri-states
Static test signals	<code>test_mode</code> must be constant during test; <code>scan_en</code> must be timed	Top-level pins or a TAP data register (JTAG) drive them

```
// DFT-friendly reset and clock control: the tester owns both in test mode
module dft_ctrl (
    input wire func_clk,      // e.g. PLL output / divided clock
    input wire test_clk,     // dedicated tester clock pin
    input wire rst_n_sync,   // functional reset (after the reset synchronizer)
    input wire test_rst_n,   // reset pin driven directly by the tester
    input wire test_mode,    // static: tied by the tester for the whole test
    output wire core_clk,
    output wire core_rst_n
);
    // In a real netlist these are library clock-mux cells, instantiated and
    // marked dont_touch; the RTL form below is for simulation and lint.
    assign core_clk = test_mode ? test_clk : func_clk;
    assign core_rst_n = test_mode ? test_rst_n : rst_n_sync;
endmodule
```

Compiled and simulated with iverilog (a small testbench checks both modes). In functional mode STA sets `test_mode = 0` with `set_case_analysis`; a separate scan-mode SDC sets it to 1.

PITFALL: THE CLOCK DIVIDER THAT EATS YOUR COVERAGE

A block clocked by `clk_div2` from a flop-based divider has no controllable clock in scan mode: the divider flop is itself in a scan chain, so shifting scrambles the derived clock. Every flop downstream is lost to ATPG. The fix is the same as above: a test-clock mux after the divider, planned in the RTL.

20.4 Memory BIST

Embedded SRAMs occupy a large fraction of an SoC's area and are the densest - hence most defect-prone - structures on it. They are tested by **MBIST**: an on-chip controller that writes and reads algorithmic patterns at speed and compares the results. Failures can drive **repair** (spare rows/columns programmed by eFuses). The standard algorithms are **March tests**: sequences of read/write elements applied to every address in ascending or descending order.

March C- : {c(w0); up(r0,w1); up(r1,w0); dn(r0,w1); dn(r1,w0); c(r0)}

operations = 1 + 2 + 2 + 2 + 2 + 1 = 10 per address -> 10N

detects: stuck-at, transition, address-decoder and most coupling faults

c = either address order. A March test with N = 64k words costs 655,360 cycles - under 1 ms at 1 GHz.

```
// March C- MBIST engine: {c(w0); up(r0,w1); up(r1,w0); dn(r0,w1); dn(r1,w0); c(r0)}
// One operation per clock; the memory model here has a combinational read.
module mbist_marchc #(parameter int AW = 4, parameter int DW = 8) (
    input wire      clk, rst_n, start,
    output reg      busy, done, fail,
    output reg [AW-1:0] fail_addr,
    output wire     mem_we,
    output wire [AW-1:0] mem_addr,
    output wire [DW-1:0] mem_wdata,
    input wire [DW-1:0] mem_rdata
);
    reg [2:0] elem;          // March element 0..5
    reg      op;             // operation index inside the element
    reg [AW-1:0] cnt;        // address counter (always counts up)
    reg      nops2, up, rd, val;
    always @* begin         // element table
        nops2 = 1'b1; up = 1'b1;
        case (elem)
            3'd0: begin nops2 = 1'b0; rd = 1'b0; val = 1'b0; end // c(w0)
```

```

        3'd1: begin rd = (op == 1'b0); val = op;          end // up(r0,w1)
        3'd2: begin rd = (op == 1'b0); val = ~op;        end // up(r1,w0)
        3'd3: begin rd = (op == 1'b0); val = op;  up = 1'b0; end // dn(r0,w1)
        3'd4: begin rd = (op == 1'b0); val = ~op; up = 1'b0; end // dn(r1,w0)
        default: begin nops2 = 1'b0; rd = 1'b1; val = 1'b0; end // c(r0)
    endcase
end
assign mem_addr = up ? cnt : ~cnt;
assign mem_we   = busy & ~rd;
assign mem_wdata = {DW{val}};
wire last_op    = !nops2 || op;
wire last_addr  = &cnt;
always @(posedge clk or negedge rst_n)
    if (!rst_n) begin
        busy <= 0; done <= 0; fail <= 0; fail_addr <= '0;
        elem <= 0; op <= 0; cnt <= '0;
    end else if (start && !busy) begin
        busy <= 1; done <= 0; fail <= 0; elem <= 0; op <= 0; cnt <= '0;
    end else if (busy) begin
        if (rd && mem_rdata != {DW{val}} && !fail) begin
            fail <= 1; fail_addr <= mem_addr;          // log the first failure
        end
        if (!last_op) op <= 1'b1;
        else begin
            op <= 1'b0;
            cnt <= cnt + 1'b1;
            if (last_addr) begin
                if (elem == 3'd5) begin busy <= 0; done <= 1; end
                else elem <= elem + 1'b1;
            end
        end
    end
end
endmodule

```

// 16x8 RAM model with an optional stuck-at-1 fault on bit 3 of word 5

```

module ram_model #(parameter bit FAULT = 0) (
    input wire clk, we, input wire [3:0] addr, input wire [7:0] wdata,
    output wire [7:0] rdata
);
    reg [7:0] m [0:15];
    always @(posedge clk) if (we) m[addr] <= wdata;
    assign rdata = (FAULT && addr == 4'd5) ? (m[addr] | 8'h08) : m[addr];
endmodule

module tb_mbist;
    reg clk = 0, rst_n = 0, start = 0;
    always #5 clk = ~clk;
    wire b0, d0, f0, b1, d1, f1, we0, we1; wire [3:0] fa0, fa1, a0, a1;
    wire [7:0] wd0, wd1, rd0, rd1;
    mbist_marchc u0 (.clk(clk), .rst_n(rst_n), .start(start), .busy(b0), .done(d0),
        .fail(f0), .fail_addr(fa0), .mem_we(we0), .mem_addr(a0),
        .mem_wdata(wd0), .mem_rdata(rd0));
    ram_model #(0) m0 (.clk(clk), .we(we0), .addr(a0), .wdata(wd0), .rdata(rd0));
    mbist_marchc u1 (.clk(clk), .rst_n(rst_n), .start(start), .busy(b1), .done(d1),
        .fail(f1), .fail_addr(fa1), .mem_we(we1), .mem_addr(a1),
        .mem_wdata(wd1), .mem_rdata(rd1));
    ram_model #(1) m1 (.clk(clk), .we(we1), .addr(a1), .wdata(wd1), .rdata(rd1));
    integer cyc = 0;
    always @(posedge clk) if (b0) cyc = cyc + 1;
    initial begin
        #12 rst_n = 1;
    end
endmodule

```

```

@(negedge clk) start = 1; @(negedge clk) start = 0;
wait (d0 && d1);
$display("good RAM : done=%b fail=%b (%0d cycles = 10N, N=16)", d0, f0, cyc);
$display("faulty RAM: done=%b fail=%b first fail at word %0d", d1, f1, fa1);
$finish;
end
endmodule

```

```

good RAM : done=1 fail=0 (160 cycles = 10N, N=16)
faulty RAM: done=1 fail=1 first fail at word 5

```

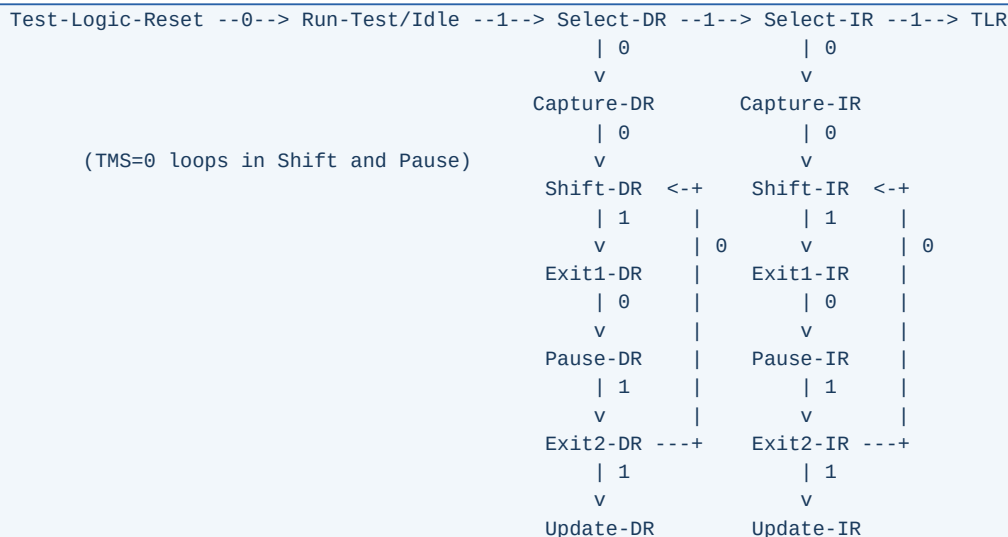
The engine executes exactly 10N operations and locates the injected stuck-at-1 fault. Production MBIST (Tessent MemoryBIST, Synopsys STAR Memory System) adds pipelined compare for synchronous-read SRAMs, multiple data backgrounds, diagnosis and repair analysis.

20.5 Logic BIST

LBIST puts the tester on the chip: an LFSR-based **PRPG** (pseudo-random pattern generator) feeds the scan chains, and a **MISR** (multiple-input signature register) compacts the responses into a signature compared against a golden value - the STUMPS architecture. It needs no external patterns, so it runs in the field: automotive (ISO 26262) parts run LBIST at every key-on and periodically to detect latent faults. Costs: **X-bounding** (a single X corrupts the signature, so every X source must be blocked), **test points** to reach random-pattern-resistant logic (wide AND trees), and the area of the controller.

20.6 JTAG: the TAP controller and boundary scan

IEEE 1149.1 (JTAG) defines a 4/5-wire serial port - TCK, TMS, TDI, TDO and optional TRST - and a **Test Access Port** (TAP) controller: a 16-state FSM advanced by TMS on every rising TCK. It was created for **boundary scan** (testing board interconnect), and became the universal access port for everything else: scan control, MBIST start/status, eFuse programming and processor debug.



Also: Capture-xR --1--> Exit1-xR; Exit1-xR --1--> Update-xR;
Update-xR --1--> Select-DR, --0--> Run-Test/Idle; TLR and RTI loop on 1 / 0.

Figure 20.2 - The 1149.1 TAP state diagram (the DR and IR columns are identical). Capture loads the selected register, Shift moves it one bit per TCK, Update transfers it to the parallel output. From any state, five TCKs with TMS = 1 reach Test-Logic-Reset.

The following implementation encodes the states with the widely used 4-bit codes from the standard's examples and adds a minimal instruction register with IDCODE (selected at reset, as the standard requires when IDCODE exists) and BYPASS (all ones).

```

// IEEE 1149.1 TAP controller: 16 states, advanced by TMS on rising TCK
module tap_ctrl (

```

```

input wire      tck, trst_n, tms,
output reg [3:0] state
);
localparam [3:0]
    TLR = 4'hF, RTI = 4'hC,
    SEL_DR = 4'h7, CAP_DR = 4'h6, SH_DR = 4'h2, EX1_DR = 4'h1,
    PAU_DR = 4'h3, EX2_DR = 4'h0, UPD_DR = 4'h5,
    SEL_IR = 4'h4, CAP_IR = 4'hE, SH_IR = 4'hA, EX1_IR = 4'h9,
    PAU_IR = 4'hB, EX2_IR = 4'h8, UPD_IR = 4'hD;
reg [3:0] nxt;
always @* begin
    case (state)
        TLR:    nxt = tms ? TLR    : RTI;
        RTI:    nxt = tms ? SEL_DR : RTI;
        SEL_DR: nxt = tms ? SEL_IR : CAP_DR;
        CAP_DR: nxt = tms ? EX1_DR : SH_DR;
        SH_DR:  nxt = tms ? EX1_DR : SH_DR;
        EX1_DR: nxt = tms ? UPD_DR : PAU_DR;
        PAU_DR: nxt = tms ? EX2_DR : PAU_DR;
        EX2_DR: nxt = tms ? UPD_DR : SH_DR;
        UPD_DR: nxt = tms ? SEL_DR : RTI;
        SEL_IR: nxt = tms ? TLR    : CAP_IR;
        CAP_IR: nxt = tms ? EX1_IR : SH_IR;
        SH_IR:  nxt = tms ? EX1_IR : SH_IR;
        EX1_IR: nxt = tms ? UPD_IR : PAU_IR;
        PAU_IR: nxt = tms ? EX2_IR : PAU_IR;
        EX2_IR: nxt = tms ? UPD_IR : SH_IR;
        UPD_IR: nxt = tms ? SEL_DR : RTI;
        default: nxt = TLR;
    endcase
end
always @(posedge tck or negedge trst_n)
    if (!trst_n) state <= TLR;
    else          state <= nxt;
endmodule

```

```

// Minimal TAP: 4-bit IR, IDCODE (default after reset) and BYPASS
module tap_top #(parameter [31:0] IDCODE = 32'h1234_5ACF) ( // bit 0 must be 1
    input wire tck, trst_n, tms, tdi,
    output reg tdo
);
localparam [3:0] I_IDCODE = 4'b0001, I_BYPASS = 4'b1111;
wire [3:0] st;
tap_ctrl_u_fsm (.tck(tck), .trst_n(trst_n), .tms(tms), .state(st));
reg [3:0] ir, ir_sh;
reg [31:0] id_sh;
reg      byp;
always @(posedge tck or negedge trst_n)
    if (!trst_n) begin
        ir <= I_IDCODE; ir_sh <= 4'b0; id_sh <= 32'b0; byp <= 1'b0;
    end else begin
        case (st)
            4'hF: ir <= I_IDCODE; // Test-Logic-Reset
            4'hE: ir_sh <= 4'b0101; // Capture-IR: ...01 pattern
            4'hA: ir_sh <= {tdi, ir_sh[3:1]}; // Shift-IR
            4'hD: ir <= ir_sh; // Update-IR
            4'h6: begin id_sh <= IDCODE; byp <= 1'b0; end // Capture-DR
            4'h2: begin id_sh <= {tdi, id_sh[31:1]}; byp <= tdi; end // Shift-DR
            default: ;
        endcase
    end
end

```

```
// TDO changes on the FALLING edge of TCK
always @(negedge tck or negedge trst_n)
  if (!trst_n)      tdo <= 1'b0;
  else if (st == 4'hA) tdo <= ir_sh[0];
  else if (st == 4'h2) tdo <= (ir == I_BYPASS) ? byp : id_sh[0];
endmodule
```

Simplifications for brevity: a real TAP drives TDO only in Shift states (tri-state otherwise), decodes all IR values (unknown opcodes select BYPASS), implements the mandatory SAMPLE/PRELOAD and EXTEST instructions with a boundary-scan register, and uses the state names instead of raw codes.

```
module tb_tap;
  reg tck = 0, trst_n = 0, tms = 1, tdi = 0;
  wire tdo;
  tap_top dut (.*);
  always #50 tck = ~tck;
  task clk_tms(input t, input d); begin
    @(negedge tck) begin tms = t; tdi = d; end
    @(posedge tck);
  end endtask
  integer i, k, fails = 0;
  reg [31:0] id; reg [7:0] bo;
  initial begin
    #120 trst_n = 1;
    // 1) read IDCODE: TLR -> RTI -> Sel-DR -> Cap-DR -> Shift-DR
    clk_tms(0,0); clk_tms(1,0); clk_tms(0,0); clk_tms(0,0);
    for (i = 0; i < 32; i = i + 1) begin
      @(negedge tck) tms = (i == 31);
      @(posedge tck) id[i] = tdo;
    end
    clk_tms(1,0); clk_tms(0,0);          // Update-DR -> RTI
    $display("IDCODE read = %h", id);
    // 2) load BYPASS: Sel-DR -> Sel-IR -> Cap-IR -> Shift-IR (4 ones)
    clk_tms(1,0); clk_tms(1,0); clk_tms(0,0); clk_tms(0,0);
    for (i = 0; i < 4; i = i + 1) clk_tms(i == 3, 1);
    clk_tms(1,0); clk_tms(0,0);          // Update-IR -> RTI
    clk_tms(1,0); clk_tms(0,0); clk_tms(0,0); // -> Shift-DR
    for (i = 0; i < 8; i = i + 1) begin    // shift 8'b1011_0010 through
      @(negedge tck) begin tms = 0; tdi = (8'b1011_0010 >> i) & 1; end
      @(posedge tck) bo[i] = tdo;
    end
    $display("BYPASS: tdi 01001101 -> tdo %b (one TCK of delay)",
      {bo[0], bo[1], bo[2], bo[3], bo[4], bo[5], bo[6], bo[7]});
    // 3) from any state, five TMS=1 clocks reach Test-Logic-Reset
    for (k = 0; k < 200; k = k + 1) begin
      for (i = 0; i < ($random & 15); i = i + 1) clk_tms($random, 0);
      for (i = 0; i < 5; i = i + 1) clk_tms(1, 0);
      #1 if (dut.st !== 4'hF) fails = fails + 1;
    end
    $display("5 x TMS=1 from 200 random states -> TLR, failures = %0d", fails);
    $finish;
  end
endmodule
```

```
IDCODE read = 12345acf
BYPASS: tdi 01001101 -> tdo 00100110 (one TCK of delay)
5 x TMS=1 from 200 random states -> TLR, failures = 0
```

The IDCODE shifts out LSB first; BYPASS is the one-bit register that shortens the chain when many devices share a JTAG port; and the reset property holds from 200 random states.

IDCODE field	Bits	Content
Version	[31:28]	Silicon revision

IDCODE field	Bits	Content
Part number	[27:12]	Assigned by the manufacturer
Manufacturer ID	[11:1]	JEDEC JEP106 code (bank + ID)
Marker	[0]	Always 1 - distinguishes IDCODE from BYPASS (which captures 0)

Boundary scan and the JTAG family of standards

Boundary scan places a scan cell between each pad and the core. With **EXTEST** the TAP drives values onto the board from one chip's output cells and captures them in another chip's input cells, testing solder joints and traces without any functional software; **SAMPLE/PRELOAD** snapshots pin values in normal operation. Each device is described by a **BSDL** file that board-test tools read.

Standard	Scope	Key structures
IEEE 1149.1	Board/chip test access; the TAP	TAP FSM, IR, BYPASS, IDCODE, boundary-scan register, BSDL
IEEE 1149.6	AC-coupled and differential high-speed IO	EXTEST_PULSE/TRAIN
IEEE 1500	Test wrappers for embedded cores	Wrapper instruction register (WIR), wrapper boundary register (WBR), WBY, serial and parallel ports - lets a core be tested in isolation from its neighbours
IEEE 1687 (IJTAG)	Access to on-chip instruments (MBIST, sensors, PLL tuning, trim registers)	Reconfigurable scan networks with SIBs (segment insertion bits); ICL describes the network, PDL the procedures, retargeted automatically
IEEE 1838	Test access for 3D stacked dies	Die wrapper and flexible parallel port

INTERVIEW INSIGHT: WHY DOES TDO CHANGE ON THE FALLING EDGE?

TDI and TMS are sampled on the rising edge of TCK and TDO is launched on the falling edge, giving half a TCK period of hold margin between chained devices on a board whose TCK arrives with different skews. It is the same reason scan-chain lock-up latches are inserted between flops in different clock domains on chip.

20.7 Summary

- Manufacturing test targets fault models - stuck-at, transition, path delay, bridging, cell-aware - and is measured by fault and test coverage.
- Scan replaces flops with mux-D scan flops in chains: shift, capture, shift. Compression and ATPG make it scale to millions of flops.
- DFT-friendly RTL: controllable clocks and resets in test mode, no latches or loops, ICG **te** connected, memories and X sources handled.
- MBIST runs March algorithms (March C- = 10N) at speed; LBIST uses PRPG + MISR for in-field self-test.
- The JTAG TAP is a 16-state FSM; IDCODE, BYPASS, boundary scan, IEEE 1500 wrappers and IEEE 1687 networks build on it.

20.8 Exercises

- 1 List the stuck-at faults of **inc4_scan**'s bit-1 adder logic that the pattern 0111 does **not** detect, and find a second pattern that detects them.
- 2 Estimate the test time of a 5M-flop design with 800 internal chains, 6,000 patterns and a 100 MHz shift clock. What happens if compression is removed and 16 external chains are used?
- 3 Extend **mbist_marchc** for a synchronous-read SRAM (data valid one cycle after the address). Where must the compare be pipelined?

- 4 Add a 4-bit user data register (opcode 4'b0100) to `tap_top` that drives an output `test_mode`, and a testbench that sets it through JTAG.
- 5 Show that four TMS=1 clocks are **not** always enough to reach Test-Logic-Reset by finding a starting state that needs five.

21. PPA Trade-offs and the Hand-Off to Physical Design

Every design decision is ultimately judged on **PPA** - power, performance and area - under the constraints of schedule and risk. The previous four chapters looked at each axis through a tool: synthesis (area), STA (performance), power analysis and UPF (power), and DFT (testability, which costs all three). This chapter puts them together: how to reason about trade-offs while the RTL is still cheap to change, how to write RTL that physical design can implement, what you hand over, and what happens between hand-off and tapeout.

21.1 The PPA triangle

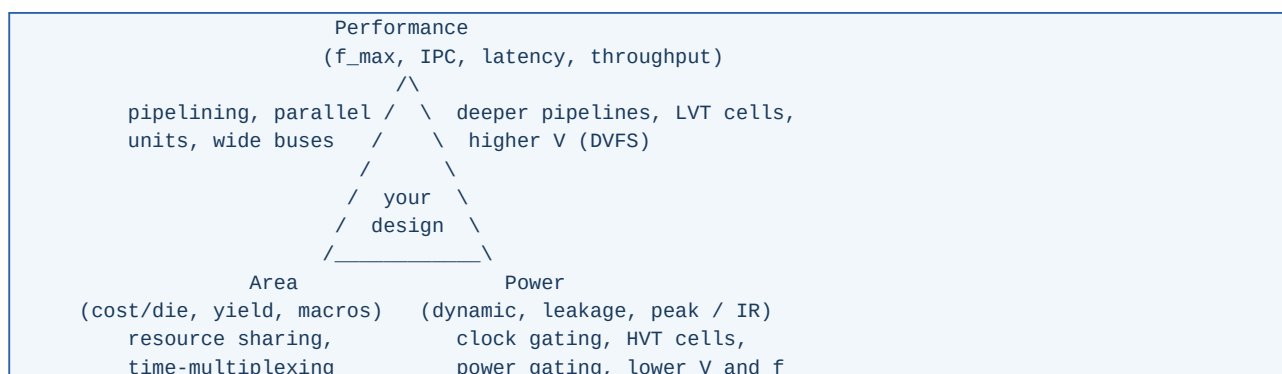


Figure 21.1 - Improving one corner usually costs another. The art is choosing the corner your product needs and not paying for the others.

Decision	Performance	Power	Area
Add a pipeline stage	+ f_max, - latency	+ flops & clock load, - glitching	+ flops
Duplicate a unit (2 MACs)	+ throughput	+ (or - via lower V/f for the same work)	+ ~2x unit
Share one unit (time-mux)	- throughput	- leakage, + muxes	- area
LVT instead of HVT cells	+ speed	++ leakage	~0
Wider SRAM word, fewer accesses	+ bandwidth	- energy per bit	+ macro width
Clock gating	~0 (enable timing)	-- dynamic	+ ICGs
Power gating	- wake-up latency	-- leakage when idle	+ switches, isolation, retention (5-15% of domain)
Reduce datapath precision	+ (shorter carry chains)	- (fewer toggles)	- (quadratic for multipliers)

ENERGY PER OPERATION IS THE METRIC THAT MATTERS

For battery devices and ML accelerators, compare architectures on **energy per task** (pJ/op, mJ/inference), not on power. A design that draws twice the power but finishes in a third of the time and then power-gates wins. This 'race to idle' argument is why fast cores with aggressive power gating often beat slow always-on ones.

21.2 Estimating area from RTL

You should be able to predict a block's area to within ~30% before synthesis. Count the three things that dominate: flops, big arithmetic, and memories; add a multiplier for control logic.

```
cell area ~ (N_flops x GE_flop + GE_arith + GE_logic) x A_GE + A_macros
block area ~ cell area / utilization      (utilization ~ 0.6 - 0.8)
```

```
example: 40,000 scan flops x 7 GE          = 280 kGE
         4 x (16x16 multiplier ~ 2 kGE)    =   8 kGE
         control + muxes (~1.2x flop GE)    =  336 kGE
         total ~ 624 kGE x 0.5 um^2 (28nm-class GE, assumed) = 0.31 mm^2
         / 0.7 utilization                  = 0.45 mm^2 + SRAM macros
```

The GE area is illustrative (roughly 28 nm class); take the real NAND2X1 area from your library. SRAM area comes from the memory compiler's datasheet, including its periphery and required halo/keep-out.

Structure	Rough cost	Remark
Flop (scan, with reset)	6-8 GE	Often 30-50% of a control-heavy block
N-bit adder	~7-10 GE per bit (fast adders more)	Grows when timing is tight
N x N multiplier	~6-8 x N ² GE	A 32x32 multiplier is ~7-8 kGE before timing upsizing
M:1 mux, W bits	~(M-1) x W x 2 GE	Plus wires - see congestion below
Flop-based register file, D x W	D x W flops + read muxes	Above a few kbit an SRAM macro is smaller and lower power
SRAM macro	From compiler: bit cell ~0.1-0.2 um ² at 7-16 nm plus periphery	Small macros are periphery-dominated

21.3 Floorplan-aware RTL

Below ~28 nm, wires, not gates, dominate delay across a chip: gate delay shrinks with each node while the resistance of thin wires grows. A signal crossing a large SoC may need more than one clock cycle even with optimal repeaters. RTL written without the floorplan in mind produces paths that no amount of physical optimisation can fix.

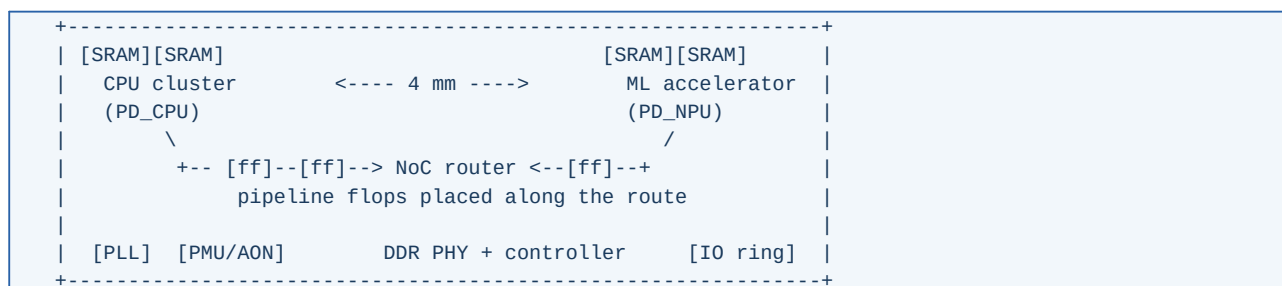


Figure 21.2 - A floorplan dictates which paths are long. Register slices along inter-block routes make them timeable; hierarchy that mirrors the floorplan lets each partition be implemented alone.

- **Hierarchy that matches the floorplan.** Physical partitions (hard or soft blocks) are cut along module boundaries. A module that mixes logic destined for opposite corners of the die forces a flat implementation or long wires.
- **Register at partition boundaries.** 'Flop in, flop out' makes inter-partition paths start and end at flops; the budget is the whole cycle for the wire. Combinational feed-throughs between partitions are the hardest paths on a chip.
- **Plan pipeline stages for distance.** A rough rule is that a well-buffered wire covers only a few millimetres per GHz-class cycle at advanced nodes (and much less on thin lower metals). Parameterise the number of stages so the physical team can tune them late without redesign.

- **Keep logic near its memory.** SRAM macros are placed first, usually at partition edges. Memory output data should be registered close to the macro before travelling.
- **Localise high-fanout control.** Broadcast configuration bits and stall signals should be re-registered locally (Chapter 18); a single global stall across a large accelerator rarely closes timing - use elastic buffers or credits instead.

```
// Parameterized "wire pipeline": N flop stages that physical design can
// spread along a long route (one stage per ~1-2 mm in an advanced node).
module wire_pipe #(parameter int W = 32, parameter int N = 2) (
    input  wire      clk, rst_n,
    input  wire      in_vld,
    input  wire [W-1:0] in_dat,
    output wire      out_vld,
    output wire [W-1:0] out_dat
);
    reg [W-1:0] dat [0:N];
    reg        vld [0:N];
    always @* begin dat[0] = in_dat; vld[0] = in_vld; end
    genvar i;
    for (i = 1; i <= N; i = i + 1) begin : g_stage
        always @(posedge clk or negedge rst_n)
            if (!rst_n) vld[i] <= 1'b0;
            else       vld[i] <= vld[i-1];
        always @(posedge clk)
            if (vld[i-1]) dat[i] <= dat[i-1];    // data flops need no reset
        end
        assign out_vld = vld[N];
        assign out_dat = dat[N];
    endmodule

module tb_wpipe;
    reg clk = 0, rst_n = 0, in_vld = 0; reg [31:0] in_dat = 0;
    wire out_vld; wire [31:0] out_dat;
    wire_pipe #(.W(32), .N(3)) dut (.*);
    always #5 clk = ~clk;
    integer t_in, t_out;
    initial begin
        #12 rst_n = 1;
        @(negedge clk) begin in_vld = 1; in_dat = 32'hCAFE_F00D; t_in = $time; end
        @(negedge clk) in_vld = 0;
        wait (out_vld); t_out = $time;
        $display("out_dat=%h after %0d cycles", out_dat, (t_out - t_in + 5) / 10);
        $finish;
    end
endmodule
```

```
out_dat=cafef00d after 3 cycles
```

A valid-only pipeline (no back-pressure). The data flops are enabled by the valid bit, which also gives synthesis a clean clock-gating enable (Chapter 19). If the path also carries **ready**, pipeline it with skid buffers or convert to credit-based flow control (Chapter 10) - a combinational **ready** across 4 mm is the same long wire in the other direction.

21.4 Congestion: when RTL makes the chip unroutable

A netlist can meet timing on paper and still be impossible to route: there are more wires crossing a region than routing tracks. Congestion forces the placer to spread cells (worse timing, more area) or the router to detour (worse timing, DRC violations). Its root cause is usually a structure in the RTL.

RTL structure	Why it congests	RTL-level remedy
Wide, many-input muxes (e.g. 64:1 x 128 bits)	All 8,192 input wires converge on one small area	Split into staged muxes placed near their sources; one-hot AND-OR trees distributed along a bus
Full crossbars (N x M x W)	Wires grow as N x M x W; every master reaches every slave	Partial crossbar, hierarchical interconnect, NoC (Chapter 14), narrower paths for rare masters
Barrel shifters / permutation networks	log N stages of long criss-crossing wires	Restrict shift amounts; pipeline; use a logarithmic layout-friendly structure
Big register files with many ports	Each read port is a full mux over all entries	Bank the file; replicate for read ports; use SRAM macros
Fully associative lookups (CAMs)	Every entry compares against a broadcast key	Set-associative structures; hash
High-fanout broadcast	One net to thousands of pins	Local re-registering, tree distribution
Pin-dense small modules	A tiny block with 2,000 ports cannot be placed without a wire jam	Rebalance hierarchy; narrower interfaces

ASK FOR AN EARLY FLOORPLAN TRIAL

Run a quick placement (DC-Topographical/NXT, Genus iSpatial, Fusion Compiler, or OpenROAD global placement) on early RTL and look at the congestion map. Hot spots almost always point to one module and one construct. Fixing it in RTL costs a day; fixing it after the floorplan is frozen costs weeks.

21.5 The hand-off to physical design

Whether you hand off RTL (the physical team runs synthesis) or a netlist, the deliverable is not just code. The package must let someone who has never read your RTL implement it correctly, and it must be **consistent**: the same revision of RTL, constraints, power intent and DFT setup.

Deliverable	Content	Checked by
RTL + file list	Tagged revision, include paths, defines, top-level parameters	Lint, CDC/RDC clean (Chapter 25)
Gate-level netlist	Synthesised, scan-inserted Verilog	Formal equivalence RTL vs netlist (LEC)
SDC per mode	Functional, scan shift, scan capture, MBIST, low-power modes; exceptions documented	Constraint linting (e.g. Synopsys TCM/GCA, Cadence Conformal Constraint Designer); timing review
UPF	Power intent matching the netlist's hierarchy	Static low-power checks; power-aware simulation
Scan DEF + DFT files	Scan chain definition (so P&R can reorder chains by placement), test protocol (.spf), ATPG setup	DFT DRC; ATPG coverage report
Memory and IP list	Macro configurations, compiler versions, LEF/Liberty views, hard-IP integration notes	View consistency checks
Clock specification	Clock tree roots, balancing groups, non-default rules, skew targets, generated clocks	CTS team review
Floorplan guidance	Partitioning, macro placement preferences, pin placement, long-route pipeline stages	Early trial placement
Activity files	SAIF/VCD from representative workloads	Power signoff
Known issues / waivers	Every lint/CDC/LEC waiver with justification	Design review

21.6 ECOs: changing a design after hand-off

An **engineering change order** (ECO) modifies the netlist directly instead of re-running synthesis and implementation, preserving months of timing closure. There are two kinds:

- **Timing ECOs** (resize, buffer, swap Vt) made by the physical team to close the last paths; function unchanged, checked by equivalence.
- **Functional ECOs** fix a bug found late. The RTL is fixed first; then a tool (Cadence Conformal ECO, Synopsys Formality ECO) compares the old netlist with the new RTL and generates a minimal **patch**. The patch is implemented with the smallest possible physical disturbance and LEC must pass against the new RTL.
- **Pre-mask vs post-mask**. Before masks are made, any cell can be added. After the base layers are fabricated, a **metal-only ECO** can rewire only metal layers, using **spare cells** (NAND, NOR, inverters, muxes, flops sprinkled across the die and tied off) or **ECO filler / gate-array cells** that can be personalised in metal. A metal-only fix saves the cost and weeks of new base-layer masks.

DESIGN FOR ECO FROM THE RTL

Experienced teams add **chicken bits** - configuration register bits that disable new or risky features (a prefetcher, an optimisation, a bypass path) - so a silicon bug can be worked around in software. They also keep ECO-prone logic in its own hierarchy with boundary optimisation off, reserve spare register bits in CSR blocks, and write RTL whose synthesised names are predictable, so that the patch touches only a few cells.

21.7 Signoff, seen from the RTL seat

Signoff check	What it verifies	How RTL can cause a failure
STA (MCM, SI)	Setup/hold in every mode and corner including crosstalk delta-delay	Too many logic levels, wrong exceptions, unregistered block boundaries
Formal equivalence (LEC)	RTL == netlist after every transformation	Sim/synth mismatches, x-assignments, retiming without setup
IR drop (static/dynamic)	Supply droop under current draw	Massive simultaneous switching (all MACs start on one edge, clock-gate wake-up of a whole domain, scan shift); stagger enables
Electromigration (EM)	Long-term wire wear from current density	Very high-activity nets, big clock drivers
DRC / LVS / antenna	Layout obeys foundry rules and matches the netlist	Rarely RTL - but congestion makes DRC closure hard
Low-power static checks	Isolation, level shifting, retention and always-on paths consistent with UPF	Unisolated outputs, signals crossing domains outside the plan
DFT signoff	Coverage targets, pattern count, test time	Uncontrollable clocks/resets, X sources
Gate-level simulation	Reset, X-propagation, timing (SDF) sanity	Missing resets, reliance on initial values (Chapter 26)

21.8 The tapeout checklist

RTL owner's tapeout checklist

- ☐ RTL frozen and tagged; every late change covered by regression and LEC against the tagged netlist.
- ☐ Lint, CDC and RDC clean with all waivers reviewed and signed.
- ☐ Functional and code coverage closed against the verification plan (Chapters 22-24).
- ☐ Every SDC exception traceable to an RTL mechanism; constraint checks clean in all modes.
- ☐ UPF reviewed: domains, isolation clamp values, retention list and power-state table match the architecture; power-aware sim of all transitions passed.

- ☐ DFT: coverage targets met (stuck-at, transition), MBIST passing on all memories, JTAG IDCODE correct, test modes documented.
- ☐ Gate-level simulations with SDF (min/max) passed for boot, reset and key test cases.
- ☐ Chicken bits and spare cells in place; ECO strategy agreed.
- ☐ Register map, IDCODE, version registers and documentation match the silicon revision.
- ☐ Silicon bring-up plan written: first tests, debug access, known risks and their workarounds.

21.9 Summary

- PPA decisions are made in the architecture and the RTL; compare alternatives on energy per task and area per throughput.
- Estimate area from flop count, arithmetic and memories before synthesis; know the cost of muxes and multipliers.
- Write floorplan-aware RTL: hierarchy that matches partitions, registered boundaries, parameterised wire pipelines, local control.
- Wide muxes, crossbars, shifters and multi-port register files cause congestion; fix them in RTL.
- The hand-off is a consistent package: RTL/netlist, SDC, UPF, scan DEF, memory list, clock spec and documented waivers.
- Functional ECOs use generated patches and spare cells; chicken bits provide software escape hatches; signoff covers timing, equivalence, power integrity, physical rules and test.

21.10 Exercises

- 1 Estimate the area of a block with 25k flops, eight 16x16 multipliers, a 32:1 x 64-bit mux and 256 kbit of SRAM. State your assumptions.
- 2 A 4x4 crossbar with 128-bit data is congested. Propose two RTL alternatives and compare their wire counts and latency.
- 3 Extend `wire_pipe` with a `ready` input and skid buffers so that it supports back-pressure; verify that no data is lost or duplicated under random stalls.
- 4 A metal-only ECO must invert the polarity of an interrupt output. Which spare cells are needed, and what checks must run afterwards?
- 5 Write the hand-off README for the `dot2_2c` block of Chapter 18, listing every deliverable in the table above that applies to it.

PART V

Verification

Verification consumes more engineering effort than design on every modern chip. This part teaches the craft from the ground up: self-checking testbenches, assertions and coverage, the UVM methodology used across the industry, the static sign-off checks (lint, CDC, formal, equivalence) that catch what simulation misses, and the debug, gate-level, FPGA-prototyping and emulation techniques that carry a design all the way to silicon.

22. Self-Checking Testbenches in SystemVerilog

A design that has not been verified does not work - it only has not failed yet. On a typical SoC project there are two to three verification engineers for every designer, and the verification code base is larger than the RTL it checks. Even so, the most important verification skill for an RTL designer is not UVM or formal: it is the ability to write a small, **self-checking** testbench in an afternoon that proves a block does what the spec says, and keeps proving it every night in regression.

A testbench is *self-checking* when it decides pass or fail by itself, by comparing what the DUT (device under test) does against an independent prediction. Nobody looks at waveforms to decide whether the test passed; waveforms are for debugging a test that has already failed. This chapter builds that discipline with plain SystemVerilog that runs on any simulator, including the open-source Icarus Verilog used throughout this book. Chapter 23 adds assertions and coverage, and Chapter 24 scales the same architecture up to UVM.

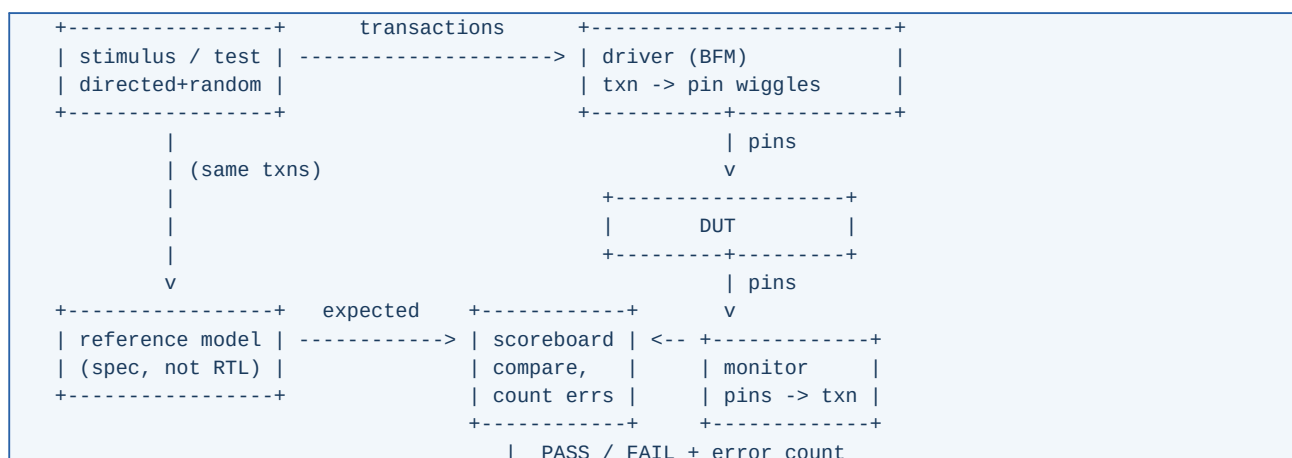
22.1 Why self-checking, and what it must check

Approach	How pass/fail is decided	Scales to
Eyeball the waveform	A human looks at signals once	One block, one afternoon, never again
Golden log diff	Compare a printed log with a saved 'good' log	Breaks on every harmless change of print format or timing
Self-checking	TB predicts every output and compares automatically; ends with PASS/FAIL and an error count	Thousands of seeds in nightly regression, CI gating
Self-checking + assertions + coverage	Plus protocol rules checked every cycle and a measure of what was exercised	Sign-off of an IP or SoC (Chapters 23-24)

A good self-checking testbench answers three questions, and a weak one answers only the first: **(1)** Did every output that should have appeared appear, with the right value? **(2)** Did nothing appear that should not have (spurious outputs, duplicated beats, wrong flags)? **(3)** Did the test actually exercise the interesting cases (full, empty, overflow, back-pressure, reset in the middle)? Question 3 is the job of functional coverage (Chapter 23).

22.2 Testbench architecture

Every serious testbench - from a 60-line module to a million-line UVM environment - has the same layers. Learning them once, on a small example, is the fastest way to understand UVM later, because UVM is only this picture expressed as reusable classes.



The layered testbench. The monitor observes pins passively; the reference model predicts from the spec; the scoreboard compares.

Component	Responsibility	Typical form in plain SV
Clock/reset generator	Free-running clock; reset asserted, then released synchronously	<code>always #5 clk = ~clk;</code> + an <code>initial</code> block
Stimulus (test)	Decides WHAT to send: directed sequences, random mixes, corner cases	<code>initial</code> block calling tasks
Driver / BFM	Decides HOW: converts a transaction into cycle-accurate pin activity, obeys the protocol	A <code>task</code> per transaction type (<code>push</code> , <code>apb_write</code>)
Monitor	Passively watches pins and reconstructs transactions; never drives	<code>always @(posedge clk)</code> sampling handshakes
Reference model	Predicts correct behaviour from the spec, at transaction level	Queue, associative array, function, or a C model via DPI
Scoreboard	Compares observed with predicted, counts errors, reports at the end	Queue compare + <code>\$error</code> + counter
Checkers	Protocol rules every cycle	Immediate or concurrent assertions (Chapter 23)

THE REFERENCE MODEL MUST BE INDEPENDENT OF THE RTL

If the same engineer writes the RTL and the model from the same mental picture, the two will agree on the same misunderstanding of the spec. Write the model at a higher level of abstraction (a queue for a FIFO, a `+` for an adder, a dictionary for a register file), from the spec rather than from the RTL, and ideally have someone else write it. A model that copies the RTL's pointer arithmetic verifies nothing.

22.3 Clock and reset generation, and driving without races

```

`timescale 1ns/1ps
localparam real CLK_PERIOD = 10.0;           // 100 MHz
logic clk = 1'b0, rst_n = 1'b0;
always #(CLK_PERIOD/2) clk = ~clk;

initial begin
    rst_n = 1'b0;
    repeat (5) @(posedge clk);                // hold reset for several cycles
    @(negedge clk) rst_n = 1'b1;              // release away from the active edge
end

```

A generic clock/reset block. Releasing reset on the inactive edge avoids a race with the DUT's flops sampling it.

The single most common testbench bug is a **race** between the testbench driving an input and the DUT sampling it on the same clock edge (Chapter 4). If the TB does `@(posedge clk) wr_en = 1;` with a blocking assignment, whether the DUT's `always_ff` sees the old or the new value depends on process ordering - it can differ between simulators and even between two runs of one simulator after an unrelated edit. Three race-free conventions are used in practice:

- **Drive on the opposite edge** (`@(negedge clk)`), sample on the active edge. Simple and robust; used in all examples here.
- **Drive with non-blocking assignments** at the active edge (`@(posedge clk) wr_en <= 1;`), so the update lands in the NBA region after every flop has sampled.
- **Clocking blocks** (section 22.10) with explicit input and output skews - the standard in UVM agents.

22.4 Tasks for bus transactions

The driver layer is a set of tasks, one per transaction type, that hide the protocol from the test. The test then reads like the spec: `apb_write(addr, data)`, `apb_check(addr, expected)`. Below is a complete, verified register test for a small APB slave (the APB protocol itself is covered in Chapter 13).

```

`timescale 1ns/1ps
module apb_regs (
    input  logic      pclk, presetn,
    input  logic      psel, penable, pwrite,
    input  logic [3:0] paddr,
    input  logic [31:0] pwidth,
    output logic [31:0] prdata,
    output logic      pready
);
    logic [31:0] regs [4];
    assign pready = 1'b1; // zero-wait-state slave
    always_ff @(posedge pclk or negedge presetn)
        if (!presetn) foreach (regs[i]) regs[i] <= '0;
        else if (psel && penable && pwrite) regs[paddr[3:2]] <= pwidth;
    assign prdata = regs[paddr[3:2]];
endmodule

```

The DUT: four 32-bit registers on APB, zero wait states.

```

`timescale 1ns/1ps
module tb_apb;
    logic pclk = 0, presetn = 0, psel = 0, penable = 0, pwrite = 0, pready;
    logic [3:0] paddr = '0;
    logic [31:0] pwidth = '0, prdata;
    int errors = 0;
    apb_regs dut (. *);
    always #5 pclk = ~pclk;

    // One APB write: SETUP phase, then ACCESS phase until PREADY.
    task automatic apb_write(input logic [3:0] addr, input logic [31:0] data);
        @(negedge pclk); psel = 1; penable = 0; pwrite = 1; paddr = addr; pwidth = data;
        @(negedge pclk); penable = 1;
        @(posedge pclk); while (!pready) @(posedge pclk);
        @(negedge pclk); psel = 0; penable = 0;
    endtask

    task automatic apb_read(input logic [3:0] addr, output logic [31:0] data);
        @(negedge pclk); psel = 1; penable = 0; pwrite = 0; paddr = addr;
        @(negedge pclk); penable = 1;
        @(posedge pclk); while (!pready) @(posedge pclk);
        data = prdata; // sample in ACCESS
        @(negedge pclk); psel = 0; penable = 0;
    endtask

    // Read-and-compare wrapper: the building block of every register test.
    task automatic apb_check(input logic [3:0] addr, input logic [31:0] exp);
        logic [31:0] got;
        apb_read(addr, got);
        if (got != exp) begin
            errors++;
            $error("APB rd @%h: got %h exp %h", addr, got, exp);
        end
    endtask

    initial begin // watchdog
        #10us $fatal(1, "TIMEOUT - DUT hung (pready never asserted?)");
    end

    initial begin
        $timeformat(-9, 0, " ns", 0); // %t prints in ns
        #22 presetn = 1;
        apb_check(4'h0, 32'h0); // reset value
    end
endmodule

```

```

    apb_write(4'h4, 32'hCAFE_F00D);
    apb_write(4'hC, 32'h1234_5678);
    apb_check(4'h4, 32'hCAFE_F00D);
    apb_check(4'hC, 32'h1234_5678);
    apb_check(4'h8, 32'h0); // untouched register
    $display("[%t] APB register test: %0d errors", $time, errors);
    $finish;
end
endmodule

```

An APB testbench built from transaction tasks, with a watchdog and a read-compare wrapper.

```

[200 ns] APB register test: 0 errors
tb_apb.sv:50: $finish called at 200000 (1ps)

```

To prove the testbench can fail - always do this once - we mutate the DUT so that writes keep only the low 16 bits (`regs[...] <= pwwdata & 32'h0000_FFFF`) and rerun:

```

ERROR: tb_apb.sv:32: APB rd @4: got 0000f00d exp cafef00d
      Time: 140000 Scope: tb_apb.apb_check
ERROR: tb_apb.sv:32: APB rd @c: got 00005678 exp 12345678
      Time: 170000 Scope: tb_apb.apb_check
[200 ns] APB register test: 2 errors
tb_apb.sv:50: $finish called at 200000 (1ps)

```

Mutation testing: a testbench that has never been seen to fail has not been shown to check anything.

DETAILS THAT SEPARATE A PROFESSIONAL TASK FROM A FRAGILE ONE

Declare tasks `automatic` so that concurrent calls from two threads get separate copies of their locals. Always wait for the handshake (`while (!pready) @(posedge pclk);`) rather than assuming zero wait states - the next DUT revision will add one. Put a **watchdog** in every test: a hung handshake otherwise burns a regression slot until the farm's wall-clock limit kills it, with no useful message.

22.5 Reporting: \$display, \$error, \$fatal and friends

System task	Severity	Effect	Use for
<code>\$display</code> / <code>\$write</code>	-	Print (with / without newline)	Progress, summaries
<code>\$info</code>	info	Print with file:line and scope	Milestones in long tests
<code>\$warning</code>	warning	Print; simulation continues	Suspicious but legal behaviour
<code>\$error</code>	error	Print; continues; tools count it	A mismatch - keep going to collect more evidence
<code>\$fatal(n, ...)</code>	fatal	Print, then end simulation (exit code n on most tools)	Unrecoverable: timeout, bad config, file not found
<code>\$finish</code> / <code>\$stop</code>	-	End simulation / break to the interactive prompt	Normal end of test / debugging

Regression scripts decide pass/fail by scanning the log, so make the verdict machine-readable: print exactly one `TEST PASSED` or `TEST FAILED` line, and make the script treat any `ERROR/FATAL` string, a missing verdict line, or a non-zero exit code as a failure. A test that crashes before printing its verdict must never count as a pass.

PITFALL: THE TEST THAT PASSES BECAUSE IT DID NOTHING

A scoreboard that compares zero transactions reports zero errors. Always count checks as well as errors, and fail the test if the count is below what the stimulus should have produced. Likewise, a queue-based scoreboard must check at the end that the expected queue is **empty** - leftover entries are outputs the DUT dropped.

22.6 File I/O: vectors in, logs out

Task	Purpose
<code>\$readmemh(file, mem) / \$readmemb</code>	Load a memory array from a hex/binary text file: test vectors, ROM contents, golden results produced by a Python or C model
<code>fd = \$fopen(file, "w")</code>	Open a file; returns 0 on failure - check it
<code>\$fdisplay(fd, ...) / \$fwrite</code>	Write a formatted line
<code>\$fscanf(fd, fmt, ...) / \$fgets</code>	Read formatted data or a line; returns the number of items matched, and <code>\$feof(fd)</code> tells you when to stop
<code>\$fclose(fd)</code>	Flush and close
<code>\$sformatf(fmt, ...)</code>	Format into a <code>string</code> (for messages)
<code>\$value\$plusargs("SEED=%d", s)</code>	Read <code>+SEED=42</code> from the simulator command line - how regressions pass knobs to tests

A common industrial pattern is to generate vectors and expected results with a bit-accurate Python or C model, dump both as hex files, and let the testbench `$readmemh` them. It is the cheapest way to reuse an algorithm team's golden model; DPI-C (Chapter 24) is the next step when the model must run in lock-step.

22.7 Randomization: \$urandom and constrained-random classes

Directed tests check the cases you thought of. **Random** tests find the ones you did not - the overflow at the same cycle as a read, the back-pressure during a burst. The simplest source of randomness works everywhere:

```

wr_en = ($urandom_range(99) < 60);    // 60% probability
wdata = $urandom;                    // 32 random bits, truncated
len   = $urandom_range(16, 1);        // uniform in [1:16]
if (!$value$plusargs("SEED=%d", seed)) seed = 1;
void'($urandom(seed));                // seed the thread's generator

```

Procedural randomization - supported by every simulator, including Icarus.

SystemVerilog's real strength is **constrained randomization**: you declare the legal space with `rand` variables and `constraint` blocks, and a constraint solver picks values. Icarus Verilog does not implement constraints (sorry: `Constraint declarations not supported`), so the following is shown without output; it is standard IEEE 1800 and runs on VCS, Xcelium, Questa and Riviera.

```

class bus_txn;
  typedef enum bit [1:0] {READ, WRITE, IDLE} kind_e;
  rand kind_e kind;
  rand bit [31:0] addr;
  rand bit [31:0] data;
  rand bit [3:0] len;
  randc bit [2:0] id;          // cyclic: all 8 values before any repeats

  constraint c_align { addr[1:0] == 2'b00; }
  constraint c_range { addr inside {[32'h4000_0000 : 32'h4000_0FFF]}; }
  constraint c_kind { kind dist {READ := 45, WRITE := 45, IDLE := 10}; }
  constraint c_len { (kind == IDLE) -> (len == 0);
                    len <= 8; }
  constraint c_order { solve kind before len; }

  function void post_randomize();
    if (kind == READ) data = '0;          // don't-care for reads
  endfunction
endclass

```

```
// in the test
bus_txn t;
t = new();
repeat (1000) begin
    if (!t.randomize()) $fatal(1, "randomize() failed");
    drive(t);
end
// inline constraints for a directed-random corner case
if (!t.randomize() with { kind == WRITE; addr == 32'h4000_0FFC; })
    $fatal(1, "inline randomize failed");
t.c_range.constraint_mode(0);           // switch a constraint off
t.id.rand_mode(0);                     // freeze one field
```

Constrained-random transaction (not runnable on Icarus).

Construct	Meaning
<code>rand / randc</code>	Random each call / random-cyclic (a permutation before repeating)
<code>inside {[a:b], c}</code>	Set membership, including ranges
<code>dist {v := w} / {[a:b] := w}</code>	Weighted distribution: <code>:=</code> gives each value weight <code>w</code> ; <code>:=</code> splits <code>w</code> across the range
<code>a -> b</code>	Implication: if <code>a</code> then <code>b</code> must hold
<code>solve a before b</code>	Changes the probability distribution, not the legal space
<code>randomize() with {...}</code>	Add constraints for one call
<code>pre_/post_randomize()</code>	Hooks to compute derived fields
return value of <code>randomize()</code>	0 if the constraints are unsatisfiable - always check it

	Directed testing	Constrained-random testing
Written as	Explicit sequences for each feature	Legal space + a few knobs; seeds do the rest
Finds	Bugs in cases you anticipated	Bugs in interactions you did not anticipate
Checking	Can be hard-coded per test	Needs a reference model and scoreboard
Knowing when done	Test plan checklist	Functional coverage (Chapter 23)
Best for	Bring-up, reset values, specific corners, regressions of fixed bugs	Bulk of verification of any block with a rich input space

COVERAGE-DRIVEN VERIFICATION IN ONE SENTENCE

Randomize within legal constraints, check everything automatically, measure what was hit with coverage, and write directed or further-constrained tests only for the holes. The methodology, not the language, is what makes it work.

22.8 A complete self-checking testbench: synchronous FIFO

We now put every layer together for a 4-deep synchronous FIFO with show-ahead (first-word-fall-through) reads, the kind designed in Chapter 9. The reference model is a SystemVerilog **queue**; the monitor and scoreboard are merged in one clocked block that updates the model with exactly the handshake rules of the spec.

```
module sync_fifo #(parameter int W = 8, parameter int DEPTH = 4) (
    input logic      clk, rst_n,
    input logic      wr_en, rd_en,
    input logic [W-1:0] wdata,
    output logic [W-1:0] rdata,
    output logic      full, empty
);
```

```

localparam int AW = $clog2(DEPTH);
logic [W-1:0] mem [DEPTH];
logic [AW:0] wptr, rptr;          // extra MSB distinguishes full/empty
assign empty = (wptr == rptr);
assign full = (wptr[AW] != rptr[AW]) && (wptr[AW-1:0] == rptr[AW-1:0]);
assign rdata = mem[rptr[AW-1:0]]; // show-ahead (FWFT) read
always_ff @(posedge clk or negedge rst_n)
  if (!rst_n) begin
    wptr <= '0; rptr <= '0;
  end else begin
    if (wr_en && !full) begin
      mem[wptr[AW-1:0]] <= wdata;
      wptr <= wptr + 1'b1;
    end
    if (rd_en && !empty) rptr <= rptr + 1'b1;
  end
endmodule

```

DUT: 4-entry FIFO with an extra pointer bit to distinguish full from empty.

```

module tb_fifo;
  localparam int W = 8, DEPTH = 4;
  logic clk = 0, rst_n = 0;
  logic wr_en = 0, rd_en = 0;
  logic [W-1:0] wdata = '0, rdata;
  logic full, empty;
  int errors = 0, checks = 0;
  logic [W-1:0] model_q[$];          // reference model: a queue

  sync_fifo #(.W(W), .DEPTH(DEPTH)) dut (.*);

  always #5 clk = ~clk;              // 100 MHz clock

  // ----- monitor + scoreboard (samples at the active edge)
  always @(posedge clk) if (rst_n) begin
    if (full != (model_q.size() == DEPTH)) begin
      errors++; $error("full=%b but model holds %0d", full, model_q.size());
    end
    if (empty != (model_q.size() == 0)) begin
      errors++; $error("empty=%b but model holds %0d", empty, model_q.size());
    end
    if (rd_en && !empty) begin
      checks++;
      if (rdata != model_q[0]) begin
        errors++; $error("rdata=%h expected %h", rdata, model_q[0]);
      end
      void'(model_q.pop_front());
    end
    if (wr_en && !full) model_q.push_back(wdata);
  end

  // ----- driver tasks (drive on the inactive edge)
  task automatic push(input logic [W-1:0] d);
    @(negedge clk); wr_en = 1; wdata = d; rd_en = 0;
    @(negedge clk); wr_en = 0;
  endtask

  task automatic pop();
    @(negedge clk); rd_en = 1; wr_en = 0;
    @(negedge clk); rd_en = 0;
  endtask

```



```

initial begin
  repeat (2) @(negedge clk);
  rst_n = 1;
  // ---- directed tests: fill to full, overflow attempt, drain, underflow
  for (int i = 0; i < DEPTH + 1; i++) push(8'hA0 + i); // 5th push is dropped
  if (!full) $fatal(1, "FIFO not full after %0d pushes", DEPTH);
  repeat (DEPTH + 1) pop(); // 5th pop is ignored
  $display("[%0t] directed phase done: checks=%0d errors=%0d", $time, checks, errors);
  // ---- constrained-random phase: random mix of push/pop every cycle
  repeat (2000) begin
    @(negedge clk);
    wr_en = ($urandom_range(99) < 60); // 60% write
    rd_en = ($urandom_range(99) < 50); // 50% read
    wdata = $urandom;
  end
  @(negedge clk); wr_en = 0; rd_en = 0;
  @(negedge clk);
  $display("[%0t] random phase done: checks=%0d errors=%0d", $time, checks, errors);
  if (errors == 0) $display("TEST PASSED");
  else $display("TEST FAILED");
  $finish;
end
endmodule

```

Self-checking FIFO testbench: directed phase (fill, overflow, drain, underflow) then 2000 constrained-random cycles.

```

[220] directed phase done: checks=4 errors=0
[20240] random phase done: checks=931 errors=0
TEST PASSED
tb_fifo.sv:63: $finish called at 20240 (1s)

```

Note what the scoreboard checks: not only the data, but the **full** and **empty** flags **every cycle** against the model's occupancy, and it updates the model only when the DUT is allowed to accept the operation (**wr_en && !full**). The random mix of 60% writes and 50% reads keeps the FIFO oscillating around full, where the bugs live. Now inject the classic bug - dropping the MSB comparison from the full flag, so that **full** is also true when the FIFO is empty:

```

assign full = (wptr[AW-1:0] == rptr[AW-1:0]); // BUG: MSB check removed

```

```

ERROR: tb_fifo.sv:17: full=1 but model holds 0
Time: 25 Scope: tb_fifo
ERROR: tb_fifo.sv:17: full=1 but model holds 0
Time: 35 Scope: tb_fifo
ERROR: tb_fifo.sv:17: full=1 but model holds 0
Time: 45 Scope: tb_fifo

```

First lines of the failing run: the bug is caught on the first clock after reset.

22.9 Class-based transactions, a reference function and file vectors

The second example checks a combinational ALU. The transaction is a class (Icarus supports classes, but not **randomize()**, so a **rand_fill()** method stands in for it and biases 10% of **b** values to the **8'hFF** corner). Directed vectors come from a hex file, results go to a log file, and the reference model is a function written from the spec.

```

module alu (
    input  logic [1:0] op,          // 0:ADD 1:SUB 2:AND 3:XOR
    input  logic [7:0] a, b,
    output logic [7:0] y,
    output logic      c            // carry-out / borrow-out
);
always_comb begin
    c = 1'b0;
    unique case (op)
        2'd0: {c, y} = a + b;
        2'd1: {c, y} = {1'b0, a} - {1'b0, b};
        2'd2: y = a & b;
        2'd3: y = a ^ b;
    endcase
end
endmodule

```

DUT: 8-bit ALU with carry/borrow.

```

0_ff_01
1_00_01
2_f0_3c
3_aa_55

```

vectors.hex - {op, a, b}; underscores are legal in \$readmemh files.

```

class alu_txn;
    bit [1:0] op;
    bit [7:0] a, b;
    function void rand_fill();          // stand-in for randomize()
        op = $urandom_range(3);
        a = $urandom;
        b = ($urandom_range(9) == 0) ? 8'hFF : $urandom; // bias 10% to a corner
    endfunction
    function string sprint();
        return $sformatf("op=%0d a=%h b=%h", op, a, b);
    endfunction
endclass

module tb_alu;
    logic [1:0] op; logic [7:0] a, b, y; logic c;
    logic [17:0] vec [4];              // {op[1:0], a[7:0], b[7:0]}
    int fd, errors = 0, n = 0;
    alu dut (. *);

    // Reference model: written independently of the RTL, from the spec.
    function automatic logic [8:0] alu_ref(logic [1:0] o, logic [7:0] x, z);
        case (o)
            2'd0: return {1'b0, x} + {1'b0, z};
            2'd1: return {1'b0, x} - {1'b0, z};
            2'd2: return {1'b0, x & z};
            default: return {1'b0, x ^ z};
        endcase
    endfunction

    task automatic check(alu_txn t);
        logic [8:0] exp;
        op = t.op; a = t.a; b = t.b;
        #1; // let combinational logic settle
        exp = alu_ref(t.op, t.a, t.b);
        n++;
        if ({c, y} !== exp) begin
            errors++;
        end
    endtask
endmodule

```

```

        $error("%s: got %h exp %h", t.sprint(), {c, y}, exp);
    end
    $fdisplay(fd, "%s -> c=%b y=%h", t.sprint(), c, y);
endtask

initial begin : main
    alu_txn t;
    t = new();
    fd = $fopen("alu_log.txt", "w");
    if (fd == 0) $fatal(1, "cannot open log file");
    $readmemh("vectors.hex", vec);          // directed vectors from a file
    foreach (vec[i]) begin
        t.op = vec[i][17:16]; t.a = vec[i][15:8]; t.b = vec[i][7:0];
        check(t);
    end
    repeat (1000) begin                    // random vectors
        t.rand_fill();
        check(t);
    end
    $fclose(fd);
    $display("ALU: %0d vectors, %0d errors -> %s", n, errors, errors ? "FAIL" : "PASS");
    $finish;
end
endmodule

```

ALU testbench: class transaction, reference function, \$readmemh, \$fopen/\$fdisplay.

```

ALU: 1004 vectors, 0 errors -> PASS
tb_alu.sv:59: $finish called at 1004 (1s)

```

```

op=0 a=ff b=01 -> c=1 y=00
op=1 a=00 b=01 -> c=1 y=ff
op=2 a=f0 b=3c -> c=0 y=30
op=3 a=aa b=55 -> c=0 y=ff

```

First lines of alu_log.txt: the four directed vectors, including carry-out and borrow corners.

Swapping the operands of the subtraction in the RTL (`{1'b0, b}` - `{1'b0, a}`) is caught immediately:

```

ERROR: tb_alu.sv:38: op=1 a=00 b=01: got 001 exp 1ff
      Time: 2 Scope: tb_alu.check
ERROR: tb_alu.sv:38: op=1 a=c5 b=e5: got 020 exp 1e0
      Time: 9 Scope: tb_alu.check

```

PITFALL: == VERSUS === IN CHECKERS

`if (y != exp)` is **false** when `y` is `X`, because `!=` returns `X` and an `if` treats `X` as false - an `X` output silently passes. Always compare with the case-equality operators `===` / `!==`, which treat `X` and `Z` as values. Every check in this chapter uses `!==`.

ICARUS-SPECIFIC NOTES

Icarus 12 aborted on a `case` statement over a class property inside a class method, and did not assign a concatenation of class properties used as an lvalue; both are simulator limitations, which is why the reference model is a module-level function and the vector fields are assigned by slices. Commercial simulators accept either form.

22.10 Interfaces, modports, clocking blocks and virtual interfaces

An **interface** bundles the signals of one protocol, so that a valid/ready channel is one port instead of three, and it can carry the protocol's tasks with it. Icarus supports interfaces with tasks accessed hierarchically, which is enough for this verified example:

```

interface vr_if #(parameter int W = 8) (input logic clk);
    logic        valid, ready;
    logic [W-1:0] data;
    // Driver task lives with the signals it wiggles.
    task automatic send(input logic [W-1:0] d);
        @(negedge clk); valid = 1; data = d;
        @(posedge clk); while (!ready) @(posedge clk); // wait for the handshake
        @(negedge clk); valid = 0;
    endtask
endinterface

module acc_sink (input logic        clk, rst_n, valid,
                 input logic [7:0] data,
                 output logic [15:0] sum);
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) sum <= '0;
        else if (valid) sum <= sum + data; // ready is driven by the TB below
endmodule

module tb_if;
    logic clk = 0, rst_n = 0;
    logic [15:0] sum;
    int exp = 0;
    always #5 clk = ~clk;
    vr_if #(.W(8)) bus (.clk(clk));
    acc_sink u_sink (.clk, .rst_n, .valid(bus.valid && bus.ready),
                    .data(bus.data), .sum);

    // Backpressure: ready on a pseudo-random ~70% of cycles.
    always @(negedge clk) bus.ready <= ($urandom_range(9) < 7);

    initial begin
        bus.valid = 0; bus.data = '0;
        repeat (2) @(negedge clk); rst_n = 1;
        for (int i = 1; i <= 20; i++) begin bus.send(8'(i)); exp += i; end
        @(negedge clk);
        if (sum !== exp) $fatal(1, "sum=%0d expected %0d", sum, exp);
        $display("[%0t] sum=%0d matches model - PASS", $time, sum);
        $finish;
    end
endmodule

```

An interface that carries its own driver task, exercised under random back-pressure.

```

[480] sum=210 matches model - PASS
tb_vr_if.sv:39: $finish called at 480 (1s)

```

The full SystemVerilog feature set goes further. **Modports** give each side of the interface a direction view; **clocking blocks** define when the testbench samples inputs and drives outputs relative to the clock, which removes races by construction; and a **virtual interface** is a handle to an interface instance that a class can hold, which is how class-based drivers and monitors (and all of UVM) reach the pins. Icarus does not support clocking blocks, interface ports or virtual interfaces, so this is shown without output:

```

interface vr_if #(parameter int W = 8) (input logic clk, input logic rst_n);
    logic        valid, ready;
    logic [W-1:0] data;
    clocking drv_cb @(posedge clk);
        default input #1step output #1; // sample just before, drive just after
        output valid, data;
        input ready;
    endclocking
    clocking mon_cb @(posedge clk);

```

```

    default input #1step;
    input valid, ready, data;
endclocking
modport dut_snk (input valid, data, output ready);
modport tb_drv  (clocking drv_cb, input rst_n);
modport tb_mon  (clocking mon_cb, input rst_n);
endinterface

class vr_driver #(int W = 8);
    virtual vr_if #(W) vif;                // handle to the real interface
    function new(virtual vr_if #(W) vif);
        this.vif = vif;
    endfunction
    task send(bit [W-1:0] d);
        vif.drv_cb.valid <= 1'b1;          // clocking drives use <=
        vif.drv_cb.data  <= d;
        do @(vif.drv_cb); while (!vif.drv_cb.ready); // handshake on this edge
        vif.drv_cb.valid <= 1'b0;
    endtask
endclass

module tb_top;
    logic clk = 0, rst_n = 0;
    always #5 clk = ~clk;
    vr_if #(8) bus (.clk(clk), .rst_n(rst_n));
    my_sink dut (.s(bus.dut_snk));          // DUT sees only its modport
    vr_driver #(8) drv;
    initial begin
        drv = new(bus);                    // actual -> virtual interface
        repeat (2) @(negedge clk); rst_n = 1;
        repeat (10) drv.send($urandom);
    end
endmodule

```

Clocking blocks and a virtual-interface driver (IEEE 1800; runs on commercial simulators, not on Icarus).

```

clk  ____/~~~~~\_____/~~~~~\____
      ^ edge      ^ edge
input #1step: | sampled in the Preponed region (value just before edge)
output #1    : |--1ns--> driven here, well after every flop sampled

=> TB reads what the DUT flops saw; TB drives never race DUT sampling

```

Clocking-block skews: #1step input sampling and a small output skew.

Program blocks (`program ... endprogram`) are a related SystemVerilog construct whose `initial` blocks execute in the Reactive region, after the design has settled, so testbench code cannot race the design even without clocking blocks. They were popular in VMM-era testbenches; UVM does not use them, and most teams today rely on clocking blocks and non-blocking drives instead. Know what they are for interviews; you will rarely write one.

INTERVIEW ANGLE: WHY DO CLASSES NEED VIRTUAL INTERFACES?

Classes are dynamic objects created at run time; interfaces and modules are static hierarchy elaborated before time zero. A class cannot contain an interface instance, so it holds a **reference** to one - a virtual interface - assigned at run time (in UVM, through `uvm_config_db`). That one level of indirection is what lets the same driver class be reused on any number of interface instances.

22.11 Summary

- A self-checking TB decides pass/fail itself by comparing DUT behaviour with an independent reference model; waveforms are for debug only.
- Layers: stimulus, driver, DUT, monitor, reference model, scoreboard - the same picture UVM implements with classes.
- Avoid TB/DUT races: drive on the inactive edge, with non-blocking assignments, or through clocking blocks.
- Hide protocols in `automatic` tasks; add a watchdog to every test; count checks as well as errors; use `!=` in comparisons.
- Mix directed tests (corners, bring-up) with constrained-random stimulus; always check the return value of `randomize()`.
- Prove every testbench can fail by mutating the DUT at least once.
- Interfaces bundle protocols; virtual interfaces connect class-based testbenches to them.

22.12 Exercises

- 1 Extend the FIFO testbench with a final check that the model queue is empty after draining, and a minimum-checks threshold that fails a test which compared fewer than 500 reads.
- 2 Add a `+SEED=` plusarg to `tb_fifo` and run 20 seeds from a shell loop; grep the logs for the verdict line.
- 3 Write `apb_write/apb_read` for an APB slave with random wait states (PREADY low for 0-3 cycles) and show that the tasks still pass while a version that assumes zero wait states fails.
- 4 Inject three different bugs into the ALU (wrong carry for ADD, XOR implemented as OR, `unique case` missing an item) and confirm the testbench catches each. Which one needs the biased corner values?
- 5 Rewrite `bus_txn` constraints so that 4 KB boundary crossings are impossible for a burst of `len` words starting at `addr`.
- 6 Explain, with a timing sketch, what goes wrong if the FIFO testbench drives `wr_en` with a blocking assignment at `@(posedge clk)`.

23. SystemVerilog Assertions and Functional Coverage

A scoreboard checks the **end-to-end** result of a transaction: the right data came out. It says nothing about **how** the data got there - whether valid was withdrawn and re-asserted, whether a one-hot state vector briefly had two bits set, whether a grant was issued without a request. Assertions check those internal and protocol-level rules every cycle, at the exact place and time they are violated. Functional coverage is their counterpart: it measures which of the interesting scenarios the tests actually exercised. Together they turn 'we ran a lot of tests' into 'we can show what was checked and what was exercised'.

23.1 Why assertions pay for themselves

Benefit	Explanation
Observability	A bug inside a block may take thousands of cycles to reach an output, or be masked entirely. An assertion fires at the cycle and the signal where the rule broke
Localization	The message names the violated property, file and line - debug starts at the root cause, not at a corrupted packet 10 microseconds later
Executable specification	<code>valid && !ready => valid</code> is unambiguous in a way that a paragraph of spec is not
Reuse across engines	The same properties run in simulation, in formal property verification (Chapter 25) and, synthesized, in emulation and FPGA prototypes (Chapter 26)
Integration safety	Assertions bound to an IP's interface catch misuse by the SoC team who integrates it

WHO WRITES WHICH ASSERTIONS

Designers write **white-box** assertions about their own implementation (FSM encodings, pointer invariants, no FIFO overflow inside the block) as they write the RTL - they know the assumptions and it costs minutes. Verification engineers write **black-box** interface and end-to-end properties from the spec. A good rule of thumb for designers: every time you think 'this can never happen', write an assertion that says so.

23.2 Immediate versus concurrent assertions

	Immediate	Concurrent
Syntax	<code>assert (expr) else ...;</code> inside procedural code	<code>assert property (@(posedge clk) ...)</code> as a module item
Evaluated	When the statement executes, on current values	At each clock tick, on values sampled in the Preponed region (just before the edge)
Spans time?	No - one instant	Yes - sequences over many cycles
Glitch sensitivity	Plain immediate: fires on transient zero-time glitches; deferred <code>assert #0/</code> <code>assert final</code> report only the settled value	None - sampled once per clock
Formal tools	Supported (mostly as combinational invariants)	Native form
Icarus Verilog 12	Plain immediate supported; deferred not	Not supported

Because immediate assertions run on every simulator, they are the portable way to put checks into RTL and testbenches. With a few registered copies of past values they can even express simple multi-cycle rules, as the next verified example shows.

23.3 Immediate assertions in practice: a protocol checker

The checker below enforces the two AXI-style valid/ready rules from Chapter 10: once **valid** is asserted it must stay asserted until **ready**, and the payload must not change while stalled. It also checks that the control signals are never X after reset. It is a separate module, so it can be instantiated next to any valid/ready port - or attached with **bind** (section 23.9).

```
// Protocol checker for a valid/ready channel, written with immediate
// assertions so that it runs on any simulator (including Icarus).
module vr_checker #(parameter int W = 8) (
    input logic clk, rst_n, valid, ready,
    input logic [W-1:0] data
);
    logic          pend;          // "valid && !ready" seen last cycle
    logic [W-1:0] data_q;
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) pend <= 1'b0;
        else begin
            pend    <= valid && !ready;
            data_q <= data;
        end

    always @(posedge clk) if (rst_n) begin
        // No X/Z on control signals out of reset (XOR-reduce is X if any bit is)
        assert ((^ {valid, ready}) !== 1'bx)
            else $error("X on valid/ready");
        // Once offered, a transfer must stay offered with stable data (AXI rule)
        if (pend) begin
            assert (valid)
                else $error("valid dropped before ready (data %h lost)", data_q);
            assert (!valid || data == data_q)
                else $error("data changed %h -> %h while stalled", data_q, data);
        end
    end
endmodule
```

A portable valid/ready protocol checker using immediate assertions and registered history.

```
module tb_chk;
    logic clk = 0, rst_n = 0, valid = 0, ready = 0;
    logic [7:0] data = '0;
    always #5 clk = ~clk;
    vr_checker #(.W(8)) u_chk (.*);

    initial begin
        repeat (2) @(negedge clk); rst_n = 1;
        // Legal: offer 8'h11, stall two cycles, then accept
        @(negedge clk); valid = 1; data = 8'h11; ready = 0;
        repeat (2) @(negedge clk);
        ready = 1;
        @(negedge clk); valid = 0; ready = 0;
        // Illegal #1: change data while stalled (t=85 edge)
        @(negedge clk); valid = 1; data = 8'h22;
        @(negedge clk); data = 8'h23;
        // Illegal #2: withdraw valid while stalled (t=95 edge)
        @(negedge clk); valid = 0;
        @(negedge clk);
        $finish;
    end
endmodule
```

A directed test: one legal stalled transfer, then two deliberate violations.


```

ERROR: vr_checker.sv:25: data changed 22 -> 23 while stalled
      Time: 85 Scope: tb_chk.u_chk
ERROR: vr_checker.sv:23: valid dropped before ready (data 23 lost)
      Time: 95 Scope: tb_chk.u_chk
tb_chk.sv:20: $finish called at 100 (1s)

```

The legal stall passes silently; each violation is reported at the clock edge where it becomes observable.

PITFALL: X-CHECKS ON CONCATENATIONS

The natural form `assert (!$isunknown({valid, ready}))` returned a wrong result for a concatenation of nets in Icarus 12 and fired on every clock. The portable Verilog-2001 idiom used above - XOR-reduce and compare with `1'bx` - works everywhere, because an XOR reduction is X if and only if some input bit is X or Z. On commercial simulators `$isunknown` is the idiomatic choice. Either way: test your checkers with a known-good stimulus before trusting a failure.

The same check as a concurrent assertion is one line per rule, and the sampling semantics take care of the history registers and of the reset: this is exactly why SVA exists.

```

a_hold:  assert property (@(posedge clk) disable iff (!rst_n)
              valid && !ready |>= valid);
a_stable: assert property (@(posedge clk) disable iff (!rst_n)
              valid && !ready |>= $stable(data));

```

The concurrent equivalents (not runnable on Icarus).

23.4 Anatomy of a concurrent assertion

```

// label    directive    clock    reset
a_req_gnt: assert property (@(posedge clk) disable iff (!rst_n)
              $rose(req) |>= ##[1:4] gnt)           // antecedent |>= consequent
              else $error("gnt did not follow req within 4 cycles");

// The same, with named sequence and property for reuse:
sequence s_req; $rose(req); endsequence
property p_handshake(int max);
  @(posedge clk) disable iff (!rst_n) s_req |>= ##[1:max] gnt;
endproperty
a_req_gnt4: assert property (p_handshake(4));

```

Each clock tick **starts a new attempt** of the property. An attempt whose antecedent does not match succeeds **vacuously** and is discarded; an attempt whose antecedent matches spawns a thread that tracks the consequent over the following cycles. Several attempts can be in flight at once, which is why SVA is far more compact than the equivalent hand-written FSM checker.

cycle	1	2	3	4	5	6	7	8
req	___/~~~~~_____/_~~~~~_____							
gnt	_____/_~~~~_____ (never for 2nd req)							

```

attempt @2: $rose(req)=1 -> look for gnt in cycles 3..6 -> gnt@4: PASS
attempt @3: $rose(req)=0 -> vacuous success
attempt @6: $rose(req)=1 -> no gnt in 7..10 -> FAIL at 10

```

Evaluation of `$rose(req) |>= ##[1:4] gnt`: one attempt per clock; values are sampled just before each edge.

SAMPLED VALUES: THE SOURCE OF MOST SVA CONFUSION

A concurrent assertion sees each signal's value from the **Preponed** region - the value it had just before the clock edge, i.e. the same value the design's flops sampled. If a waveform viewer shows `gnt` rising exactly at edge 4, the assertion sees it at edge 5. When an assertion 'fires one cycle late', this is almost always why.

23.5 Sequence operators

Operator	Meaning	Example
<code>##n</code>	Exactly n clock ticks later	<code>a ##2 b</code> : b two cycles after a
<code>##[m:n], ##[m:\$]</code>	Between m and n ticks; <code>\$</code> = eventually	<code>req ##[1:4] gnt</code>
<code>s [*n], [*m:n]</code>	Consecutive repetition	<code>busy [*3]</code> : busy for exactly 3 consecutive cycles
<code>b [->n]</code>	Goto repetition: the n-th occurrence of b (not necessarily consecutive); ends on the n-th	<code>start ##1 done [->3]</code>
<code>b [=n]</code>	Non-consecutive repetition; may end after the n-th	<code>wr [=2] ##1 flush</code>
<code>s1 and s2, s1 or s2</code>	Both / either sequence matches	-
<code>s1 intersect s2</code>	Both match with the same start and end	-
<code>b throughout s</code>	b holds at every cycle of s	<code>!rst throughout (req ##[1:\$] gnt)</code>
<code>s1 within s2</code>	s1 matches somewhere inside s2	-
<code>first_match(s)</code>	Only the earliest match of s	Bounds open-ended ranges in antecedents

cycle	1	2	3	4	5	6	7	
done	0	1	0	1	0	1	0	
done [->3]		-----	-----	-----	-----	-----	-----	matches at cycle 6
done [=3]		-----	-----	-----	-----	-----	-----	matches at 6 and 7 (and later, while done stays 0)
busy [*3]	needs busy=1 in three consecutive cycles							

Goto versus non-consecutive repetition.

23.6 Implication: `|->` and `|=>`

A `|->` B (overlapping): if A matches, B must start in the **same** cycle in which A ended. **A `|=>` B (non-overlapping)**: B starts in the **next** cycle; it is exactly `A |-> ##1 B`. If A does not match, the property is vacuously true.

cycle	1	2	3	
valid&&!ready	1	.	.	
<code> -></code>	consequent checked at 1			e.g. 'valid -> !\$isunknown(data)'
<code> =></code>	consequent checked at 2			e.g. 'valid && !ready => valid'

PITFALL: VACUOUS SUCCESS HIDES DEAD ASSERTIONS

An assertion whose antecedent never occurs 'passes' forever. If the reset polarity in `disable iff` is wrong, or a signal name is bound to a constant, the assertion checks nothing and nobody notices. Every important assertion should have a matching **cover property** on its antecedent, and assertion reports should list attempts, vacuous and real successes. Formal tools report vacuity directly.

PITFALL: IMPLICATION INSIDE COVER

cover property (a `|->` b) is satisfied vacuously by every cycle in which a is false, so it proves nothing. **Cover sequences**: `cover property (a ##1 b)`.

23.7 Sampled-value functions

Function	True when (sampled at this tick)	Note
<code>\$rose(x)</code>	LSB of x went 0 -> 1 since the previous tick	On a bus uses the LSB only - use <code>\$changed</code> or compare values for buses
<code>\$fell(x)</code>	LSB went 1 -> 0	-
<code>\$stable(x)</code>	x equals its value at the previous tick	Whole vector
<code>\$changed(x)</code>	<code>!\$stable(x)</code>	SV-2009
<code>\$past(x, n)</code>	Value of x n ticks ago (default 1)	Before n ticks have elapsed it returns the initial value - guard with reset
<code>\$onehot(x), \$onehot0(x)</code>	Exactly one / at most one bit set	FSM encodings, grants, mux selects
<code>\$isunknown(x)</code>	Any bit is X or Z	X-checks on control
<code>\$countones(x)</code>	Number of 1 bits	Credits, occupancy

23.8 assert, assume, cover - and disable iff

Directive	In simulation	In formal verification
<code>assert property</code>	Error if the property fails	Goal to prove for all legal inputs
<code>assume property</code>	Checked like an assert (flags illegal stimulus)	Constraint on inputs: only traces that satisfy it are explored
<code>cover property</code>	Counts matches; shows a scenario happened	Tool must find a trace that reaches it (reachability)
<code>restrict property</code>	Ignored	Constraint, but not checked in simulation

`disable iff (expr)` asynchronously aborts every in-flight attempt while the expression is true - almost always the reset. A module can declare `default clocking` and `default disable iff` once, so the individual assertions stay short, as in the library below. The same interface properties are **assumptions** when you formally verify the block that receives the interface and **assertions** when you verify the block that drives it - which is why good assertion IP is written with a parameter that switches between the two.

23.9 Binding assertions without touching the RTL

`bind` instantiates a checker module inside another module (or one instance of it) from outside, as if the instantiation were written in the target's source. The checker's ports can connect to any signal in the target's scope, including internal ones. Verification teams use it to add assertions to frozen or third-party RTL, and to keep checkers out of the files that go to synthesis.

```
// In a verification-only file, compiled with the testbench:
bind sync_fifo fifo_sva #(.DEPTH(DEPTH)) u_fifo_sva (
    .clk    (clk),   .rst_n (rst_n),
    .wr_en  (wr_en), .rd_en (rd_en),
    .full   (full),  .empty (empty),
    .count  (wptr - rptr) // expression over internal signals
);
// bind to one instance only:
bind tb_top.u_soc.u_dma.u_fifo fifo_sva #(.DEPTH(16)) u_chk (.*);
```

bind: every `sync_fifo` instance gets a `fifo_sva` checker.

23.10 A library of useful assertions

The following modules collect properties that appear on almost every project. They parse cleanly with Verilator 5 (which reports several operators as unsupported for simulation) and are standard IEEE 1800-2017; run them on a commercial simulator or formal tool.

```
module vr_sva #(parameter int W = 8) (
    input logic clk, rst_n, valid, ready,
    input logic [W-1:0] data
);
    default clocking cb @(posedge clk); endclocking
    default disable iff (!rst_n);

    // 1. Once asserted, valid stays high until ready (no withdrawal)
    a_valid_hold: assert property (valid && !ready |>= valid)
        else $error("valid withdrawn before handshake");
    // 2. Payload stable while stalled
    a_data_stable: assert property (valid && !ready |>= $stable(data));
    // 3. No X on valid after reset
    a_no_x: assert property (!$isunknown(valid));
    // 4. Liveness-style bound: a stalled transfer completes within 16 cycles
    a_no_starve: assert property (valid |>-> ##[0:16] ready);
    // 5. Cover: see a back-to-back transfer and a 3-cycle stall
    c_b2b: cover property (valid && ready ##1 valid && ready);
    c_stall: cover property ((valid && !ready) [*3] ##1 (valid && ready));
endmodule

module fifo_sva #(parameter int DEPTH = 4) (
    input logic clk, rst_n, wr_en, rd_en, full, empty,
    input logic [$clog2(DEPTH):0] count
);
    default clocking cb @(posedge clk); endclocking
    default disable iff (!rst_n);
    a_no_ovf: assert property (full |>-> !wr_en); // env obligation
    a_no_udf: assert property (empty |>-> !rd_en);
    a_excl: assert property (!(full && empty));
    a_cnt_up: assert property (wr_en && !rd_en && !full |>= count == $past(count) + 1);
    a_full_ok: assert property (count == DEPTH |>-> full);
    c_full: cover property (full ##1 !full);
endmodule

module misc_sva (input logic clk, rst_n, req, gnt, start, done,
                input logic [3:0] state, input logic [7:0] irq);
    default clocking cb @(posedge clk); endclocking
    default disable iff (!rst_n);
    a_onehot: assert property ($onehot(state));
    a_req_gnt: assert property ($rose(req) |>-> ##[1:4] gnt);
    a_gnt_req: assert property (gnt |>-> req);
    a_done_3rd: assert property (start |>-> ##1 done [->3] ##1 !done);
    a_irq_pls: assert property ($rose(irq[0]) |>= $fell(irq[0]));
    m_env: assume property (start |>= !start);
endmodule
```

Reusable valid/ready, FIFO and miscellaneous properties.

Property	Catches
a_valid_hold , a_data_stable	Sources that drop or change a transfer under back-pressure - the most common AXI/stream bug
a_no_starve	Deadlock or starvation (bounded liveness; unbounded ##[1:\$] is a true liveness property, better handled by formal)

Property	Catches
<code>a_no_ovf</code> , <code>a_no_udf</code>	Writes when full / reads when empty - use as assertions on the user of a FIFO whose spec forbids them (the FIFO of Chapter 22 tolerates them by design)
<code>a_excl</code> , <code>a_full_ok</code> , <code>a_cnt_up</code>	Pointer arithmetic errors inside the FIFO
<code>a_onehot</code>	FSM corruption, glitchy state updates, missing default in a one-hot FSM
<code>a_req_gnt</code> , <code>a_gnt_req</code>	Arbiter response bound; grant without request
<code>a_done_3rd</code>	Protocols that count events (bursts, credits)
<code>a_irq_pls</code>	Pulse-type signals that must last exactly one cycle

23.11 Functional coverage: covergroups, coverpoints, bins and crosses

Code coverage tells you which lines ran. **Functional coverage** tells you which **features** were exercised, as defined by a human from the spec and the test plan: every opcode, operands at their extremes, the FIFO being full while a read and a write arrive together. A **covergroup** samples variables on an event and counts hits in **bins**; a **cross** counts combinations.

```

module cg_demo (input logic clk, rst_n, valid, ready, input logic [1:0] op,
                input logic [7:0] a, b);
  covergroup alu_cg @(posedge clk iff (rst_n && valid && ready));
    option.per_instance = 1;
    cp_op: coverpoint op {
      bins add = {2'd0};
      bins sub = {2'd1};
      bins log[] = {2'd2, 2'd3};
    }
    cp_a: coverpoint a {
      bins zero = {8'h00};
      bins max = {8'hFF};
      bins low = {[8'h01:8'h7F]};
      bins high = {[8'h80:8'hFE]};
    }
    cp_b: coverpoint b {
      bins zero = {0};
      bins max = {255};
      bins other = default;
    }
    x_op_a: cross cp_op, cp_a {
      ignore_bins logic_extremes = binsof(cp_op.log) && binsof(cp_a.low);
    }
  endgroup
  alu_cg cg = new();
endmodule

```

A covergroup for the ALU of Chapter 22 (IEEE 1800; not supported by Icarus or Verilator 5.020).

```

covergroup fifo_cg @(posedge clk iff rst_n);
  cp_occ: coverpoint count {
    bins empty = {0};
    bins mid[] = {[1:DEPTH-1]}; // one bin per value
    bins full = {DEPTH};
    illegal_bins over = {[DEPTH+1:$]}; // error if ever sampled
  }
  cp_ops: coverpoint {wr_en, rd_en} {
    bins idle = {2'b00}; bins rd = {2'b01}; bins wr = {2'b10}; bins both = {2'b11};
  }
  x_occ_ops: cross cp_occ, cp_ops; // e.g. 'both' while full
  cp_seq: coverpoint count { // transition bins
    bins fill_up = (DEPTH-1 => DEPTH);
  }
endgroup

```

```

    bins drain      = (DEPTH ==> DEPTH-1);
    bins blip       = (0 ==> 1 ==> 0);
    bins stay_full  = (DEPTH [*3]);           // full for 3 samples
  }
endgroup
fifo_cg cg = new();           // instantiate; sample() can also be called by hand

```

FIFO coverage with illegal bins, a cross and transition bins.

Bin kind	Syntax	Purpose
Automatic	<code>coverpoint x;</code>	One bin per value (up to <code>auto_bin_max</code> , default 64 buckets)
Explicit / range	<code>bins lo = {[0:15]};</code>	One bin for a set
Array	<code>bins v[] = {[0:7]};</code>	One bin per value in the set
Default	<code>bins other = default;</code>	Catch-all, excluded from the coverage score
Transition	<code>bins t = (1 ==> 2 ==> 3);</code>	Value sequences across samples
ignore_bins	<code>ignore_bins x = {...};</code>	Excluded - unreachable or irrelevant
illegal_bins	<code>illegal_bins x = {...};</code>	Run-time error if hit - a cheap checker
Cross	<code>cross a, b;</code>	Cartesian product; trim with <code>binsof(...) intersect {...}</code> and ignore/illegal bins

CROSSES EXPLODE - DESIGN THEM

A cross of three 16-bin coverpoints has 4096 bins, most of them meaningless. Cross only the dimensions whose **interaction** matters in the spec (opcode x operand corner, occupancy x operation), and remove impossible combinations with `ignore_bins` so the report shows real holes instead of noise.

`cover property` complements covergroups for **temporal** scenarios (a 3-cycle stall followed by a transfer, back-to-back bursts); covergroups are better for **value** spaces and crosses.

23.12 Coverage closure

- Test plan first.** List features from the spec; for each, decide how it is checked (scoreboard, assertion) and how it is measured (coverpoint, cover property). This 'vplan' is reviewed with designers and architects.
- Run constrained-random regressions** with many seeds; merge the coverage databases of all runs.
- Analyse holes.** For each unhit bin decide: the stimulus cannot reach it (tighten or add constraints, write a directed test), it is unreachable by design (exclude with a documented reason), or the coverage model is wrong (fix the model).
- Iterate** until the agreed goals are met - typically 100% functional coverage with reviewed exclusions, and high code coverage with every unhit line explained.
- Rank tests** by unique coverage contribution to build a fast 'smoke' regression and to drop redundant tests.

COVERAGE MEASURES STIMULUS, NOT CORRECTNESS

A bin is hit when the scenario occurred, whether or not anything checked the result. 100% coverage with a scoreboard that compares nothing is 0% verification. Coverage is only meaningful together with checkers - and the checkers must be proven to fire (mutation testing, Chapter 22).

23.13 Code coverage types

Metric	Measures	Blind spot
Line / statement	Each executable statement ran	Says nothing about values or combinations

Metric	Measures	Blind spot
Branch	Each if/else arm and case item was taken	Missing branches (the unwritten <code>else</code>) cannot be measured
Condition / expression	Each sub-condition independently flipped the outcome (FEC/MC-DC-like)	Expensive; noisy on wide expressions
Toggle	Every bit went 0->1 and 1->0	A toggling bus can still carry wrong values; key for connectivity at SoC level
FSM	Every state and every transition visited	Tools extract FSMs heuristically - check the extraction
Assertion	Each assertion attempted, succeeded non-vacuously, failed	Only as good as the assertions

INTERVIEW ANGLE: CODE VERSUS FUNCTIONAL COVERAGE

Code coverage is automatic and objective but only tells you what the **implementation** did; it cannot detect a missing feature, because missing code has no lines. Functional coverage is written by hand from the **spec**, so it can reveal that a feature was never exercised - or never implemented - but it is only as good as the person who wrote it. Sign-off needs both: high code coverage with reviewed exclusions and 100% of the agreed functional coverage.

23.14 Summary

- Assertions check rules every cycle where they are violated; coverage measures which scenarios were exercised.
- Immediate assertions run anywhere (including Icarus); concurrent assertions sample values before the clock edge and express multi-cycle behaviour compactly.
- Know the operators: `##n`, `##[m:n]`, `[*n]`, `[->n]`, `[=n]`, `throughout`, and the implications `|->` (same cycle) and `|=>` (next cycle).
- `$rose/$fell` use only the LSB; `$past` needs a reset guard; `disable iff` aborts attempts during reset.
- `assert/assume/cover` mean different things to simulation and formal; watch for vacuous passes and never cover an implication.
- `bind` attaches checkers without editing RTL.
- Close coverage from a reviewed test plan; coverage without checking is meaningless.

23.15 Exercises

- 1 Write concurrent assertions for APB (Chapter 13): `PENABLE` only in the cycle after `PSEL` rises, `PADDR/PWRITE/PWDATA` stable from `SETUP` to the end of `ACCESS`, and `PSEL` held until `PREADY`.
- 2 Extend `vr_checker` with an immediate-assertion check that `data` is not X whenever `valid` is high. Verify it with Icarus using a stimulus that drives `8'hxx` with `valid`.
- 3 For `$rose(req) |-> ##[1:4] gnt`, draw a waveform where two attempts are in flight at the same time and state when each passes or fails.
- 4 Design a covergroup for a round-robin arbiter with 4 requesters: which crosses are meaningful, and which bins should be illegal?
- 5 Explain why `assert property (@(posedge clk) a |-> b)` passes in a simulation where `a` is stuck at 0, and add the cover property that would expose it.

24. UVM Methodology

The testbenches of Chapter 22 work, but they do not scale. When ten engineers verify a 50-block SoC, each block's testbench must be reusable at subsystem and chip level, agents for standard protocols must be shared across projects, and tests must be selectable from the command line without recompiling. The **Universal Verification Methodology** (UVM, standardized as IEEE 1800.2) is the industry's answer: a SystemVerilog class library plus a set of conventions for building layered, configurable, reusable testbenches. Nearly every ASIC verification job description asks for it.

UVM does not change **what** a testbench does - it is still stimulus, driver, monitor, reference model and scoreboard. It standardizes **how** those pieces are built, connected, configured and run. If Chapter 22 made sense, UVM is mostly vocabulary. Icarus Verilog cannot run UVM, so the code in this chapter is shown without output; it is written against the IEEE 1800.2 / UVM 1.2 API and runs on Questa, VCS, Xcelium and Riviera-PRO.

24.1 Why UVM

Problem in ad-hoc testbenches	UVM mechanism
Each block's TB is monolithic and cannot be reused at SoC level	Agents (driver + monitor + sequencer per interface) and environments that nest
Changing stimulus means editing and recompiling the TB	Sequences run on sequencers; tests chosen with <code>+UVM_TESTNAME=</code>
Swapping a component (error-injecting driver, extended transaction) needs code edits	Factory overrides by type or instance
Passing parameters and interface handles through levels of constructors	uvm_config_db - hierarchical configuration database
Deciding when the test is finished	Objections - the run phase ends when nobody objects
Ordering build, connect and run across hundreds of components	Phases executed in lock-step by the UVM kernel
Inconsistent messages and verbosity	Report server : <code>uvm_info/warning/error/fatal</code> with IDs and verbosity control
Register tests rewritten for every block	RAL register model with built-in sequences

24.2 The class hierarchy

```

uvm_void
+-- uvm_object                (data: copy/compare/print/pack, factory)
    +-- uvm_transaction
        |   +-- uvm_sequence_item    <- your transactions
        |   +-- uvm_sequence_base
        |   +-- uvm_sequence #(REQ,RSP)  <- your sequences
+-- uvm_reg, uvm_reg_block, uvm_reg_field ... (RAL)
+-- uvm_report_object
    +-- uvm_component          (hierarchy, phases, config)
        +-- uvm_driver #(REQ)    uvm_monitor    uvm_sequencer #(REQ)
        +-- uvm_agent           uvm_scoreboard  uvm_env
        +-- uvm_test

```

Objects are transient data; components form the static testbench tree built during the build phase.

	uvm_object	uvm_component
Lifetime	Created and discarded freely during the run	Created once in build_phase, lives for the whole simulation

	uvm_object	uvm_component
Hierarchy	None (no parent)	Has a parent and a full hierarchical name, e.g. <code>uvm_test_top.env.in_agt.drv</code>
Phases	No	Yes
Constructor	<code>new(string name = "...")</code>	<code>new(string name, uvm_component parent)</code>
Registration macro	<code>uvm_object_utils(T)</code>	<code>uvm_component_utils(T)</code>
Examples	Sequence items, sequences, configuration objects	Drivers, monitors, agents, envs, tests, scoreboards

24.3 Phases

Phase	Kind	Order	Typical work
<code>build_phase</code>	function	top-down	Create children with the factory; read config_db
<code>connect_phase</code>	function	bottom-up	Connect TLM ports (driver-sequencer, monitor-scoreboard)
<code>end_of_elaboration_phase</code>	function	bottom-up	Print topology, final sanity checks
<code>start_of_simulation_phase</code>	function	bottom-up	Print configuration, open files
<code>run_phase</code>	task	parallel	All time-consuming activity; runs in parallel with the 12 run-time sub-phases (reset, configure, main, shutdown ... - rarely used)
<code>extract_phase</code>	function	bottom-up	Collect final state, coverage
<code>check_phase</code>	function	bottom-up	End-of-test checks (scoreboard queues empty)
<code>report_phase</code>	function	bottom-up	Print results
<code>final_phase</code>	function	top-down	Close files

Build is top-down because a parent must exist - and must have published its configuration - before its children are created. Every other function phase is bottom-up. `run_phase` of every component starts at the same time and the phase ends when all **objections** have been dropped.

24.4 Testbench topology

```

+----- uvm_test_top (vr_base_test) -----+
| +----- env (vr_env) -----+ |
| | +---- in_agt (active) ----+ | | | | | | |
| | | sqr <-> drv      mon    | | |
| | | +-----+-----+ | | |
| | | |          | ap      +-----+ | | |
| | | |          +----->| sb  |<-----+ | | |
| | | |          exp_imp +-----+ act_imp | | |
| | +-----+-----+ | | |
| +-----+-----+ | | |
+-----+-----+ | | |
          in_if -----> [ DUT: skid buffer ] -----> out_if

```

The minimal environment built in this chapter: two valid/ready agents around a pass-through DUT (e.g. the skid buffer of Chapter 10) and an in-order scoreboard.

24.5 Sequence items and sequences

A **sequence item** is one transaction. A **sequence** is a class whose `body()` task generates items and hands them to a sequencer through the `start_item` / `finish_item` handshake. Sequences can start other sequences, which is how complex scenarios are layered from simple ones.

```
`include "uvm_macros.svh"
package vr_pkg;
import uvm_pkg::*;

class vr_item extends uvm_sequence_item;
  `uvm_object_utils(vr_item)
  rand bit [7:0] data;
  rand int unsigned delay; // idle cycles before the transfer
  constraint c_delay { delay dist {0 := 6, [1:3] := 3, [4:10] := 1}; }

  function new(string name = "vr_item");
    super.new(name);
  endfunction
  virtual function string convert2string();
    return $sformatf("data=%02h delay=%0d", data, delay);
  endfunction
  virtual function void do_copy(uvm_object rhs);
    vr_item t;
    super.do_copy(rhs);
    if (!$cast(t, rhs)) `uvm_fatal("CAST", "do_copy: wrong type")
    data = t.data;
    delay = t.delay;
  endfunction
  virtual function bit do_compare(uvm_object rhs, uvm_comparer comparer);
    vr_item t;
    if (!$cast(t, rhs)) return 0;
    return super.do_compare(rhs, comparer) && (data == t.data); // not delay
  endfunction
endclass

class vr_rand_seq extends uvm_sequence #(vr_item);
  `uvm_object_utils(vr_rand_seq)
  rand int unsigned n_items = 20;
  constraint c_n { n_items inside {[1:1000]}; }
  function new(string name = "vr_rand_seq");
    super.new(name);
  endfunction
  virtual task body();
    repeat (n_items) begin
      req = vr_item::type_id::create("req"); // factory, never new()
      start_item(req); // wait for the driver
      if (!req.randomize()) `uvm_fatal("RAND", "vr_item randomize failed")
      finish_item(req); // wait for item_done()
    end
  endtask
endclass
```

Transaction and sequence (vr_pkg, part 1). `do_compare` deliberately ignores the timing-only field `delay`.

FIELD MACROS VERSUS DO_* METHODS

`uvm_field_int(data, UVM_ALL_ON)` inside `uvm_object_utils_begin/end` generates copy, compare, print and pack automatically. It is convenient but slow and hard to debug; many companies' coding guidelines require hand-written `do_copy`, `do_compare` and `convert2string`, as above.

24.6 Driver, monitor and the sequencer handshake

```
interface vr_if (input logic clk, input logic rst_n);
    logic        valid, ready;
    logic [7:0] data;
    clocking src_cb @(posedge clk);           // used by a source driver
        default input #1step output #1;
        output valid, data; input ready;
    endclocking
    clocking snk_cb @(posedge clk);           // used by a sink driver
        default input #1step output #1;
        input valid, data; output ready;
    endclocking
    clocking mon_cb @(posedge clk);           // passive sampling
        default input #1step;
        input valid, ready, data;
    endclocking
endinterface
```

The interface shared by both agents (compiled before vr_pkg).

```
class vr_driver extends uvm_driver #(vr_item);
    `uvm_component_utils(vr_driver)
    virtual vr_if vif;
    bit is_sink;                               // 1: drive ready, not valid/data
    function new(string name, uvm_component parent);
        super.new(name, parent);
    endfunction
    virtual function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if (!uvm_config_db#(virtual vr_if)::get(this, "", "vif", vif))
            `uvm_fatal("NOVIF", "virtual interface 'vif' not set")
        void'(uvm_config_db#(bit)::get(this, "", "is_sink", is_sink));
    endfunction
    virtual task run_phase(uvm_phase phase);
        if (is_sink) drive_ready(); else drive_items();
    endtask
    task drive_items();
        vif.src_cb.valid <= 1'b0;
        @(posedge vif.rst_n);
        forever begin
            seq_item_port.get_next_item(req);           // blocks until a sequence sends
            repeat (req.delay) @(vif.src_cb);
            vif.src_cb.valid <= 1'b1;
            vif.src_cb.data  <= req.data;
            do @(vif.src_cb); while (!vif.src_cb.ready);
            vif.src_cb.valid <= 1'b0;
            seq_item_port.item_done();                   // unblocks finish_item()
        end
    endtask
    task drive_ready();                               // random back-pressure
        forever begin
            @(vif.snk_cb);
            vif.snk_cb.ready <= ($urandom_range(3) != 0);
        end
    endtask
endclass

class vr_monitor extends uvm_monitor;
    `uvm_component_utils(vr_monitor)
    virtual vr_if vif;
    uvm_analysis_port #(vr_item) ap;               // broadcast to any subscribers
```

```

function new(string name, uvm_component parent);
    super.new(name, parent);
endfunction
virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    ap = new("ap", this);
    if (!uvm_config_db#(virtual vr_if)::get(this, "", "vif", vif))
        `uvm_fatal("NOVIF", "virtual interface 'vif' not set")
    endfunction
virtual task run_phase(uvm_phase phase);
    forever begin
        @(vif.mon_cb);
        if (vif.rst_n && vif.mon_cb.valid && vif.mon_cb.ready) begin
            vr_item t = vr_item::type_id::create("t"); // NEW object each time
            t.data = vif.mon_cb.data;
            ap.write(t);
        end
    end
endtask
endclass

```

Driver and monitor (vr_pkg, part 2).

```

sequence (body)          sequencer (arbitration)          driver
start_item(req)  -----> grant when driver asks <----- get_next_item(req)
randomize req
finish_item(req) -----> item delivered -----> drive pins ...
    | blocks                                     ...
    +<----- done <----- item_done()

```

The sequence-sequencer-driver handshake. Late randomization (between start_item and finish_item) lets an item react to the current state.

PITFALL: THE MONITOR THAT REUSES ONE OBJECT

Analysis ports pass a **handle**. A monitor that creates one `vr_item` in build_phase and overwrites its fields for every transfer corrupts every item already queued in the scoreboard, which then compares an object with itself. Create a new object per transaction (or clone it before `write`).

24.7 Agent, scoreboard, environment and TLM connections

```

class vr_agent extends uvm_agent;
    `uvm_component_utils(vr_agent)
    vr_driver          drv;
    vr_monitor          mon;
    uvm_sequencer #(vr_item) sqr;
    function new(string name, uvm_component parent);
        super.new(name, parent);
    endfunction
    virtual function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        mon = vr_monitor::type_id::create("mon", this);
        if (get_is_active() == UVM_ACTIVE) begin
            drv = vr_driver::type_id::create("drv", this);
            sqr = uvm_sequencer#(vr_item)::type_id::create("sqr", this);
        end
    endfunction
    virtual function void connect_phase(uvm_phase phase);
        if (get_is_active() == UVM_ACTIVE)
            drv.seq_item_port.connect(sqr.seq_item_export);
    endfunction
endclass

```

```

`uvm_analysis_imp_decl(_exp)          // creates uvm_analysis_imp_exp
`uvm_analysis_imp_decl(_act)
class vr_scoreboard extends uvm_scoreboard;
    `uvm_component_utils(vr_scoreboard)
    uvm_analysis_imp_exp #(vr_item, vr_scoreboard) exp_imp;
    uvm_analysis_imp_act #(vr_item, vr_scoreboard) act_imp;
    vr_item exp_q[$];
    int      n_match, n_err;
    function new(string name, uvm_component parent);
        super.new(name, parent);
    endfunction
    virtual function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        exp_imp = new("exp_imp", this);
        act_imp = new("act_imp", this);
    endfunction
    // Reference model: a pass-through DUT must deliver the same data in order
    function void write_exp(vr_item t);
        exp_q.push_back(t);
    endfunction
    function void write_act(vr_item t);
        vr_item e;
        if (exp_q.size() == 0) begin
            `uvm_error("SB", {"unexpected output: ", t.convert2string()})
            n_err++;
            return;
        end
        e = exp_q.pop_front();
        if (t.compare(e)) n_match++;
        else begin
            `uvm_error("SB", $sformatf("got %s exp %s",
                t.convert2string(), e.convert2string()))
            n_err++;
        end
    endfunction
    virtual function void check_phase(uvm_phase phase);
        if (exp_q.size() != 0)
            `uvm_error("SB", $sformatf("%0d items never came out", exp_q.size()))
        if (n_match == 0) `uvm_error("SB", "no transfers were checked")
    endfunction
    virtual function void report_phase(uvm_phase phase);
        `uvm_info("SB", $sformatf("matches=%0d errors=%0d", n_match, n_err), UVM_LOW)
    endfunction
endclass

class vr_env extends uvm_env;
    `uvm_component_utils(vr_env)
    vr_agent      in_agt, out_agt;
    vr_scoreboard sb;
    function new(string name, uvm_component parent);
        super.new(name, parent);
    endfunction
    virtual function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        uvm_config_db#(bit)::set(this, "out_agt.drv", "is_sink", 1'b1);
        in_agt = vr_agent::type_id::create("in_agt", this);
        out_agt = vr_agent::type_id::create("out_agt", this);
        sb = vr_scoreboard::type_id::create("sb", this);
    endfunction
    virtual function void connect_phase(uvm_phase phase);
        in_agt.mon.ap.connect(sb.exp_imp);

```

```

        out_agt.mon.ap.connect(sb.act_imp);
    endfunction
endclass

```

Agent, scoreboard and environment (vr_pkg, part 3).

TLM element	Direction / semantics
uvm_seq_item_pull_port (seq_item_port)	Driver pulls items from the sequencer: <code>get_next_item</code> , <code>item_done</code> , <code>try_next_item</code>
uvm_analysis_port #(T)	One-to-many broadcast; <code>write()</code> is a function (no blocking); zero subscribers is legal
uvm_analysis_imp #(T, IMP)	Terminates a connection; calls <code>IMP::write(T)</code> . <code>uvm_analysis_imp_decl(_sfx)</code> makes variants with <code>write_sfx</code> for components with several inputs
uvm_tlm_analysis_fifo #(T)	Unbounded FIFO with an analysis export; lets a scoreboard <code>get()</code> items in its own run_phase
uvm_blocking_put/get_port	Point-to-point, may block - used between stimulus layers

24.8 Tests, config_db, the factory and objections

```

class vr_base_test extends uvm_test;
    `uvm_component_utils(vr_base_test)
    vr_env env;
    function new(string name, uvm_component parent);
        super.new(name, parent);
    endfunction
    virtual function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        env = vr_env::type_id::create("env", this);
    endfunction
    virtual function void end_of_elaboration_phase(uvm_phase phase);
        uvm_top.print_topology();
    endfunction
    virtual task run_phase(uvm_phase phase);
        vr_rand_seq seq;
        phase.raise_objection(this);           // keep run_phase alive
        seq = vr_rand_seq::type_id::create("seq");
        if (!seq.randomize() with { n_items == 200; })
            `uvm_fatal("RAND", "sequence randomize failed")
        seq.start(env.in_agt.sqr);
        phase.get_objection().set_drain_time(this, 200ns); // let outputs drain
        phase.drop_objection(this);
    endtask
endclass

// A test that changes the stimulus without touching the environment:
class vr_b2b_item extends vr_item;
    `uvm_object_utils(vr_b2b_item)
    constraint c_b2b { delay == 0; }           // back-to-back transfers
    function new(string name = "vr_b2b_item");
        super.new(name);
    endfunction
endclass

class vr_b2b_test extends vr_base_test;
    `uvm_component_utils(vr_b2b_test)
    function new(string name, uvm_component parent);
        super.new(name, parent);
    endfunction

```

```

    virtual function void build_phase(uvm_phase phase);
        vr_item::type_id::set_type_override(vr_b2b_item::get_type());
        super.build_phase(phase);
    endfunction
endclass
endpackage : vr_pkg

```

Tests (vr_pkg, part 4): the factory override replaces every `vr_item` the sequence creates with a `vr_b2b_item`.

```

module tb_top;
    import uvm_pkg::*;
    import vr_pkg::*;
    logic clk = 1'b0, rst_n = 1'b0;
    always #5 clk = ~clk;
    initial #50 rst_n = 1'b1;

    vr_if in_if (.clk(clk), .rst_n(rst_n));
    vr_if out_if (.clk(clk), .rst_n(rst_n));

    skid_buffer #(W(8)) dut (
        .clk, .rst_n,
        .s_valid(in_if.valid), .s_ready(in_if.ready), .s_data(in_if.data),
        .m_valid(out_if.valid), .m_ready(out_if.ready), .m_data(out_if.data));

    initial begin
        uvm_config_db#(virtual vr_if)::set(null, "uvm_test_top.env.in_agt*",
                                           "vif", in_if);
        uvm_config_db#(virtual vr_if)::set(null, "uvm_test_top.env.out_agt*",
                                           "vif", out_if);
        run_test(); // test named by +UVM_TESTNAME=...
    end
endmodule

// Run: <sim> ... +UVM_TESTNAME=vr_b2b_test +UVM_VERBOSITY=UVM_MEDIUM

```

The static top: the only module in a UVM testbench besides the DUT and interfaces.

Mechanism	Key API	Remember
config_db	<code>uvm_config_db#(T)::set(ctx, "path", "field", val) / ::get(this, "", "field", var)</code>	Type, path and field name must all match exactly; <code>get</code> returns 0 on a miss - check it. Higher-level (earlier) sets win in <code>build_phase</code>
Factory	<code>T::type_id::create(name, parent); set_type_override, set_inst_override</code>	Only objects created through <code>type_id::create</code> can be overridden; <code>new()</code> bypasses the factory silently
Objections	<code>phase.raise_objection(this) / drop_objection(this), drain time</code>	No objection raised -> <code>run_phase</code> ends at time 0 and the test 'passes' having done nothing
Reporting	<code>uvm_info(ID, msg, UVM_LOW), uvm_error, uvm_fatal</code>	Test status = no <code>UVM_ERROR/UVM_FATAL</code> in the report summary

24.9 The register abstraction layer (RAL)

Every block has control/status registers (Chapter 15), and every register must be tested for reset values, access policies and connectivity. UVM's **register layer** models the register map as objects - `uvm_reg_field` inside `uvm_reg` inside `uvm_reg_block` - normally **generated** from the same IP-XACT/SystemRDL source that generates the RTL. Tests then access registers by name, independent of the bus protocol.

```

class ctrl_reg extends uvm_reg;
    `uvm_object_utils(ctrl_reg)
    rand uvm_reg_field en, mode;
    function new(string name = "ctrl_reg");

```

```

    super.new(name, 32, UVM_NO_COVERAGE);
endfunction
virtual function void build();
    en = uvm_reg_field::type_id::create("en");
    //      parent size lsb access volatile reset has_rst is_rand indiv
    en.configure(this, 1, 0, "RW", 0, 1'b0, 1, 1, 0);
    mode = uvm_reg_field::type_id::create("mode");
    mode.configure(this, 2, 1, "RW", 0, 2'b00, 1, 1, 0);
endfunction
endclass

// in a sequence or test:
uvm_status_e status; uvm_reg_data_t v;
regmodel.ctrl.write(status, 32'h3);           // frontdoor: real bus cycles
regmodel.ctrl.read (status, v);
regmodel.ctrl.mirror(status, UVM_CHECK);       // read and compare with model
regmodel.ctrl.peek(status, v);                 // backdoor: HDL path, zero time

```

A generated-style register class and typical accesses.

RAL piece	Role
Register model (<code>uvm_reg_block</code>)	Mirror of the DUT's registers with reset values and access policies (RW, RO, W1C ...)
Adapter (<code>uvm_reg_adapter</code>)	<code>reg2bus</code> / <code>bus2reg</code> : converts generic register operations to the bus agent's sequence items (APB, AXI-Lite)
Predictor (<code>uvm_reg_predictor</code>)	Updates the mirror from transactions observed by the bus monitor (explicit prediction)
Frontdoor / backdoor	Access through the bus / directly through HDL paths (fast init, checking without bus traffic)
Built-in sequences	<code>uvm_reg_hw_reset_seq</code> , <code>uvm_reg_bit_bash_seq</code> , <code>uvm_reg_access_seq</code> , <code>uvm_mem_walk_seq</code> - free register tests for every block

24.10 Debugging UVM testbenches

Symptom	Likely cause	Tool / fix
Test ends at time 0, passes	No objection raised in any run_phase	<code>+UVM OBJECTION_TRACE</code>
Simulation hangs forever	Driver never calls <code>item_done</code> ; handshake waits for a ready that never comes	Global timeout <code>+UVM TIMEOUT=</code> , watchdog, check the driver loop
NOVIF fatal / null handle	config_db path, field name or type mismatch (<code>virtual vr_if</code> vs <code>virtual vr_if#(8)</code>)	<code>+UVM CONFIG_DB_TRACE</code> , <code>print_config()</code>
Override has no effect	Object created with <code>new()</code> , or override set after creation	<code>factory.print()</code> , set overrides before <code>super.build_phase</code>
Scoreboard compares object with itself	Monitor reuses one item handle	Create or clone per transaction
Components missing	Typo in create name; agent passive	<code>uvm_top.print_topology()</code>
Too much / too little log	Verbosity	<code>+UVM VERBOSITY=</code> , <code>set_report_verbosity_level_hier</code> , per-ID <code>+uvm_set_verbosity</code>
Phase ordering surprises	Work done in the wrong phase	<code>+UVM PHASE_TRACE</code>

WHAT INTERVIEWERS PROBE ON UVM

Explain the difference between `uvm_object` and `uvm_component`; why `build_phase` is top-down; what happens if you forget `raise_objection`; how the sequencer and driver hand items back and forth (`start_item/finish_item`, `get_next_item/item_done`); type versus instance overrides; `p_sequencer` versus `m_sequencer`; how a virtual interface gets from `tb_top` into a driver; and how RAL prediction works. Being able to sketch this chapter's topology on a whiteboard from memory covers most of it.

UVM FOR A DESIGNER'S OWN BLOCK TESTS

UVM's overhead pays off for reusable, multi-engineer environments. For a designer's quick unit tests, the plain SystemVerilog of Chapter 22 or Python-based **cocotb** are often faster to write. Knowing all three - and when each fits - is the mark of a senior engineer.

24.11 Summary

- UVM standardizes how testbench components are built, connected, configured and run; the architecture is the same as Chapter 22's.
- Objects (items, sequences) are transient data; components (driver, monitor, sequencer, agent, env, test) form the static tree.
- Phases: build (top-down), connect, ..., run (task, objection-controlled), check, report.
- Sequences generate items; the driver pulls them via `seq_item_port`; monitors broadcast via analysis ports.
- `config_db` distributes virtual interfaces and settings; the factory allows overrides; always create with `type_id::create`.
- RAL models registers and gives free, protocol-independent register tests.

24.12 Exercises

- 1 Add a functional covergroup to `vr_monitor` (data value ranges, delay 0 versus non-zero) and sample it on every transfer.
- 2 Write a `vr_burst_seq` that runs `vr_rand_seq` three times with different `n_items`, using `uvm_do_with`-free code (`start_item/finish_item` only).
- 3 Make the out agent configurable as a **passive** agent (monitor only) and explain what then must drive `ready`.
- 4 Replace the `uvm_analysis_imp_decl` scoreboard with two `uvm_tlm_analysis_fifos` and a `run_phase` that compares `get()` results. What changes about when mismatches are reported?
- 5 Write the `reg2bus` and `bus2reg` functions of an APB adapter for an APB agent whose item has fields `addr`, `data`, `write`.

25. Static Sign-off: Lint, CDC/RDC, Formal and Equivalence Checking

Simulation answers 'does the design do the right thing **for the stimulus I applied?**'. Static methods answer questions simulation structurally cannot: is there **any** input sequence that breaks this property; is **every** clock-domain crossing synchronized; is the netlist **equivalent** to the RTL for **all** inputs. They read the design rather than run it, so they need no testbench and they are exhaustive within their scope. On a real tape-out, clean lint, CDC, RDC and equivalence reports - with every waiver reviewed - are mandatory sign-off gates alongside timing (Chapter 18) and power (Chapter 19).

25.1 Where static checks fit

Check	Question answered	Runs on	When
Lint	Is the RTL well-formed, synthesizable, free of suspicious constructs?	RTL	Every commit (seconds)
CDC	Is every signal crossing between asynchronous clocks correctly synchronized?	RTL (+ netlist)	Weekly, then at every milestone
RDC	Can an asynchronous reset in one domain corrupt flops in another reset domain?	RTL (+ netlist)	With CDC
Formal (FPV)	Does a property hold for all legal input sequences?	RTL + SVA	Block level, during design
LEC	Is the netlist logically equivalent to the RTL?	RTL vs netlist, netlist vs netlist	After synthesis, DFT, every ECO
X-prop	Can X values hide bugs in simulation or appear in silicon?	RTL	Block and SoC level

WHY STATIC CHECKS CATCH WHAT SIMULATION CANNOT

A CDC bug is a **probability**: a synchronizer-less crossing may work in a billion simulated cycles because RTL simulation has no metastability and no skew between bits. A lint issue such as an incomplete sensitivity list makes simulation **disagree with the hardware**, so the simulation that passed is irrelevant. Only a tool that reasons about structure or about all inputs finds these.

25.2 Lint: rule categories

Category	Examples of rules	Why it matters
Synthesizability	Delays #5 in RTL, initial blocks, unsupported loops, real types	Tool silently ignores or rejects
Sim/synth mismatch	Incomplete sensitivity list, full_case/parallel_case pragmas, X assignments used as don't-care	RTL simulation no longer predicts the gates
Inferred latches	Incomplete if/case in combinational blocks	Unintended storage, timing problems (Chapter 5)
Width	Truncation, extension, sign mismatch, unsized constants	Silent arithmetic bugs
Assignments	Blocking in sequential blocks, non-blocking in combinational, multiple drivers	Races and ordering bugs (Chapter 4)
Clocks and resets	Clock used as data, gated clocks built from logic, mixed async/sync resets, reset not synchronized	Glitches, timing and DFT problems
Structure	Combinational loops, undriven/unloaded nets, floating inputs, implicit nets	Oscillation, X, dead logic
FSM	Unreachable states, no default, terminal states	Lock-ups

Category	Examples of rules	Why it matters
Naming / style	Suffix conventions (<u>_n</u> , <u>_q</u>), one module per file, header present	Reviewability, automation
DFT readiness	Uncontrollable async resets/clocks, latches in scan paths	Scan insertion problems (Chapter 20)

The file below contains five classic mistakes. We run it through the two free tools available in this book's environment: Icarus with **-Wall**, and Verilator's dedicated linter.

```

module lint_bait (
    input wire      clk,
    input wire [1:0] sel,
    input wire [7:0] a, b,
    output reg  [7:0] y,
    output reg  [3:0] q,
    output wire [7:0] sum
);
    reg [7:0] t;
    // (1) incomplete case in combinational block -> latch on y
    always @(*)
        case (sel)
            2'd0: y = a;
            2'd1: y = b;
        endcase
    // (2) incomplete sensitivity list -> sim/synth mismatch
    always @(a)
        t = a & b;
    // (3) width truncation: 9-bit result into 8-bit net
    assign sum = a + b + 9'd1;
    // (4) blocking assignment in a clocked block
    always @(posedge clk)
        q = t[3:0];
    // (5) implicit net from a typo
    assign carr = a[7] & b[7];
endmodule

```

lint_bait.v - five deliberate problems.

```

$ iverilog -g2012 -Wall -o /dev/null lint_bait.v
$ verilator --lint-only -Wall lint_bait.v

```

```
lint_bait.v:25: warning: implicit definition of wire 'carr'.
```

Icarus -Wall: a simulator's warnings are not a linter - only the implicit net is reported.

```

%Warning-IMPLICIT: lint_bait.v:25:10: Signal definition not found, creating implicitly: 'carr'
%Warning-WIDTHTRUNC: lint_bait.v:20:14: Operator ASSIGNW expects 8 bits on the Assign RHS, but
                    Assign RHS's ADD generates 9 bits.
%Warning-UNUSED SIGNAL: lint_bait.v:9:13: Bits of signal are not used: 't'[7:4]
%Warning-UNUSED SIGNAL: lint_bait.v:25:10: Signal is not used: 'carr'
%Warning-CASEINCOMPLETE: lint_bait.v:12:5: Case values incompletely covered (example pattern 0x2)
%Warning-BLKSEQ: lint_bait.v:18:7: Blocking assignment '=' in sequential logic process
%Warning-BLKSEQ: lint_bait.v:23:7: Blocking assignment '=' in sequential logic process
%Error: Exiting due to 7 warning(s)

```

Verilator 5.020 --lint-only -Wall (one long line wrapped for the page).

Verilator found four of the five problems, but read the report critically. It reports the latch-producing **case** as CASEINCOMPLETE rather than as a latch; it classifies **always @(a)** as a **sequential** process (hence the BLKSEQ at line 18) instead of flagging the missing **b** in the sensitivity list, which is the real bug; and it has no opinion about the async/sync reset style. Commercial linters (Synopsys SpyGlass/VC SpyGlass, Real Intent Ascent Lint, Siemens Questa Lint, Cadence HAL/JasperGold Superlint) apply thousands of configurable rules, including the company's own coding guidelines (Chapter 27).

CHEAP HABITS THAT REMOVE WHOLE LINT CATEGORIES

Put `default_nettype none` at the top of every RTL file (and `default_nettype wire` at the end) - implicit nets become compile errors. Use `always_comb/always_ff`, which make sensitivity lists automatic and let tools check intent. Write explicitly sized constants and use `unique/priority` instead of synthesis pragmas.

WAIVERS

Not every lint message is a bug; the width warning on a deliberate carry drop is intended. Such messages are **waived** in a waiver file, with the rule, the exact location or signal, a reason and a reviewer. Blanket waivers ('turn off WIDTH everywhere') are how real bugs ship. Waivers are re-reviewed when the RTL changes.

25.3 CDC verification

Chapter 11 designed the synchronizers; CDC sign-off proves that they are present and used correctly on **every** crossing of a chip that may have hundreds of clocks and tens of thousands of crossings. The flow of every commercial tool (Synopsys VC SpyGlass CDC, Siemens Questa CDC, Cadence Conformal/Jasper CDC, Real Intent Meridian CDC) is the same:

- 1 **Setup.** Define clocks and their relationships (asynchronous, synchronous, divided), resets, constant/static signals (mode pins that never toggle in function), and the synchronizer cells of the library. Much of this comes from the SDC (Chapter 18). A wrong setup gives either thousands of false violations or, worse, silently missed ones.
- 2 **Structural analysis.** Find every crossing and classify it: synchronized by a recognized scheme (2-flop, pulse, handshake, async FIFO, MUX-enable) or not. Report missing synchronizers, combinational logic before a synchronizer, fan-out of a synchronizer input to several synchronizers (divergence), re-convergence of separately synchronized signals, and multi-bit buses crossing without a safe scheme.
- 3 **Functional analysis.** Structural recognition assumes a protocol: the data of a MUX/handshake crossing is stable while sampled; a Gray-coded pointer changes one bit at a time; a pulse is wide enough for the destination clock. The tool generates assertions for these and checks them formally or in simulation.
- 4 **Review and waive.** Every remaining violation is fixed or waived with a reason (for example, a quasi-static configuration register written only while the destination is idle).
- 5 **Netlist CDC** repeats the analysis after synthesis, because synthesis can insert glitch-prone logic in front of a synchronizer or duplicate a synchronizer flop.

```
module cdc_bad (
    input logic      clk_a, clk_b, rst_b_n,
    input logic      req_a, en_a,           // both in the clk_a domain
    input logic [3:0] cnt_a,               // binary counter in clk_a domain
    output logic      go_b,
    output logic [3:0] cnt_b
);
    // VIOLATION 1: combinational logic in front of a synchronizer.
    // req_a & en_a can glitch between clk_a edges; clk_b may capture the glitch.
    logic s1, s2;
    always_ff @(posedge clk_b or negedge rst_b_n)
        if (!rst_b_n) {s2, s1} <= '0;
        else          {s2, s1} <= {s1, req_a & en_a};
    assign go_b = s2;

    // VIOLATION 2: multi-bit binary value through per-bit synchronizers.
    // On 0111 -> 1000 each bit may resolve differently: clk_b can see 1111.
    logic [3:0] c1, c2;
    always_ff @(posedge clk_b or negedge rst_b_n)
        if (!rst_b_n) {c2, c1} <= '0;
        else          {c2, c1} <= {c1, cnt_a};
    assign cnt_b = c2;
endmodule
```

Two violations every CDC tool reports (compiles and simulates fine - which is exactly the problem).

```

module cdc_good (
    input logic      clk_a, rst_a_n, clk_b, rst_b_n,
    input logic      req_a, en_a,
    input logic [3:0] cnt_a,
    output logic      go_b,
    output logic [3:0] cnt_gray_b           // decode to binary in clk_b if needed
);
    // FIX 1: register the combinational term in the source domain first.
    logic go_a, s1, s2;
    always_ff @(posedge clk_a or negedge rst_a_n)
        if (!rst_a_n) go_a <= 1'b0;
        else          go_a <= req_a & en_a;
    always_ff @(posedge clk_b or negedge rst_b_n)
        if (!rst_b_n) {s2, s1} <= '0;
        else          {s2, s1} <= {s1, go_a};
    assign go_b = s2;

    // FIX 2: Gray-code in the source domain (one bit changes per increment),
    // register it, then synchronize. Valid only if cnt_a moves by +/-1 per clk_a.
    logic [3:0] gray_a, g1, g2;
    always_ff @(posedge clk_a or negedge rst_a_n)
        if (!rst_a_n) gray_a <= '0;
        else          gray_a <= cnt_a ^ (cnt_a >> 1);
    always_ff @(posedge clk_b or negedge rst_b_n)
        if (!rst_b_n) {g2, g1} <= '0;
        else          {g2, g1} <= {g1, gray_a};
    assign cnt_gray_b = g2;
endmodule

```

The fixes: register in the source domain, and Gray-code multi-bit counters before synchronizing.

clk_a domain	clk_b domain
en_a --+	
+++[AND]--X-->[s1]-->[s2]--> go_b	X = glitch can be captured
req_a -+	
en_a --+	
+++[AND]-->[go_a]-->[s1]-->[s2]--> go_b	registered: glitch-free
req_a -+	clk_a clk_b
re-convergence: a_q --sync--> x_b --+	
	+++[logic]--> may see (x_b,y_b) never
b_q --sync--> y_b --+	produced in clk_a

Combinational logic before a synchronizer, its fix, and re-convergence of independently synchronized signals.

PITFALL: 'IT PASSED IN SIMULATION' IS NOT CDC SIGN-OFF

RTL simulation models a synchronizer as two ideal flops: a 1-cycle latency uncertainty and bit-to-bit skew simply do not exist. Re-convergence bugs therefore never appear in RTL simulation. Some simulators offer **metastability injection** (randomly adding a cycle of delay at synchronizers) to expose them - use it, but it is a complement to structural CDC, not a replacement.

25.4 RDC verification

Reset-domain crossing (Chapter 12) is CDC's less famous sibling. Even with a single clock, when an **asynchronous reset** asserts in one reset domain, the flops it resets change at an arbitrary time relative to the clock. A flop in a different reset domain that is **not** being reset may sample that change during its setup/hold window and go metastable.

```

module rdc_example (
    input  logic clk, por_n, sw_rst_n,      // same clock, two reset domains
    input  logic d,
    output logic q_b
);
    // Flop A is reset by a software-controlled reset; flop B only by power-on reset.
    logic q_a;
    always_ff @(posedge clk or negedge sw_rst_n)
        if (!sw_rst_n) q_a <= 1'b0;
        else          q_a <= d;
    // RDC: when sw_rst_n asserts asynchronously, q_a changes at an arbitrary time
    // relative to clk, and flop B (NOT in reset) may sample it while it changes.
    always_ff @(posedge clk or negedge por_n)
        if (!por_n) q_b <= 1'b0;
        else       q_b <= q_a;
endmodule

```

An RDC violation: sw_rst_n asserting can corrupt q_b, which is still in functional mode.

Fix	How
Reset ordering	Guarantee by design that the receiving domain is also in reset (or idle) whenever the source reset asserts - e.g. software resets only asserted after traffic is quiesced; RDC tools accept this as a constraint
Isolation	Gate the crossing data (AND with an 'isolate' signal) or gate the receiving clock while the source reset asserts
Synchronization	Treat the crossing like a CDC path and synchronize it in the receiving domain
Same domain	Put both flops in the same reset domain if the architecture allows

RDC tools (Synopsys VC SpyGlass RDC, Siemens Questa RDC, Real Intent Meridian RDC) need the reset tree, reset sequencing and isolation signals as setup, exactly as CDC tools need clocks.

25.5 Formal property verification

A formal tool translates the RTL and its properties into a mathematical problem (usually Boolean satisfiability, SAT, or SMT) and asks: **is there any sequence of legal inputs that violates an assertion?** If yes, it returns a **counterexample** trace - typically a few cycles long, which is far easier to debug than a random simulation failure. If no, the property is **proven** for all inputs, for all time.

Concept	Meaning
BMC (bounded model checking)	Search all input sequences up to k cycles from reset. Finds bugs fast; a pass only means 'no bug within k cycles' - a bounded proof
Induction	Base: property holds for the first k cycles from reset. Step: if it holds for any k consecutive cycles, it holds on the next. Both pass -> full proof
Induction failure	The step can fail from an unreachable state - a false counterexample. Fix by adding helper assertions (invariants) or increasing k
Assumptions	assume property restricts inputs to legal behaviour. Too few: false failures. Too many (over-constraint): real bugs hidden, even vacuous proofs
Cover	cover property asks the tool to reach a scenario - the sanity check that assumptions still allow interesting behaviour
State explosion	Proof effort grows with state; deep counters and memories are handled with abstraction, cut-points and black-boxing

We can demonstrate both outcomes with the open-source Yosys SAT engine. The design is a BCD counter with one safety property: it never leaves 0-9. Properties go inside **ifdef FORMAL** so that synthesis and simulation ignore them.

```

module bcd_cnt (
    input logic    clk, rst,
    input logic    en,
    output logic [3:0] q
);
    always_ff @(posedge clk)
        if (rst)      q <= 4'd0;
        else if (en)   q <= (q == 4'd9) ? 4'd0 : q + 4'd1;

`ifdef FORMAL
    // Safety property: the counter never leaves the BCD range 0..9.
    always_comb if (!rst) assert (q <= 4'd9);
`endif
endmodule

```

A BCD counter with an embedded immediate assertion (Yosys treats it as a formal property).

```

$ yosys -p "read_verilog -sv -formal bcd.sv; prep -top bcd_cnt; \
    sat -tempinduct -prove-asserts -set-at 1 rst 1 -maxsteps 20"

```

k-induction, with reset forced in the first cycle.

```

[base case 1] Solving problem with 167 variables and 442 clauses..
Base case for induction length 1 proven.
[induction step 1] Solving problem with 340 variables and 911 clauses..
Induction step proven: SUCCESS!

```

Excerpt of the Yosys log: a full proof by 1-induction - q stays in 0..9 for every input sequence, forever.

Without `-set-at 1 rst 1` the base case fails immediately with `init q = 10`: the tool is free to start in any state, and an unreset flop can power up as 10. This is the formal view of why reset matters. Now introduce a bug - wrap at 10 instead of 9:

```

    else if (en)      q <= (q == 4'd10) ? 4'd0 : q + 4'd1;    // BUG

```

```

Base case for induction length 11 proven.
[induction step 11] Solving problem with 2969 variables and 8493 clauses..
Induction step failed. Incrementing induction length.
[base case 12] Solving problem with 2985 variables and 8544 clauses..
SAT temporal induction proof finished - model found for base case: FAIL!

```

Time	Signal Name	Dec	Hex	Bin
init	\q	12	c	1100
1	\q	12	c	1100
2	\q	0	0	0000
3	\q	1	1	0001
11	\q	9	9	1001
12	\q	10	a	1010

Abridged log (separator lines and steps 4-10 removed): the 12-cycle counterexample - reset, count 0..9, then the illegal 10.

Notice the induction steps that failed first: from an arbitrary (possibly unreachable) state the step fails until k is long enough to force the trace back through reset. This is the typical 'induction failure is not a bug' situation; production tools report such steps as **undetermined** and the engineer adds an invariant to close the proof.

SymbiYosys and the commercial tools

For real projects, the open-source front end is **SymbiYosys** ([sby](#)), which drives Yosys and SMT solvers (Yices, Boolector, Z3) with BMC, k-induction and PDR/IC3 engines and full SVA support in its commercial variant. A job file looks like this (shown without output):

```

[tasks]
prove
cover

[options]
prove: mode prove

```

```

prove: depth 20
cover: mode cover
cover: depth 40

[engines]
smtbmc yices

[script]
read -formal fifo.sv fifo_props.sv
prep -top sync_fifo

[files]
fifo.sv
fifo_props.sv

```

fifo.sby - a SymbiYosys job with a proof task and a cover task.

Industrial formal is done with Cadence JasperGold, Synopsys VC Formal and Siemens Questa Formal. Beyond hand-written properties they offer push-button **apps**: connectivity checking (SoC pin-mux and top-level wiring), X-propagation, register (CSR) checking, sequential equivalence, unreachable analysis to exclude dead coverage bins, security/information-flow and deadlock checks.

WHERE FORMAL BEATS SIMULATION - AND WHERE IT DOES NOT

Formal excels on **control-dominated** blocks with deep corner cases: arbiters, FIFOs and credit logic, cache-coherence and interrupt controllers, CDC protocols, and anything where one counterexample in a trillion is still a bug. It struggles with **wide datapaths** (multipliers, floating point) and very deep sequential behaviour, where specialized datapath equivalence (C-to-RTL) or simulation are better. Mature teams plan which blocks are 'formal-signed-off' at the start of the project.

25.6 Equivalence checking

Logic equivalence checking (LEC) proves that two representations of a design implement the same Boolean function: RTL versus the synthesized netlist, pre- versus post-scan-insertion netlist, netlist versus post-layout netlist, and before versus after every ECO. It is the check that allows the team to trust that the gates sent to the foundry are the RTL that was verified. Tools: Cadence Conformal LEC, Synopsys Formality, Siemens FormalPro.

golden (RTL)	revised (netlist)
inputs --+--[comb cone]--> flop A	inputs --+--[gate cone]--> flop A'
+--[comb cone]--> output	+--[gate cone]--> output

1. map key points: flop A <-> flop A', output <-> output (by name/structure)
2. for every compare point prove: cone(golden) == cone(revised) for all inputs
3. non-equivalent -> counterexample input vector

Combinational equivalence: registers become cut points, so only combinational cones are compared.

Combinational LEC maps registers one-to-one, which is why it is fast and why it breaks when synthesis **retimes** registers, re-encodes FSMs, merges or duplicates flops, or inserts clock gating. Those optimizations need **sequential equivalence checking (SEC)**, which compares behaviour at the outputs over time without a register mapping - also the method for verifying a hand-optimized or low-power version of a block against the original RTL. The synthesis tool helps LEC by writing a 'guidance' file (SVF in Synopsys flows) that records its transformations.

A miniature LEC with Yosys: prove that a hand-optimized ALU with one shared adder for ADD and SUB is equivalent to the reference ALU of Chapter 22. The miter compares the outputs of both for identical inputs and the SAT solver searches for any input that makes them differ.

```

// Hand-optimized ALU: one shared adder does both ADD and SUB.
module alu_opt (

```



```

input  logic [1:0] op,
input  logic [7:0] a, b,
output logic [7:0] y,
output logic      c
);
    logic      sub;
    logic [8:0] sum;
    assign sub = (op == 2'd1);
    assign sum = {1'b0, a} + {1'b0, b ^ {8{sub}}} + 9'(sub);    // a + ~b + 1
    always_comb begin
        c = 1'b0;
        case (op)
            2'd0, 2'd1: begin y = sum[7:0]; c = sum[8] ^ sub; end    // borrow = !carry
            2'd2:      y = a & b;
            default:   y = a ^ b;
        endcase
    end
endmodule

```

Revised design: one adder computes $a + b$ or $a + \sim b + 1$ (also passes the Chapter 22 testbench).

```

# Combinational equivalence: golden RTL vs hand-optimized RTL
read_verilog -sv alu.sv alu_opt.sv
proc; opt_clean
miter -equiv -flatten -make_assert alu alu_opt miter
sat -verify -prove-asserts -show-inputs miter

```

lec.ys

```

Solving problem with 951 variables and 2573 clauses..
SAT proof finished - no model found: SUCCESS!

```

Excerpt: no input distinguishes the designs - proven for all 2^{18} input combinations at once.

If the optimizer forgets that the borrow of a subtraction is the **inverted** carry ($c = \text{sum}[8]$ instead of $\text{sum}[8] \wedge \text{sub}$), the check returns a counterexample in milliseconds:

```

SAT proof finished - model found: FAIL!

```

Signal Name	Dec	Hex	Bin
\in_a	42	2a	00101010
\in_b	50	32	00110010
\in_op	1	1	01
\trigger	1	1	1

42 - 50 must borrow; the buggy design reports no borrow. A random test would find this too - but LEC proves the fixed version has no other such input.

Finally the real use: RTL versus the gate-level netlist produced by synthesis.

```

# 1. Synthesize the RTL to a generic gate netlist (the "revised" design)
read_verilog -sv alu.sv
synth -flatten -top alu
abc -g AND,NAND,OR,NOR,XOR,XNOR,MUX
stat
rename alu alu_gate
write_verilog -noattr alu_gate.v
design -reset
# 2. Compare golden RTL against the netlist
read_verilog -sv alu.sv
read_verilog alu_gate.v
proc; opt_clean
miter -equiv -flatten -make_assert alu alu_gate miter
sat -verify -prove-asserts miter

```

lec_gate.ys - synthesize, then prove the netlist equivalent to the RTL.

```

Number of cells:          139
  $_AND_                 30
  $_NAND_                 67
  $_NOR_                   1
  $_NOT_                   9
  $_OR_                   15
  $_XNOR_                  3
  $_XOR_                  14

```

Solving problem with 1482 variables and 3866 clauses..

SAT proof finished - no model found: SUCCESS!

Excerpt: 139 generic gates, formally equivalent to the 16-line RTL.

PITFALL: LEC SETUP THAT PROVES THE WRONG THING

LEC must be run with the **functional** mode constrained: scan enable and test mode tied inactive, otherwise the scan-inserted netlist is (correctly) not equivalent. Conversely, over-constraining - tying off a pin that is not really constant - makes LEC pass on a broken netlist. Every LEC constraint and every 'black-boxed' module (memories, analog macros) must be reviewed; unmapped and unreachable key points are warnings to read, not to ignore.

25.7 X-propagation checking

RTL simulation handles unknown values optimistically: an **if** with an X condition takes the else branch, a **case** with an X selector matches nothing, so an uninitialized flop can silently produce a **clean** output in RTL simulation while real silicon powers up to a random 0 or 1 (Chapter 26 demonstrates this with Icarus). The opposite, **X-pessimism**, appears in gate-level simulation, where a MUX with an X select and equal data inputs outputs X although hardware would output the data value.

Technique	What it does
Simulator X-prop modes	VCS -xprop, Xcelium X-propagation options: if/case with X conditions merge both branches (T-merge/X-merge), making RTL simulation pessimistic like hardware
Formal X-prop apps	Prove that no X from uninitialized registers or X-assignments can reach outputs or specified control points after reset
Reset-less flop review	List every flop without reset (common in datapaths for area/power) and prove its value is written before it is used - typically with a 'valid' qualifier
Assertions	!\$isunknown(ctrl) checks on every control signal and interface (Chapter 23)
Randomized initialization	Simulator options that initialize unreset flops to random 0/1 instead of X, across seeds

25.8 A sign-off checklist

Static sign-off before RTL freeze and tape-out

- ☐ Lint clean against the project rule set; every waiver has a reason and a reviewer.
- ☐ CDC setup reviewed (clocks, resets, constants, synchronizer cells); zero unwaived structural violations; functional CDC assertions proven or simulated.
- ☐ RDC clean, with reset sequencing constraints documented.
- ☐ Formal: planned properties proven or bounded to an agreed depth; covers reachable; no vacuous proofs.
- ☐ LEC: RTL vs synthesis netlist, synthesis vs DFT netlist, DFT vs post-layout netlist, and after every ECO - all equivalent, with reviewed constraints and black boxes.
- ☐ X-prop: unreset flops reviewed; X-prop simulation or formal app clean.
- ☐ Netlist-level CDC and gate-level simulation (Chapter 26) run on the final netlist.

25.9 Summary

- Static checks are exhaustive within their scope and need no testbench; they are mandatory sign-off gates.
- Lint catches synthesizability, sim/synth mismatch, width, latch and style issues; free tools find a subset - read reports critically and waive precisely.
- CDC flow: setup -> structural -> functional -> review/waive -> netlist CDC. RDC checks asynchronous-reset crossings the same way.
- Formal: BMC finds bugs and gives bounded proofs; induction gives full proofs; assumptions constrain inputs and must be covered to avoid over-constraint.
- LEC proves RTL and netlists equivalent after every transformation; sequential equivalence handles retiming, re-encoding and clock gating.
- X-propagation checking closes the gap between optimistic RTL simulation and real power-up behaviour.

25.10 Exercises

- 1 Fix all five problems in `lint_bait.v` and get Verilator `--lint-only -Wall` to report nothing, without any `lint_off` pragma.
- 2 Write the formal property 'the FIFO of Chapter 22 is never full and empty at once' inside `ifdef FORMAL`, and prove it with `sat -tempinduct`. Does 1-induction succeed? If not, which helper invariant closes it?
- 3 In `cdc_good.sv`, the Gray-code fix is valid only if `cnt_a` changes by at most one per `clk_a` cycle. Write an assertion that checks this assumption.
- 4 Modify `alu_opt.sv` so that it is equivalent for ADD, SUB and AND but not XOR, and read the LEC counterexample. How many counterexamples exist in total?
- 5 List three synthesis transformations that break combinational LEC key-point mapping and explain how each is handled in a production flow.

26. Debug, Gate-Level Simulation, FPGA Prototyping and Emulation

The previous chapters were about **finding** that something is wrong. This one is about finding **why**, and about the verification platforms that take over when RTL simulation becomes too slow or too idealized: gate-level simulation of the real netlist, FPGA prototypes and hardware emulators that run billions of cycles, hardware/software co-verification, and finally debug on the silicon itself. Debug skill is the most under-taught and most valued ability in the industry: a senior engineer is, above all, someone who reliably finds root causes quickly.

26.1 A debugging methodology

- 1 **Reproduce.** Rerun the failing test with the same seed, the same build and the same command line. A bug you cannot reproduce you cannot fix - log the seed of every random run.
- 2 **Find the first symptom.** The scoreboard error is usually far downstream of the cause. Look for the **earliest** assertion, error or divergence from the model; with transaction logs from every monitor, compare expected and actual streams to find the first bad transaction and its time.
- 3 **Localize in time and space.** Dump waves only around that time and only for the relevant hierarchy. Trace the wrong value **backwards**: which driver produced it, from which inputs, in which cycle? Repeat until you reach a value that is wrong while its inputs are right - that is the bug.
- 4 **Form a hypothesis and test it.** Predict what a change should do before making it ('if the full flag is late by one cycle, forcing it here will make the error disappear').
- 5 **Fix, then generalize.** After the fix, ask: where else did the same pattern get written? Add an assertion that would have caught it at the source, and add the seed or a directed test to the regression.
- 6 **Classify.** Is it a DUT bug, a testbench bug, a model bug or a spec ambiguity? Roughly half of all failures in a young environment are testbench bugs.

26.2 Waveforms: \$dumpfile, \$dumpvars and GTKWave

System task	Purpose
<code>\$dumpfile("f.vcd")</code>	Name the VCD (Value Change Dump) file
<code>\$dumpvars(depth, scope)</code>	Record changes in scope and depth levels below it; depth 0 = everything
<code>\$dumpoff / \$dumpon</code>	Pause and resume recording (skip long idle or initialization phases)
<code>\$dumplimit(bytes)</code>	Cap the file size
<code>\$dumpall / \$dumpflush</code>	Checkpoint all values / flush to disk (useful before a crash)

```

`timescale 1ns/1ps
module tb_vcd;
  logic clk = 0, rst_n = 0, wr_en = 0, rd_en = 0;
  logic [7:0] wdata = 0, rdata;
  logic full, empty;
  sync_fifo #(.W(8), .DEPTH(4)) dut (.*);
  always #5 clk = ~clk;
  initial begin
    $dumpfile("fifo.vcd");           // open the VCD file
    $dumpvars(0, tb_vcd);           // depth 0 = everything below tb_vcd
    #12 rst_n = 1;
    repeat (3) begin @(negedge clk) wr_en = 1; wdata = wdata + 8'h11; end
    @(negedge clk) begin wr_en = 0; rd_en = 1; end
    #20 $dumpoff;                   // stop dumping (e.g. skip a long idle phase)
    #100 $dumpon;
    #20 $finish;
  end
endmodule

```

Dumping the FIFO of Chapter 22 to a VCD file.

```

VCD info: dumpfile fifo.vcd opened for output.
tb_vcd.sv:16: $finish called at 190000 (1ps)

```

```

$timescale
  1ps
$end
$scope module tb_vcd $end
$var wire 8 ! rdata [7:0] $end
$var wire 1 " full $end
$var wire 1 # empty $end
$var reg 1 $ clk $end
...
$enddefinitions $end
#0
$dumpvars
bx /
b0 *

```

Start of fifo.vcd (abridged): a header that maps short identifier codes to signals, then time stamps (#t) and value changes.

Open the file with `gtkwave fifo.vcd`, drag signals from the hierarchy into the wave pane, and save the signal list as a `.gtkw` file so the next debug session starts where this one ended. VCD is plain text and very large; Icarus can write the compact **FST** format (`vvp sim -fst`) that GTKWave also reads. Commercial flows use their own compressed databases - FSDB (Synopsys Verdi), SHM (Cadence SimVision) and WLF (Siemens Visualizer/Questa) - with driver/load tracing, schematic views and the ability to trace a value backwards through the design with one click.

DUMP STRATEGICALLY

Dumping every signal of a SoC for a long test can slow simulation by 2-10x and produce hundreds of gigabytes. Dump only the hierarchy under suspicion, start dumping shortly before the first symptom (`$dumpon` at a time given by a plusarg), and rely on transaction logs to find that time first.

26.3 Common bug patterns

Pattern	Typical symptom	Where to look
Off-by-one in full/empty/count	Overflow or lost data only at exact capacity	Pointer compare, count update on simultaneous read+write

Pattern	Typical symptom	Where to look
Missing or wrong reset	X after reset; works in one simulator only; GLS fails	Flops without reset feeding control
Blocking/non-blocking misuse	Behaviour changes with code order or simulator	= in clocked blocks (Chapter 4)
Width truncation / sign	Wrong results only for large or negative values	Lint width warnings, signed mixing
Inferred latch	Holds stale value in some states	Incomplete if/case in comb logic
Handshake violations	Duplicated or dropped beats under back-pressure	valid depending on ready; data changing while stalled
Pipeline hazard	Wrong result when two operations are back-to-back	Forwarding, stall/valid alignment
FSM lock-up	Hang after a rare sequence	Missing transitions, unhandled simultaneous events
CDC	Rare, seed- or silicon-only failures	Unsynchronized or re-converging crossings (Chapter 25)
Endianness / bit order	Bytes swapped at an interface	Packing, [0:7] vs [7:0]
Testbench race or model bug	DUT looks right in the waves	Sampling edge, reference model assumptions

26.4 X: optimism in RTL, pessimism in gates

Unknown values are the source of the most insidious gap between simulation and hardware. The experiment below leaves a select signal uninitialized - exactly what a flop without reset looks like at time zero - and evaluates the same multiplexer written four ways.

```

module xprop_demo;
  logic      sel;                // never initialized: X, like a flop without reset
  logic [3:0] a = 4'b1100, b = 4'b1010;
  logic [3:0] y_if, y_case, y_tern, y_and;

  always_comb if (sel) y_if = a; else y_if = b;    // if: X treated as FALSE
  always_comb case (sel)                          // case: X matches no item
    1'b1:    y_case = a;
    default: y_case = b;
  endcase
  assign y_tern = sel ? a : b;                    // ?: merges a and b bitwise
  assign y_and  = {4{sel}} & a | {4{~sel}} & b;    // what gates would compute

  initial begin
    #1 $display("sel=%b if:%b case:%b ?::%b gates:%b",
      sel, y_if, y_case, y_tern, y_and);
    sel = 1'b0;
    #1 $display("sel=%b if:%b case:%b ?::%b gates:%b",
      sel, y_if, y_case, y_tern, y_and);
  end
endmodule

```

Four descriptions of one 2:1 multiplexer, evaluated with an X select.

```

sel=x  if:1010  case:1010  ?::1xx0  gates:xxx0
sel=0  if:1010  case:1010  ?::1010  gates:1010

```

Form	Result with sel = X	Behaviour
if (sel)	1010 (= b)	X-optimism: X treated as false; the uncertainty vanishes and a missing reset is hidden

Form	Result with sel = X	Behaviour
<code>case (sel)</code>	1010 (= b)	X matches no item, so <code>default</code> is taken - also optimistic
<code>sel ? a : b</code>	1xx0	LRM merge: bits where a and b agree are known, others X - the most accurate
AND-OR gates	xxx0	X-pessimism : bit 3 is 1 in both inputs, so real hardware outputs 1, but gate evaluation gives X
Real silicon	a or b	sel powers up as 0 or 1 - never X

WHY THIS MATTERS FOR SIGN-OFF

RTL simulation can pass while silicon fails because an `if` swallowed an X from an unreset flop (optimism). Gate-level simulation can fail while silicon works because gate evaluation cannot resolve reconvergent X (pessimism). The cures are the X-propagation tools and reviews of Chapter 25, and resetting every flop whose value is used before it is written.

26.5 Gate-level simulation

Gate-level simulation (GLS) runs the testbench on the synthesized or post-layout **netlist** instead of the RTL. Equivalence checking (Chapter 25) already proves the netlist's logic, and STA (Chapter 18) proves its timing, so GLS is run selectively for what they do not cover:

- **Initialization and reset**: X-pessimism and unreset flops, reset sequencing through the real reset tree.
- **DFT structures**: scan chains, BIST controllers and test-mode muxing (pattern simulation of ATPG vectors).
- **Timing-dependent behaviour** STA does not model: asynchronous interfaces, multicycle and false-path exceptions that might be wrong (GLS with SDF will fail if an exception was not really safe), clock-gating and clock-switching glitches.
- **Power-aware** GLS with the UPF: isolation, retention and power-switch behaviour in the real netlist (Chapter 19).
- A **sanity check** that the netlist boots: a handful of tests, not the full regression - GLS is 10-100x slower than RTL.

In its simplest, **zero-delay** form, GLS just swaps the netlist in. The ALU netlist synthesized by Yosys in Chapter 25 (139 generic gates) runs unmodified with the Chapter 22 testbench:

```
ALU: 1004 vectors, 0 errors -> PASS
tb_alu_gate.sv:59: $finish called at 1004 (1s)
```

The Chapter 22 ALU testbench run on `alu_gate.v` (only the module name in the instantiation changed).

Timing annotation with SDF

For timed GLS, every standard cell's simulation model has a `specify` block with path delays and timing checks, and the extraction/STA tool writes an **SDF** (Standard Delay Format) file with the actual delays of every cell and net instance in the layout. `$sdf_annotate` replaces the model's default delays with those values. The miniature example below - a toggle flop built from an inverter and a D flip-flop - runs in Icarus with `-gspecify`.

```

`timescale 1ns/1ps
module DFFQ_X1 (input CK, input D, output reg Q);
  always @(posedge CK) Q <= D;
  specify
    (posedge CK => (Q += D)) = (0.10, 0.12); // clk-to-q rise, fall
    $setup(D, posedge CK, 0.05);
    $hold (posedge CK, D, 0.03);
  endspecify
endmodule

module INV_X1 (input A, output ZN);
  assign ZN = ~A;
  specify
    (A => ZN) = (0.02, 0.02);
  endspecify
endmodule

```

cells.v - simplified library cell models with specify blocks.

```

`timescale 1ns/1ps
module toggle_net (input clk, output q);
  wire d;
  INV_X1 u_inv (.A(q), .ZN(d));
  DFFQ_X1 u_ff (.CK(clk), .D(d), .Q(q));
endmodule

```

net.v - the 'netlist'.

```

(DELAYFILE
(SDFVERSION "3.0")
(DESIGN "toggle_net")
(TIMESCALE 1ns)
(CELL (CELLTYPE "INV_X1") (INSTANCE u_inv)
  (DELAY (ABSOLUTE (IOPATH A ZN (0.35) (0.30)))))
(CELL (CELLTYPE "DFFQ_X1") (INSTANCE u_ff)
  (DELAY (ABSOLUTE (IOPATH (posedge CK) Q (0.25) (0.28)))))
)

```

net.sdf - back-annotated delays (normally written by the STA tool).

```

`timescale 1ns/1ps
module tb;
  reg clk = 0; wire q;
  toggle_net dut (.clk(clk), .q(q));
  always #5 clk = ~clk;
  initial begin
`ifdef SDF
    $sdf_annotate("net.sdf", dut);
`endif
`ifdef INIT
    force dut.u_ff.Q = 1'b0; #1 release dut.u_ff.Q; // emulate a reset
`endif
    $monitor("%8.3f ns  clk=%b q=%b", $realtime, clk, q);
    #22 $finish;
  end
endmodule

```

tb.v - INIT emulates a reset; SDF enables annotation.

```

$ iverilog -gspecify -DINIT -o g tb.v net.v cells.v && vvp g
$ iverilog -gspecify -DINIT -DSDF -o g tb.v net.v cells.v && vvp g
$ iverilog -gspecify -DSDF -o g tb.v net.v cells.v && vvp g

```



```

5.000 ns  clk=1 q=0
5.100 ns  clk=1 q=1
10.000 ns clk=0 q=1
15.000 ns clk=1 q=1
15.120 ns clk=1 q=0

```

Run 1 (excerpt): the library's default clock-to-Q delays, 100 ps rise and 120 ps fall.

```

5.000 ns  clk=1 q=0
5.250 ns  clk=1 q=1
10.000 ns clk=0 q=1
15.000 ns clk=1 q=1
15.280 ns clk=1 q=0

```

Run 2 (excerpt): SDF-annotated delays of 250/280 ps from the layout.

```

0.000 ns  clk=0 q=x
5.000 ns  clk=1 q=x
10.000 ns clk=0 q=x
15.000 ns clk=1 q=x
20.000 ns clk=0 q=x
tb.v:14: $finish called at 22000 (1ps)

```

Run 3: no reset. The flop starts at X and the loop X -> NOT -> X never resolves. In silicon the flop would simply toggle from whatever value it powered up with.

Run 3 is the textbook GLS problem: RTL written as `always @(posedge clk) q <= ~q;` has the same X-lock in RTL simulation, but a slightly different RTL coding (an `if` on the value) would have hidden it by optimism and only GLS would show it. The right fix is a reset in the design, not a `force` in the testbench; for datapath flops that legitimately have no reset, simulators offer options to initialize them to random values.

GLS issue	Explanation and handling
Timing-check violations	<code>\$setup</code> , <code>\$hold</code> , <code>\$width</code> , <code>\$recovery</code> in cell models report violations and drive the flop output to X through a notifier . Real violations are bugs; violations on the first flop of a synchronizer are expected and are disabled for those instances only
X-pessimism	Reconvergent X through gates never resolves; requires resets or targeted initialization - never global 'force everything to 0'
Zero-delay races	Without SDF, clock-tree buffers make the netlist race (data and clock change in the same time step); use unit-delay mode or SDF
Corner selection	Run SDF from min (hold) and max (setup) corners, as STA does
Speed	Run a small, targeted test list; save-and-restore after reset to skip initialization

PITFALL: GLS AS A SUBSTITUTE FOR STA

Timed GLS only checks the paths the test happens to exercise, with the vectors it happens to apply. STA checks every path. GLS complements STA by validating the **constraints** (a wrong false path shows up as a GLS failure) - it never replaces it.

26.6 FPGA prototyping of ASIC RTL

An FPGA prototype maps the ASIC RTL onto one or more large FPGAs and runs it at 10-100 MHz - thousands of times faster than simulation - so that software teams can boot operating systems and run real workloads months before silicon, and so that the design can talk to real interfaces (Ethernet, USB, cameras). The RTL is the same, but ASIC-specific structures must be converted:

ASIC construct	FPGA conversion
Gated clocks / ICG cells	Gated-clock conversion: the prototyping synthesis tool (or a wrapper) turns clock gates into clock enables on the flops, keeping the design on a few global clocks; or map to <code>BUFGCE</code> -type global buffers

ASIC construct	FPGA conversion
PLLs, clock dividers	FPGA MMCM/PLL primitives; generated clocks as enables where possible
Memory-compiler SRAMs/ROMs	Behavioural models inferring block RAM behind the same wrapper interface
Standard-cell instances, pads, analog	Behavioural models, FPGA I/O buffers, or external daughter cards for the analog
Asynchronous resets everywhere	Supported but costly; many teams convert to synchronous resets for the prototype
Design too large for one FPGA	Partitioning across FPGAs with pin multiplexing (TDM of many signals over one wire), which lowers the achievable clock
DFT, power-management logic	Tied off or removed (<code>ifdef FPGA</code>)

The key design-for-prototyping practice is to put every technology-specific element behind a **wrapper** with one RTL interface and two implementations selected at build time, so that neither the ASIC nor the prototype needs forked RTL.

```

module sram_sp #(parameter int AW = 10, parameter int DW = 32) (
    input  logic      clk, ce, we,
    input  logic [AW-1:0] addr,
    input  logic [DW-1:0] wdata,
    output logic [DW-1:0] rdata
);
`ifdef ASIC
    // Foundry memory-compiler macro (active-low enables, as most macros)
    SRAM1RW1024X32 u_macro (.CLK(clk), .CEB(~ce), .WEB(~we),
                           .A(addr), .D(wdata), .Q(rdata));
`else
    // FPGA and RTL simulation: behavioural model that infers block RAM.
    // Same interface and 1-cycle read latency as the macro.
    logic [DW-1:0] mem [2**AW];
    always_ff @(posedge clk)
        if (ce) begin
            if (we) mem[addr] <= wdata;
            rdata <= mem[addr];           // read-before-write, like the macro
        end
`endif
endmodule

```

A single-port SRAM wrapper: foundry macro for the ASIC, inferred block RAM for FPGA and simulation.

```

module clk_gate (
    input  logic clk, en, test_en,
    output logic gclk
);
`ifdef FPGA
    // FPGA: gate with a global clock buffer (or convert to enables, see text)
    BUFGCE u_bufgce (.I(clk), .CE(en | test_en), .O(gclk));
`else
    // ASIC: latch-based clock gate. In a real flow this is the library ICG cell.
    logic en_lat;
    always_latch
        if (!clk) en_lat = en | test_en;    // transparent while clk is low
    assign gclk = clk & en_lat;             // glitch-free: en only changes when clk=0
`endif
endmodule

```

A clock-gate wrapper: latch-based ICG for the ASIC build, global clock buffer for the FPGA build.

The simulation branches of both wrappers were verified together (the testbench writes and reads back two SRAM locations and toggles the gate enable in the middle of a high clock phase, which must not produce a glitch):

```
sram readback ok, gated clock pulses = 3 (expect 3)
tb_wrap.sv:25: $finish called at 115 (1s)
```

Multi-FPGA prototyping systems (Synopsys HAPS, Cadence Protium, S2C Prodigy, Siemens proFPGA) come with partitioning software, pin multiplexing IP and debug instrumentation. On-FPGA debug uses embedded logic analyzers - Xilinx ILA (ChipScope), Intel SignalTap - which capture a limited window of selected signals into on-chip RAM when a trigger fires; visibility is the prototype's main weakness.

26.7 Hardware emulation

An **emulator** is a purpose-built machine that compiles the RTL (billions of gates) into a massively parallel hardware fabric and runs it at around 1 MHz, with **full visibility** of every signal and fast compile times compared with FPGA prototyping. Three architectures dominate:

System	Vendor	Architecture
Palladium	Cadence	Custom processor-based : thousands of Boolean processors evaluate the design; fast compile, high capacity
Veloce	Siemens	Custom emulation chips (Veloce Strato), plus FPGA-based Primo for prototyping
ZeBu	Synopsys	FPGA-based emulation with commercial FPGAs: higher speed, longer compile

	RTL sim	Emulation	FPGA prototype	Silicon
Speed	10 Hz - 1 kHz (SoC)	~1 MHz	10-100 MHz	GHz
Visibility	Full	Full (with trace depth limits)	Very limited	Minimal
Capacity	Unlimited, slow	Billions of gates	Needs partitioning	-
Turnaround after RTL change	Minutes	Hours	Hours-days	Months
Typical use	Block/SoC verification	SoC verification, SW bring-up, power analysis	SW development, real I/O	Validation

Emulators are used in two modes. In **in-circuit emulation (ICE)** the emulated design is cabled to real hardware through **speed bridges** that buffer traffic between the slow emulator and full-speed peripherals. In **transaction-based acceleration** the testbench stays on a workstation and talks to synthesizable transactors (BFMs) in the emulator through SCE-MI or DPI-style function calls - the whole UVM environment gets faster because only transactions, not every clock edge, cross the link. Designs must therefore keep synthesizable assertions and BFM in mind.

26.8 Hardware/software co-verification

A modern SoC is not correct until its firmware and drivers run on it. Co-verification starts long before silicon:

- **Embedded C tests in RTL simulation:** the real CPU RTL executes a small C program compiled for the target and loaded into memory with `$readmemh`; the program writes pass/fail to a mailbox register the testbench monitors. This verifies integration, boot code, address maps and interrupts.
- **Virtual platforms:** instruction-accurate models (QEMU, SystemC TLM-2.0, Arm Fast Models) run the software stack at hundreds of MIPS before the RTL exists.
- **Hybrid emulation:** the CPU subsystem runs as a fast virtual model on the host while the new hardware runs in the emulator - Linux boots in minutes instead of days.
- **Portable Stimulus (Accellera PSS):** one abstract scenario description generates UVM sequences for block level and C tests for SoC level, emulation and silicon.

26.9 Post-silicon debug

When first silicon returns, visibility collapses to the pins. Everything that will be needed to debug it must have been designed in (Chapter 20):

Feature	What it provides
JTAG (IEEE 1149.1) + debug access port	Halt, step and inspect CPUs (Arm CoreSight DAP, RISC-V Debug Module); read and write memory and registers through the TAP
On-chip trace	Program-flow and data trace (Arm ETM/ETE, STM for software instrumentation) into an on-chip buffer (ETB/ETR) or off-chip through a trace port
Embedded logic analyzers	Trigger logic and trace RAM on selected internal buses, configured through JTAG
Performance counters	Count events (cache misses, stalls, bus retries) to debug performance, not just function
Scan dump	Stop the clocks and shift out every flop's state through the scan chains - a full snapshot of the chip
Spare cells and metal ECO	Unused gates scattered in the layout, so a fix can be made by changing only metal masks
Shmoo plots	Pass/fail across voltage and frequency to separate logic bugs from timing/power-integrity problems

THE COST CURVE OF A BUG

A bug found by the designer's own unit test costs minutes. In SoC regression it costs days of several people's time. In emulation or on the prototype, weeks. In silicon, a metal ECO costs months and a mask set; a full respin of an advanced-node chip costs tens of millions of dollars and a market window. Every technique in Part V exists to move bug discovery to the left of this curve.

26.10 Summary

- Debug systematically: reproduce, find the first symptom, trace backwards to the source, test a hypothesis, fix and generalize.
- Dump waves selectively (`$dumpvars`, `$dumpoff`) and view them in GTKWave or a commercial debugger; lead with transaction logs.
- RTL simulation is X-optimistic (`if`, `case`), gate evaluation is X-pessimistic; silicon is neither - reset what you use.
- GLS checks initialization, DFT, SDF-timed asynchronous behaviour and constraints; it complements, never replaces, LEC and STA.
- FPGA prototypes need wrappers for memories, clock gating and technology cells; emulators trade speed for full visibility.
- Post-silicon debug relies on JTAG, trace, embedded analyzers, scan dump and spare cells designed in from the start.

26.11 Exercises

- 1 Add a `+DUMP_START=<ns>` plusarg to `tb_vcd.sv` that starts dumping only at the given time, and verify the VCD begins there.
- 2 Rewrite the X experiment with `unique case` and with `priority if`. Does your simulator warn when the selector is X?
- 3 Add an active-low reset to the toggle flop cell model and rerun run 3 with a reset pulse instead of `force`. What changes in the SDF file if the reset-to-Q path has a delay?
- 4 List everything in the FIFO of Chapter 22 that would need an FPGA wrapper or conversion if the FIFO were built with a foundry register-file macro and a clock-gated write port.

- 5 Estimate how long a 10^{10} -cycle Linux boot takes in RTL simulation (100 Hz), emulation (1 MHz) and on a 50 MHz FPGA prototype.

PART VI

Projects and the Roadmap

The habits that make RTL reviewable and reusable; two complete, simulated projects - an APB UART subsystem and an AXI-Stream int8 MAC accelerator for on-device ML - that put every earlier chapter to work; and a staged career roadmap with a study plan, portfolio ideas and an honest look at how RTL interviews are run.

27. RTL Coding Guidelines and Design Reviews

Every serious chip company has an RTL coding guideline, and on the surface they look like a list of style preferences: suffixes, indentation, where the reset goes. They are not. Each rule exists because someone, somewhere, shipped a bug, lost a week in a CDC review, or taped out a latch. A guideline is a compressed record of expensive mistakes, and the design review is the process that enforces it where tools cannot.

This chapter collects the rules that almost every company shares, explains the reason behind each, and then turns them into a working review process: a micro-architecture spec template, a review checklist, the findings reviewers raise most often, and the git and regression discipline that keeps a design healthy for the two years between the first commit and tapeout.

27.1 Why guidelines exist

Audience	What consistent RTL buys them
Reviewers	A reviewer who knows that every <code>_q</code> is a flop and every <code>_d</code> is its next-state value can read a 2000-line module in an hour instead of a day, and spots the one signal that breaks the pattern
Lint, CDC and RDC tools (Ch. 25)	Tools classify signals by name and structure: clock and reset naming lets them infer domains, and consistent synchronizer cells let CDC tools recognize approved crossings instead of flagging hundreds of false violations
Synthesis and STA (Ch. 17-18)	Predictable structure gives predictable netlists: register names survive into the netlist, so <code>u_rx/st_q_reg[1]</code> in a timing report maps straight back to the RTL
Verification (Ch. 22-24)	Uniform handshakes and register conventions let one agent or one register model serve every block
Your future self	Blocks are reused across three or four chip generations. The engineer who modifies your FIFO in 2029 may be you

THE ONE-SENTENCE PHILOSOPHY

Write RTL so that **the hardware is obvious from the text**: a reader should be able to draw every flop, every mux and every clock domain without simulating anything. Every rule below serves that goal.

27.2 Naming conventions

Names carry type information that Verilog itself does not. The table is a composite of widely used industrial conventions (lowRISC/OpenTitan, many in-house guides); individual companies differ in details, but **pick one convention per project and apply it without exception** - a convention applied 90% of the time is worse than none, because readers trust it.

Pattern	Meaning	Example
<code>_q</code>	Output of a flop (current state)	<code>cnt_q, st_q</code>
<code>_d</code>	Next-state value computed combinatorially, input of the <code>_q</code> flop	<code>cnt_d</code>
<code>_n</code> / <code>_ni</code>	Active-low signal (lowRISC uses <code>_ni</code> / <code>_no</code> for active-low ports)	<code>rst_n, cs_n</code>
<code>i_</code> / <code>o_</code> / <code>io_</code> prefix, or <code>_i</code> / <code>_o</code> suffix	Port direction, visible at every use inside the module	<code>i_valid, o_ready</code>
<code>clk_<domain></code> , <code>rst_<domain>_n</code>	One name per clock and reset domain; the domain name travels with every signal of that domain	<code>clk_axi</code> , <code>rst_apb_n</code>

Pattern	Meaning	Example
<code>_sync, _meta</code>	Output and first stage of a synchronizer; CDC reviewers search for these	<code>rx_d_sync</code>
<code>_async</code>	Signal not yet synchronized to the local clock	<code>rx_d_async</code>
<code>UPPER_CASE</code>	Parameters, localparams, enum literals, macros	<code>DEPTH, IDLE</code>
<code>_t</code>	User-defined type (typedef, enum, struct)	<code>state_t</code>
<code>u_ / g_</code>	Instance name / generate-block label	<code>u_txf, g_lane</code>
<code><ip>_</code> module prefix	All modules of one IP share a prefix, so names never collide at SoC integration	<code>uart_tx, uart_rx</code>

A representative naming convention. Direction prefixes and suffixes are equivalent; mixing them is not.

The `_q/_d` pair deserves a sentence of its own. It forces the **two-process** style (Ch. 6): one `always_comb` computes every `_d` from the current `_qs` and the inputs, one `always_ff` does nothing but `x_q <= x_d`. The payoff is that every flop's next-state logic is in one place, the register boundary is explicit, and the netlist names match. Many teams relax this for simple counters and pipeline registers written in a single `always_ff`, which is fine as long as the `_q` suffix still marks every flop - the Chapter 28 UART does exactly that.

PITFALL - NAMES THAT LIE

A signal called `valid_q` that is actually combinational, or `rst_n` that is active high after an inverter somebody added later, is worse than an arbitrary name. Reviewers read names, not drivers. When a signal's nature changes, **rename it** in the same commit.

27.3 File and module organization

- **One module per file, file named after the module** (`uart_tx.sv` holds `module uart_tx`). Tools, filelists, code search and reviewers all assume it; a second module hidden at the bottom of a file is how two versions of the same module end up in one build.
- **Packages for shared types and constants** (`uart_pkg.sv`), compiled before their users. Prefer package parameters over global text macros (`define`), which leak across files in compilation order.
- The `default_nettype none` directive at the top of each RTL file (restored to `wire` at the bottom) **turns every typo in a port connection into a compile error** instead of a silently created 1-bit net.
- Include files (`.svh`) **get include guards** (`ifndef/define` directives), and contain declarations only.
- Filelists (`.f`) **are the single source of truth** for what is in the design; simulation, lint, synthesis and CDC all read the same list.
- **No testbench code in RTL files.** Assertions go in a separate bind file (Ch. 23) or behind a clearly marked `ifdef`.

```

uart/                                one IP = one self-contained tree
|-- doc/      uart_uarch.md          micro-architecture spec (27.9)
|-- rtl/      uart_pkg.sv  uart_apb.sv  uart_tx.sv  uart_rx.sv  uart_fifo.sv
|-- rdl/      uart.rdl          register description -> generated RTL/C/UVM
|-- sva/      uart_sva.sv  uart_bind.sv
|-- tb/      tb_uart.sv      (UVM env in tb/uvm/)
|-- lint/    waivers.tcl      every waiver has a reason and an owner
|-- syn/     uart.sdc         block-level constraints (Ch. 18)
|-- uart.f   filelist: package first, then leaf modules
`-- Makefile  make sim | lint | cdc | syn | regress

```

A typical IP directory. The point is that an integrator can take the whole directory and nothing else.

27.4 Parameterization

Parameters make a block reusable; badly chosen parameters make it unverifiable. The rules: give every parameter a **type** and a sensible **default**; expose only what an integrator should change; compute everything else as a **localparam**; and **check illegal values at elaboration** so a bad configuration fails the build rather than the silicon.

```
module fifo_cfg #(
    parameter int unsigned DEPTH = 8,
    parameter int unsigned W      = 32
) (
    input  wire logic      i_clk,
    input  wire logic [W-1:0] i_data,
    output      logic [W-1:0] o_data
);
    localparam int unsigned AW = $clog2(DEPTH);          // derived, not overridable
    // Elaboration-time check (IEEE 1800-2009): stops the build in lint/synthesis
    if (DEPTH < 2 || (DEPTH & (DEPTH - 1)) != 0) begin : g_bad_depth
        $error("fifo_cfg: DEPTH=%0d must be a power of two >= 2", DEPTH);
    end
    // Simulation-time twin for simulators that ignore elaboration tasks
    // synopsys translate_off
    initial if (DEPTH < 2 || (DEPTH & (DEPTH - 1)) != 0)
        $fatal(1, "fifo_cfg: DEPTH=%0d must be a power of two >= 2", DEPTH);
    // synopsys translate_on
    // ... pointers and storage ...
endmodule
```

Typed parameters, a derived localparam, and two layers of checking.

```
FATAL: snip.sv:19: fifo_cfg: DEPTH=6 must be a power of two >= 2
Time: 0 Scope: top.bad
```

Instantiating `fifo_cfg #(.DEPTH(6))` in Icarus Verilog 12. Icarus silently ignores the elaboration-time `$error` in the generate block, which is exactly why the `initial` twin exists; Verilator, lint tools and commercial simulators honor the elaboration check.

- Every parameter combination you allow is a design you must verify. Document the **supported** range and regress at least the corners (minimum, maximum, one typical value).
- Never let a parameter change the **interface protocol**; change widths and depths, not behavior. Two behaviors belong in two modules or an explicit mode port.
- Use `$clog2` for pointer widths, and remember `$clog2(1) = 0` - a zero-width vector is illegal, which is one reason `DEPTH >= 2` is enforced.

27.5 Reset and clock rules

Rule	Why
One clock per <code>always_ff</code> , named <code>clk_<domain></code>	Each flop belongs to exactly one domain; STA, CDC and DFT tools all depend on it
No logic, muxes or dividers on clock nets in RTL; use the library clock-gating (ICG) cell through a wrapper	Gated clocks built from AND gates glitch and break STA and scan (Ch. 19-20)
Asynchronous assert, synchronous de-assert resets (Ch. 12)	Reset works with no clock, and release never violates recovery/removal
Reset control flops; do not reset pure datapath flops	Reset flops are larger and add reset-tree load; data qualified by a reset valid bit does not need a reset
Never mix reset and non-reset flops in one <code>always_ff</code>	The reset becomes a load-enable on the non-reset flops (see below)
No <code>initial</code> blocks or declaration initializers in RTL	FPGA tools honor them, ASIC synthesis ignores them: the simulation then lies about power-up state

Rule	Why
Every CDC goes through an approved synchronizer cell or module	CDC tools recognize the cell; hand-written double flops get optimized, retimed or mis-constrained
No # delays in RTL	Ignored by synthesis; they hide races that reappear in gate-level simulation

```

module mix_bad (input wire logic clk, rst_n, in_valid, input wire logic [7:0] in_data,
                output logic valid_q, output logic [7:0] data_q);
    // BAD: data_q has no reset but shares a block with a reset flop
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) valid_q <= 1'b0;
        else begin
            valid_q <= in_valid;
            data_q <= in_data;      // rst_n silently becomes a hold-enable
        end
    endmodule

module mix_good (input wire logic clk, rst_n, in_valid, input wire logic [7:0] in_data,
                 output logic valid_q, output logic [7:0] data_q);
    always_ff @(posedge clk or negedge rst_n)    // control: reset
        if (!rst_n) valid_q <= 1'b0;
        else    valid_q <= in_valid;
    always_ff @(posedge clk)                    // datapath: no reset
        if (in_valid) data_q <= in_data;
    endmodule

```

The mixed-reset bug and its fix (both compiled with iverilog -Wall).

PITFALL - THE RESET THAT BECAME AN ENABLE

In `mix_bad`, `data_q` has no reset branch, so while `rst_n` is low the block must **hold** `data_q`. Synthesis implements that with a feedback mux controlled by `rst_n` - an extra gate per bit, a timing path from the reset tree into the datapath, and a flop that is neither reset nor free-running for DFT. Lint tools flag it (for example as a 'reset used as data' or 'partial reset' rule); good reviewers flag it faster. The fix is always two blocks.

27.6 A module written to the guideline

The small timeout counter below applies every rule so far: a header block, typed parameter, direction prefixes, documented ports, `_d/_q` two-process style with defaults at the top of `always_comb`, an async-assert reset used only on control state, and a registered output.

```

`timescale 1ns/1ps
// -----
// Module   : tmr_timeout
// Purpose  : Counts enabled cycles; o_expired rises when the count hits i_limit
// Clocking : single domain i_clk
// Reset    : i_rst_n, active low, async assert / sync deassert (see Ch. 12)
// -----
module tmr_timeout #(
    parameter int unsigned CNT_W = 16          // counter width in bits
) (
    input  logic      i_clk,
    input  logic      i_rst_n,
    input  logic      i_en,                    // count enable
    input  logic      i_clr,                    // sync clear, beats i_en
    input  logic [CNT_W-1:0] i_limit,           // must be static while i_en
    output logic      o_expired                 // registered level output
);
    logic [CNT_W-1:0] cnt_q, cnt_d;
    logic      expired_q, expired_d;

```

```

// Next-state logic: combinational only, every output defaulted first
always_comb begin
    cnt_d      = cnt_q;
    expired_d  = expired_q;
    if (i_clr) begin
        cnt_d      = '0;
        expired_d  = 1'b0;
    end else if (i_en && !expired_q) begin
        cnt_d      = cnt_q + 1'b1;
        expired_d  = (cnt_d == i_limit);
    end
end

// State: flops only, nothing else in this block
always_ff @(posedge i_clk or negedge i_rst_n)
    if (!i_rst_n) begin
        cnt_q      <= '0;
        expired_q <= 1'b0;
    end else begin
        cnt_q      <= cnt_d;
        expired_q <= expired_d;
    end

    assign o_expired = expired_q;
endmodule

```

tmr_timeout.sv - a reference module in house style.

```

`timescale 1ns/1ps
module tb_tmr;
    logic clk = 0, rst_n = 0, en = 0, clr = 0, expired;
    tmr_timeout #(.CNT_W(4)) dut (.i_clk(clk), .i_rst_n(rst_n), .i_en(en),
                                   .i_clr(clr), .i_limit(4'd5), .o_expired(expired));

    always #5 clk = ~clk;
    always @(posedge expired) $display("%0d ns: expired, cnt=%0d", $time, dut.cnt_q);
    initial begin
        #12 rst_n = 1;
        @(negedge clk) en = 1;
        repeat (8) @(negedge clk);
        $display("%0d ns: still expired=%0b, cnt=%0d (saturates)", $time, expired, dut.cnt_q);
        clr = 1; @(negedge clk) clr = 0;
        $display("%0d ns: after clr expired=%0b cnt=%0d", $time, expired, dut.cnt_q);
        repeat (6) @(negedge clk);
        $finish;
    end
end
endmodule

```

A minimal directed testbench.

```

65 ns: expired, cnt=5
100 ns: still expired=1, cnt=5 (saturates)
110 ns: after clr expired=0 cnt=0
155 ns: expired, cnt=5
tb_tmr.sv:16: $finish called at 170000 (1ps)

```

The counter saturates at the limit, **i_clr** wins over **i_en**, and counting resumes after the clear.

WHY THE DEFAULTS SIT AT THE TOP OF ALWAYS_COMB

Assigning **cnt_d = cnt_q** before any **if** makes it impossible to leave a path unassigned, so no latch can be inferred no matter how the conditions are later edited. It also documents the flop's default behavior - hold - in one line. **always_comb** then adds a tool check: lint and synthesis warn if a latch is still inferred.

27.7 Rules that prevent silicon bugs

Rule	What goes wrong without it
<code>always_ff</code> with <code><=</code> ; <code>always_comb</code> with <code>=</code>	Simulation races and simulation/synthesis mismatch (Ch. 4)
Default every output of <code>always_comb</code>	Inferred latches; timing loops; X in gate-level simulation
<code>case</code> with <code>default</code> ; no <code>casex</code> ; <code>casez</code> only with <code>?</code> wildcards	<code>casex</code> treats X from the input as don't-care and hides real X bugs
<code>unique/priority</code> only when the condition is truly guaranteed	They grant synthesis permission to optimize; if the assumption is false, silicon differs from RTL
Match widths explicitly; size every literal	Silent truncation or zero-extension; <code>a + b</code> losing its carry
Cast signed arithmetic deliberately (<code>\$signed</code> , <code>33'(x)</code>)	One unsigned operand makes the whole expression unsigned (Ch. 8)
One driver per signal; one <code>always</code> block per signal	Multiple drivers are X in simulation and a short circuit or error in synthesis
No combinational loops, including through module boundaries	Oscillation, untimeable paths; often created by valid/ready handshakes that depend on each other (Ch. 10)
Register module outputs where practical	Unbounded combinational paths across block boundaries make top-level timing unpredictable
Full FSM encoding with a safe <code>default</code> transition	An illegal state reached by an upset or a bug locks the FSM forever (Ch. 7)

27.8 Comments and documentation

- **Comment the why, not the what.** `cnt_q <= cnt_q + 1 // increment` is noise; `// count to 15: 16x oversampling = one bit time` is the sentence the next engineer needs.
- **Every port gets a one-line comment** with its meaning, units and timing constraints ("level", "1-cycle pulse", "must be static while enabled").
- **Every module gets a header:** purpose, clocks, resets, parameters, and a pointer to the micro-architecture spec section.
- **Every waiver, magic number and workaround gets a ticket reference.** `// TODO` without an owner and a ticket is a bug nobody will fix.
- **Registers are documented from a single source** (SystemRDL or IP-XACT, Ch. 15) that generates the RTL, the C header, the UVM register model and the HTML documentation. Hand-maintained register tables drift from the RTL within weeks.
- **Diagrams live next to the code** (in `doc/`), under version control, in a text-diffable format where possible (WaveDrom JSON for waveforms, Markdown/AsciiDoc for specs).

27.9 The micro-architecture specification

The architecture spec says **what** a block does; the micro-architecture spec says **how** - the pipeline, the FSMs, the storage, the corner cases - at a level where a reviewer can find bugs **before** a line of RTL exists. Writing it is not bureaucracy: it is the cheapest verification you will ever do. A template that works:

Section	Contents
1. Overview	Purpose, features, non-goals, target PPA, standards followed (AMBA version, UART framing)
2. Interfaces	Every port: direction, width, protocol, timing; reference to the bus spec
3. Clocks and resets	Domains, frequencies, crossings and how each is synchronized; reset sequence and values
4. Block diagram	Sub-blocks, the datapath, every flop stage on the critical path
5. Datapath	Widths, number formats, rounding and saturation, pipeline timing diagram

Section	Contents
6. Control	FSM state diagrams, arbitration, flow control and back-pressure behavior
7. Registers	Generated register map; side effects (W1C, read-to-pop); reset values
8. Performance	Latency, throughput, buffering calculations (e.g. FIFO depth vs. interrupt latency)
9. Errors and corner cases	Overflow, underflow, illegal configuration, what software sees
10. Power, DFT, safety	Clock-gating points, power domains, scan exceptions, parity/ECC
11. Verification hooks	Assertions to write, coverage points, features that need formal
12. Open issues	Decisions still to make, with owners

27.10 Design reviews

A project runs several kinds of review, each with a different question. The **spec review** asks 'is this the right block?'; the **micro-architecture review** asks 'will this design meet its requirements?'; the **RTL code review** asks 'does this code implement the micro-architecture cleanly and safely?'; the **verification plan review** asks 'how will we know?'; the **sign-off review** asks 'is every report clean or waived with a reason?'. Mixing these questions in one meeting is the most common reason reviews are long and useless.

RTL design-review checklist

- ☐ Micro-architecture spec exists, is current, and matches the RTL block diagram
- ☐ Every port documented; directions, widths and active levels match the spec
- ☐ Naming convention followed: `_q`, `_d`, `_n`, domain names on clocks and resets
- ☐ Every clock-domain crossing uses an approved synchronizer; CDC report clean
- ☐ Reset strategy correct: async assert / sync release; no mixed reset blocks
- ☐ No latches, no combinational loops, no multiple drivers (lint clean)
- ☐ All `case` statements have `default`; FSMs recover from illegal states
- ☐ Arithmetic widths, signedness, overflow and saturation reviewed
- ☐ Valid/ready: no combinational ready-to-valid dependency; data stable while valid and not ready
- ☐ Register map generated from the single source; side effects (W1C, RC) match the spec
- ☐ Every parameter value range checked at elaboration
- ☐ Critical paths estimated (logic levels per stage) against the target clock
- ☐ Clock-gating enables and power-domain boundaries identified (Ch. 19)
- ☐ No DFT blockers: no gated clocks outside ICGs, resets controllable in test mode
- ☐ Assertions for key protocol and FSM properties exist and are bound
- ☐ Every lint/CDC waiver has a reason, an owner and a ticket
- ☐ Code coverage and functional coverage targets agreed with verification

HOW TO RUN A CODE REVIEW THAT FINDS BUGS

Keep each review under about 400 lines of change and 60-90 minutes; defect-finding rate drops sharply beyond that. Have the author walk through the **data path first**, then the control. Reviewers should read with the spec open, and ask 'what happens when...' about every input: back-pressure during the last beat, reset in the middle of a frame, a write and an event on the same W1C bit in the same cycle.

27.11 Common review findings

Finding	Symptom	Fix
Incomplete <code>always_comb</code> assignment	Latch inferred; lint warning	Default assignments at the top of the block
Reset and non-reset flops in one block	Reset used as data enable	Split into two <code>always_ff</code> blocks
Combinational <code>ready</code> depends on <code>valid</code>	Loop across modules at integration	Register ready or insert a skid buffer (Ch. 10)
Unsynchronized async input	Metastability; random field failures	2-flop synchronizer; CDC tool sign-off
Multi-bit bus crossing through 2-flop syncs	Bits arrive in different cycles: corrupted values	Gray code, handshake or async FIFO (Ch. 11)
W1C bit loses an event that arrives with the clear	Missed interrupts under load	Make set win over clear: <code>(q & ~w1c) evt</code>
Counter or pointer width off by one	Wrap at wrong value; FIFO full/empty confusion	Extra MSB pointer scheme; assertions
Signed/unsigned mix	Wrong results only for negative values	Explicit casts; directed negative tests
Output driven combinational from inputs	Top-level timing failure	Register outputs or budget the path
Magic numbers	Next change breaks one of three copies	<code>localparam</code> or package constant
Clock gating written as <code>clk & en</code>	Glitches, STA and scan break	Library ICG through a wrapper module

27.12 A git workflow for RTL

RTL is software in its workflow even if it is hardware in its result. The practices that matter most are small, reviewed changes; a CI pipeline that runs lint and a smoke regression on every push; and **tags on milestones** so that any netlist can be traced back to the exact RTL it came from.

```
# feature branch per change, named after the ticket
git switch -c uart/1234-rx-overflow-irq
# ... edit RTL, testbench and the register description together ...
make lint sim TEST=uart_overflow          # local gate before pushing
git commit -m "uart: add RX overflow interrupt (W1C, set wins) [#1234]"
git push -u origin uart/1234-rx-overflow-irq
# CI: lint + CDC + smoke regression; reviewer approves; squash-merge

# milestone: freeze the RTL that goes to synthesis
git tag -a uart_rtl_v1.0_freeze -m "RTL freeze for block synthesis"
```

A typical change flow. The tag is what synthesis, DFT and STA reports reference.

- **Commit the source, generate the rest:** register RTL, C headers and filelists generated in CI from the single source - unless your flow requires the generated RTL to be frozen and reviewed, in which case commit it with the generator version recorded.
- **Never commit simulator databases, waveforms or netlists;** keep them in `.gitignore` and store sign-off artifacts in a release area.
- **Change RTL, testbench and spec in the same pull request** - a spec that lags the RTL by one commit is already wrong.
- **Write commit messages for the bug hunter** two years from now: which block, what behavior changed, which ticket.

27.13 Regression discipline

Tier	When	Contents	Budget
Smoke	Every push (CI)	Lint, compile, 10-50 directed tests	< 15 min
Nightly	Every night on main	Full directed + constrained-random suite, many seeds, coverage merge	Hours
Weekly	Weekend	Long random runs, formal proofs, CDC/RDC, gate-level smoke (Ch. 26)	Days
Milestone	Before each freeze	Everything above, plus equivalence (Ch. 25) and power-aware simulation	As needed

- **Record the seed** of every random test and make any failure reproducible from one command line.
- **A red regression is the top priority** of the team that broke it. Two days of ignored failures and nobody knows which commit broke what.
- **Never accept 'flaky'**. A test that fails 1 time in 50 is a race in the testbench or a real corner case in the design - both must be fixed.
- **Track trends, not snapshots**: pass rate, coverage and bug arrival rate over time are what tell management the design is converging toward tapeout.

27.14 Summary

- Coding guidelines are compressed experience: each rule prevents a known class of bug or tool problem.
- Names carry hardware meaning: `_q/_d` for flops and next state, `_n` for active low, direction prefixes, domain-tagged clocks and resets.
- One module per file, typed parameters with elaboration checks, `default_nettype none`, and filelists as the single source of truth.
- Reset and clock rules exist for STA, CDC and DFT; never mix reset and non-reset flops in one block.
- A micro-architecture spec and focused reviews find bugs when they are cheapest; checklists keep reviews complete.
- Small reviewed changes, milestone tags and tiered regressions keep a design converging for its whole life.

27.15 Exercises

- 1 Rewrite `tmr_timeout` in single-process style (one `always_ff`). Which names change, and what does a reviewer lose?
- 2 Take any 100-line module you have written and review it against the checklist in 27.10. List every finding with a proposed fix.
- 3 Draw the gate-level circuit synthesis produces for `mix_bad` for one bit of `data_q`, and explain why it hurts scan insertion.
- 4 Write a micro-architecture spec (sections 1-9 of the template) for the synchronous FIFO of Chapter 9, including the full/empty timing diagram.
- 5 Design a regression plan for a 6-person team working on an AXI interconnect: which tests in which tier, and how do you keep the smoke tier under 15 minutes?

28. Project: An APB UART Subsystem, Designed and Verified

A UART is the 'hello world' of SoC peripherals, and for good reason: it is small enough to write in an afternoon, yet it contains nearly every idea of Parts II and III - an FSM, a datapath, FIFOs, an asynchronous input that must be synchronized, a clock-enable baud generator, a bus slave with side-effect registers, and an interrupt with W1C semantics. Getting all of them right **together** is what distinguishes a peripheral from a collection of exercises.

This chapter builds the complete subsystem in about 210 lines of SystemVerilog, verifies it with a self-checking testbench that sends 53 bytes through the serial line and reads them back over APB, and then lists what a production UART adds. Every listing is the exact code that was simulated.

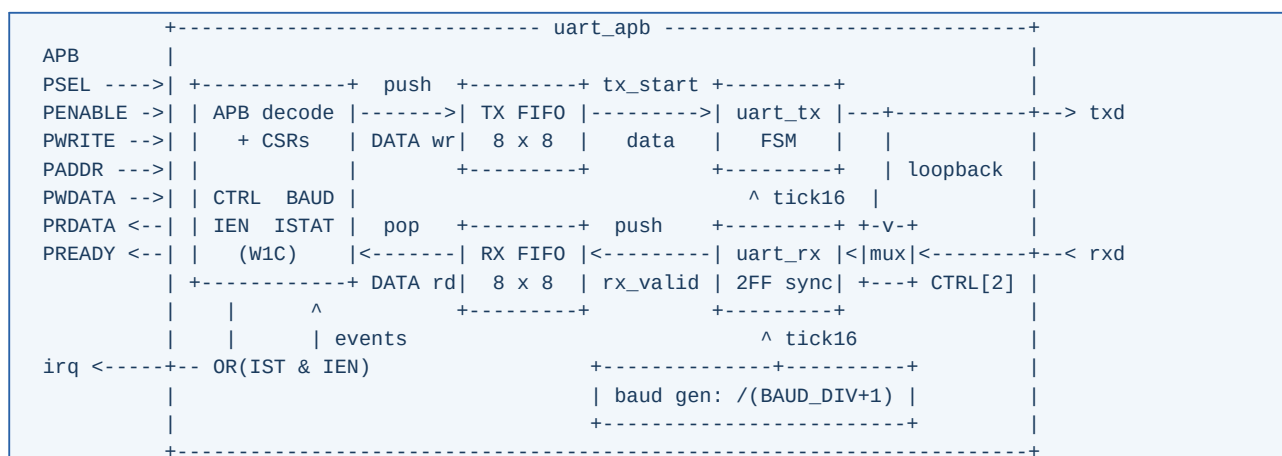
28.1 Specification

- **Bus:** AMBA APB slave (Ch. 13), 32-bit data, zero wait states, `PSLVERR` tied low, 5-bit word-aligned address space.
- **Line format:** 8N1 - one start bit (0), 8 data bits LSB first, one stop bit (1); idle line high.
- **Baud rate:** programmable divider producing a 16x oversampling enable; `baud = f_pclk / (16 x (BAUD_DIV + 1))`.
- **Buffering:** 8-entry TX FIFO and 8-entry RX FIFO.
- **Receiver:** 2-flop synchronizer, false-start rejection, mid-bit sampling, framing-error detection, overrun detection.
- **Interrupt:** one level-sensitive `irq` = OR of enabled sticky events; events are cleared by writing 1 (W1C).
- **Test feature:** internal loopback (TX line fed to RX inside the IP).

Offset	Name	Access	Fields
0x00	DATA	W: push TX FIFO / R: pop RX FIFO	[7:0] byte. A read has a side effect : it pops the RX FIFO
0x04	STATUS	RO	[0] RX_AVAIL [1] RX_FULL [2] TX_EMPTY [3] TX_FULL [4] TX_BUSY
0x08	CTRL	RW, reset 0x3	[0] TX_EN [1] RX_EN [2] LOOPBACK
0x0C	BAUD_DIV	RW, reset 26	[15:0] divider (26 = 115200 baud at 50 MHz)
0x10	INT_EN	RW, reset 0	[3:0] enable per INT_STATUS bit
0x14	INT_STATUS	RW1C, reset 0	[0] RX_DONE [1] TX_DONE [2] RX_OVERRUN [3] FRAME_ERR (sticky)

UART register map. In a real project this table is generated from a SystemRDL description (Ch. 15).

28.2 Block diagram



The subsystem. Everything runs on `pc1k`; the only asynchronous signal is `rx_d`, and it is synchronized inside `uart_rx`.

ONE CLOCK, ONE ASYNCHRONOUS INPUT

The baud generator produces a **clock enable** (`tick16`), not a clock. All flops run on `pclk`, so there is exactly one clock domain, STA is trivial, and the only CDC in the design is the `rx_d` pin - handled by one 2-flop synchronizer. Generating a divided clock would create a second domain, a crossing on every FIFO access, and a clock-tree to build (Ch. 11, 17).

28.3 Baud generation and the 16x oversampling budget

```
f_tick16 = f_pclk / (BAUD_DIV + 1)      baud = f_tick16 / 16
```

$$\text{BAUD DIV} = \text{round}(f_{\text{pc\text{lk}}} / (16 \times \text{baud})) - 1$$

50 MHz, 115200 baud: $50e6 / (16 \times 115200) = 27.13 \rightarrow \text{BAUD_DIV} = 26$

actual = 50e6 / (16 x 27) = 115741 baud error = +0.47 %

Integer division sets the baud error: production UARTs add a fractional divider to reduce it.

Why 16x? The receiver cannot see the transmitter's clock, so it must find the **middle** of each bit by counting from the falling edge of the start bit. With 16 samples per bit, the edge is located to within 1/16 of a bit (plus two **clk** cycles of synchronizer latency), and every later sample is taken 16 ticks after the previous one. The last sample - the stop bit - is taken about 9.5 bit-times after the edge. If the two ends' clocks differ by a fraction e , that sample drifts by 9.5 e bit-times; it must stay within the bit, so roughly $9.5e < 0.5 - 1/16$, i.e. **e below about 4.6% total** between the two ends. That is the real reason a +0.47% divider error is harmless and why cheap RC oscillators are not.

28.4 The FIFOs

```
timescale 1ns/1ps
// Synchronous first-word-fall-through FIFO: rdata shows the head entry.
module uart_fifo #(parameter int W = 8, parameter int AW = 3) (
    input  logic          clk, rst_n,
    input  logic          push, pop,
    input  logic [W-1:0] wdata,
    output logic [W-1:0] rdata,
    output logic          empty, full
);
    logic [W-1:0] mem [2**AW];
    logic [AW:0]  wptr_q, rptr_q;          // extra MSB tells full from empty
    assign empty = (wptr_q == rptr_q);
    assign full  = (wptr_q == {~rptr_q[AW], rptr_q[AW-1:0]});
    assign rdata = mem[rptr_q[AW-1:0]];
endmodule
```

```

always_ff @(posedge clk or negedge rst_n)
  if (!rst_n) begin
    wptr_q <= '0;
    rptr_q <= '0;
  end else begin
    if (push && !full) wptr_q <= wptr_q + 1'b1;
    if (pop && !empty) rptr_q <= rptr_q + 1'b1;
  end
always_ff @(posedge clk) // storage: no reset needed
  if (push && !full) mem[wptr_q[AW-1:0]] <= wdata;
endmodule

```

uart_fifo.sv - first-word-fall-through synchronous FIFO (the Ch. 9 design, specialized).

- **Extra pointer MSB:** equal pointers mean empty; pointers equal except the MSB mean full. No separate counter, no ambiguity at 8 entries.
- **First-word-fall-through:** `rdata` always shows the head entry, so the TX FSM can load it and the APB read can return it in the same cycle as the pop.
- **Storage has no reset,** pointers do - the reset rule from Ch. 27. The entries are meaningless until the pointers say otherwise.
- Push when full and pop when empty are **ignored** rather than corrupting state; the RX overrun interrupt reports the lost byte.

28.5 The transmitter

```

      start (FIFO not empty & TX_EN)
+-----+-----+-----+-----+-----+ 16 ticks +-----+ 8 bits x +-----+
| IDLE | txd<=0, load shifter | START | -----> | DATA | -16 ticks-> | STOP |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
      txd=1                      16 ticks with txd=1                      txd=1

txd: ~~~~~|_start_|_d0_|_d1_|_d2_|_d3_|_d4_|_d5_|_d6_|_d7_|~stop~|~~~~~ idle
      16 tk  16   16   ...                      16

```

TX FSM and the resulting 8N1 frame.

```

`timescale 1ns/1ps
// 8N1 transmitter. tick16 is a 1-cycle enable at 16x the baud rate.
module uart_tx (
  input logic clk, rst_n,
  input logic tick16,
  input logic start, // 1-cycle: load data and begin a frame
  input logic [7:0] data,
  output logic busy,
  output logic txd // registered: glitch-free pad output
);
typedef enum logic [1:0] {IDLE, START, DATA, STOP} state_t;
state_t st_q;
logic [3:0] os_q; // oversample count 0..15 = one bit time
logic [2:0] bit_q;
logic [7:0] sh_q;
assign busy = (st_q != IDLE);
always_ff @(posedge clk or negedge rst_n)
  if (!rst_n) begin
    st_q <= IDLE; os_q <= '0; bit_q <= '0; sh_q <= '0; txd <= 1'b1;
  end else case (st_q)
    IDLE: if (start) begin
      sh_q <= data; os_q <= '0; txd <= 1'b0; st_q <= START;
    end
    START: if (tick16) begin
      os_q <= os_q + 1'b1;
      if (os_q == 4'd15) begin
        txd <= sh_q[0]; bit_q <= '0; st_q <= DATA;

```

```

end
end
DATA: if (tick16) begin
    os_q <= os_q + 1'b1;
    if (os_q == 4'd15) begin
        sh_q <= {1'b0, sh_q[7:1]};           // LSB first
        if (bit_q == 3'd7) begin txd <= 1'b1; st_q <= STOP; end
        else begin txd <= sh_q[1]; bit_q <= bit_q + 1'b1; end
    end
end
end
STOP: if (tick16) begin
    os_q <= os_q + 1'b1;
    if (os_q == 4'd15) st_q <= IDLE;
end
default: st_q <= IDLE;
endcase
endmodule

```

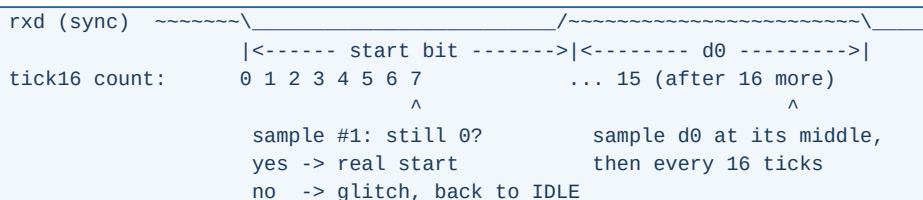
uart tx.sv

The oversample counter `os_q` counts `tick16` enables; 16 of them make one bit time. `txd` is a **flop**, not a decode of the state: an output pad driven from combinational logic glitches whenever the state changes, and a glitch on a serial line is a false start bit at the other end.

A DELIBERATE 1/16-BIT IMPRECISION

`txd` falls as soon as `start` arrives, but the START state counts `tick16` enables, the first of which arrives 1 to `BAUD_DIV+1` cycles later. The start bit is therefore between 15 and 16 ticks long - at most 1/16 of a bit short, well inside the receiver's budget. Aligning `start` to a tick would remove it at the cost of up to one tick of added latency; both choices are defensible, and the spec should state which one was made.

28.6 The receiver: synchronize, find the edge, sample the middle



Mid-bit sampling. The first check at tick 7-8 rejects short glitches; every later sample lands near the centre of its bit.

```
timescale 1ns/1ps
// 8N1 receiver: 2-flop synchronizer, 16x oversampling, mid-bit sampling.
module uart_rx (
    input logic      clk, rst_n,
    input logic      tick16,
    input logic      rxd_async,    // straight from the pad: asynchronous
    output logic     valid,        // 1-cycle pulse with a good byte
    output logic [7:0] data,
    output logic     frame_err     // 1-cycle pulse: stop bit was 0
);
    logic [1:0] sync_q;
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) sync_q <= 2'b11;           // idle line is high
        else      sync_q <= {sync_q[0], rxd_async};
    wire rxd = sync_q[1];

    typedef enum logic [1:0] {IDLE, START, DATA, STOP} state_t;
    state_t    st_q;
    logic [3:0] os_q;
    logic [2:0] bit_q;
```

```

logic [7:0] sh_q;
always_ff @(posedge clk or negedge rst_n)
  if (!rst_n) begin
    st_q <= IDLE; os_q <= '0; bit_q <= '0; sh_q <= '0;
    valid <= 1'b0; data <= '0; frame_err <= 1'b0;
  end else begin
    valid <= 1'b0; frame_err <= 1'b0;
    case (st_q)
      IDLE: if (!rx_d) begin os_q <= '0; st_q <= START; end
      START: if (tick16) begin
        if (os_q == 4'd7) begin // middle of start bit
          if (rx_d) st_q <= IDLE; // glitch: false start
          else begin os_q <= '0; bit_q <= '0; st_q <= DATA; end
        end else os_q <= os_q + 1'b1;
      end
      DATA: if (tick16) begin
        if (os_q == 4'd15) begin // 16 ticks later = mid-bit
          os_q <= '0; sh_q <= {rx_d, sh_q[7:1]};
          if (bit_q == 3'd7) st_q <= STOP;
          else bit_q <= bit_q + 1'b1;
        end else os_q <= os_q + 1'b1;
      end
      STOP: if (tick16) begin
        if (os_q == 4'd15) begin
          st_q <= IDLE; data <= sh_q;
          valid <= rx_d; frame_err <= !rx_d;
        end else os_q <= os_q + 1'b1;
      end
      default: st_q <= IDLE;
    endcase
  end
endmodule

```

uart_rx.sv

- **2-flop synchronizer** (`sync_q`, reset to 1 = idle): `rx_d` is asynchronous to `pclk`, and the first flop may go metastable. The second flop gives it a full cycle to resolve (Ch. 11). Every other flop sees only the clean `rx_d`. The flops reset to 1 so reset release is not mistaken for a start bit.
- **False-start rejection**: a start bit must still be low at its middle; noise shorter than half a bit returns the FSM to IDLE.
- **Framing error**: a 0 where the stop bit should be means the receiver lost bit alignment, the baud rates differ, or the far end sent a BREAK. The byte is not pushed; `frame_err` pulses instead.
- Returning to IDLE in the **middle** of the stop bit is intentional: the receiver is ready half a bit early, which absorbs a transmitter that runs slightly fast.

PITFALL - SYNCHRONIZING INSIDE THE FSM

A common bug is to use the raw pin in the IDLE test (`if (!rx_d_async)`) and the synchronized one elsewhere. The IDLE branch then samples an asynchronous signal into several flops (`st_q`, `os_q`) at once: if it changes near the clock edge, some flops see 0 and some see 1, and the FSM lands in an illegal combination. **Exactly one flop** may sample an asynchronous signal. CDC tools report this as a 'divergence' or 'unsynchronized fan-out'.

28.7 The APB interface, registers and interrupt

```

// APB UART subsystem: registers, baud generator, FIFOs, TX, RX, interrupt.
module uart_apb (
  input  logic      pclk, presetn,
  input  logic      psel, penable, pwrite,
  input  logic [4:0] paddr,
  input  logic [31:0] pwrite,

```

```

output logic [31:0] prdata,
output logic      pready, pslverr,
output logic      irq,
output logic      txd,
input  logic      rxd
);
localparam logic [4:0] A_DATA = 5'h00, A_STAT = 5'h04, A_CTRL = 5'h08,
                      A_BAUD = 5'h0C, A_IEN  = 5'h10, A_ISTAT = 5'h14;
logic [2:0]  ctrl_q;           // [0] TX_EN [1] RX_EN [2] LOOPBACK
logic [15:0] baud_q, bcnt_q;
logic [3:0]  ien_q, ist_q;
logic        tick16, tx_busy, tx_busy_q, rx_valid, rx_ferr;
logic        txf_empty, txf_full, rxf_empty, rxf_full;
logic [7:0]  txf_dout, rxf_dout, rx_data;

wire wr = psel & penable & pwrite;           // APB access phase
wire rd = psel & penable & ~pwrite;
assign pready  = 1'b1;                       // zero-wait-state slave
assign pslverr = 1'b0;

```

uart_apb.sv (1/4): ports, declarations and APB decode.

APB is simple enough that the decode is two AND gates. A transfer spends one cycle in SETUP (**PSEL=1, PENABLE=0**) and one in ACCESS (**PSEL=1, PENABLE=1**); with **PREADY** tied high, every access completes in its first ACCESS cycle, so writes and read side effects happen exactly once, on **psel & penable**.

```

// Baud generator: one tick16 every (BAUD_DIV + 1) pclk cycles
assign tick16 = (bcnt_q >= baud_q);
always_ff @(posedge pclk or negedge presetn)
  if (!presetn)   bcnt_q <= '0;
  else if (tick16) bcnt_q <= '0;
  else           bcnt_q <= bcnt_q + 1'b1;

```

uart_apb.sv (2/4): the baud generator is a counter that emits an enable.

The comparison is **>=** rather than **==** on purpose: if software lowers BAUD_DIV while the counter is above the new value, **==** would let the counter run to 65535 and wrap - a 1-ms hiccup at 50 MHz that a **==** design only shows when someone reprograms the baud rate on the fly.

```

// TX path: APB write to DATA pushes; the FSM pops when idle
wire tx_start = ctrl_q[0] & ~txf_empty & ~tx_busy;
uart_fifo u_txf (.clk(pclk), .rst_n(presetn), .push(wr && paddr == A_DATA),
               .pop(tx_start), .wdata(pwdata[7:0]), .rdata(txf_dout),
               .empty(txf_empty), .full(txf_full));
uart_tx u_tx (.clk(pclk), .rst_n(presetn), .tick16(tick16), .start(tx_start),
             .data(txf_dout), .busy(tx_busy), .txd(txd));

// RX path: the FSM pushes; an APB read of DATA pops
wire rx_in  = ctrl_q[2] ? txd : rxd;           // internal loopback mux
wire rx_push = rx_valid & ctrl_q[1];
uart_rx u_rx (.clk(pclk), .rst_n(presetn), .tick16(tick16), .rx_async(rx_in),
             .valid(rx_valid), .data(rx_data), .frame_err(rx_ferr));
uart_fifo u_rxf (.clk(pclk), .rst_n(presetn), .push(rx_push),
               .pop(rd && paddr == A_DATA), .wdata(rx_data), .rdata(rxf_dout),
               .empty(rxf_empty), .full(rxf_full));

```

uart_apb.sv (3/4): TX and RX datapaths, and the loopback mux.

```

// Interrupts: sticky event bits, write-1-to-clear, set wins over clear
// [0] RX_DONE [1] TX_DONE [2] RX_OVERRUN [3] FRAME_ERR
wire [3:0] evt = {rx_ferr, rx_push & rxf_full, tx_busy_q & ~tx_busy & txf_empty,
                  rx_push & ~rxf_full};
wire [3:0] w1c = (wr && paddr == A_ISTAT) ? pwdata[3:0] : 4'h0;
always_ff @(posedge pclk or negedge presetn)
  if (!presetn) begin ist_q <= '0; tx_busy_q <= 1'b0; end

```

```

else begin
    ist_q    <= (ist_q & ~w1c) | evt;
    tx_busy_q <= tx_busy;
end
assign irq = |(ist_q & ien_q);

// Read/write control registers
always_ff @(posedge pclk or negedge presetn)
    if (!presetn) begin
        ctrl_q <= 3'b011; baud_q <= 16'd26; ien_q <= '0;    // 115200 @ 50 MHz
    end else if (wr) begin
        case (paddr)
            A_CTRL: ctrl_q <= pwrdata[2:0];
            A_BAUD: baud_q <= pwrdata[15:0];
            A_IEN:  ien_q  <= pwrdata[3:0];
            default: ;
        endcase
    end

always_comb
    case (paddr)
        A_DATA: prdata = {24'h0, rxf_dout};
        A_STAT: prdata = {27'h0, tx_busy, txf_full, txf_empty, rxf_full, ~rxf_empty};
        A_CTRL: prdata = {29'h0, ctrl_q};
        A_BAUD: prdata = {16'h0, baud_q};
        A_IEN:  prdata = {28'h0, ien_q};
        A_ISTAT: prdata = {28'h0, ist_q};
        default: prdata = 32'h0;
    endcase
endmodule

```

uart_apb.sv (4/4): W1C interrupt status, control registers and read mux.

- **Set wins over clear:** `ist_q <= (ist_q & ~w1c) | evt`. If a new event arrives in the same cycle software clears the bit, the event survives. The opposite priority loses interrupts under load - one of the review findings of Chapter 27.
- **Level interrupt:** `irq` stays high until software clears the cause; a level is robust to an interrupt controller that is busy, whereas a pulse can be missed (Ch. 15).
- **TX_DONE** fires on the falling edge of `tx_busy` when the FIFO is empty - 'everything you gave me has left the pin', which is what a driver needs before it disables the transmitter or powers down.
- **Read side effect:** reading DATA pops the RX FIFO. Debuggers that read all registers for display will steal bytes - which is why many designs provide a side-effect-free debug alias, and why the register description must mark such fields (SystemRDL `rc1r/rset`, or a user property for read-to-pop).

28.8 A self-checking testbench

The testbench behaves as software would: it programs the UART through APB, writes bytes to DATA, polls STATUS and reads bytes back. The serial line is looped back outside the IP (and in test 3, inside it), so every byte crosses the whole TX path, the line, the synchronizer and the RX path. A queue holds the bytes in flight - a one-line reference model (Ch. 22).

```

`timescale 1ns/1ps
module tb_uart;
    logic          pclk = 0, presetn = 0;
    logic          psel = 0, penable = 0, pwrite = 0;
    logic [4:0]    paddr = '0;
    logic [31:0]   pwrdata = '0, prdata;
    logic          pready, pslverr, irq, txd, ext_lb = 1;
    wire           rxd = ext_lb ? txd : 1'b1;          // external loopback wire
    int            errors = 0, checked = 0;
    byte           exp_q[$];

    uart_apb dut (.*);
    always #5 pclk = ~pclk;                          // 100 MHz

```

tb_uart.sv (1/3): signals, the external loopback wire, DUT.

```

task automatic apb_write(input logic [4:0] a, input logic [31:0] d);
    @(posedge pclk); psel <= 1; penable <= 0; pwrite <= 1; paddr <= a; pwrdata <= d;
    @(posedge pclk); penable <= 1;
    @(posedge pclk); psel <= 0; penable <= 0;
endtask
task automatic apb_read(input logic [4:0] a, output logic [31:0] d);
    @(posedge pclk); psel <= 1; penable <= 0; pwrite <= 0; paddr <= a;
    @(posedge pclk); penable <= 1;
    @(negedge pclk); d = prdata;                      // sample before the edge
    @(posedge pclk); psel <= 0; penable <= 0;
endtask
// Send one byte: wait for TX FIFO space, write DATA, remember the byte
task automatic send(input byte b);
    logic [31:0] st;
    do apb_read(5'h04, st); while (st[3]);             // STATUS.TX_FULL
    apb_write(5'h00, {24'h0, b});
    exp_q.push_back(b);
endtask
// Drain: pop every received byte and compare against the model queue
task automatic drain(input int n);
    logic [31:0] st, d;
    repeat (n) begin
        do apb_read(5'h04, st); while (!st[0]);      // STATUS.RX_AVAIL
        apb_read(5'h00, d);
        checked++;
        if (d[7:0] !== exp_q[0]) begin
            errors++;
            $display(" MISMATCH: got %h expected %h", d[7:0], exp_q[0]);
        end
        void'(exp_q.pop_front());
    end
endtask
endtask

```

tb_uart.sv (2/3): APB bus-functional tasks and the checking tasks.

PITFALL - TWO THREADS ON ONE BUS

A first version of this testbench forked `send` and `drain` as two parallel threads. Both called `apb_read`, both drove `psel`, `paddr` and `penable` in the same cycles, and the result was two corrupted bytes and a hung test. A bus-functional model is a **single master**: either serialize access (one thread, as below), or guard it with a semaphore - which is what a UVM sequencer does for you (Ch. 24).

```

initial begin
    logic [31:0] st;
    repeat (3) @(posedge pclk);
    presetn = 1;
    apb_write(5'h0C, 32'd3);                          // BAUD_DIV: tick16 every 4 clk

```

```

apb_write(5'h10, 32'h1); // INT_EN: RX_DONE
// Test 1: directed bytes, one at a time; check irq and W1C
send(8'h55);
wait (irq);
$display("[%0d ns] irq after first byte, INT_STATUS read next", $time);
apb_read(5'h14, st);
$display("  INT_STATUS = %b", st[3:0]);
apb_write(5'h14, 32'h1); // W1C the RX_DONE bit
@(posedge pclk);
$display("  after W1C irq = %0b", irq);
drain(1);
// Test 2: 40 random bytes, streamed through both 8-deep FIFOs
apb_write(5'h10, 32'h0);
for (int i = 0; i < 40; i++) begin // one bus master thread:
    send($urandom); // interleave TX writes
    apb_read(5'h04, st); // with opportunistic
    if (st[0]) drain(1); // RX reads
end
drain(exp_q.size());
$display("[%0d ns] random stream done, %0d bytes checked so far", $time, checked);
// Test 3: internal loopback, external wire held idle
ext_lb = 0;
apb_write(5'h08, 32'b111);
repeat (4) send($urandom);
drain(4);
// Test 4: overrun - send 10 bytes without reading; FIFO keeps the first 8
ext_lb = 1; apb_write(5'h08, 32'b011);
repeat (10) send($urandom);
do apb_read(5'h04, st); while (st[4] || !st[2]); // TX_BUSY or !TX_EMPTY
repeat (100) @(posedge pclk);
apb_read(5'h14, st);
$display("[%0d ns] after 10 unread bytes: STATUS.RX_FULL=%0b OVERRUN=%0b",
    $time, dut.rxf_full, st[2]);
drain(8);
exp_q.delete();
$display("RESULT: %0d bytes checked, %0d errors -> %s", checked, errors,
    errors == 0 ? "PASS" : "FAIL");
$finish;
end
initial begin #5ms $display("TIMEOUT"); $finish; end
endmodule

```

tb_uart.sv (3/3): four tests - interrupt and W1C, a 40-byte random stream, internal loopback, and overrun.

```

iverilog -g2012 -Wall -o sim tb_uart.sv uart_fifo.sv uart_tx.sv \
    uart_rx.sv uart_apb.sv && vvp -n sim

```

```

[6275 ns] irq after first byte, INT_STATUS read next
  INT_STATUS = 0001
  after W1C irq = 0
[262335 ns] random stream done, 41 bytes checked so far
[353185 ns] after 10 unread bytes: STATUS.RX_FULL=1 OVERRUN=1
RESULT: 53 bytes checked, 0 errors -> PASS
tb_uart.sv:90: $finish called at 353665000 (1ps)

```

Real Icarus Verilog 12 output. With BAUD_DIV=3 a bit lasts 64 cycles (640 ns), so one frame takes 6.4 us - the first interrupt at 6275 ns is one frame after the write, as expected.

Reading the result critically: the first byte raised `irq` and the W1C write dropped it; 40 random bytes streamed through both FIFOs while the testbench interleaved writes and reads; internal loopback worked with the external line held idle; and ten unread bytes filled the 8-entry RX FIFO and set `RX_OVERFLOW`, after which the first eight bytes were still intact. What it does **not** prove: framing-error detection, tolerance to baud mismatch, glitch rejection, or reset in the middle of a frame. Those are the exercises - and exactly what a verification plan

review (Ch. 27) would demand before sign-off.

28.9 What it costs

Block	Flops (approx.)	Notes
TX FIFO + RX FIFO	$2 \times (64 + 8) = 144$	Storage dominates; at 16+ entries a latch array or small SRAM macro wins on area
uart_tx	~18	State, oversample and bit counters, shifter, txd
uart_rx	~29	Synchronizer, state, counters, shifter, output byte
Registers, baud counter, IRQ	~50	BAUD_DIV and its counter are 32 of them
Total	~240 flops	A few thousand gate equivalents - negligible next to a CPU, which is why SoCs carry several UARTs

Area estimate. Critical path: the 16-bit compare plus increment in the baud generator - trivially fast.

28.10 From project to production

Feature	Why production UARTs have it
Parity (even/odd/stick), 5-8 data bits, 1/1.5/2 stop bits	Compatibility with legacy equipment and the 16550 register model that operating-system drivers expect
Break detection and generation	A line held low for a whole frame is a BREAK, used as an attention signal (e.g. LIN, debug consoles); must be distinguished from a framing error
Majority-vote sampling (3 samples around mid-bit)	Rejects single-sample noise on long cables
RX timeout interrupt	Tells software that fewer than a threshold of bytes are waiting and the line has gone quiet
FIFO threshold interrupts	Interrupt at 1/4, 1/2, 3/4 full: fewer interrupts per byte at high baud rates
Fractional baud divider	Reduces baud error at high rates and odd clock frequencies
RTS/CTS hardware flow control	Prevents overrun when the far end cannot keep up
DMA request/acknowledge handshake	Lets a DMA engine move bytes without CPU interrupts per byte (Ch. 16)
Separate UART core clock with CDC	Baud accuracy independent of bus-clock scaling (DVFS); requires async FIFOs (Ch. 11)
PSLVERR on bad offsets, PPROT checks, PSTRB	Security and robust software error handling
Clock gating and a wake-on-start-bit feature	Low power while idle (Ch. 19)
UVM environment: APB agent + UART agent + RAL + scoreboard, SVA, coverage, formal on the FIFO	The verification that turns a demo into IP someone else will tape out (Ch. 23-25)

28.11 Summary

- A UART exercises FSMs, datapaths, FIFOs, CDC, bus slaves and interrupts at once; building one end to end is the best first project.
- Generate a 16x clock **enable**, not a clock: one domain, easy STA, one synchronizer.
- 16x oversampling finds the middle of each bit and tolerates about 4-5% total clock mismatch.
- Synchronize the asynchronous input into exactly one flop chain before any logic uses it.
- W1C interrupt status with set-wins-over-clear, a level **irq**, and documented read side effects are what make a peripheral usable by software.
- A self-checking testbench that acts like software, with a queue as reference model, verified 53 bytes across four scenarios.

28.12 Exercises

- 1 Add a test that drives `rx_d` directly from the testbench with a frame whose stop bit is 0, and check that `FRAME_ERR` is set and no byte is pushed.
- 2 Drive frames from the testbench at a baud rate 3% fast and 3% slow; then find experimentally the mismatch at which bytes start failing and compare it with the 4.6% estimate.
- 3 Add even parity with a `CTRL` enable bit and a `PARITY_ERR` interrupt. Update the register table, RTL and testbench together.
- 4 Replace mid-bit sampling with a 3-sample majority vote (ticks 7, 8 and 9). How much logic does it cost?
- 5 Write SVA properties (Ch. 23) for: `tx_d` is high whenever the TX FSM is IDLE; the RX FIFO never pushes when full; `irq` implies some enabled status bit is set.
- 6 Sketch the UVM environment for this UART: agents, sequences, register model and scoreboard (Ch. 24).

29. Project: An AXI-Stream MAC Accelerator for On-Device ML

This book lives in a repository about **on-device training, quantization and pruning** - the techniques that shrink neural networks until they fit on phones, microcontrollers and sensors. Those techniques only pay off if the hardware exploits them: an int8 model on a float32 datapath saves memory but not energy, and a 75%-pruned model on a dense engine saves nothing at all. This chapter closes the loop by building the hardware side: a pipelined int8 dot-product engine with requantization and zero-skipping for pruned weights, fed and drained by AXI4-Stream, configured through a register interface, and verified bit-exactly against a golden model.

29.1 Why int8, and what the hardware must compute

Integer-only inference (the scheme popularized by Jacob et al., CVPR 2018, and used by TensorFlow Lite and most NPUs) represents each real value r by an 8-bit integer q with a scale S and a zero point Z . With symmetric weights ($Z_w = 0$), a layer output becomes an integer dot product followed by a single rescale:

$$r = S \times (q - Z) \quad (\text{affine quantization})$$

$$y_q = Z_y + M \times (\text{bias}_q + \text{SUM}_k a_q[k] \times w_q[k]) \quad M = S_a \times S_w / S_y$$

M is a real number in $(0, 1)$: hardware stores $M \sim \text{mult} \times 2^{\text{-shift}}$
 int8 x int8 -> int16 products, summed in an int32 accumulator

The activation zero point Z_a is folded into the bias offline (its contribution is $Z_a \times \text{SUM } w$, a per-channel constant), so the engine sees plain signed products.

Operation (45 nm, Horowitz ISSCC 2014)	Energy	Relative to 8-bit add
8-bit integer add	0.03 pJ	1x
32-bit integer add	0.1 pJ	3x
8-bit integer multiply	0.2 pJ	7x
32-bit float multiply	3.7 pJ	~120x
32-bit read from on-chip SRAM (cache)	5 pJ	~170x
32-bit read from DRAM	640 pJ	~21000x

Widely quoted energy figures. An int8 multiply is ~18x cheaper than a float32 multiply, and **every** arithmetic operation is cheap next to moving data.

THE TWO LESSONS OF THE ENERGY TABLE

First, quantization pays twice: int8 arithmetic is an order of magnitude cheaper, and int8 data is 4x smaller to move. Second, data movement dominates - which is why accelerator design is mostly the art of **reusing** each operand fetched from memory many times (29.8), and why skipping a pruned weight saves more if you never fetch it at all.

29.2 Specification

Interface	Signals	Protocol
Input stream	<code>s_tvalid</code> , <code>s_tready</code> , <code>s_tdata[16L-1:0]</code> , <code>s_tlast</code>	AXI4-Stream. One beat = L activations (low half) and L weights (high half), int8 each. <code>s_tlast</code> marks the last beat of a vector
Output stream	<code>m_tvalid</code> , <code>m_tready</code> , <code>m_tdata[7:0]</code>	AXI4-Stream. One int8 result per input vector

Interface	Signals	Protocol
CSR port	<code>csr_we</code> , <code>csr_addr[4:0]</code> , <code>csr_wdata</code> , <code>csr_rdata</code>	Single-cycle write, combinational read; an APB or AXI-Lite bridge (Ch. 13) adapts it to the SoC bus

Offset	Name	Access	Description
0x00	CTRL	RW, reset 1	[0] ZSKIP_EN: skip lanes whose weight is 0
0x04	MULT	RW, reset 1	[15:0] requant multiplier (unsigned)
0x08	SHIFT	RW, reset 0	[4:0] requant right shift (rounding)
0x0C	ZP	RW, reset 0	[7:0] signed output zero point
0x10	BIAS	RW, reset 0	[31:0] signed bias added to the accumulator
0x14	SKIP_CNT	RO, write clears	Number of skipped MACs
0x18	MAC_CNT	RO, write clears	Number of MAC slots processed (L per accepted beat)

Accelerator CSRs. Configuration registers must only change while the engine is idle.

```

out = sat8( ((acc + BIAS) x MULT + 2^(SHIFT-1)) >>> SHIFT + ZP )
sat8(v) = min(127, max(-128, v))    (no rounding term when SHIFT = 0)

```

The requantization the RTL and the golden model both implement (round half toward +infinity).

29.3 Micro-architecture

s_tdata	--->	nz = (w != 0) !ZSKIP	-----+	
			v	
	S1: operand regs	S2: multiply + sum	S3: requant	S4: round,
	(load only if nz)	+ int32 accumulate	multiply	shift, sat
	+-----+	+-----+	+-----+	+-----+
+-->	a1_q, w1_q	-----> SUM nz?a*w : 0	--> (acc2 +	--> +2^(s-1)
	x L lanes	+ acc_q	BIAS) x	>>> SHIFT
	nz1_q, v1_q	on last: acc2_q,	MULT	+ ZP, sat
	+-----+	acc_q <= 0	prod3_q	m_tdata
		+-----+	+-----+	+-----+
en = !m_tvalid m_tready	---	enables every stage;	s_tready = en	

Four register stages. The result is valid three clock edges after the edge that accepts the last beat; throughput is one beat (L MACs) per cycle.

The **number formats** drive the widths. An int8 x int8 product lies in [-16256, 16384] and needs 16 signed bits (the extreme +16384 = (-128) x (-128) is why an 8x8 signed multiplier's output cannot be 15 bits). A sum of 4 products needs 18 bits. The int32 accumulator can then absorb at least $2^{31} / 2^{14} = 131072$ worst-case products - far longer than any realistic layer - which is why int32 is the universal accumulator width for int8 inference. The requantization multiply takes a 33-bit biased sum times a 17-bit (zero-extended) multiplier: a 50-bit product.

29.4 The RTL, stage by stage

```

// int8 x int8 dot-product engine with requantization and zero-skipping.
// s_tdata = {w[L-1],...,w[0], a[L-1],...,a[0]}; s_tlast ends a vector.
module mac_accel #(parameter int L = 4) (
    input  logic          clk, rst_n,
    input  logic          csr_we,           // simple CSR port
    input  logic [4:0]    csr_addr,
    input  logic [31:0]   csr_wdata,
    output logic [31:0]   csr_rdata,
    input  logic          s_tvalid,        // AXI4-Stream slave
    output logic          s_tready,
    input  logic [16*L-1:0] s_tdata,

```

```

input  logic      s_tlast,
output logic      m_tvalid,          // AXI4-Stream master
input  logic      m_tready,
output logic [7:0] m_tdata
);
localparam logic [4:0] A_CTRL = 5'h00, A_MULT = 5'h04, A_SHIFT = 5'h08,
                      A_ZP    = 5'h0C, A_BIAS = 5'h10, A_SKIP  = 5'h14,
                      A_MACS  = 5'h18;

logic      zskip_q;          // CTRL[0]: zero-skip enable
logic [15:0] mult_q;         // requant multiplier (Q0.16)
logic [4:0] shift_q;         // requant right shift
logic signed [7:0] zp_q;      // output zero point
logic signed [31:0] bias_q;
logic [31:0] skip_cnt_q, mac_cnt_q;

```

mac_accel.sv (1/7): ports and CSR state. **L** is the number of lanes.

```

// Global-stall pipeline: every stage advances unless the output is blocked
wire en  = !m_tvalid || m_tready;
assign s_tready = en;
wire take = s_tvalid && s_tready;

```

mac_accel.sv (2/7): flow control.

This is the simplest correct valid/ready pipeline: **one global enable**. Every stage advances unless the output register holds a result the consumer has not taken. It is correct by construction - no stage can overwrite data that has not moved on - but it has two costs that a reviewer should raise: **s_tready** depends combinatorially on **m_tready** (a path through the whole engine, and a potential loop if the consumer's ready depends on our valid), and **en** fans out to every flop. Chapter 10's skid buffer on the output removes both; exercise 3 asks you to add it.

```

// ----- S1: operand registers with operand isolation -----
logic [L-1:0]      nz_in, nz1_q;          // lane has a non-zero weight
logic signed [7:0] a1_q [L], w1_q [L];
logic              v1_q, last1_q;
int                nskip;
always_comb begin
  nskip = 0;
  for (int i = 0; i < L; i++) begin
    nz_in[i] = !zskip_q || (s_tdata[8*(L+i) +: 8] != 8'h00);
    nskip    = nskip + {31'b0, !nz_in[i]};
  end
end
always_ff @(posedge clk or negedge rst_n)
  if (!rst_n) begin v1_q <= 1'b0; last1_q <= 1'b0; nz1_q <= '0; end
  else if (en) begin
    v1_q <= take; last1_q <= s_tlast;
    if (take) nz1_q <= nz_in;
  end
always_ff @(posedge clk)                // skipped lanes hold their
  for (int i = 0; i < L; i++)            // operands: the multiplier
    if (take && nz_in[i]) begin           // inputs do not toggle
      a1_q[i] <= s_tdata[8*i      +: 8];
      w1_q[i] <= s_tdata[8*(L+i) +: 8];
    end
end

```

mac_accel.sv (3/7): stage 1 - operand registers with operand isolation.

WHAT ZERO-SKIPPING SAVES IN THIS DESIGN - AND WHAT IT DOES NOT

When ZSKIP_EN is set and a lane's weight is zero, that lane's operand registers are simply **not loaded**. The multiplier inputs keep their old values, so the multiplier's gates do not toggle and consume almost no dynamic power; **nz1_q** forces the lane's contribution to zero. This is **operand isolation**, a classic low-power technique (Ch. 19). It saves energy but not **time**: the stream is dense, so a beat with four zero weights still takes a cycle. Throughput gains from sparsity require the zeros to be removed from the stream itself (29.7).

```
// ----- S2: multiply, adder tree, int32 accumulator -----
logic signed [31:0] sum2, acc_q, acc2_q;
logic
    v2_q;
always_comb begin
    sum2 = '0;
    for (int i = 0; i < L; i++)
        if (nz1_q[i]) sum2 = sum2 + a1_q[i] * w1_q[i];    // 8x8 -> 16 bit, signed
end
always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) begin acc_q <= '0; acc2_q <= '0; v2_q <= 1'b0; end
    else if (en) begin
        v2_q <= v1_q && last1_q;
        if (v1_q && last1_q) begin acc2_q <= acc_q + sum2; acc_q <= '0; end
        else if (v1_q) acc_q <= acc_q + sum2;
    end
end
```

mac_accel.sv (4/7): stage 2 - multipliers, adder tree and accumulator.

The **for** loop describes L multipliers and an adder chain; synthesis rebalances the chain into a tree, and for large L a designer would write an explicit tree or use a carry-save compressor followed by one carry-propagate adder (Ch. 8). On the last beat the stage writes the **final** sum to **acc2_q** and clears **acc_q** in the same cycle, so the next vector can start on the very next beat - no dead cycle between vectors.

```
// ----- S3: requant multiply (acc + bias) * M -----
logic signed [49:0] prod3_q;
logic
    v3_q;
wire signed [32:0] biased = 33'(acc2_q) + 33'(bias_q);
always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) begin prod3_q <= '0; v3_q <= 1'b0; end
    else if (en) begin
        v3_q <= v2_q;
        if (v2_q) prod3_q <= biased * $signed({1'b0, mult_q});
    end
end
```

mac_accel.sv (5/7): stage 3 - the requantization multiply gets its own stage.

```
// ----- S4: round, shift, add zero point, saturate -----
logic signed [49:0] rnd, shd;
always_comb begin
    rnd = (shift_q == 5'd0) ? prod3_q : prod3_q + (50'sd1 <<< (shift_q - 5'd1));
    shd = (rnd >>> shift_q) + 50'(zp_q);
end
always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) begin m_tvalid <= 1'b0; m_tdata <= '0; end
    else if (en) begin
        m_tvalid <= v3_q;
        if (v3_q) m_tdata <= (shd > 50'sd127) ? 8'h7F :
            (shd < -50'sd128) ? 8'h80 : shd[7:0];
    end
end
```

mac_accel.sv (6/7): stage 4 - rounding shift, zero point, saturation.

PITFALL - SIGNEDNESS IN THE REQUANT STAGE

Three details here each produce wrong answers only for negative values: `mult_q` is unsigned, so it is zero-extended with `$signed({1'b0, mult_q})` before multiplying a signed value - otherwise the whole product becomes unsigned; the shift must be `>>>` on a **signed** operand to be arithmetic; and the widening casts `33'(acc2_q)` preserve the sign because the operand is declared signed. Each of these is a finding in the Chapter 27 table ('signed/unsigned mix'), and each is caught only by tests with negative accumulators.

```
// ----- CSRs and performance counters -----
always_ff @(posedge clk or negedge rst_n)
  if (!rst_n) begin
    zskip_q <= 1'b1; mult_q <= 16'd1; shift_q <= '0; zp_q <= '0; bias_q <= '0;
    skip_cnt_q <= '0; mac_cnt_q <= '0;
  end else begin
    if (take) begin
      mac_cnt_q <= mac_cnt_q + L;
      skip_cnt_q <= skip_cnt_q + nskip;
    end
    if (csr_we)
      case (csr_addr)
        A_CTRL: zskip_q <= csr_wdata[0];
        A_MULT: mult_q <= csr_wdata[15:0];
        A_SHIFT: shift_q <= csr_wdata[4:0];
        A_ZP: zp_q <= csr_wdata[7:0];
        A_BIAS: bias_q <= csr_wdata;
        A_SKIP: skip_cnt_q <= '0; // write = clear (wins)
        A_MACS: mac_cnt_q <= '0;
        default: ;
      endcase
    end
  end
always_comb
  case (csr_addr)
    A_CTRL: csr_rdata = {31'b0, zskip_q};
    A_MULT: csr_rdata = {16'b0, mult_q};
    A_SHIFT: csr_rdata = {27'b0, shift_q};
    A_ZP: csr_rdata = 32'(zp_q); // sign-extends
    A_BIAS: csr_rdata = bias_q;
    A_SKIP: csr_rdata = skip_cnt_q;
    A_MACS: csr_rdata = mac_cnt_q;
    default: csr_rdata = '0;
  endcase
endmodule
```

mac_accel.sv (7/7): configuration registers and performance counters.

The two counters are what make the zero-skipping measurable: software reads SKIP_CNT / MAC_CNT after a layer and learns the effective sparsity the hardware exploited - the number a pruning algorithm designer wants to see. Writing a counter clears it, and the clear deliberately wins over a simultaneous increment.

29.5 Verification against a golden model

An arithmetic engine is verified by comparing it, bit for bit, with a **reference model** of the same arithmetic written in the testbench language (Ch. 22). Here the model is ten lines of 64-bit integer arithmetic. Stimulus is constrained-random: vectors of 1-16 beats, random int8 activations, weights zeroed with a chosen probability to mimic pruning, random idle gaps on the input and random back-pressure on the output, so the global-stall logic is exercised in every cycle combination.

```
`timescale 1ns/1ps
module tb_mac;
  localparam int L = 4;
  logic clk = 0, rst_n = 0;
```

```

logic csr_we = 0; logic [4:0] csr_addr = '0; logic [31:0] csr_wdata = '0, csr_rdata;
logic s_tvalid = 0, s_tready, s_tlast = 0; logic [16*L-1:0] s_tdata = '0;
logic m_tvalid, m_tready = 0; logic [7:0] m_tdata;
mac_accel #(.L(L)) dut (.*);
always #5 clk = ~clk;

int mult, shift, zp, bias, ready_pct;          // current configuration
byte exp_q[$];
int errors = 0, outs = 0, sats = 0;
longint exp_skip;

// Golden model: exactly the arithmetic the spec defines, in 64-bit integers
function automatic byte requant(longint acc);
    longint v = (acc + bias) * mult;
    if (shift > 0) v = v + (longint'(1) <<< (shift - 1)); // round half up
    v = (v >>> shift) + zp;
    if (v > 127 || v < -128) sats++;
    return (v > 127) ? 8'sd127 : (v < -128) ? -8'sd128 : byte'(v);
endfunction

```

tb_mac.sv (1/3): the golden requantization model.

```

task automatic csr_write(input logic [4:0] a, input logic [31:0] d);
    @(posedge clk); csr_we <= 1; csr_addr <= a; csr_wdata <= d;
    @(posedge clk); csr_we <= 0;
endtask

task automatic csr_read(input logic [4:0] a, output logic [31:0] d);
    @(posedge clk); csr_addr <= a; @(negedge clk); d = csr_rdata;
endtask

// Drive one vector of n beats; weights are zero with probability sp_pct
task automatic send_vec(input int n, input int sp_pct, input bit zskip);
    longint acc = 0;
    for (int b = 0; b < n; b++) begin
        logic [16*L-1:0] d;
        for (int i = 0; i < L; i++) begin
            byte a = $urandom, w = ($urandom_range(0, 99) < sp_pct) ? 0 : $urandom;
            d[8*i+:8] = a; d[8*(L+i):8] = w;
            acc += a * w;
            if (zskip && w == 0) exp_skip++;
        end
        while ($urandom_range(0, 99) < 30) @(posedge clk); // random idle gaps
        s_tvalid <= 1; s_tdata <= d; s_tlast <= (b == n - 1);
        do @(posedge clk); while (!s_tready); // hold until handshake
        s_tvalid <= 0;
    end
    exp_q.push_back(requant(acc));
endtask

```

tb_mac.sv (2/3): CSR tasks and the AXI-Stream driver. The driver holds `s_tvalid` and data stable until the handshake, as AXI4-Stream requires.

```

always @(posedge clk) begin // random backpressure
    m_tready <= ($urandom_range(0, 99) < ready_pct);
    if (m_tvalid && m_tready) begin // scoreboard
        outs++;
        if (m_tdata != exp_q[0]) begin
            errors++;
            if (errors < 5) $display(" MISMATCH got %0d exp %0d", $signed(m_tdata), exp_q[0]);
        end
        void'(exp_q.pop_front());
    end
end
end

```



```

task automatic run(string name, bit zs, int m, int s, int z, int bi, int sp, int nv);
    logic [31:0] skips, macs; int t0 = $time, e0 = errors, o0 = outs, s0 = sats;
    mult = m; shift = s; zp = z; bias = bi; exp_skip = 0;
    csr_write(5'h00, zs); csr_write(5'h04, m); csr_write(5'h08, s);
    csr_write(5'h0C, z); csr_write(5'h10, bi);
    csr_write(5'h14, 0); csr_write(5'h18, 0); // clear counters
    repeat (nv) send_vec($urandom_range(1, 16), sp, zs);
    while (exp_q.size() != 0) @(posedge clk); // drain the pipeline
    csr_read(5'h14, skips); csr_read(5'h18, macs);
    $display("%-9s vec=%0d out=%0d err=%0d sat=%0d MACs=%0d skipped=%0d (exp %0d) %0d ns",
            name, nv, outs - o0, errors - e0, sats - s0, macs, skips, exp_skip,
            ($time - t0));
    if (skips != exp_skip) errors++;
endtask

initial begin
    repeat (2) @(posedge clk); rst_n = 1;
    ready_pct = 100;
    run("dense", 0, 16384, 24, 0, 0, 0, 200); // scale 1/1024
    ready_pct = 60;
    run("pruned", 1, 16384, 24, 3, -500, 75, 200); // 75% zero weights
    run("saturate", 1, 65535, 23, -5, 1000, 50, 100);
    $display("RESULT: %0d vectors checked, %0d errors -> %s", outs, errors,
            errors == 0 ? "PASS" : "FAIL");
    $finish;
end
initial begin #10ms $display("TIMEOUT"); $finish; end
endmodule

```

tb_mac.sv (3/3): back-pressure, the scoreboard, and three test phases.

```

dense      vec=200 out=200 err=0 sat=0 MACs=6196 skipped=0 (exp 0) 22295 ns
pruned     vec=200 out=200 err=0 sat=0 MACs=6436 skipped=4887 (exp 4887) 24300 ns
saturate   vec=100 out=100 err=0 sat=42 MACs=3328 skipped=1647 (exp 1647) 12080 ns
RESULT: 500 vectors checked, 0 errors -> PASS
tb_mac.sv:88: $finish called at 586900000 (1ps)

```

Real output of `iverilog -g2012 -Wall -o sim tb_mac.sv mac_accel.sv && vvp -n sim`.

The three phases check different things. **dense** (zero-skipping off, no back-pressure) checks the arithmetic; SKIP_CNT must stay 0. **pruned** (75% zero weights, 40% back-pressure, non-zero bias and zero point) checks that skipping never changes a result and that the counter is exact: 4887 of 6436 MAC slots (76%) were skipped - the hardware saw the sparsity the pruning created. **saturate** raises the scale so that 42 of 100 outputs clip, exercising both saturation limits.

Does the testbench actually catch bugs?

A testbench that has never failed has not been shown to work. A quick check is **mutation**: inject a plausible bug and confirm the scoreboard fires. Here the rounding term was deleted (`rnd = prod3_q`), i.e. truncation instead of round-half-up - exactly the mismatch that occurs when RTL and a framework's reference disagree on rounding:

```

MISMATCH got 50 exp 51
MISMATCH got -25 exp -24
MISMATCH got -42 exp -41
MISMATCH got -15 exp -14
dense      vec=200 out=200 err=105 sat=0 MACs=6196 skipped=0 (exp 0) 22295 ns
pruned     vec=200 out=200 err=106 sat=0 MACs=6436 skipped=4887 (exp 4887) 24300 ns
saturate   vec=100 out=100 err=31 sat=42 MACs=3328 skipped=1647 (exp 1647) 12080 ns
RESULT: 500 vectors checked, 242 errors -> FAIL
tb_mac.sv:88: $finish called at 586900000 (1ps)

```

Same testbench, mutated RTL. Off-by-one errors in about half the outputs.

BIT-EXACTNESS WITH THE ML FRAMEWORK

The golden model in a real project is not hand-written: it is the framework's own integer kernel (e.g. TFLite's reference int8 kernels), and the RTL must match it bit for bit. TFLite rounds with `SaturatingRoundingDoublingHighMul` and a rounding divide by a power of two (round half away from zero) using a 32-bit multiplier - not this chapter's 16-bit multiplier and round-half-up. Off-by-one differences like the ones above are invisible in accuracy metrics for one layer but compound across layers, and they are the most common accelerator bring-up bug. Agree on the rounding **in the spec**.

29.6 PPA of the engine

Aspect	Estimate / trade-off
Area	Dominated by the L 8x8 multipliers (a few hundred gates each), the requant 33x17 multiplier (shared by all lanes, used once per vector - a candidate for a slower multi-cycle or serial implementation) and ~250 flops at L = 4
Timing	Critical path candidates: 8x8 multiply + 4-input add + 32-bit accumulate in S2; the 33x17 multiply in S3. Splitting S2 (products registered, then accumulate) is the first retiming step
Throughput	L MACs per cycle; at 500 MHz and L = 4: 2 GMAC/s. Beats per vector set the overhead: none, thanks to back-to-back accumulation
Power	Multipliers and the accumulator toggle every beat; operand isolation removes multiplier toggling on pruned weights; clock-gating the pipeline when <code>s_tvalid</code> is low removes the rest (Ch. 19)
Memory bandwidth	Each beat consumes 2L bytes, and no operand is reused : a dot-product engine is bandwidth-bound. This is the limitation the systolic array removes

29.7 Exploiting sparsity for speed, not just power

Sparsity form	How hardware exploits it	Cost
Unstructured (any weight can be 0)	Compressed formats (bitmap, CSR/CSC indices); zeros never fetched; gather logic selects matching activations	Index storage and irregular access; load imbalance between lanes
Structured N:M (e.g. 2:4)	Store only 2 of every 4 weights plus 2-bit indices; a 4:2 mux picks activations - a 2x MAC reduction with a regular datapath (the scheme used by NVIDIA's sparse tensor cores)	Pruning must respect the pattern; small mux cost
Block / channel pruning	Remove whole rows, filters or channels: the layer is simply smaller	Coarser pruning, more accuracy loss per removed weight
Operand isolation (this chapter)	Gate the multiplier on zeros	Power only; no speedup

29.8 Scaling up: from a dot-product engine to a systolic array

29.10 Summary

- Integer-only inference reduces a layer to int8 x int8 products, an int32 accumulator and a single requantization (multiply, rounding shift, zero point, saturation).
- A 4-stage valid/ready pipeline with a global stall is simple and correct; a skid buffer removes its combinational ready path.
- Zero-skipping by operand isolation saves dynamic power on pruned weights; speedups need compressed or N:M-structured sparsity.
- Signedness and rounding are where arithmetic RTL goes wrong; a bit-exact golden model and negative-value tests catch them.
- A constrained-random testbench with back-pressure checked 500 vectors bit-exactly, and a mutation test proved it can fail.
- Dot-product engines are bandwidth-bound; systolic arrays add operand reuse, and DMA-based integration keeps the CPU out of the data path.

29.11 Exercises

- 1 Make `L` = 8 and 16 and rerun the testbench. What changes in the RTL, and which path becomes critical?
- 2 Change the requantization to TFLite's scheme: a 32-bit multiplier with a rounding doubling high multiply and round-half-away-from-zero shift. Update the golden model first and watch the RTL fail, then fix it.
- 3 Add a 2-entry skid buffer on the output so `s_tready` no longer depends combinationaly on `m_tready`. Prove it with the unchanged testbench and measure throughput under 60% back-pressure.
- 4 Implement 2:4 structured sparsity: each beat carries 2 weights and 2-bit indices per group of 4 activations. How many multipliers do you save?
- 5 Add an accumulator overflow sticky flag and a directed test that triggers it. Is overflow possible for vectors under 131072 MACs?
- 6 Write the Linux-style driver pseudo-code for one layer using the integration steps above, including cache maintenance.

30. The RTL/SoC Career Roadmap: Skills, Study Plan, Portfolio and Interviews

The first 29 chapters describe what an RTL engineer knows. This one describes how people actually get there: what each career stage expects, a study plan that turns this book into a year of deliberate practice, the portfolio projects that get a resume read, the tools worth learning, and how RTL interviews are really structured. Titles and timelines vary between companies and countries; the progression of **scope** - from a block, to a subsystem, to a chip, to a product line - is remarkably consistent.

30.1 The four stages

Stage	Typical scope	Core skills	You are ready to move on when...
Beginner (student, career changer)	Exercises and personal projects	Digital logic, Verilog/SV syntax and simulation semantics, FSMs, basic testbenches, Linux, git, one scripting language	You can write a FIFO, an FSM and a UART from a spec and verify them with a self-checking testbench
Junior RTL engineer (0-3 years)	Blocks and features inside an IP, under review	Coding guidelines, bus protocols, CDC basics, lint/CDC cleanup, reading STA reports, debugging failing regressions	Your blocks pass review with few findings and you close your own lint, CDC and timing issues
Senior RTL / IP owner (3-8 years)	A whole IP or subsystem, from micro-architecture to sign-off	Micro-architecture, PPA trade-offs, timing closure with PD, low power, DFT collaboration, mentoring, schedule estimation	You own an IP through tapeout and silicon bring-up, and others ask you to review their designs
Architect / principal (8+ years)	Chip or product-line architecture	Performance modeling, workload analysis, memory systems, interconnect and coherency, hardware/software co-design, roadmaps, cross-team negotiation	- (the scope keeps growing: platform, company technology strategy)

Years are indicative only; scope and independence define the stage, not time.

THE ONE SKILL THAT MATTERS AT EVERY STAGE

Debugging. A beginner debugs syntax, a junior debugs a failing test, a senior debugs a silicon failure from a few register dumps, and an architect debugs why a product misses its performance target. Every stage rewards the same habit: form a hypothesis, find the cheapest experiment that can falsify it, and never guess twice without new data (Ch. 26).

30.2 Skills by stage, in detail

Area	Beginner	Junior	Senior	Architect
RTL coding	Syntax, blocking vs. non-blocking	Guideline-clean, reusable blocks	Parameterized IP, PPA-aware structure	Reviews for architecture fit
Micro-architecture	Reads block diagrams	Implements a given uarch	Writes the uarch spec	Defines block partitioning
Buses and SoC	APB	AXI4-Lite/Stream, CSRs, interrupts	AXI4 full, interconnect, DMA	Coherency, NoC, QoS
Clocks and resets	Single clock	2-flop sync, async FIFO usage	CDC/RDC architecture and sign-off	Clock/power domain plan
Implementation	What synthesis is	Reads timing reports	Closes timing with PD, low power, DFT	PPA budgets per block

Area	Beginner	Junior	Senior	Architect
Verification	Directed self-checking TB	SVA, constrained random, coverage reading	Verification planning with DV	Performance and system validation
Software	Python, shell	Makefiles, regression scripts, Tcl	Firmware interface, drivers, bring-up	HW/SW partitioning

30.3 A 48-week study plan built on this book

The plan assumes about 10-12 hours per week alongside study or a job. Compressed to six months at 20+ hours a week it still works; the order matters more than the pace. Every block ends in a **deliverable** that goes into a public repository - by week 48 that repository is your portfolio.

Weeks	Topics	Chapters	Deliverable
1-3	Flow, digital logic refresher, Verilog/SV for design	1-3	Toolchain installed (Icarus/Verilator, GTKWave, Yosys); 20 small modules compiled
4-5	Simulation semantics, races	4	Write and fix three deliberate race conditions
6-9	Combinational, sequential, FSMs, datapath	5-8	ALU, counters, sequence detectors, a multiplier - each with a self-checking TB
10-12	Memories, FIFOs, pipelines, valid/ready	9-10	Sync FIFO + skid buffer, verified with random back-pressure
13-15	Clocks, CDC, resets	11-12	Async FIFO with Gray pointers; reset synchronizer
16-19	Buses, interconnect, CSRs, peripherals	13-16	APB and AXI4-Lite slaves; project: APB UART (Ch. 28)
20-23	Synthesis, STA, low power, DFT, PPA	17-21	Run Yosys + OpenSTA on your UART; fix a timing path; read a gate netlist
24-28	Self-checking TBs, SVA, coverage	22-23	Add assertions and coverage to the UART and FIFO; formal proof of the FIFO with SymbiYosys
29-32	UVM, static sign-off, debug, FPGA	24-26	A small UVM environment (EDA Playground or a commercial trial); UART on an FPGA board talking to a PC
33-40	Big project: a RISC-V core	3-10, 22-23	RV32I 5-stage pipeline with forwarding and hazards; passes riscv-tests
41-46	Big project: the ML accelerator	29, 10, 16	MAC engine + DMA + CPU on FPGA running a quantized layer
47-48	Interview preparation	27, 30, App. C	100 interview questions answered aloud; two mock interviews; portfolio polished

MAKE THE PLAN SURVIVE REAL LIFE

Track it in the same repository (a checklist in the README). Skip a week, not a topic. When a chapter feels easy, do its exercises anyway - interviews probe exactly the corner cases the exercises contain.

30.4 Portfolio projects that get attention

Project	What it demonstrates	Level
APB/AXI-Lite UART, SPI or I2C controller with a real register map	Spec-to-silicon discipline: CSRs, interrupts, CDC of an async pin, self-checking TB	Beginner-junior
Parameterized async FIFO with formal proofs	CDC mastery; SymbiYosys or a commercial formal tool	Junior

Project	What it demonstrates	Level
RISC-V RV32I core (single-cycle, then 5-stage pipelined)	Datapath, hazards, forwarding; passes riscv-tests; optionally riscv-formal	Junior-senior
AXI4 DMA engine or AXI crossbar	Full AXI (bursts, IDs, outstanding transactions) - the protocol most interviews probe	Senior
ML accelerator (Ch. 29) extended to a systolic tile on FPGA	PPA thinking, HW/SW co-design, the topic of this repository	Senior
Contribution to OpenTitan , CVA6 , Ibex , PULP or OpenHW cores	Working in an industrial-quality codebase with strict review, lint and DV - the closest thing to a job	Junior-senior
A Tiny Tapeout design through OpenLane to real silicon	The full RTL-to-GDS flow on an open PDK (SkyWater SKY130, GF180MCU or IHP SG13G2) and a chip you can hold	Any

Open-silicon shuttles have changed quickly: efabless, which ran the Google-sponsored OpenMPW and chipignite shuttles, ceased operations in 2025, and Tiny Tapeout has since used other shuttle providers. Check the project's site for the current PDK and deadlines; the skills - OpenLane/OpenROAD, DRC/LVS, timing sign-off - transfer regardless.

- **Present each project like IP:** a README with a block diagram, the register map, a verification summary (what is covered and what is not), synthesis area and Fmax numbers, and one command that runs the tests.
- **CI on every push** (GitHub Actions with Verilator/Icarus and Verible lint) shows discipline in one green badge.
- **Depth beats breadth:** one well-verified, documented core with formal proofs is worth more than ten half-finished repositories.
- **Be ready to defend every line** - interviewers pick a random module and ask why it is written that way.

30.5 Tools to learn

Category	Free / open source	Commercial (industry standard)
Simulation	Icarus Verilog, Verilator, GHDL (VHDL)	Synopsys VCS, Cadence Xcelium, Siemens Questa
Waveforms and debug	GTKWave, Surfer	Synopsys Verdi, Cadence SimVision / Indago
Lint	Verilator --lint-only , Verible	Synopsys SpyGlass, Real Intent Ascent, Cadence HAL / Jasper Superlint
CDC / RDC	(limited open options)	SpyGlass CDC, Questa CDC, Real Intent Meridian, Jasper CDC
Formal	SymbiYosys (Yosys + SMT solvers)	Cadence Jasper, Synopsys VC Formal, Siemens Questa Formal
Verification frameworks	cocotb (Python), SV UVM on EDA Playground	UVM with commercial simulators and VIP
Synthesis	Yosys	Synopsys Design Compiler / Fusion Compiler, Cadence Genus
STA	OpenSTA	Synopsys PrimeTime, Cadence Tempus
Physical design	OpenROAD / OpenLane	Synopsys ICC2 / Fusion Compiler, Cadence Innovus
FPGA	Yosys + nextpnr; free editions of Vivado and Quartus	Full Vivado / Quartus; emulation: Palladium, ZeBu, Veloce
Glue	Linux shell, Python, Tcl, Make, git	(same - these are universal)

OPEN-SOURCE TOOLS ARE NOT A LESSER PATH

The concepts - elaboration, scheduling, synthesis, STA, formal - are the same in every tool. Engineers who learned on Yosys and OpenSTA switch to commercial tools in weeks; what takes years is the judgment the tools cannot give you. Tcl (for every commercial tool) and Python (for everything around them) are the two languages to be fluent in.

30.6 How RTL interviews are structured

Round	What happens	What they are really testing
Recruiter screen	Background, interests, logistics	Communication; match to the team
Technical phone screen	Digital logic, Verilog semantics, a small design on a shared editor	Fundamentals: blocking vs. non-blocking, latches, setup/hold, FSM encoding
RTL coding	Write synthesizable RTL for a FIFO, arbiter, edge detector, clock divider, sequence detector or credit counter	Clean code, correct reset, no latches, handling of corner cases (full and empty at once, back-to-back requests)
CDC and clocks	Synchronize a pulse, a bus, a reset; design an async FIFO; explain metastability and MTBF	Whether you know why each technique works (Ch. 11-12)
Timing / STA	Setup/hold equations, fix a violation, max frequency of a circuit, effect of skew	Physical intuition about paths (Ch. 18)
Architecture / micro-architecture	Design a cache, a DMA, a pipelined unit; discuss trade-offs	Structured thinking, PPA awareness, asking clarifying questions
Puzzles	Divisible-by-3 detector, divide-by-3 clock with 50% duty, count ones, gray code, one-hot checks	Reasoning under pressure; state-machine thinking
Project deep-dive	Walk through your resume project in detail	Ownership: can you defend decisions and explain bugs you found?
Behavioral	Conflicts, failures, deadlines	Team fit, honesty, learning from mistakes

```

setup:  T_clk >= t_clk-q + t_logic,max + t_setup - t_skew

hold:   t_clk-q + t_logic,min >= t_hold + t_skew

(t_skew = capture clock arrival - launch clock arrival)

f_max = 1 / (t_clk-q + t_logic,max + t_setup - t_skew)

```

The two inequalities every timing question reduces to. Note that positive skew helps setup and hurts hold, and a hold violation cannot be fixed by slowing the clock.

A classic puzzle, solved as an interviewer wants to see it

'Bits of a binary number arrive serially, MSB first. Assert an output whenever the number received so far is divisible by 3.' The key insight is that you never need the number - only its **remainder**. Appending a bit *b* maps value *v* to $2v + b$, so the remainder *r* becomes $(2r + b) \bmod 3$: a 3-state FSM.

```

`timescale 1ns/1ps
// Serial input, MSB first: o_div3 is 1 when the bits seen so far form a
// number divisible by 3. State = remainder; next = (2*rem + bit) mod 3.
module div3_detect (
    input  logic i_clk, i_rst_n, i_valid, i_bit,
    output logic o_div3
);
    logic [1:0] rem_q, rem_d;
    always_comb
        case ({rem_q, i_bit})
            3'b00_0: rem_d = 2'd0;   3'b00_1: rem_d = 2'd1;
            3'b01_0: rem_d = 2'd2;   3'b01_1: rem_d = 2'd0;
        endcase
    rem_q <- rem_d;
endmodule

```



```

        3'b10_0: rem_d = 2'd1;    3'b10_1: rem_d = 2'd2;
        default: rem_d = 2'd0;    // 2'b11 unreachable: recover
    endcase
    always_ff @(posedge i_clk or negedge i_rst_n)
        if (!i_rst_n) rem_q <= 2'd0;
        else if (i_valid) rem_q <= rem_d;
    assign o_div3 = (rem_q == 2'd0);
endmodule

```

div3_detect.sv - the remainder FSM, with a safe default for the unreachable encoding.

```

module tb_div3;
    logic clk = 0, rst_n = 0, valid = 0, b = 0, div3;
    int errors = 0;
    div3_detect dut (.i_clk(clk), .i_rst_n(rst_n), .i_valid(valid), .i_bit(b),
                    .o_div3(div3));
    always #5 clk = ~clk;
    initial begin
        logic [7:0] n;
        #12 rst_n = 1;
        for (int k = 0; k < 256; k++) begin
            n = k[7:0];
            rst_n = 0; #1 rst_n = 1;                // restart for each number
            for (int i = 7; i >= 0; i--) begin
                @(negedge clk) valid = 1; b = n[i];
            end
            @(negedge clk) valid = 0;
            if (div3 !== (n % 3 == 0)) errors++;
            if (k == 9 || k == 10 || k == 255) $display("n=%3d bits=%b div3=%0b", n, n, div3);
        end
        $display("all 256 8-bit numbers checked, %0d errors", errors);
        $finish;
    end
endmodule

```

Exhaustive check of every 8-bit number.

```

n= 9 bits=00001001 div3=1
n= 10 bits=00001010 div3=0
n=255 bits=11111111 div3=1
all 256 8-bit numbers checked, 0 errors
tb_div3.sv:22: $finish called at 23050000 (1ps)

```

HOW TO BEHAVE IN A CODING ROUND

Restate the problem and ask about the interface (reset polarity, valid signal, MSB or LSB first). Draw the state diagram or datapath **before** writing code. Write the reset and the defaults first. Then test out loud: walk one input sequence through your design, including a corner case. Interviewers score the process as much as the answer - a candidate who finds their own bug scores higher than one who never checked.

30.7 Soft skills that separate good engineers from great ones

- **Writing:** specs, review comments and bug reports that others can act on without a meeting. The micro-architecture spec (Ch. 27) is the most visible artifact of your thinking.
- **Reviewing well and being reviewed well:** comment on the code, not the author; accept findings without defending; thank the reviewer who found your bug before silicon did.
- **Estimating honestly:** break work into verifiable steps, add time for verification and sign-off, and report slips early.
- **Working across disciplines:** DV, PD, DFT, firmware and architecture all depend on your RTL. Learn enough of their language to ask the right questions (Parts IV and V exist for this reason).

- **Reading standards:** the AMBA specifications, IEEE 1800, and the RISC-V ISA manual are primary sources; an engineer who quotes the spec ends arguments.
- **Teaching:** mentoring juniors and writing internal notes is how seniors multiply their impact - and how they are recognized as senior.

30.8 Specializations

Specialization	What you do	Key skills	Why RTL helps
Design verification (DV)	Prove the design matches the spec before tapeout	SV/UVM, constrained random, coverage, SVA, formal, Python	You know where designers make mistakes
DFT	Make the chip testable: scan, ATPG, MBIST, JTAG, test-mode clocking	Test compression, fault models, ATE patterns, Tcl	DFT fixes are RTL changes (Ch. 20)
Physical design (PD)	Floorplan, place, CTS, route, timing/power/IR sign-off	STA, SDC, floorplanning, ECOs, scripting	You can push back on - or fix - unroutable RTL
Architecture / performance	Define what to build; model performance and power	C++/SystemC/Python models, workload analysis, statistics	Your models are credible because you know what hardware costs
FPGA design	Designs that ship on FPGAs; prototyping and emulation	Vendor tools, IP integration, timing on FPGA fabric, high-speed I/O	Same RTL, different PPA rules
Static sign-off (lint/CDC/RDC)	Own methodology and sign-off for a chip	Tool expertise, waiver discipline, clock/reset architecture	Every violation is an RTL structure
Post-silicon validation	Bring up and debug real chips	Lab equipment, debug infrastructure, firmware	Root-causing needs the RTL in your head

30.9 Final checklist

Ready for a junior RTL role?

- ☐ I can explain blocking vs. non-blocking assignment in terms of the event scheduler, and show the race that the wrong choice causes (Ch. 4)
- ☐ I write latch-free combinational logic and correctly reset sequential logic without thinking about it (Ch. 5-6)
- ☐ I can design and verify an FSM, a synchronous FIFO and a valid/ready pipeline stage with a skid buffer from scratch (Ch. 7, 9-10)
- ☐ I can explain metastability and design a pulse synchronizer and an async FIFO (Ch. 11)
- ☐ I can write an APB slave and describe AXI4 channels and handshake rules (Ch. 13)
- ☐ I can read a timing report and state the setup and hold equations (Ch. 18)
- ☐ I have written a self-checking testbench with a reference model, and SVA assertions for a protocol (Ch. 22-23)
- ☐ I have at least one complete, documented, verified project in a public repository (Ch. 28 or 29 extended)
- ☐ I have run a design through synthesis and looked at the netlist and the area/timing reports (Ch. 17)
- ☐ I use git, Linux, Make and Python or Tcl comfortably
- ☐ I have answered Appendix C's interview questions aloud
- ☐ I can walk through my portfolio project and defend every design decision

30.10 Summary

- Careers progress by scope: block, IP, chip, product line; each stage has distinct skills and signals of readiness.
- A 48-week plan through this book, with a public deliverable per block, builds both the skills and the portfolio.
- The strongest portfolios are few, deep and well-verified: a peripheral, a RISC-V core, an accelerator, an open-source contribution, a tapeout.
- Learn the concepts on free tools; commercial tools follow quickly. Tcl and Python are mandatory.
- Interviews test fundamentals (semantics, CDC, STA, FSMs) and the process of design as much as the answer.
- Specializations - DV, DFT, PD, architecture, FPGA, sign-off, post-silicon - all build on a solid RTL foundation.

30.11 Exercises

- 1 Place yourself in the skills table of 30.2. For each area, write the one concrete action that would move you one column to the right.
- 2 Adapt the 48-week plan to your available hours and publish it as the README of your portfolio repository.
- 3 Solve the divide-by-3 **clock** puzzle: design a divide-by-3 clock divider with 50% duty cycle, and explain why it needs both clock edges and what that means for STA and DFT.
- 4 Write RTL in 30 minutes, without a simulator, for a 4-requester round-robin arbiter. Then simulate it and count your bugs.
- 5 Pick one specialization from 30.8 and find three open-source projects or tools where you could practice it.
- 6 Run a mock interview with a friend using five questions from Appendix C; record yourself and review how you explained your reasoning.

APPENDICES

Appendices

A Verilog/SystemVerilog quick reference, a glossary of about 150 terms, 100 interview questions with answers, and the tools, standards and resources that make up the RTL engineer's world.

Appendix A. Verilog/SystemVerilog Quick Reference

A dense, look-it-up-fast summary of the language as used for design (Chapters 3-4) and testbenches (Chapter 22). Everything marked *synth* is accepted by mainstream synthesis tools; the rest is simulation-only.

Data types

Type	States / width	Signed ?	Use
<code>logic</code>	4-state, user width (default 1)	no	The default for all RTL signals; one driver (procedural or one continuous assign)
<code>wire / tri</code>	4-state net, resolved	no	Multiple drivers (tri-state buses), port connections; <code>wire</code> is a net kind, <code>logic</code> is a data type
<code>reg</code>	4-state variable	no	Legacy Verilog name for a variable - not necessarily a register
<code>bit</code>	2-state, user width	no	Testbench / modelling; X and Z become 0 - hides X bugs in RTL
<code>byte shortint int longint</code>	2-state, 8/16/32/64	yes	Loop indices, TB counters
<code>integer</code>	4-state, 32	yes	Legacy loop variable
<code>time / realtime / real</code>	64-bit / double	-	Simulation only (delays, measurements)
<code>string</code>	dynamic	-	TB only; <code>\$sprintf</code> , <code>.len()</code> , <code>.substr()</code>
<code>enum</code>	base type (default <code>int</code>)	base	FSM states, opcodes; <code>.name()</code> , <code>.next()</code> , strong typing
<code>struct packed</code>	sum of fields, MSB = first field	opt.	Bus payloads; can be sliced as a vector
<code>union packed</code>	all members same width	opt.	Several views of one vector
Packed array <code>logic [3:0][7:0] w</code>	contiguous bits	no	32-bit vector viewed as 4 bytes; <code>w[0]</code> is the LS byte
Unpacked array <code>logic [7:0] m [0:255]</code>	array of elements	-	Memories, register files (Chapter 9)
Dynamic <code>[]</code> , <code>queue [\$]</code> , <code>assoc [key]</code>	resizable	-	TB only (scoreboards, sparse memories)

Literal	Meaning
<code>8'hA5, 4'b10x1, 12'o777, 16'd255</code>	Sized, unsigned, based; <code>x/z/?</code> allowed in b/o/h
<code>8'sh80</code>	Sized signed literal (-128)
<code>255</code>	Unsize decimal: 32-bit signed
<code>'hFF</code>	Unsize based: at least 32 bits, unsigned
<code>'0 '1 'x 'z</code>	Fill every bit of the context width (SV)
<code>w'(expr), signed'(x), type_t'(x)</code>	Size, sign and type casts (SV)
<code>1_000_000, 32'hDEAD_BEEF</code>	Underscores are ignored

Operators and precedence

Prec.	Operators	Notes
1 (highest)	<code>() [] :: .</code>	Grouping, select, scope, member
2	unary <code>+ - ! ~ & ~& ~ ^ ~^ , ++ --</code>	Unary <code>&/ ^</code> are reductions -> 1 bit
3	<code>**</code>	Power; exponent is self-determined
4	<code>* / %</code>	Division/modulo by non-constant is expensive in HW
5	binary <code>+ -</code>	
6	<code><< >> <<< >>></code>	<code>>>></code> is arithmetic only if the operand is signed
7	<code>< <= > >= inside dist</code>	Relational; result 1 bit
8	<code>== != === !== ==? !=?</code>	<code>===</code> compares X/Z exactly (TB only); <code>==?</code> wildcard
9	binary <code>&</code>	
10	binary <code>^ ~^ ^~</code>	
11	binary <code> </code>	
12	<code>&&</code>	Logical, short-circuit
13	<code> </code>	
14	<code>?:</code>	Right-associative; X select merges both arms
15	<code>-> <-></code>	Logical implication / equivalence (constraints)
16 (lowest)	<code>= += -= ... <= and { } {{ }}</code>	Assignments; concatenation / replication

PITFALL - PRECEDENCE TRAPS

`a & b == c` parses as `a & (b == c)`; `x << 1 + y` shifts by `1 + y`; `!a == b` is `(!a) == b`. When in doubt, parenthesise - reviewers will thank you.

Expression sizing and signedness rules

Rule	Consequence
Context-determined width = max of all operand widths and the LHS	<code>sum = a + b</code> with 8-bit a, b and 9-bit sum keeps the carry; <code>{c, s} = a + b</code> also works
Self-determined operands: concatenation members, shift amount, <code>**</code> exponent, reduction/relational/logical operands	<code>{a + b}</code> is only max(La, Lb) bits wide - the carry is lost
Comparison operands are sized to each other, result is 1 bit	<code>if (cnt == 4'd10)</code> - fine; comparing 4-bit to 32-bit int extends the 4-bit one
If any operand is unsigned the whole expression is unsigned	<code>s8 + u8</code> is unsigned; <code>-1 > 8'd0</code> evaluates true
Part-selects and concatenations are always unsigned	<code>a[7:0]</code> of a signed <code>a</code> is unsigned; use <code>\$signed()</code>
Extension: signed operands sign-extend, unsigned zero-extend	Happens before evaluation, based on the final expression type
Truncation on assignment keeps the LSBs	<code>-Wall</code> / lint flags width mismatches - fix every one

Procedural blocks and assignments

Construct	Infers	Rule of thumb
<code>always_ff @(posedge clk or negedge rst_n)</code>	Flip-flops	Only <code><=</code> ; one clock; tool checks one-event-control
<code>always_comb</code>	Combinational logic	Only <code>=</code> ; default every output first; runs at time 0; no sensitivity list to forget
<code>always_latch</code>	Latches	Intentional latches only (ICGs, latch-based designs)
<code>always @(*)</code>	Comb (Verilog-2001)	Prefer <code>always_comb</code>
<code>assign y = expr;</code>	Comb wire	Continuous; one per net bit
<code>initial</code>	Nothing (FPGA: init values)	TB only in ASIC RTL
<code>final</code>	-	End-of-sim reporting
<code>a = b; (blocking)</code>	-	Comb blocks; TB sequential code
<code>a <= b; (non-blocking)</code>	-	Every clocked assignment (Chapter 4 explains the scheduler)
<code>function automatic</code>	Comb logic	No timing controls; <code>return</code> ; <code>void</code> functions allowed
<code>task</code>	Usually TB only	May consume time (<code>@</code> , <code>#</code> , <code>wait</code>)

case variants

Form	Matching	Synthesis meaning
<code>case</code>	Exact 4-state (<code>x</code> matches only <code>x</code>)	Priority mux unless items are mutually exclusive
<code>casez</code>	<code>z/?</code> in items or expression are don't-care	Priority encoders: <code>4'b1???</code>
<code>casex</code>	<code>x</code> and <code>z</code> both don't-care	Avoid - an X on the selector matches anything and hides bugs
<code>case (...)</code> <code>inside</code>	Wildcards only in items, ranges <code>[0:7]</code>	The safe modern <code>casez</code>
<code>unique case</code>	Error if 0 or >1 items match (at run time)	Parallel mux, no priority; replaces <code>parallel_case full_case</code>
<code>unique0 case</code>	Error if >1 match; no-match allowed	Parallel, outputs keep defaults when nothing matches
<code>priority case</code>	Error if no item matches	Keeps priority but asserts completeness

System tasks and functions

Group	Tasks / functions
Printing	<code>\$display</code> (newline), <code>\$write</code> , <code>\$strobe</code> (end of time step, sees NBA results), <code>\$monitor</code> , <code>\$sformatf</code> ; formats <code>%d %h %b %o %t %s %e %m %0d</code>
Control	<code>\$finish</code> , <code>\$stop</code> , <code>\$fatal(1, msg)</code> , <code>\$error</code> , <code>\$warning</code> , <code>\$info</code>
Time	<code>\$time</code> (64-bit int), <code>\$realtime</code> , <code>\$stime</code> , <code>\$timeformat</code> , <code>\$prnttimescale</code>
Random	<code>\$urandom</code> , <code>\$urandom_range(max, min)</code> , <code>\$random(seed)</code> , <code>std::randomize()</code> , <code>obj.randomize()</code>
Files	<code>\$fopen</code> , <code>\$fclose</code> , <code>\$fdisplay</code> , <code>\$fwrite</code> , <code>\$fscanf</code> , <code>\$fgets</code> , <code>\$feof</code> , <code>\$readmemh</code> , <code>\$readmemb</code> , <code>\$writememh</code>

Group	Tasks / functions
Waveforms	<code>\$dumpfile</code> , <code>\$dumpvars(0, top)</code> , <code>\$dumpoff/on</code> (VCD)
Plusargs	<code>\$test\$plusargs("DBG")</code> , <code>\$value\$plusargs("SEED=%d", s)</code>
Math / sizing (elaboration-time, synth)	<code>\$clog2</code> , <code>\$bits</code> , <code>\$size</code> , <code>\$left</code> , <code>\$right</code> , <code>\$high</code> , <code>\$low</code> , <code>\$signed</code> , <code>\$unsigned</code>
Bit queries (synth in most tools)	<code>\$countones</code> , <code>\$countbits(v, 1'b1)</code> , <code>\$onehot</code> , <code>\$onehot0</code> , <code>\$isunknown</code>
Assertions (SVA, Chapter 23)	<code>\$past</code> , <code>\$rose</code> , <code>\$fell</code> , <code>\$stable</code> , <code>\$changed</code> , <code>\$sampled</code> , <code>\$assertoff/\$asserton</code>
Types	<code>\$cast(dst, src)</code> , <code>\$typename</code>

Modules, parameters and generate

```

module fifo #(
    parameter int W      = 32,          // overridable
    parameter int DEPTH = 16,
    localparam int AW    = $clog2(DEPTH) // derived, not overridable
) (
    input logic      clk, rst_n,
    input logic [W-1:0] wdata,
    output logic [W-1:0] rdata
);
// instantiation: named parameter and port binding; .clk is shorthand for .clk(clk)
fifo #(.W(64), .DEPTH(8)) u_fifo (.clk, .rst_n, .wdata(din), .rdata(dout));

```

ANSI header with parameters, and an instance. `.*` connects every same-named port but hides mistakes; prefer explicit binding.

```

module gen_ex #(parameter int N = 4, parameter bit REG_OUT = 1) (
    input logic      clk,
    input logic [N-1:0] a, b,
    output logic [N-1:0] y
);
    logic [N-1:0] s;
    for (genvar i = 0; i < N; i++) begin : g_bit // generate-for
        assign s[i] = a[i] ^ b[i];
    end
    if (REG_OUT) begin : g_reg // generate-if
        always_ff @(posedge clk) y <= s;
    end else begin : g_comb
        assign y = s;
    end
endmodule

```

Generate loops and conditionals (the `generate` keyword is optional in SV). Always name generate blocks: hierarchical paths become `g_bit[2]`. instead of `genblk1[2]`.. Verified with Icarus.

Packages, typedef, enum, struct

```

package bus_pkg;
    localparam int AW = 32;
    typedef logic [AW-1:0] addr_t;
    typedef enum logic [1:0] {OKAY = 2'b00, EXOKAY = 2'b01,
                             SLVERR = 2'b10, DECERR = 2'b11} resp_e;

    typedef struct packed {
        addr_t    addr; // bits [36:5]
        logic     write; // bit [4]
        logic [3:0] strb; // bits [3:0]
    } req_t;

```



```

    function automatic logic is_err(resp_e r);
        return r[1];
    endfunction
endpackage

module pkg_user
    import bus_pkg::*;          // header import: types usable in the port list
    (
        input  req_t  req,
        input  resp_e resp,
        output addr_t word_addr,
        output logic  bad
    );
    assign word_addr = {req.addr[AW-1:2], 2'b00};
    assign bad       = is_err(resp);
endmodule

```

A package shared by RTL and TB. Compile packages first; avoid `import *` in `$unit` (file) scope. Verified with Icarus.

Interfaces and modports

```

interface vr_if #(parameter int W = 8) (input logic clk);
    logic      valid, ready;
    logic [W-1:0] data;
    modport src (output valid, data, input  ready);
    modport snk (input  valid, data, output ready);
endinterface

module sink (vr_if.snk s, output logic [7:0] last);
    assign s.ready = 1'b1;
    always_ff @(posedge s.clk)
        if (s.valid && s.ready) last <= s.data;
endmodule

// in the parent: vr_if #(8) bus (.clk);  sink u_sink (.s(bus), .last);

```

An interface bundles a protocol's wires; modports give each side its direction. IEEE 1800-legal and supported by all commercial tools and Verilator; Icarus 12 does not accept interface ports, so this listing was not simulated.

Synthesizable subset at a glance

Synthesizable	Not synthesizable (TB / model only)
<code>always_ff/_comb/_latch</code> , <code>assign</code> , module instances	<code>initial</code> (except FPGA init / ROM <code>\$readmemh</code> in some flows)
<code>if</code> , <code>case</code> family, <code>for</code> with constant bounds, <code>generate</code>	<code>#</code> delays (ignored by synthesis - a sim/synth mismatch source)
<code>function automatic</code> , tasks without timing	<code>wait</code> , <code>@</code> inside tasks, <code>fork/join</code> , <code>forever</code> without a clock
Packed/unpacked arrays, structs, unions, enums, packages	Classes, dynamic arrays, queues, associative arrays, <code>string</code>
Arithmetic <code>+</code> <code>-</code> <code>*</code> , shifts, compares; <code>/</code> and <code>%</code> by constants or powers of 2	<code>real</code> , <code>\$random</code> , file I/O, <code>\$display</code> (ignored)
Parameters, <code>localparam</code> , <code>\$clog2</code> , <code>\$bits</code>	<code>===/!==(X is not a hardware value)</code> , <code>force/release</code>
Interfaces with modports (most tools)	SVA concurrent assertions (ignored by synthesis, used by formal)

Common idioms (all verified with Icarus)

Flop with asynchronous active-low reset and enable

```
module dff_en #(parameter int W = 8) (
  input  logic      clk, rst_n, en,
  input  logic [W-1:0] d,
  output logic [W-1:0] q
);
  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) q <= '0;
    else if (en) q <= d;          // enable -> mux or clock gate
endmodule
```

Combinational block with defaults (no latches)

```
always_comb begin
  onehot = '0;                // defaults first: every path assigns
  err    = 1'b0;
  if (valid) begin
    unique case (sel)
      2'd0: onehot = 4'b0001;
      2'd1: onehot = 4'b0010;
      2'd2: onehot = 4'b0100;
      2'd3: onehot = 4'b1000;
      default: err = 1'b1;      // reached only on X/Z in simulation
    endcase
  end
end
```

FSM template (two processes, enumerated state)

```
typedef enum logic [1:0] {IDLE, RUN, FLUSH} state_e;
state_e state_q, state_d;

always_ff @(posedge clk or negedge rst_n)
  if (!rst_n) state_q <= IDLE;
  else       state_q <= state_d;

always_comb begin
  state_d = state_q;          // default: stay
  busy    = 1'b0;            // default outputs
  case (state_q)
    IDLE: if (start) state_d = RUN;
    RUN:  begin busy = 1'b1; if (done) state_d = FLUSH; end
    FLUSH: state_d = IDLE;
    default: state_d = IDLE;
  endcase
end
```

Two-flop synchronizer (single-bit level signal only)

```
(* ASYNC_REG = "TRUE" *) logic [1:0] ff; // keep flops adjacent (FPGA)
always_ff @(posedge clk or negedge rst_n)
  if (!rst_n) ff <= '0;
  else       ff <= {ff[0], d_async};
assign q_sync = ff[1];
```

Modulo-N counter with terminal-count pulse

```
module cnt_mod #(parameter int N = 10) (
    input  logic          clk, rst_n, en,
    output logic [$clog2(N)-1:0] cnt,
    output logic          tc
);
    assign tc = en && (cnt == $clog2(N)'(N-1));
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) cnt <= '0;
        else if (tc) cnt <= '0;
        else if (en) cnt <= cnt + 1'b1;
endmodule
```

Other one-liners

Need	Idiom
Sign-extend 8 -> 32	{{24{a[7]}}, a} or 32'(signed'(a))
Rising-edge pulse	rise = a & ~a_q; (a_q is a delayed one clock)
Isolate lowest set bit / fixed-priority grant	gnt = req & (~req + 1'b1);
Power of two?	(x != 0) && ((x & (x - 1)) == 0)
Binary -> Gray	g = b ^ (b >> 1);
Round up to multiple of 4	(x + 3) & ~32'd3
Bit reverse	for (int i=0;i<W;i++) r[i] = a[W-1-i]; or {<<{a}} (streaming)
Parity	^data (even parity bit)
Saturating add (unsigned)	{c, s} = a + b; y = c ? '1 : s;
Register slice without reset (datapath)	always_ff @(posedge clk) if (v_in) d_q <= d_in;

Appendix B. Glossary

Terms used across the book, in alphabetical order. Numbers in parentheses point to the chapter where the concept is treated in depth.

Term	Definition
ACE / CHI	AMBA coherency protocols: ACE extends AXI with snoop channels; CHI is a packetised, layered protocol for scalable coherent meshes.
Activity factor (alpha)	Probability that a node toggles per clock; dynamic power = $\alpha C V^2 f$.
AHB	AMBA Advanced High-performance Bus: pipelined address/data phases, single outstanding transfer, bursts, HREADY wait states.
Always-on domain	A power domain that stays powered while others are gated; hosts the power controller, wake-up logic and retention control.
Antenna effect	Charge collected by long metal during fabrication that can damage thin gate oxide; fixed by diodes or layer jumping.
APB	AMBA Advanced Peripheral Bus: two-phase (setup/access), unpipelined, for low-bandwidth register access.
Arbiter	Logic that grants one of several requesters a shared resource: fixed priority, round-robin, weighted, or QoS-based.
ASIC	Application-Specific Integrated Circuit: a chip fabricated for one design, as opposed to a programmable FPGA.
Assertion	An executable property of the design (SVA); checked in simulation and proven or refuted by formal tools.
Async FIFO	FIFO with read and write in different clock domains; Gray-coded pointers are synchronized across (Chapter 11).
At-speed test	Testing with launch/capture at functional clock frequency to detect delay (transition, path-delay) faults.
ATPG	Automatic Test Pattern Generation: computes scan patterns that detect modelled manufacturing faults.
AXI4	AMBA Advanced eXtensible Interface: five independent valid/ready channels (AW, W, B, AR, R), bursts, IDs, outstanding and out-of-order transactions.
AXI4-Lite	Single-beat, 32/64-bit subset of AXI4 for registers; no bursts or IDs.
AXI4-Stream	Unidirectional data streaming with TVALID/TREADY , TDATA , TLAST , TKEEP , TUSER ; no addresses.
Back-annotation	Loading extracted delays (SDF) or parasitics (SPEF) into simulation or STA.
Backpressure	A downstream stage stalling an upstream one, e.g. by deasserting ready .
BIST	Built-In Self-Test: on-chip pattern generation and response compaction - MBIST for memories, LBIST for logic.
Blocking assignment	=: updates the variable immediately in the active region; use in combinational blocks.
Boundary scan	IEEE 1149.1 chain of cells at the chip pins, controlled through the JTAG TAP, for board interconnect test.
Burst	A multi-beat bus transaction issued with a single address (AXI: INCR, WRAP, FIXED; max 4 KB boundary).
CDC	Clock-Domain Crossing: any signal launched in one clock domain and captured in an asynchronous one; needs a synchronization scheme.
Clock gating (ICG)	Stopping the clock to idle registers with an Integrated Clock Gating cell (latch + AND) - the main dynamic-power lever.
Clock jitter	Cycle-to-cycle variation of the clock edge; modelled as clock uncertainty.

Term	Definition
Clock skew	Difference in clock arrival time at two flops; helps setup or hold depending on its sign.
Clock tree synthesis (CTS)	Building the buffered clock distribution network to meet skew, latency and transition targets.
Clock uncertainty	SDC margin (<code>set_clock_uncertainty</code>) covering jitter and pre-CTS skew estimates.
Combinational loop	A feedback path with no storage element; oscillates or latches, breaks STA; lint error.
Congestion	Routing demand exceeding available tracks in a region; caused by dense, highly connected logic (big muxes, crossbars).
Constrained random	Stimulus generated randomly within declared constraints, steered by coverage feedback.
Corner (PVT)	A process/voltage/temperature combination at which timing and power are analysed (e.g. SS 0.72 V 125C).
Coverage	Metrics of verification completeness: code (line, branch, toggle, FSM, expression) and functional (covergroups, cover properties).
Covergroup	SystemVerilog construct sampling values and crosses to measure functional coverage.
Credit-based flow control	Sender holds credits equal to receiver buffer space; sends only with credit - tolerates long wire latency.
Critical path	The timing path with the worst slack; sets the maximum clock frequency.
Crosstalk (SI)	Coupling between adjacent wires changing delay or causing glitches; analysed in signal-integrity STA.
CSR	Control and Status Register: software-visible register mapped into the address space (Chapter 15).
Decap	Decoupling capacitor cells that stabilize the local supply against dynamic IR drop.
DEF / LEF	Design Exchange Format (placed/routed design) and Library Exchange Format (cell abstracts and technology rules).
Delta cycle	A zero-time simulation iteration; many deltas can occur in one time step while events settle.
DFT	Design for Test: scan, compression, BIST, JTAG and test access so manufactured parts can be screened.
DMA	Direct Memory Access engine: moves data between memory and peripherals without CPU involvement.
DRC	Design Rule Check: verifies layout geometry against foundry rules. Also: electrical design rules (max cap/transition/fanout).
DVFS	Dynamic Voltage and Frequency Scaling: lowering V and f under light load; power scales roughly with $V^2 f$.
ECC	Error-Correcting Code (e.g. SECDED Hamming) protecting memories and buses against bit flips.
ECO	Engineering Change Order: a late, minimal netlist or layout patch, often using spare cells.
Elaboration	Building the design hierarchy: resolving parameters, generate blocks and instances after parsing.
Electromigration (EM)	Metal atoms displaced by high current density over time; limits wire widths and via counts.
Emulation	Running the design mapped onto a dedicated hardware platform (Palladium, ZeBu, Veloce) at MHz speeds.
Equivalence checking (LEC)	Formal proof that two representations (RTL vs netlist, netlist vs netlist) implement the same function.
False path	A structurally present path that never propagates in operation; excluded from STA with <code>set_false_path</code> .
Fanout	Number of inputs driven by one output; high fanout nets are buffered into trees.
Fault coverage	Detected faults divided by total modelled faults; production targets are typically > 99% stuck-at.

Term	Definition
FIFO	First-In First-Out buffer decoupling producer and consumer rates (Chapter 9).
FinFET	Multi-gate 3D transistor used from ~22/16 nm; followed by gate-all-around (nanosheet) devices at 3/2 nm.
Flip-flop	Edge-triggered storage element: samples D at the clock edge; characterised by setup, hold and clock-to-Q.
Floorplan	Placement of macros, IO, power grid and block outlines before standard-cell placement.
Formal verification	Mathematical proof of properties over all input sequences (model checking), without test vectors.
Foundry	The company that fabricates wafers (TSMC, Samsung, Intel Foundry, GlobalFoundries, SkyWater, IHP ...).
FPGA prototyping	Mapping the ASIC RTL onto FPGAs to run software at tens of MHz before silicon.
FSM	Finite State Machine: state register plus next-state and output logic (Moore or Mealy).
Functional coverage	User-defined coverage of features and scenarios from the verification plan.
Gate-level simulation (GLS)	Simulating the synthesized or post-layout netlist, optionally with SDF timing.
GDSII / OASIS	Binary layout formats delivered to the foundry at tapeout.
Glitch	A spurious short pulse from unequal path delays; harmless on synchronous data, fatal on clocks, resets and async inputs.
Glitch-free clock mux	Clock switch that disables the old clock on its own falling edge before enabling the new one.
Gray code	Encoding where consecutive values differ in one bit; makes multi-bit counters safe to synchronize.
Handshake synchronizer	Req/ack protocol across domains, each direction through a 2-flop synchronizer; for multi-bit data.
HLS	High-Level Synthesis: generating RTL from C/C++/SystemC.
Hold time	Minimum time data must be stable after the clock edge; hold violations are frequency-independent and kill silicon.
IJTAG	IEEE 1687: network of instruments accessed via reconfigurable scan paths (SIBs) described in ICL/PDL.
Interconnect	The fabric (crossbar, bus matrix, NoC) that routes transactions between managers and subordinates.
Interrupt controller	Collects, prioritizes and routes interrupts to cores (RISC-V PLIC/CLIC, Arm GIC/NVIC).
IP-XACT	IEEE 1685 XML schema describing IP interfaces, registers and memory maps for tool automation.
IR drop	Voltage loss across the power grid (static or dynamic) that slows cells.
Isolation cell	Clamps outputs of a powered-down domain to a known value so live domains do not see X.
JTAG / TAP	IEEE 1149.1 Test Access Port: TCK, TMS, TDI, TDO, optional TRST; a 16-state controller selects IR/DR scans.
Latch	Level-sensitive storage; transparent while enabled. Unintended latches come from incomplete combinational assignment.
Latency	Cycles (or time) from input to corresponding output.
Leakage power	Static power from subthreshold and gate leakage; grows with temperature and lower Vt.
Level shifter	Cell translating a signal between voltage domains.
Liberty (.lib)	Standard-cell library timing/power/area models (NLDM, CCS, ECSM) used by synthesis and STA.
Lint	Static rule checking of RTL for coding, synthesis and simulation-mismatch issues.

Term	Definition
Lockup latch	Latch inserted between scan flops of different clock domains/skews to avoid hold failures during shift.
LVS	Layout Versus Schematic: checks the layout's extracted netlist matches the source netlist.
Macro	A large pre-built block placed as a unit: SRAM, PLL, SerDes, analog IP.
Mask / reticle	The photomask set used per layer; full-mask costs are millions of dollars at advanced nodes.
Memory compiler	Foundry/IP tool generating SRAM/ROM/register-file macros of a requested size with views (.lib, LEF, GDS, Verilog).
Memory map	Assignment of address ranges to subordinates and registers (Chapter 14).
Metastability	A flop left between 0 and 1 after a setup/hold violation; resolves in a random time. Synchronizers make failure improbable, not impossible.
MPW / shuttle	Multi-Project Wafer: many designs share one mask set to cut prototype cost.
MTBF	Mean Time Between Failures of a synchronizer: $\exp(t_r / \tau) / (T_0 f_{clk} f_{data})$.
Multi-Vt	Libraries with low/standard/high threshold cells traded between speed and leakage.
Multicycle path	A path allowed more than one clock period, declared with <code>set_multicycle_path</code> (adjust hold too).
Netlist	Structural description of cells and nets produced by synthesis.
NoC	Network-on-Chip: packet-switched interconnect with routers and links, scaling beyond crossbars.
Non-blocking assignment (NBA)	<code><=></code> : RHS sampled now, LHS updated in the NBA region; models flops without races.
OCC	On-Chip Clock Controller: generates at-speed launch/capture pulses from a PLL during scan test.
OCV / AOCV / POCV	On-Chip Variation models: flat derates, depth/distance-based, or statistical (per-cell sigma).
One-hot	Encoding with exactly one bit set; fast decode, more flops; popular for FSMs and grants.
Operand isolation	Gating datapath inputs when the result is unused to cut toggle power.
Outstanding transactions	Requests issued but not yet completed; AXI allows many, bounded by ID and buffer resources.
Parasitic extraction (SPEF)	Computing wire R and C from layout for sign-off timing and power.
PDK	Process Design Kit: device models, rules, cell libraries and tech files for a foundry process.
Pipelining	Inserting registers to split long logic, trading latency for clock frequency and throughput.
Place and route (P&R)	Placement, CTS, routing and optimization that turn a netlist into layout.
Power domain	A group of logic sharing a supply that can be switched or scaled independently (UPF).
Power gating	Cutting a domain's supply with header/footer switch cells to eliminate leakage.
PPA	Power, Performance, Area - the three competing metrics of an implementation.
Priority encoder	Outputs the index of the highest (or lowest) priority set bit.
Pulse synchronizer	Converts a source pulse to a toggle, synchronizes the toggle, and edge-detects it in the destination.
QoS	Quality of Service: priority/bandwidth/latency controls in interconnects (AXI <code>AxQoS</code>).
RDC	Reset-Domain Crossing: paths from flops reset by one reset to flops on another, which can go metastable on asynchronous assertion (Chapter 12).
Reconvergence	Separately synchronized signals recombined in the destination; they may arrive in different cycles.
Recovery / removal	Setup/hold-like checks for an asynchronous reset's deassertion relative to the clock.

Term	Definition
Register file	Small multi-ported storage array, often built from flops or custom bitcells.
Reset synchronizer	Asserts reset asynchronously, deasserts it synchronously through two flops.
Retention register	Flop with a shadow latch on always-on supply that keeps state while its domain is gated.
Retiming	Moving registers across combinational logic to balance stages without changing behaviour at the boundaries.
RTL	Register-Transfer Level: describing hardware as registers and the combinational transfers between them.
Scan chain	Flops converted to scan flops (mux-D) and stitched into shift registers for test.
Scan compression	Decompressor/compactor around many short internal chains to cut test time and pins (e.g. EDT, DFTMAX).
Scoreboard	Testbench component comparing DUT outputs with a reference model's predictions.
SDC	Synopsys Design Constraints: Tcl-based clocks, IO delays and exceptions used by synthesis, P&R and STA.
SDF	Standard Delay Format (IEEE 1497): cell and interconnect delays for timing simulation.
Setup time	Minimum time data must be stable before the clock edge; violations are fixed by slowing the clock or the logic.
Sign-off	Final verification against all criteria (timing, power, DRC, LVS, EM/IR) before tapeout.
Skid buffer	Two-entry buffer that registers ready in a valid/ready pipeline without losing throughput (Chapter 10).
Slack	Required time minus arrival time; negative slack is a timing violation.
SoC	System-on-Chip: processors, memories, interconnect and peripherals integrated on one die.
Spare cells	Unused gates scattered in the layout for metal-only ECOs.
SRAM	Static RAM macro; single- or dual-port, synchronous read with one-cycle latency.
STA	Static Timing Analysis: exhaustive path-based timing check without vectors (Chapter 18).
Standard cell	Pre-characterized logic gate or flop of fixed height placed in rows.
Stuck-at fault	Fault model where a node is permanently 0 or 1.
Synchronizer	Chain of flops in the destination domain giving a metastable value time to resolve.
Synthesis	Translating RTL into an optimized gate-level netlist under constraints (Chapter 17).
SystemRDL	Accellera register description language from which RTL, headers, docs and UVM register models are generated.
Tapeout	Releasing the final layout database to the foundry for mask making.
Testbench	The non-synthesizable environment that drives, checks and measures the DUT.
Throughput	Results per cycle (or per second) in steady state.
Timing arc	A modelled delay/check between two pins of a cell in Liberty.
Toggle coverage	Whether each bit has transitioned 0->1 and 1->0 in simulation.
Transition fault	Delay fault model: a node is slow to rise or fall; needs at-speed launch/capture.
Unique / priority	SystemVerilog case/if qualifiers that add run-time checks and guide synthesis.
UPF	Unified Power Format (IEEE 1801): power domains, supplies, isolation, retention, level shifters (Chapter 19).
Utilization	Cell area divided by core area; typical targets 60-80% to leave room for routing and buffering.
UVM	Universal Verification Methodology (IEEE 1800.2): class library for reusable SystemVerilog testbenches (Chapter 24).

Term	Definition
Valid/ready	Two-signal handshake; a transfer happens on each clock where both are high.
Verilator	Open-source compiler of SystemVerilog to C++; very fast, cycle-based, 2-state.
Wafer	Silicon disc (300 mm for advanced nodes) carrying hundreds of dies.
Well tap / endcap	Physical-only cells tying wells and ending rows to prevent latch-up and edge effects.
WNS / TNS	Worst and Total Negative Slack - the headline timing metrics.
Write strobe	Byte-enable mask on a write (WSTRB , PSTRB).
X-propagation	How unknowns flow through logic in simulation; if is X-optimistic, which can hide reset bugs.
Yield	Fraction of dies that are functional; depends on defect density and die area.

Appendix C. 100 RTL Interview Questions with Answers

Grouped by topic, roughly from screening questions to on-site design problems. Answer out loud first, then check. Every code answer was compiled and exercised with Icarus Verilog.

Digital logic fundamentals

Q1. What is the difference between a latch and a flip-flop?

A latch is level-sensitive: transparent while its enable is active, so data can race through. A flip-flop is edge-triggered: it samples only at the clock edge. Flops make timing analysis simple (one launch, one capture); latches allow time borrowing but complicate STA and testing.

Q2. Define setup time, hold time and clock-to-Q.

Setup: data must be stable a minimum time before the capturing edge. Hold: stable a minimum time after it. Clock-to-Q: delay from the edge until Q changes. Setup: $T_{clk} \geq t_{cq} + t_{logic} + t_{setup} - \text{skew}$. Hold: $t_{cq} + t_{logic,min} \geq t_{hold} + \text{skew}$.

Q3. Why can a hold violation not be fixed by slowing the clock?

The hold check compares launch and capture on the **same** edge; the clock period does not appear in the equation. It must be fixed by adding delay on the data path (buffers) or reducing skew.

Q4. What is metastability and can it be eliminated?

When setup/hold is violated the flop may hover at an intermediate level and resolve after an unbounded, exponentially distributed time. It cannot be eliminated, only made improbable: $MTBF = \exp(t_r / \tau) / (T_0 f_{clk} f_{data})$, so each extra synchronizer stage multiplies MTBF by roughly $\exp(T / \tau)$.

Q5. Build a 2:1 mux from NAND gates only. How many?

Four: $n1 = \text{NAND}(s, s)$ gives s' ; $n2 = \text{NAND}(a, s')$; $n3 = \text{NAND}(b, s)$; $y = \text{NAND}(n2, n3) = a s' + b s$.

Q6. Implement an XOR with 2:1 muxes; implement an inverter with a mux.

XOR: $y = b ? \sim a : a$ (a mux with inputs a and $\sim a$; $\sim a$ itself is a mux with inputs 1 and 0 selected by a). Inverter: $\text{mux}(\text{sel} = a, \text{in0} = 1, \text{in1} = 0)$. Muxes are universal.

Q7. What is a glitch (hazard) and when does it matter?

A transient wrong output value caused by unequal path delays (static-1 hazard in $a + a'$ style logic). In synchronous logic it settles before the edge and is harmless; on clocks, asynchronous resets, clock-gate enables or CDC paths it causes real failures.

Q8. Compare binary, Gray and one-hot encoding.

Binary: $\log_2(N)$ flops, multi-bit transitions. Gray: $\log_2(N)$ flops, one bit changes per step - safe to synchronize and low-power for counters. One-hot: N flops, trivial decode and fast next-state logic - favoured for FSMs in FPGAs and fast ASIC control.

Q9. What limits the maximum frequency of a design?

The worst register-to-register path: $t_{cq} + t_{logic} + t_{route} + t_{setup} + \text{uncertainty} - \text{useful skew}$. Also IO paths, memory access times and clock-tree/PLL limits.

Q10. How do you convert two's complement to magnitude, and what is special about the most negative number?

Invert and add one. For N bits the range is -2^{N-1} to $2^{N-1}-1$, so negating -2^{N-1} overflows back to itself; absolute-value logic needs one extra bit or saturation.

Verilog and SystemVerilog semantics

Q11. Blocking versus non-blocking: which, where, and why?

`=` executes immediately; `<=` samples the RHS now and updates in the NBA region after all active events. Use `<=` in clocked blocks so every flop sees pre-edge values (no race between blocks); use `=` in combinational blocks so later statements see earlier results. Never mix both on the same variable (Chapter 4).

Q12. What does this swap produce: `always @(posedge clk) begin a = b; b = a; end`?

Both end up with the old `b`: after `a = b`, `a` already holds the new value. Written with `<=` it is a true swap. It also races with any other block reading `a` at the same edge.

Q13. How does an unintended latch get inferred, and how do you prevent it?

A combinational block that does not assign an output on every path (missing `else`, incomplete `case`) must remember the old value - a latch. Prevent with default assignments at the top of `always_comb`, complete `case` with `default`, and lint.

Q14. What is the difference between `wire`, `reg` and `logic`?

`wire` is a net (resolved, can have multiple drivers, continuous assignment). `reg` is a Verilog variable assigned procedurally - it does not imply a register. `logic` is SV's 4-state data type usable as either, but enforcing a single driver.

Q15. `always_comb` versus `always @(*)`?

`always_comb` runs once at time 0, is sensitive to variables read in called functions, forbids other processes writing its outputs, and lets tools warn about latches. `@(*)` misses all of these.

Q16. What is the difference between `==` and `===`?

`==` returns X if either operand has X/Z; `===` compares X and Z literally and always returns 0/1. `===` is for testbenches; it is not synthesizable in any meaningful way.

Q17. How are `casez`, `casex` and `case inside` different, and which is safe?

`casez` treats z/? as don't care in items and the selector; `casex` also treats x as don't care - an X selector then matches the first item and masks bugs. `case inside` applies wildcards only to the items and is the recommended form.

Q18. What do `unique` and `priority` do?

`unique case` asserts exactly one item matches (warning otherwise) and lets synthesis build a parallel mux; `unique0` allows none; `priority` asserts at least one matches but keeps order. They replace the dangerous `full_case/parallel_case` pragmas, which changed synthesis without changing simulation.

Q19. What is a simulation/synthesis mismatch? Give three causes.

RTL simulation behaving differently from the synthesized netlist: incomplete sensitivity lists, `#` delays (ignored by synthesis), `full_case/parallel_case` pragmas, X-optimism of `if/case`, `initial` blocks, and reading a variable before it is assigned in a combinational block.

Q20. What width is `a + b` in `{a + b}` if `a` and `b` are 8 bits?

8 bits: operands of a concatenation are self-determined, so the carry is dropped. Assigning `sum9 = a + b` keeps it because the 9-bit LHS participates in context sizing.

Q21. What does `-1 > 8'd0` evaluate to, and why?

Mixing signed and unsigned makes the whole expression unsigned: the 32-bit `-1` becomes `32'hFFFFFF_FFFF`, so the compare is true. Keep both operands signed (`$signed, 8'sd0`) when comparing signed values.

Q22. What is the difference between a task and a function?

Functions execute in zero time, return a value (or `void`) and may not contain timing controls; they synthesize to combinational logic. Tasks may consume time (`@`, `#`, `wait`) and return through output arguments; used mainly in testbenches.

Q23. What is a delta cycle and why can a zero-delay clock divider cause a race?

A delta is a zero-time iteration of the scheduler. A clock generated by a flop (`clk2 <= ~clk2`) changes one delta later than `clk`; data launched on `clk` with `=` or through continuous logic can be seen by `clk2` flops in the same time step, so results depend on event order.

Q24. Is `for` synthesizable?

Yes, when the bounds are constant at elaboration: it is unrolled into parallel hardware. It describes replicated logic, not a loop in time - a 32-iteration loop is 32 copies of the body.

Finite state machines

Q25. Moore versus Mealy?

Moore outputs depend only on the state (glitch-free if registered, one cycle later); Mealy outputs depend on state and inputs (react in the same cycle, fewer states, but create combinational input-to-output paths).

Q26. Why separate state register and next-state logic into two processes?

It keeps sequential and combinational code distinct, avoids accidental flops on outputs, and maps directly to the textbook structure. Registered outputs can then be added deliberately.

Q27. How do you make an FSM safe against illegal states?

Use a `default` that returns to a known state, enumerate all encodings, and if required by safety (radiation, ISO 26262) ask synthesis for a safe FSM (`syn_encoding safe` or equivalent), since optimizers otherwise remove unreachable-state recovery logic.

Q28. Draw/describe an FSM that detects the overlapping sequence 1011.

States S0 (nothing), S1 (1), S2 (10), S3 (101). On a 1 in S3 assert `hit` and go to S1 (the trailing 1 starts a new match); on a 0 in S3 go to S2 (the trailing 10). An equivalent shift-register form compares the last four bits:

```
module seq1011 (input logic clk, rst_n, din, output logic hit);
    logic [2:0] h;           // last three bits
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) h <= '0;
        else      h <= {h[1:0], din};
    assign hit = ({h, din} == 4'b1011); // Mealy output
endmodule
```

Q29. When would you register FSM outputs?

When they drive another block, IO pins or a clock gate: registering removes glitches and decouples timing. The cost is one cycle of latency, often absorbed by computing outputs from `state_d`.

Q30. What is a one-hot FSM's `unique case (1'b1)` idiom?

`case (1'b1) state[IDLE]: ... state[RUN]: ... endcase` with `unique` tells synthesis only one bit is set, so each branch decodes a single flop instead of a full compare.

Q31. How would you verify an FSM?

FSM state and transition coverage, assertions that states are legal (`$onehot(state)` for one-hot) and that key transitions happen (`req |-> ##[1:4] gnt`), plus formal reachability for deadlocks.

Datapath, pipelining and FIFOs

Q32. Carry-ripple versus carry-lookahead versus prefix adders?

Ripple: $O(N)$ delay, minimal area. Lookahead: groups compute generate/propagate to skip carries. Parallel prefix (Kogge-Stone, Brent-Kung, Sklansky): $O(\log N)$ delay with different area/fanout trade-offs. Synthesis picks one from `+` according to timing.

Q33. How do you implement a multiplier-accumulator at high frequency?

Pipeline the multiplier (partial products, Wallace/Dadda tree, final adder), keep the accumulator in carry-save form or register the adder, and size the accumulator for N products: $2W + \text{ceil}(\log_2 N)$ bits.

Q34. What is the difference between latency and throughput? Give an example where pipelining increases latency but helps.

Latency is the time for one result; throughput is results per unit time. A 4-stage pipelined multiplier has 4 cycles latency but delivers one product per cycle at a much higher clock.

Q35. In a valid/ready pipeline, why must valid not depend on ready?

The AXI rule: a source must not wait for **ready** before asserting **valid**, otherwise two components that each wait for the other deadlock. **ready** may depend on **valid**.

Q36. What is a skid buffer and when do you need one?

A 2-entry buffer that lets you register the **ready** path. Because the upstream sees the registered **ready** a cycle late, one beat may arrive after the stall; the skid register holds it. It breaks the long combinational **ready** chain without losing throughput.

Q37. How do you detect full and empty in a synchronous FIFO?

Use pointers one bit wider than the address. Empty: pointers equal. Full: addresses equal but MSBs differ. Alternatively keep an occupancy counter.

Q38. FIFO depth: writer 100 MHz, bursts of 80 words with no idle; reader at 50 MHz reads one word every cycle. Minimum depth?

Burst time = $80 / 100 \text{ MHz} = 800 \text{ ns}$. Words read in that time = $800 \text{ ns} \times 50 \text{ MHz} = 40$. Depth $\geq 80 - 40 = 40$ words (add margin for synchronizer latency in an async FIFO, typically a few entries).

Q39. FIFO depth: writer 80 MHz writes 1 of every 2 cycles; reader 50 MHz reads 1 of every 4 cycles; one burst of 120 words. Depth?

Write rate = 40 Mwords/s, read rate = 12.5 Mwords/s. Burst lasts $120 / 40 \text{ M} = 3 \text{ us}$; reads in that time = $3 \text{ us} \times 12.5 \text{ M} = 37.5$, so 37 complete. Depth $\geq 120 - 37 = 83$ words.

Worst case for back-to-back bursts: if bursts can follow each other without idle time the average write rate exceeds the read rate and **no** finite FIFO suffices - always state the idle assumption.

Clock-domain crossing and resets

Q40. Why is a two-flop synchronizer enough for a single bit but not for a bus?

Each bit resolves independently and may arrive a cycle apart, so a bus can be sampled with a mix of old and new bits (data incoherency). Buses need Gray coding (only one bit changes), a handshake with a stable data bus, or an async FIFO.

Q41. How do you pass a single-cycle pulse from a fast to a slow domain?

A fast pulse may fall between slow edges. Convert it to a level toggle in the source, synchronize the toggle, and XOR the last two stages in the destination. Pulses must be spaced by more than about two destination cycles, or use a handshake.

```

module pulse_sync (
    input  logic src_clk, src_rst_n, src_pulse,
    input  logic dst_clk, dst_rst_n,
    output logic dst_pulse
);
    logic      tgl;
    logic [2:0] s;
    always_ff @(posedge src_clk or negedge src_rst_n)
        if (!src_rst_n) tgl <= 1'b0;
        else if (src_pulse) tgl <= ~tgl;
    always_ff @(posedge dst_clk or negedge dst_rst_n)
        if (!dst_rst_n) s <= '0;
        else s <= {s[1:0], tgl};
    assign dst_pulse = s[2] ^ s[1];
endmodule

```

Q42. Why are async FIFO pointers Gray-coded?

An incrementing Gray pointer changes one bit, so a synchronizer captures either the old or the new value - never a wild one. The synchronized value is stale, which makes full/empty pessimistic (safe) but never wrong.

Q43. Can you Gray-code a FIFO pointer for a depth of 6?

Not with a plain Gray counter: wrapping from 5 to 0 changes more than one bit. Use a power-of-two depth, or a special symmetric Gray sequence for even depths, or a handshake-based design.

Q44. What is reconvergence and why is it a CDC bug?

Two signals synchronized separately and then combined: they may arrive one cycle apart, creating a transient state the designer never intended. Synchronize them together as one encoded value or one handshake.

Q45. What must never be in front of a synchronizer?

Combinational logic from the source domain: it can glitch and the destination may capture the glitch. Register the signal in the source domain first.

Q46. Asynchronous or synchronous reset?

Async: works without a clock, no data-path logic, but deassertion must be synchronized (recovery/removal) and it is sensitive to glitches. Sync: timed like data, glitch-filtered, but needs a running clock and adds a gate to the D path. Common practice is **async assert, sync deassert** via a reset synchronizer.

Q47. Write a reset synchronizer.

Both flops are asynchronously cleared; a constant 1 shifts in after release so deassertion is aligned to the clock:

```

always_ff @(posedge clk or negedge rst_async_n)
    if (!rst_async_n) rff <= 2'b00;
    else rff <= {rff[0], 1'b1};
assign rst_sync_n = rff[1];

```

Q48. What is a reset-domain crossing (RDC) problem?

A flop reset by reset A drives a flop that is **not** reset by A. When A asserts asynchronously the source output changes at an arbitrary time relative to the destination clock, which can go metastable. Fix with reset sequencing, isolation of the path during reset, or synchronizing on the destination side.

Q49. How do you switch between two asynchronous clocks without glitches?

Glitch-free clock mux: each select path goes through a synchronizer clocked by (the falling edge of) its own clock, and is gated by the other path's disabled state - the old clock is turned off on its own low phase before the new one is enabled on its low phase.

Static timing analysis

Q50. What are the four timing path types?

Input port to register, register to register, register to output port, and input port to output port (combinational feed-through).

Q51. What is slack? Write the setup slack equation.

Slack = required - arrival. Setup: required = $T + \text{capture clock latency} - t_{\text{setup}} - \text{uncertainty}$; arrival = launch latency + $t_{\text{cq}} + t_{\text{path}}$. Negative slack is a violation.

Q52. Positive skew helps which check and hurts which?

Capture clock later than launch (positive skew) gives setup more time and makes hold harder. Useful skew is used deliberately by CTS to borrow time.

Q53. What is a multicycle path and why adjust hold too?

`set_multicycle_path 2 -setup` moves the setup check to the second edge; the hold check moves with it by default (to one edge before), which is too strict, so `set_multicycle_path 1 -hold` brings it back to the launch edge. The RTL must really hold data for two cycles (e.g. via an enable).

Q54. False path versus clock group?

`set_false_path` removes specific paths; `set_clock_groups -asynchronous` declares whole clocks unrelated so no paths between them are timed. Both hide real bugs if the crossing is not synchronized - CDC tools, not STA, verify those crossings.

Q55. What does `set_input_delay` model?

The time after the clock edge at which external data arrives at the input port - the external launch flop's clock-to-Q plus board and logic delay - so the tool knows how much of the period is left inside the chip.

Q56. What are OCV derates and CRPR?

On-chip variation makes nominally equal paths differ; derates slow the data path and speed the capture path (or vice versa for hold). Clock Reconvergence Pessimism Removal credits back the common part of the launch and capture clock paths, which cannot be both fast and slow at once.

Q57. Which corners do you check setup and hold at?

Setup at slow corners (SS, low V, high or low T depending on temperature inversion); hold at fast corners (FF, high V) - but sign-off checks both at all corners in the MCMM set.

Q58. How do you fix a setup violation from the RTL side?

Pipeline or retime, reduce logic depth (one-hot, precompute, parallel compare), remove late-arriving signals from deep cones, duplicate high-fanout registers, and break false priority chains.

Q59. What is the difference between a generated clock and a virtual clock?

A generated clock (`create_generated_clock`) is derived from a master inside the design (divider, mux); its latency includes the source path. A virtual clock has no source in the design and references IO delays to an external clock.

Q60. Why is a clock divider made from a flop a problem, and what is better?

It creates a new clock domain with skew relative to the source and requires a generated-clock constraint and CTS balancing. Prefer a clock enable (one flop gets `en` every N cycles) or an ICG driven by a counter.

Low power

Q61. Write the dynamic power equation and name the levers.

$P_{\text{dyn}} = \alpha C V^2 f$ (+ short-circuit). Levers: lower V (DVFS, multi-voltage), lower f , lower α (clock gating, operand isolation, data encoding), lower C (smaller cells, shorter wires).

Q62. Why is clock gating done with a latch-based ICG, not an AND gate?

An enable changing while the clock is high would chop the clock pulse. The latch is transparent while the clock is low, so the enable is frozen during the high phase. The ICG is a characterized library cell so STA checks the enable against the clock:

```
always_latch
  if (!clk) en_l = en | test_en;    // transparent while clk is low
assign gclk = clk & en_l;
```

Q63. How does RTL enable clock gating automatically?

Any flop bank with a common load enable (`if (en) q <= d;`) is a candidate; synthesis replaces the feedback mux with an ICG when the bank is wide enough (typically ≥ 3 -8 bits).

Q64. What is the difference between power gating and clock gating?

Clock gating stops switching (dynamic power) but keeps leakage and state. Power gating removes the supply (dynamic and leakage), needs isolation, retention or state restore, and costs wake-up latency and rush-current management.

Q65. What goes into a UPF file?

Power domains and their elements, supply nets and ports, power switches, isolation strategies (clamp value, enable, location), retention strategies (save/restore), level shifters and the power state table.

Q66. Why do you need isolation cells?

Outputs of a switched-off domain float to unknown levels, which causes X in simulation and crowbar current in real receivers. Isolation clamps them to 0/1/latched before power-down.

Design for test

Q67. What does scan insertion do to a flop?

Replaces it by a mux-D scan flop (`SE` selects `SI` or `D`) and stitches flops into chains. In shift mode the chain is a shift register; in capture mode flops capture functional logic outputs.

Q68. What RTL styles hurt testability?

Internally generated clocks and resets without test bypass, latches, combinational loops, gated clocks without `test_en`, tri-state buses, and X sources (uninitialized memories) reaching scan flops.

Q69. Stuck-at versus transition faults?

Stuck-at: node permanently 0/1, detected with slow-speed scan. Transition: node too slow to rise/fall; detected with at-speed launch-on-capture or launch-on-shift patterns using an OCC.

Q70. What is scan compression and why is it needed?

A decompressor expands a few tester channels into many short internal chains, and a compactor merges their outputs. Test time and data volume drop by 10-100x.

Q71. How are memories tested?

By MBIST: an on-chip controller runs March algorithms (e.g. March C-) and compares data; failing addresses can drive repair with redundant rows/columns.

Q72. Name the JTAG signals and the purpose of the TAP.

TCK, TMS, TDI, TDO, optional TRST. The TAP state machine selects the instruction register or a data register (BYPASS, IDCODE, boundary scan, user registers) for serial access.

Q73. What is IEEE 1500 and how does IEEE 1687 differ?

1500 defines a wrapper around a core (wrapper boundary register, WIR) so the core can be tested in isolation. 1687 (IJTAG) defines networks of embedded instruments with reconfigurable scan paths and portable procedures (ICL/PDL).

On-chip buses

Q74. Describe an APB write.

Setup phase: **PSEL** = 1, **PENABLE** = 0, address/data/**PWRITE** valid. Access phase: **PENABLE** = 1; the transfer completes on the first clock with **PREADY** = 1, with **PSLVERR** reporting errors.

Q75. What does AHB pipelining mean?

The address phase of transfer N+1 overlaps the data phase of transfer N. A subordinate stretches the data phase by driving **HREADY** low, which also holds the next address phase.

Q76. Why does AXI have five channels?

Separate write address, write data, write response, read address and read data allow reads and writes in parallel, data before or after address, multiple outstanding transactions and out-of-order completion by ID.

Q77. What are the AXI ordering rules for IDs?

Transactions with the same ID must complete in order (per direction); different IDs may complete out of order. Write data has no ID in AXI4, so it must follow the AW order.

Q78. Why may an AXI burst not cross a 4 KB boundary?

4 KB is the minimum page/subordinate address granularity; limiting bursts keeps one burst inside one subordinate so the interconnect decodes only the first address.

Q79. Compute the addresses of an AXI WRAP burst: **AxADDR** = 0x34, **AxSIZE** = 4 bytes, **AxLEN** = 3 (4 beats).

Wrap boundary = 16 bytes (4 x 4), lower boundary 0x30. Beats: 0x34, 0x38, 0x3C, 0x30.

Q80. AXI4-Lite versus AXI4?

Lite: every transaction is one beat, full data width, no IDs, bursts, locks, cache or QoS attributes - simple register access. AXI4: bursts up to 256 beats, IDs, outstanding and out-of-order.

Q81. What is a bus deadlock and how can an interconnect cause one?

A cycle of waits no one can break, e.g. a manager with outstanding reads to two subordinates with the same ID whose responses must be reordered, or write data interleaved wrongly. Interconnects avoid it with per-ID single-subordinate rules and bounded outstanding counts (Chapter 14).

Verification

Q82. What makes a testbench self-checking?

It predicts expected results with a reference model, compares them automatically in a scoreboard, and reports pass/fail with an error count - no waveform inspection required.

Q83. Code coverage 100% - are you done?

No. Code coverage shows lines were executed, not that results were checked or that required scenarios and corner cases (crosses, back-pressure, error responses) occurred. Functional coverage and checkers close the gap.

Q84. Immediate versus concurrent assertions?

Immediate: procedural, checked when executed like an `if`. Concurrent: clocked properties over time using sequences (`req |-> ##[1:3] ack`), sampled in the preponed region and usable by formal tools.

Q85. What does `|->` versus `|=>` mean?

`a |-> b`: if `a` holds, `b` must hold in the same cycle (overlapping). `a |=> b`: `b` must hold in the next cycle; equivalent to `a |-> ##1 b`.

Q86. Write an assertion: once `valid` is high it stays high with stable data until `ready`.

Shown without simulation (concurrent SVA):

```
property p_hold;
  @(posedge clk) disable iff (!rst_n)
    valid && !ready |=> valid && $stable(data);
endproperty
a_hold: assert property (p_hold) else $error("valid dropped");
```

Q87. What are the main UVM components?

Sequence item, sequence, sequencer, driver, monitor, agent (sequencer + driver + monitor), scoreboard, coverage collector, environment and test; connected with TLM ports and configured through `uvm_config_db`.

Q88. What is the UVM phasing order?

build, connect, end_of_elaboration, start_of_simulation, run (with parallel runtime sub-phases), extract, check, report, final. Build and final are top-down; the others are bottom-up.

Q89. What does formal verification give you that simulation cannot?

Exhaustive proof over all inputs and sequences up to a depth or unbounded, counterexamples in a few cycles, and early bring-up without a testbench. Limits: state explosion on large datapaths.

Q90. Why run gate-level simulation if equivalence checking passes?

To verify things LEC does not: reset/initialization with real X behaviour, timing with SDF (async paths, CDC structures), DFT modes and power-up sequences.

Q91. Why can RTL simulation hide a reset bug that silicon exposes?

X-optimism: `if (x_signal)` takes the else branch and `case` picks a branch, so an unreset flop looks deterministic in RTL. Gate-level or X-pessimistic simulation (X-prop modes) and reset lint catch it.

Design puzzles

Q92. Design a divide-by-3 clock with 50% duty cycle.

Count 0, 1, 2 on the rising edge and produce a pulse high for one input period. Delay that pulse by half a period with a falling-edge flop, and OR the two: the output is high for 1.5 of 3 periods. The OR is glitch-free because the second pulse rises before the first falls. In a real ASIC the output needs a generated-clock constraint and the OR should be a library clock cell.

The testbench below measures the output after reset with a 10 ns input clock.

```

module div3_50 (
    input  logic clk, rst_n,
    output logic clk_div3
);
    logic [1:0] cnt;
    logic      q_pos, q_neg;
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) cnt <= '0;
        else      cnt <= (cnt == 2'd2) ? 2'd0 : cnt + 2'd1;
    always_ff @(posedge clk or negedge rst_n) // high for 1 of 3 cycles
        if (!rst_n) q_pos <= 1'b0;
        else      q_pos <= (cnt == 2'd0);
    always_ff @(negedge clk or negedge rst_n) // same, half a cycle later
        if (!rst_n) q_neg <= 1'b0;
        else      q_neg <= q_pos;
    assign clk_div3 = q_pos | q_neg;          // 1.5 of 3 cycles high
endmodule

```

```

module tb_div3;
    logic clk = 0, rst_n = 0, y;
    always #5 clk = ~clk;                // 10 ns input clock
    div3_50 dut (.clk, .rst_n, .clk_div3(y));
    realtime tr = 0;
    always @(posedge y) begin
        if (tr > 0) $display("t=%0t period=%0t", $realtime, $realtime - tr);
        tr = $realtime;
    end
    always @(negedge y) if (rst_n) $display("t=%0t high=%0t", $realtime, $realtime - tr);
    initial begin
        #12 rst_n = 1;
        #75 $finish;
    end
endmodule

```

```

t=30 high=15
t=45 period=30
t=60 high=15
t=75 period=30
div3.sv:32: $finish called at 87 (1s)

```

Icarus output: 30 ns period, 15 ns high - exactly 50% duty.

Q93. Detect a rising and a falling edge of a synchronous signal.

Delay the signal by one flop and compare. The pulses are one clock wide. If the input is asynchronous, synchronize it first and detect the edge on the synchronizer output.

```

logic a_q;
always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) a_q <= 1'b0;
    else      a_q <= a;
assign rise = a & ~a_q;
assign fall = ~a & a_q;

```

Q94. Count the number of ones in a 16-bit word.

A **for** loop of additions describes it; synthesis builds an adder tree (or use **\$countones**). For wide words or high frequency, pipeline the tree. The output needs $\lceil \log_2(W+1) \rceil$ bits: 5 for $W = 16$.

```

module popcount #(parameter int W = 16) (
    input  logic [W-1:0]      x,
    output logic [$clog2(W+1)-1:0] n
);
    always_comb begin
        n = '0;
        for (int i = 0; i < W; i++)
            n = n + x[i];
        end
    endmodule

```

Q95. Check whether a number is a power of two; find its lowest set bit.

$x \& (x - 1)$ clears the lowest set bit, so a power of two gives zero. $x \& -x$ isolates the lowest set bit - also the grant vector of a fixed-priority arbiter where bit 0 has the highest priority.

```

assign is_pow2 = (x != '0) && ((x & (x - 1'b1)) == '0);
assign lsb1    = x & (~x + 1'b1);

```

Q96. Convert binary to Gray and Gray to binary.

Binary to Gray is one XOR level; Gray to binary is a prefix XOR from the MSB down (a serial chain of W-1 XORs):

```

assign gray_out = bin_in ^ (bin_in >> 1);
always_comb begin
    bin_out[W-1] = gray_in[W-1];
    for (int i = W-2; i >= 0; i--)
        bin_out[i] = bin_out[i+1] ^ gray_in[i];
    end

```

Q97. Design a round-robin arbiter for N requesters.

Keep a one-hot pointer to the last winner. Mask requests above the pointer; if any masked request exists grant its lowest bit ($m \& -m$), else grant the lowest bit of the unmasked requests. Update the pointer only when a grant is accepted. This is two fixed-priority arbiters and a mux - $O(\log N)$ depth with prefix logic.

Q98. Design a circuit that outputs 1 when the serial input so far, read MSB first, is divisible by 3.

Keep the remainder r in $\{0, 1, 2\}$. Each new bit b updates $r = (2r + b) \bmod 3$: a 3-state FSM. Output is $r == 0$. The same trick works for any modulus.

Q99. How many flops and what logic for a 0-to-9 BCD counter that can count up and down?

Four flops. Next state = up ? ($q == 9 ? 0 : q + 1$) : ($q == 0 ? 9 : q - 1$). Carry/borrow out = $en \& (up ? q == 9 : q == 0)$, used to enable the next decade.

Q100. A signal arrives from a 200 MHz domain as a one-cycle pulse and must be counted in a 25 MHz domain. Pulses can come every cycle. What do you do?

A pulse synchronizer cannot keep up (it needs pulses spaced by several destination cycles). Count in the source domain and pass the counter across as a Gray code (sampled value may be stale but never wrong), or use an async FIFO of events. Then compute differences in the destination.

HOW TO ANSWER DESIGN QUESTIONS IN INTERVIEWS

State assumptions (clock relationship, reset, widths, throughput), draw the block diagram before code, name the hardware each line infers, and finish with how you would verify it and what limits its timing. Interviewers grade the reasoning, not the syntax.

Appendix D. Tools, Standards and Resources

IEEE and industry standards

Standard	Title / scope	Where in this book
IEEE 1364-2005	Verilog HDL (merged into 1800 since 2009; no longer maintained separately)	Ch. 3-4
IEEE 1800-2023	SystemVerilog: design, assertions, coverage, classes, DPI	Ch. 3, 22, 23
IEEE 1800.2	UVM class library reference (2017, 2020 revisions)	Ch. 24
IEEE 1801	UPF - Unified Power Format (UPF 3.x in 1801-2018; newer revisions follow)	Ch. 19
IEEE 1149.1	JTAG boundary scan and TAP (2013 revision)	Ch. 20
IEEE 1500	Embedded core test wrapper	Ch. 20
IEEE 1687	IJTAG - access to embedded instruments (ICL/PDL)	Ch. 20
IEEE 1685	IP-XACT - XML description of IP, registers, memory maps (2014, 2022)	Ch. 15
IEEE 1497	SDF - Standard Delay Format	Ch. 18, 26
IEEE 1735	IP encryption and rights management	Ch. 21
IEEE 1666	SystemC (TLM models, HLS input)	Ch. 16
IEEE 754	Floating-point arithmetic	Ch. 8
Accellera SystemRDL 2.0	Register description language	Ch. 15
Accellera PSS	Portable Test and Stimulus Standard	Ch. 24
SDC	Synopsys Design Constraints (openly licensed Tcl subset)	Ch. 18
Liberty, LEF/DEF, SPEF	Library, physical and parasitic exchange formats	Ch. 17, 21
ISO 26262	Automotive functional safety (ASIL, FMEDA, safety mechanisms)	Ch. 27

AMBA and other bus specifications

Specification	Arm document	Notes
AMBA APB (APB2 ... APB5)	IHI 0024	Registers and slow peripherals; APB4 added PPROT/PSTRB
AMBA AHB / AHB5	IHI 0033	Pipelined single-outstanding bus; AHB-Lite is the single-manager subset
AMBA AXI (AXI3, AXI4, AXI5) incl. AXI4-Lite and ACE	IHI 0022	Main SoC data bus; ACE adds coherency
AMBA AXI-Stream (AXI4-Stream, AXI5-Stream)	IHI 0051	Streaming datapaths, accelerators
AMBA CHI	IHI 0050	Coherent Hub Interface for meshes
AMBA ATB, DTI, LTI	various	Trace, SMMU translation interfaces
TileLink	SiFive / CHIPS Alliance	RISC-V ecosystem (Rocket, Chipyard); TL-UL used by OpenTitan
Wishbone B4	OpenCores	Simple open bus, common in hobby SoCs

Specification	Arm document	Notes
Avalon	Intel FPGA	Memory-mapped and streaming, Platform Designer

Arm AMBA specifications are free to download from the Arm developer site after registration; always read the issue letter in the document ID, as signal lists changed between issues.

Open-source tools

Tool	Purpose	Notes
Icarus Verilog	Event-driven Verilog/SV simulator	Used for every example in this book; good Verilog-2005, partial SV
Verilator	SV to C++ compiler/simulator and linter	Fastest open simulator; <code>--lint-only -wall</code> is an excellent free linter; supports timing and many SV features since v5
Yosys	RTL synthesis framework	Generic, FPGA (iCE40, ECP5, Gowin ...) and ASIC flows; <code>read_verilog -sv</code>
OpenROAD	Floorplan-to-GDS physical design	Placement, CTS, routing, OpenSTA timing
OpenLane / OpenLane 2 / LibreLane	Automated RTL-to-GDS flow	Wraps Yosys + OpenROAD + Magic/KLayout + Netgen for open PDKs
OpenSTA	Static timing analyser	Reads Liberty, SDC, SPEF
SymbiYosys (sby)	Formal verification front end	BMC, k-induction, cover; SVA subset via Yosys (full SVA with the commercial Tabby CAD Suite)
cocotb	Python coroutine testbenches	Drives Icarus, Verilator and commercial simulators
GTKWave	Waveform viewer	VCD, FST, LXT2
Surfer	Modern waveform viewer	Fast, runs natively or in a browser; VCD/FST/GHW
Verible	SV parser, formatter, linter	From CHIPS Alliance
slang / sv2v / Surelog	SV front ends	Full-language elaboration; <code>sv2v</code> converts SV to Verilog for older tools
KLayout, Magic, Netgen	Layout viewer/DRC, layout editor, LVS	Open-PDK sign-off tools
FuseSoC / Edalize	IP package manager and tool launcher	Used by OpenTitan and many cores

Commercial tools by flow step

Flow step	Synopsys	Cadence	Siemens EDA
RTL simulation	VCS	Xcelium	Questa
Debug	Verdi	SimVision, Verisium Debug	Visualizer
Lint	VC SpyGlass Lint	Jasper Superlint, HAL	Questa Lint
CDC / RDC	VC SpyGlass CDC/RDC	Jasper CDC App	Questa CDC / RDC
Formal property checking	VC Formal	Jasper Formal Verification Platform	Questa Formal (PropCheck)
Low-power static checks	VC LP	Conformal Low Power	Questa Power Aware
Emulation / prototyping	ZeBu / HAPS	Palladium / Protium	Veloce
Logic synthesis	Design Compiler, Fusion Compiler	Genus	Catapult (HLS); Precision (FPGA)

Flow step	Synopsys	Cadence	Siemens EDA
DFT insertion and ATPG	TestMAX (DFTMAX, TetraMAX)	Modus	Tessent
Equivalence checking	Formality	Conformal LEC	Questa SLEC (sequential)
Place and route	IC Compiler II, Fusion Compiler	Innovus	Aprisa
Sign-off STA	PrimeTime	Tempus	-
Parasitic extraction	StarRC	Quantus	Calibre xACT
Power analysis / IR-EM	PrimePower, RedHawk-SC	Joules, Voltus	PowerPro (RTL power)
Physical verification (DRC/LVS)	IC Validator	Pegasus	Calibre

TOOL NAMES CHANGE

Vendors rename and bundle products frequently (and acquire each other - Synopsys completed its acquisition of Ansys in 2025). The flow step is the stable concept; check the current product name before quoting it in a resume or interview.

Papers and books

Cliff Cummings (Sunburst Design) SNUG papers - required reading

- **Nonblocking Assignments in Verilog Synthesis, Coding Styles That Kill!** (SNUG 2000) - the eight rules behind Chapter 4.
- **Simulation and Synthesis Techniques for Asynchronous FIFO Design** (SNUG 2002) and the follow-up with Peter Alfke on asynchronous pointer comparison.
- **Clock Domain Crossing (CDC) Design & Verification Techniques Using SystemVerilog** (SNUG 2008).
- **Synthesis and Scripting Techniques for Designing Multi-Asynchronous Clock Designs** (SNUG 2001).
- **Synchronous Resets? Asynchronous Resets? I am so confused!** and **Asynchronous & Synchronous Reset Design Techniques - Part Deux** (with Mills and Golson, SNUG 2002/2003).
- **full_case parallel_case, the Evil Twins of Verilog Synthesis** (SNUG 1999) and **State Machine Coding Styles for Synthesis**.

Books

Book	Level	Why read it
Harris & Harris, <i>Digital Design and Computer Architecture</i> (RISC-V edition)	Beginner	Logic to a pipelined RISC-V in SystemVerilog
Weste & Harris, <i>CMOS VLSI Design: A Circuits and Systems Perspective</i>	Intermediate	Transistors, timing, power, sequencing - what your RTL becomes
Pong P. Chu, <i>FPGA Prototyping by SystemVerilog Examples</i>	Beginner	Hands-on RTL design with complete projects
Stuart Sutherland, <i>RTL Modeling with SystemVerilog for Simulation and Synthesis</i>	Intermediate	Synthesis-oriented SV coding
Sutherland & Mills, <i>Verilog and SystemVerilog Gotchas</i>	Intermediate	101 language traps
Chris Spear & Greg Tumbush, <i>SystemVerilog for Verification</i>	Intermediate	OOP testbenches, randomization, coverage
Ray Salemi, <i>The UVM Primer</i>	Beginner (UVM)	The shortest path into UVM
Cerny et al., <i>SVA: The Power of Assertions in SystemVerilog</i>	Advanced	Complete SVA semantics

Book	Level	Why read it
Seligman, Schubert, Kumar, <i>Formal Verification: An Essential Toolkit</i>	Advanced	Practical formal methodology
Bhasker & Chadha, <i>Static Timing Analysis for Nanometer Designs</i>	Intermediate	STA and SDC in depth
Keating et al., <i>Low Power Methodology Manual</i>	Intermediate	Clock/power gating, multi-voltage
Bushnell & Agrawal, <i>Essentials of Electronic Testing</i>	Advanced	Fault models, ATPG, BIST
Kaeslin, <i>Top-Down Digital VLSI Design</i>	Intermediate	Architecture-to-silicon methodology
Dally & Towles, <i>Principles and Practices of Interconnection Networks</i>	Advanced	NoC theory
Hennessy & Patterson, <i>Computer Architecture: A Quantitative Approach</i>	Advanced	Pipelines, caches, memory systems

Open-source SoC and processor projects to study

Project	Language	What to learn from it
OpenTitan (lowRISC)	SystemVerilog	Production-quality silicon root of trust: style guide, reggen CSRs, TL-UL, full UVM and formal sign-off - the best public example of an industrial flow
Ibex (lowRISC)	SystemVerilog	Small 2/3-stage RV32 core, extensively verified
PULP Platform (ETH Zurich / Bologna)	SystemVerilog	AXI IP library (pulp-platform/axi), common_cells, clusters, PULPissimo, Cheshire
CVA6 (OpenHW Group)	SystemVerilog	64-bit Linux-capable application core, MMU, caches
Rocket Chip / Chipyard (Berkeley)	Chisel (Scala)	Generator-based SoCs, TileLink, BOOM out-of-order core
VexRiscv	SpinalHDL	Highly configurable RV32 core via plugins
PicoRV32	Verilog	Tiny, readable size-optimised RV32 core
SERV	Verilog	Bit-serial RISC-V - the smallest core

Getting real silicon: shuttles and open PDKs

Program / PDK	Details
Tiny Tapeout	Low-cost educational shuttle: many small designs share one chip behind a mux, submitted from a GitHub template with Verilog or cocotb tests. After Efabless closed in 2025 it moved to other fabrication routes, including IHP SG13G2 shuttles.
SkyWater SKY130	Open 130 nm PDK (Google + SkyWater, 2020); powered the free Google/Efabless Open MPW shuttles and remains the most documented open PDK.
GlobalFoundries GF180MCU	Open 180 nm PDK (Google + GF); good for mixed-signal and 5 V IO experiments.
IHP SG13G2	Open 130 nm SiGe BiCMOS PDK from IHP; IHP runs open MPW runs, some free for research and education.
Europractice / MOSIS-style MPW	Commercial-PDK shuttles for universities (TSMC, GF, UMC ...) under NDA; the normal academic route to advanced nodes.

A suggested order of use

- 1 Write and simulate with **Icarus** or **Verilator**, view with **GTKWave** or **Surfer**, lint with **verilator** `--lint-only -Wall`.
- 2 Test with **cocotb** or a SystemVerilog testbench (Chapter 22); prove small properties with **SymbiYosys**.
- 3 Synthesize with **Yosys** and read the statistics and critical path (Chapter 17); run a block through **OpenLane/LibreLane** on SKY130 or IHP to see the full RTL-to-GDS picture.
- 4 Put a design on **Tiny Tapeout** - nothing on a resume beats "my RTL is in silicon" (Chapter 30).
- 5 Read OpenTitan and PULP RTL for style; read Cummings' papers for the reasons behind the style.